INSTITUT SUPÉRIEUR D'ENSEIGNEMENT PROFESSIONNEL DE DIAMNIADIO

Département Informatique — Filière Développement backend

Programmation Java moderne

pour le développement backend

Du langage aux API REST · Java 25 LTS · nouveautés jusqu'à Java 27

Cours complet

- 19 chapitres, du premier programme à Spring Boot 4
- Exemples complets compilés et exécutés avec le JDK 25
- Plus de 100 exercices gradués, tous corrigés
- Un projet fil rouge : le backend de TerangaShop

Abdoulaye MBAYE

Enseignant, Département Informatique — ISEP Diamniadio

Année universitaire 2026 – 2027

Table des matières

Avant-propos 7
Public et prérequis 7
Version de référence 7
Conventions du document 7
Pourquoi un nouveau plan ? 8
Le fil rouge : TerangaShop 9
PARTIE I — Les bases du langage 10
Chapitre 1 — Java en 2026 : plate-forme, outils et premier programme 11
1.1 Qu'est-ce que Java ? 11
1.2 Comment un programme Java s'exécute 12
1.3 Les versions de Java et les LTS 12
1.4 Installer le JDK 13
1.5 Premier programme : trois façons de faire 14
1.6 JShell : expérimenter en direct 15
1.7 Les erreurs : compilation ou exécution ? 16
1.8 Conventions de nommage 16
Exercices du chapitre 1 17
Chapitre 2 — Variables, types et opérateurs 19
2.1 Déclarer une variable 19
2.2 Les huit types primitifs 20
2.3 Types primitifs et types référence 21
2.4 Les conversions de type (*cast*) 22
2.5 Ne jamais utiliser `double` pour de l'argent 23
2.6 Les opérateurs 24
2.7 La classe `Math` 26
2.8 Affichage formaté 26
2.9 Lire des données au clavier 27
2.10 Fil rouge TerangaShop — le ticket de caisse 28
Exercices du chapitre 2 28
Chapitre 3 — Chaînes de caractères et text blocks 30
3.1 La classe `String` 30
3.2 Les méthodes essentielles 30
3.3 Comparer deux chaînes : == ou `equals` ? 32
3.4 Construire du texte : `StringBuilder` 33
3.5 Les *text blocks* 33
3.6 Les caractères : la classe `Character` 35
3.7 Un premier contact avec les expressions régulières 35
3.8 Fil rouge TerangaShop — normaliser les données clients 36

Exercices du chapitre 3 37
Chapitre 4 — Structures de contrôle 39
4.1 Blocs et portée des variables 39
4.2 L'instruction `if` 39
4.3 Le `switch` classique 40
4.4 Les switch expressions 41
4.5 Les boucles 43
Exercices du chapitre 4 46
Chapitre 5 — Tableaux, méthodes et récursivité 47
5.1 Les tableaux 47
5.2 Les méthodes 49
5.3 La récursivité 51
5.4 Fil rouge TerangaShop — statistiques du mois 52
Exercices du chapitre 5 53
PARTIE II — Programmation orientée objet moderne 55
Chapitre 6 — Classes, objets et encapsulation 56
6.1 Penser « objet » 56
6.2 Une première classe 57
6.3 Les constructeurs 59
6.4 L'encapsulation 59
6.5 Membres d'instance et membres `static` 60
6.6 Packages et modificateurs d'accès 60
6.7 `toString`, `equals` et `hashCode` 61
6.8 Fil rouge TerangaShop — le modèle objet 61
6.9 Les objets immuables 66
Exercices du chapitre 6 66
Chapitre 7 — Héritage, polymorphisme, abstraction et interfaces 68
7.1 L'héritage 68
7.2 Redéfinition de méthodes 68
7.3 Le polymorphisme 69
7.4 Les classes abstraites 69
7.5 Les interfaces 69
7.6 Fil rouge TerangaShop — les modes de paiement 70
7.7 Classes internes et imbriquées 73
7.8 Composition plutôt qu'héritage 75
7.9 Les classes scellées 75
Exercices du chapitre 7 76
Chapitre 8 — Records, enums et pattern matching 78
8.1 Les records 78

8.2 Les énumérations (`enum`) 80
8.3 Le pattern matching 82
8.4 La programmation orientée données 84
8.5 Fil rouge TerangaShop — le journal des événements 85
Exercices du chapitre 8 86
Chapitre 9 — Exceptions et gestion des erreurs 88
9.1 Qu'est-ce qu'une exception ? 88
9.2 La hiérarchie des exceptions 89
9.3 Intercepter une exception : `try` / `catch` / `finally` 90
9.4 Lever une exception : `throw` 91
9.5 Déclarer une exception : `throws`, et la propagation 91
9.6 Libérer les ressources : *try-with-resources* 92
9.7 Créer ses propres exceptions 93
9.8 Bonnes pratiques 95
9.9 Côté backend : des exceptions aux codes HTTP 96
Exercices du chapitre 9 96
PARTIE III — La bibliothèque standard du développeur backend 98
Chapitre 10 — Les génériques 99
10.1 Pourquoi les génériques ? 99
10.2 Classes et records génériques 99
10.3 Méthodes génériques et bornes 100
10.4 L'invariance et les jokers (*wildcards*) 101
10.5 L'effacement de type et ses limites 102
10.6 Fil rouge TerangaShop — un dépôt générique 102
Exercices du chapitre 10 104
Chapitre 11 — Les collections 105
11.1 Vue d'ensemble 105
11.2 Les listes 105
11.3 Collections immuables et vues non modifiables 107
11.4 Les ensembles (`Set`) 108
11.5 Les tables associatives (`Map`) 108
11.6 Files et piles : `Queue`, `Deque`, `PriorityQueue` 110
11.7 Les *Sequenced Collections* 111
11.8 Trier : `Comparable` et `Comparator` 111
11.9 Choisir la bonne collection 112
Exercices du chapitre 11 113
Chapitre 12 — Lambdas, Optional et Streams 115
12.1 Des classes anonymes aux lambdas 115
12.2 Les interfaces fonctionnelles standard 116

12.3 Les références de méthodes 117
12.4 `Optional` : représenter une valeur absente 117
12.5 L'API Stream 119
12.6 Les collecteurs 121
12.7 Les *Stream Gatherers* 123
12.8 Pièges et bonnes pratiques 124
Exercices du chapitre 12 124
Chapitre 13 — Dates, fichiers, configuration et journalisation 126
13.1 Dates et heures avec `java.time` 126
13.2 Lire et écrire des fichiers avec NIO.2 128
13.3 Externaliser la configuration 130
13.4 Journaliser au lieu d'afficher 131
Exercices du chapitre 13 132
Chapitre 14 — Concurrence moderne et threads virtuels 134
14.1 Pourquoi la concurrence en backend ? 134
14.2 Créer des threads 134
14.3 Les conditions de course 135
14.4 Les exécuteurs et les `Future` 136
14.5 Composer des traitements asynchrones : `CompletableFuture` 137
14.6 Les threads virtuels 138
14.7 Les collections concurrentes 139
14.8 Transmettre un contexte : `ScopedValue` 140
14.9 En preview : la concurrence structurée 141
Exercices du chapitre 14 142
PARTIE IV — Java côté serveur 144
Chapitre 15 — Organiser un projet : Maven et tests JUnit 145
15.1 Pourquoi un outil de construction ? 145
15.2 La structure standard d'un projet Maven 145
15.3 Le fichier `pom.xml` 145
15.4 Les commandes Maven 147
15.5 Pourquoi tester ? 147
15.6 Écrire des tests avec JUnit 148
15.7 Isoler le code testé : les doublures de test 150
15.8 Le développement piloté par les tests (TDD) 152
Exercices du chapitre 15 152
Chapitre 16 — Accéder à PostgreSQL avec JDBC 154
16.1 L'architecture de JDBC 154
16.2 Préparer la base 155
16.3 Se connecter 155

16.4 Insérer des données : `PreparedStatement` et traitement par lots 157
16.5 Lire des données : `ResultSet` 158
16.6 Les injections SQL 159
16.7 Clés générées et codes d'erreur 160
16.8 Les transactions 161
16.9 Le patron DAO et le pool de connexions 164
16.10 Au-delà de JDBC 167
Exercices du chapitre 16 167
Chapitre 17 — HTTP et API REST en Java pur 169
17.1 HTTP en bref 169
17.2 JSON et Jackson 170
17.3 Appeler une API : `HttpClient` 171
17.4 Une API REST sans framework 172
17.5 Ce qu'il reste à faire… et pourquoi utiliser un framework 176
Exercices du chapitre 17 176
Chapitre 18 — Introduction à Spring Boot 4 178
18.1 Spring et Spring Boot 178
18.2 Créer le projet 178
18.3 La configuration 180
18.4 L'injection de dépendances 181
18.5 L'entité JPA 181
18.6 Le *repository* Spring Data 182
18.7 Les DTO et la validation 183
18.8 Le service 183
18.9 Le contrôleur REST 185
18.10 La gestion centralisée des erreurs 186
18.11 Démarrer et tester l'application 187
18.12 Pour aller plus loin 188
Exercices du chapitre 18 189
PARTIE V — Projet et annexes 190
Chapitre 19 — Projet intégrateur : l'API TerangaShop 191
19.1 Le contexte 191
19.2 Les fonctionnalités attendues 191
19.3 Les routes minimales de l'API 192
19.4 Les exigences techniques 192
19.5 Le déroulement 193
19.6 La grille d'évaluation 193
Annexe A — Chronologie des nouveautés de Java 8 à Java 27 194
Annexe B — Aide-mémoire 195

B.1 Commandes du JDK 195
B.2 Commandes Maven 195
B.3 Syntaxe moderne en un coup d'œil 195
Annexe C — Corrigés des exercices 198
Corrigés du chapitre 1 198
Corrigés du chapitre 2 200
Corrigés du chapitre 3 203
Corrigés du chapitre 4 206
Corrigés du chapitre 5 211
Corrigés du chapitre 6 215
Corrigés du chapitre 7 222
Corrigés du chapitre 8 227
Corrigés du chapitre 9 232
Corrigés du chapitre 10 237
Corrigés du chapitre 11 242
Corrigés du chapitre 12 246
Corrigés du chapitre 13 250
Corrigés du chapitre 14 257
Corrigés du chapitre 15 262
Corrigés du chapitre 16 266
Corrigés du chapitre 17 272
Corrigés du chapitre 18 275

Avant-propos

Ce document est un cours complet de programmation Java destiné aux étudiants de la filière **Développement backend**. Il a un objectif clair : **former des développeurs capables d'écrire du code serveur Java propre, testé et moderne**.

Le langage a beaucoup changé depuis Java 8. Les *records*, le *pattern matching*, les *switch expressions*, les *text blocks*, les *threads virtuels* ou encore les *fichiers source compacts* de Java 25 modifient la façon d'écrire du code au quotidien. Un étudiant qui apprend Java aujourd'hui doit apprendre directement ce Java-là, tout en sachant lire le code plus ancien qu'il rencontrera en entreprise.

Public et prérequis

- Étudiants de première ou deuxième année en développement backend (BTS, DUT, Licence professionnelle, TS).
- Prérequis : algorithmique de base (variables, conditions, boucles, fonctions) et, idéalement, une première expérience en C.
- Pour les chapitres 16 à 18 : notions de SQL (PostgreSQL) et de protocole HTTP.

Version de référence

Tout le code de ce cours a été **compilé et exécuté avec le JDK 25** (version LTS, *Long-Term Support*, sortie en septembre 2025). Les nouveautés de Java 26 (mars 2026) et Java 27 (septembre 2026) sont signalées lorsqu'elles concernent le développeur backend. Les fonctionnalités encore en *preview* sont clairement identifiées : elles doivent être activées avec l'option `--enable-preview` et peuvent encore évoluer.

Conventions du document

Le cours utilise des encadrés de couleur pour structurer la lecture :

> **Nouveauté Java**
> Signale une fonctionnalité apparue récemment dans le langage, avec la version qui l'a introduite. Ces encadrés vous aident à reconnaître le code « moderne » et le code « historique ».

> **Côté backend**
> Relie la notion étudiée à une situation réelle de développement serveur : API REST, base de données, performance, sécurité.

> **Piège à éviter**
> Décrit une erreur fréquente chez les débutants (et parfois chez les développeurs expérimentés).

> **Astuce**
> Donne une bonne pratique, un raccourci ou une façon plus élégante d'écrire le code.

> **À retenir**
> Résume l'essentiel d'une section.

Les programmes complets sont présentés dans des blocs de code avec le nom du fichier ; leur **sortie console** est reproduite juste en dessous. Les exercices sont notés de ★ (application directe) à ★★★ (problème de synthèse). Les **corrigés** de tous les exercices se trouvent en annexe C : essayez toujours de chercher avant de les consulter.

Pourquoi un nouveau plan ?

Le plan des anciens supports suivait l'ordre historique du langage : syntaxe, POO, puis interfaces graphiques (Swing, Applets), threads et JDBC. Pour un développeur backend, certaines de ces parties sont devenues inutiles, et d'autres, indispensables, manquaient. Le tableau suivant résume les choix faits.

Ancien support	Décision	Où dans ce cours
Introduction, histoire, plate-forme (séance 1)	Conservé et actualisé (versions LTS, JDK 25, jshell)	Chapitre 1
Syntaxe, variables, types, cast, opérateurs (séance 2)	Conservé, enrichi (`var`, débordements, argent en FCFA)	Chapitre 2
Chaînes de caractères (UCAD module 7)	Conservé, enrichi (text blocks, `formatted`)	Chapitre 3
Conditions, switch, boucles, tableaux (séance 3)	Conservé, enrichi (switch expressions)	Chapitres 4 et 5
Méthodes, surcharge, récursivité (séance 4)	Conservé	Chapitre 5
Classes, objets, constructeurs (séance 5)	Conservé, enrichi (immutabilité, `equals`/`hashCode`)	Chapitre 6
Paquets, héritage, polymorphisme, interfaces (séance 6)	Conservé, enrichi (classes scellées)	Chapitre 7
—	**Ajouté** : records, enums, pattern matching	Chapitre 8
Exceptions (séance 7)	Conservé, enrichi (try-with-resources, exceptions métier)	Chapitre 9
Collections (UCAD module 13)	Réécrit (génériques, Sequenced Collections)	Chapitres 10 et 11
—	**Ajouté** : lambdas, Streams, Optional	Chapitre 12
Entrées/sorties (UCAD module 12)	Réécrit avec NIO.2 et `java.time`	Chapitre 13
Threads (UCAD module 10)	Réécrit (ExecutorService, threads virtuels)	Chapitre 14
—	**Ajouté** : Maven, tests JUnit 5	Chapitre 15
JDBC (séance 8)	Modernisé (plus de `Class.forName`, pool, DAO, PostgreSQL)	Chapitre 16
—	**Ajouté** : HTTP, API REST, Spring Boot	Chapitres 17 et 18
Swing, Applets (UCAD modules 11 et 14)	**Retiré** : hors du périmètre backend ; l'API Applet a été supprimée du JDK en Java 26	—
Pont JDBC-ODBC, pilotes de type 1 à 3	**Retiré** : le pont n'existe plus depuis Java 8 ; seul le pilote de type 4 est utilisé aujourd'hui	Mention au chapitre 16

Le fil rouge : TerangaShop

Pour relier les chapitres entre eux, nous construisons progressivement le backend de **TerangaShop**, une petite boutique en ligne basée à Dakar. Elle vend des produits locaux (bissap, thiakry, pagnes wax, sacs en cuir de Ngaye Mékhé…), encaisse les paiements par Wave, Orange Money ou en espèces à la livraison, et livre dans plusieurs quartiers : Plateau, Médina, Parcelles Assainies, Rufisque, Diamniadio.

Au fil du cours, TerangaShop va évoluer :

- chapitres 2 à 5 : calculs de prix, de remises et de frais de livraison ;
- chapitres 6 à 9 : modélisation objet des produits, clients et commandes, avec des exceptions métier ;
- chapitres 10 à 14 : catalogue en mémoire, statistiques de ventes avec les Streams, import de fichiers CSV, traitement concurrent des commandes ;
- chapitres 15 à 19 : persistance dans PostgreSQL, tests automatisés, puis exposition d'une API REST.

Tous les montants sont exprimés en **francs CFA (FCFA)**. Comme le franc CFA ne comporte pas de centimes, nous les représentons par des entiers de type `long` ; nous verrons au chapitre 2 pourquoi il ne faut jamais utiliser `double` pour de l'argent.

PARTIE I — Les bases du langage

- Chapitre 1 — Java en 2026 : plate-forme, outils et premier programme
- Chapitre 2 — Variables, types et opérateurs
- Chapitre 3 — Chaînes de caractères et text blocks
- Chapitre 4 — Structures de contrôle
- Chapitre 5 — Tableaux, méthodes et récursivité

Chapitre 1 — Java en 2026 : plate-forme, outils et premier programme

Objectifs du chapitre

- Expliquer le rôle de la JVM, du JDK et du bytecode.
- Connaître le rythme de sortie des versions et la notion de version LTS.
- Installer un JDK et vérifier l'installation.
- Écrire, compiler et exécuter un premier programme de trois façons différentes.

1.1 Qu'est-ce que Java ?

Java est un **langage de programmation orienté objet**, à typage statique et fort, créé chez Sun Microsystems par l'équipe de James Gosling. Le projet démarre en 1991 sous le nom *Oak*, destiné à l'électronique grand public ; il est rebaptisé Java et publié en 1995, au moment où le Web explose. Oracle rachète Sun en 2010 et pilote depuis l'évolution de la plate-forme, dont le code source est ouvert au sein du projet **OpenJDK**.

Trente ans plus tard, Java reste l'un des langages les plus utilisés au monde, et tout particulièrement **côté serveur** :

- les systèmes bancaires, les assurances, les opérateurs télécoms et les plateformes de paiement mobile reposent massivement sur Java ;
- les frameworks backend les plus répandus en entreprise sont écrits en Java : Spring Boot, Quarkus, Micronaut, Jakarta EE ;
- de nombreux outils d'infrastructure sont écrits en Java ou tournent sur la JVM : Apache Kafka, Elasticsearch, Apache Cassandra, Jenkins ;
- Android utilise le langage Java (et Kotlin, un autre langage de la JVM).

Les grandes qualités du langage expliquent ce succès :

Qualité	Signification concrète
Portable	Le même programme compilé s'exécute sur Windows, Linux ou macOS sans modification (« *Write once, run anywhere* »).
Orienté objet	Le code est organisé en classes et objets, ce qui facilite la modélisation de domaines complexes.
Typage statique et fort	Le compilateur détecte de nombreuses erreurs avant l'exécution.
Gestion automatique de la mémoire	Le *ramasse-miettes* (*Garbage Collector*) libère la mémoire inutilisée : pas de `malloc`/`free` comme en C.
Robuste et sûr	Pas d'arithmétique de pointeurs, vérification des indices de tableaux, gestion des erreurs par exceptions.
Multithread	La concurrence fait partie du langage depuis l'origine ; les threads virtuels (Java 21) la rendent très économique.
Riche bibliothèque standard	Collections, dates, réseau, HTTP, bases de données, cryptographie… sont fournis avec le JDK.
Rétrocompatible	Un programme écrit il y a 20 ans compile et tourne généralement encore aujourd'hui.

1.2 Comment un programme Java s'exécute

En C, le compilateur traduit le code source directement en **code machine** propre à un processeur et à un système d'exploitation. Java procède en deux temps.

1. Le compilateur `javac` traduit le fichier source `.java` en **bytecode**, stocké dans un fichier `.class`. Le bytecode est un code intermédiaire, indépendant du processeur et du système.
2. Au lancement, la **machine virtuelle Java** (JVM, *Java Virtual Machine*) charge ce bytecode, le vérifie, puis l'exécute. Pour aller vite, son compilateur **JIT** (*Just-In-Time*) traduit à la volée en code machine les parties du programme les plus souvent exécutées.

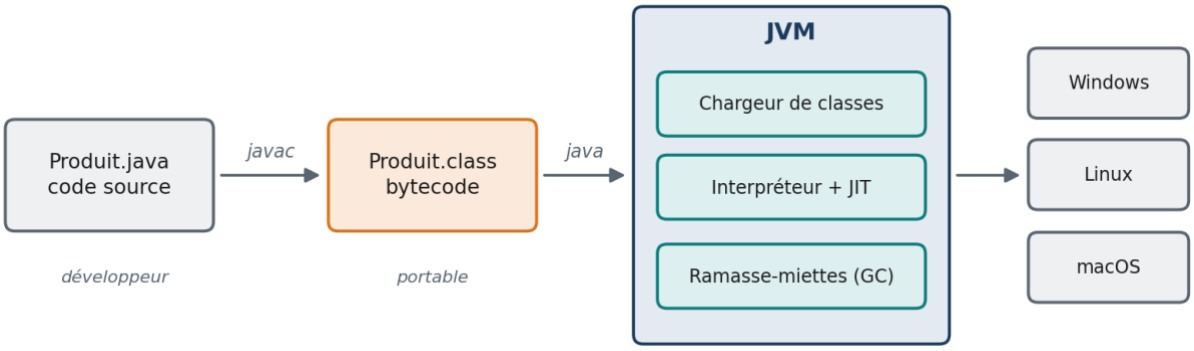


Figure 1.1 — Du code source à l'exécution : compilation en bytecode puis exécution par la JVM

C'est la JVM qui est spécifique à chaque système ; le bytecode, lui, est universel. C'est ce qui rend Java portable.

JVM, JRE et JDK

Sigle	Nom complet	Contenu	Pour qui ?
JVM	Java Virtual Machine	Le moteur qui exécute le bytecode (chargeur de classes, JIT, ramasse-miettes).	Intégrée au JDK
JRE	Java Runtime Environment	JVM + bibliothèques standard : de quoi *exécuter* un programme.	Historique : depuis Java 11, il n'est plus distribué séparément
JDK	Java Development Kit	JRE + outils de développement : `javac`, `java`, `jshell`, `jar`, `javadoc`, `jlink`…	**Le développeur : c'est ce que vous installez**

> **Côté backend**
> En production, on n'installe plus un « JRE » générique sur le serveur. On construit une image Docker contenant un JDK (ou un *runtime* minimal produit avec l'outil `jlink`) et l'application. Le choix de la version de Java fait donc partie du projet, au même titre que ses dépendances.

1.3 Les versions de Java et les LTS

Depuis Java 10 (mars 2018), une nouvelle version sort **tous les six mois**, en mars et en septembre. Certaines versions sont désignées **LTS** (*Long-Term Support*) : elles reçoivent des correctifs de sécurité pendant plusieurs années. Les entreprises utilisent presque exclusivement des versions LTS en production.

Version	Date	Statut	Apports marquants pour le développeur
Java 8	mars 2014	LTS (ancienne)	Lambdas, Streams, `Optional`, API `java.time`

Version	Date	Statut	Apports marquants pour le développeur
Java 11	sept. 2018	LTS	`var` dans les lambdas, `HttpClient`, lancement direct d'un fichier `.java`
Java 17	sept. 2021	LTS	Records, classes scellées, text blocks, switch expressions, pattern matching `instanceof`
Java 21	sept. 2023	LTS	Threads virtuels, pattern matching dans `switch`, record patterns, Sequenced Collections
Java 25	**sept. 2025**	**LTS (référence de ce cours)**	Fichiers source compacts, `main` d'instance, constructeurs flexibles, `ScopedValue`, import de modules
Java 26	mars 2026	Non-LTS	HTTP/3 dans `HttpClient`, suppression de l'API Applet
Java 27	sept. 2026	Non-LTS	G1 comme ramasse-miettes par défaut partout, en-têtes d'objets compacts par défaut, échange de clés TLS post-quantique

La prochaine LTS est attendue pour septembre 2027 (Java 29). L'annexe A récapitule toutes les nouveautés version par version.

> **Preview, incubator : de quoi parle-t-on ?**
>
> Une fonctionnalité **preview** est complète mais pas encore définitive : on peut l'essayer en ajoutant l'option `--enable-preview` à `javac` et `java`, et elle peut encore changer. Une API **incubator** est encore plus expérimentale et vit dans un module `jdk.incubator.*`. Ce cours signale toujours clairement ces fonctionnalités ; ne les utilisez pas dans un projet en production.

1.4 Installer le JDK

Plusieurs distributions du JDK existent. Elles sont toutes construites à partir d'OpenJDK et sont équivalentes pour l'apprentissage :

- **Eclipse Temurin** (Adoptium) — gratuite, la plus utilisée ; c'est celle que nous recommandons ;
- Oracle JDK, Amazon Corretto, Microsoft Build of OpenJDK, Azul Zulu, BellSoft Liberica.

Sous Windows : téléchargez l'installateur `.msi` de Temurin 25 sur le site adoptium.net et cochez l'option « Set JAVA_HOME variable » pendant l'installation.

Sous Linux (Ubuntu/Debian) :

Terminal

```
sudo apt update
sudo apt install openjdk-25-jdk
```

Sous macOS, Linux ou WSL, l'outil SDKMAN! permet d'installer et de changer de version facilement :

Terminal

```
curl -s "https://get.sdkman.io" | bash
sdk install java 25-tem      # installe Temurin 25
sdk use java 21-tem          # bascule temporairement sur Java 21
```

Vérifiez ensuite l'installation dans un terminal :

Terminal

```
java -version
javac -version
```

Sortie console

```
openjdk version "25.0.4.1" 2026-08-18
OpenJDK Runtime Environment (build 25.0.4.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 25.0.4.1+1-1-24.04.4-Ubuntu, mixed mode, sharing)
javac 25.0.4.1
```

Quel éditeur choisir ?

IntelliJ IDEA Community (gratuit) est l'environnement de référence pour Java en entreprise. **VS Code** avec l'extension « Extension Pack for Java » est une alternative plus légère. Pour les tout premiers chapitres, un simple éditeur de texte et un terminal suffisent : c'est même recommandé pour bien comprendre ce que fait l'IDE à votre place.

1.5 Premier programme : trois façons de faire

Façon 1 — Le programme « classique »

Créez un fichier nommé exactement `Bienvenue.java` :

Bienvenue.java

```java
public class Bienvenue {
    public static void main(String[] args) {
        System.out.println("Bienvenue à ISEP Diamniadio !");
        System.out.println("Filière : Développement backend");
    }
}
```

Compilez-le puis exécutez-le :

Terminal

```
javac Bienvenue.java      # produit Bienvenue.class (le bytecode)
java Bienvenue            # lance la JVM sur la classe Bienvenue (sans .class !)
```

Sortie console

```
Bienvenue à ISEP Diamniadio !
Filière : Développement backend
```

Analysons chaque élément :

Élément	Rôle
`public class Bienvenue`	Déclare une classe publique. Tout code Java vit dans une classe. Le nom de la classe publique **doit être identique au nom du fichier** (`Bienvenue.java`), majuscules comprises.
`{ … }`	Les accolades délimitent un bloc (le corps de la classe, puis celui de la méthode).
`public static void main(String[] args)`	Le point d'entrée du programme : la JVM cherche cette méthode et l'exécute. `public` : accessible de l'extérieur ; `static` : appelable sans créer d'objet ; `void` : ne renvoie rien ; `args` : les arguments de la ligne de commande.
`System.out.println(…)`	Affiche un texte suivi d'un retour à la ligne. `System` est une classe, `out` le flux de sortie standard, `println` une méthode.
`;`	Chaque instruction se termine par un point-virgule.

Façon 2 — Lancer directement le fichier source

Depuis Java 11, la commande `java` sait compiler en mémoire et exécuter un fichier source en une seule étape ; depuis Java 22, cela fonctionne même si le programme est réparti sur plusieurs fichiers.

Terminal

```
java Bienvenue.java
```

C'est très pratique pour les petits programmes et les scripts. Pour un vrai projet, on compile toujours (via Maven ou Gradle, chapitre 15).

Façon 3 — Le fichier source compact (Java 25)

> **Nouveauté Java 25 — Fichiers source compacts et méthode main d'instance (JEP 512)**
>
> Pour les débutants et les petits scripts, Java 25 permet d'écrire un programme **sans déclarer de classe**, avec une méthode `main` qui n'est ni `public` ni `static`, et sans paramètre. La nouvelle classe `java.lang.IO` offre les méthodes `IO.println`, `IO.print` et `IO.readln`. Enfin, un fichier compact importe automatiquement tout le module `java.base` : `List`, `Map`, `LocalDate`… sont utilisables sans `import`.

Salut.java

```java
void main() {
    IO.println("Salut depuis un fichier source compact !");
    var quartiers = List.of("Plateau", "Médina", "Diamniadio");
    IO.println("Zones de livraison : " + quartiers);
}
```

Terminal

```
java Salut.java
```

Sortie console

```
Salut depuis un fichier source compact !
Zones de livraison : [Plateau, Médina, Diamniadio]
```

Le compilateur génère en réalité une classe implicite (ici nommée `Salut`) autour de ce code. Cette forme est idéale pour s'exercer ; dès le chapitre 6, nous écrirons des classes complètes, comme dans tout projet professionnel.

> **Piège à éviter**
>
> Les fichiers compacts n'existent qu'à partir de Java 25. Si votre machine a un JDK plus ancien, vous obtiendrez une erreur de compilation du type `class, interface, enum, or record expected`. Vérifiez toujours votre version avec `java -version`.

1.6 JShell : expérimenter en direct

Le JDK fournit depuis Java 9 un interpréteur interactif, **JShell** (une *REPL*, *Read-Eval-Print Loop*). Il permet de tester une expression ou une méthode sans créer de fichier.

```
$ jshell
|  Welcome to JShell -- Version 25.0.4.1
jshell> 12500 * 3
$1 ==> 37500

jshell> var prix = 4500
prix ==> 4500

jshell> "bissap".toUpperCase()
$3 ==> "BISSAP"

jshell> Math.sqrt(144)
$4 ==> 12.0

jshell> /exit
```

Les commandes internes commencent par `/` : `/vars` liste les variables, `/methods` les méthodes, `/imports` les imports, `/help` l'aide.

1.7 Les erreurs : compilation ou exécution ?

On distingue deux grandes familles d'erreurs.

- **Erreurs de compilation** : le code ne respecte pas les règles du langage (point-virgule manquant, type incompatible, variable inconnue). `javac` refuse de produire le `.class` et indique le fichier, la ligne et la nature de l'erreur.
- **Erreurs d'exécution** : le programme compile mais échoue en cours d'exécution (division entière par zéro, indice de tableau hors limites…). La JVM lève une **exception** (chapitre 9).

Exemple d'erreur de compilation :

Java

```java
public class Erreur {
    public static void main(String[] args) {
        System.out.println("Il manque un point-virgule")
    }
}
```

Sortie console

```
Erreur.java:3: error: ';' expected
        System.out.println("Il manque un point-virgule")
                                                       ^
1 error
```

> **Astuce**
>
> Lisez toujours **la première** erreur signalée par le compilateur et corrigez-la en premier : les suivantes en sont souvent la conséquence.

1.8 Conventions de nommage

Java possède des conventions universellement respectées. Les suivre rend votre code immédiatement lisible par n'importe quel développeur Java.

Élément	Convention	Exemples
Classe, interface, record, enum	`PascalCase` (majuscule à chaque mot)	`Produit`, `CommandeService`, `ModePaiement`
Méthode, variable, paramètre	`camelCase` (minuscule au début)	`calculerTotal()`, `prixUnitaire`, `nomClient`
Constante (`static final`)	`MAJUSCULES_AVEC_TIRETS`	`TAUX_TVA`, `FRAIS_LIVRAISON_MIN`
Package	tout en minuscules, domaine inversé	`sn.isep.terangashop.service`

Exercices du chapitre 1

Exercice 1.1 ★ — Mon premier programme

Écrivez un programme `Presentation.java` (forme classique, avec une classe) qui affiche sur trois lignes votre prénom et nom, votre filière et la phrase « J'apprends Java 25 ». Compilez-le avec `javac`, exécutez-le avec `java`, puis relancez-le directement avec `java Presentation.java`. Observez les fichiers présents dans le dossier après chaque étape.

Exercice 1.2 ★ — Questions de cours

1. Quelle est la différence entre le JDK et la JVM ?
2. Que contient un fichier `.class` ? Peut-on l'exécuter sur un autre système d'exploitation que celui qui l'a produit ?
3. Pourquoi une entreprise choisit-elle Java 25 plutôt que Java 27 pour son application de production ?
4. Qu'est-ce qu'une fonctionnalité *preview* ?

Exercice 1.3 ★★ — Chasse aux erreurs

Le programme suivant contient **cinq erreurs de compilation**. Trouvez-les sans compilateur, corrigez-les, puis vérifiez avec `javac`.

Java

```java
public class Boutique {
    public static void main(string[] args) {
        System.out.printn("TerangaShop ouvre ses portes !")
        System.out.println("Livraison à Diamniadio);
    }
```

Exercice 1.4 ★★ — Explorer JShell

Dans JShell, évaluez les expressions suivantes et notez le résultat et son type (utilisez la commande `/vars` après avoir affecté chaque résultat à une variable) : `7 / 2`, `7 / 2.0`, `7 % 2`, `"Dakar".length()`, `"Dakar" + 221`, `221 + 1 + "Dakar"`, `"Dakar" + 221 + 1`. Expliquez les deux dernières différences.

Exercice 1.5 ★★ — Carte de visite TerangaShop

Écrivez un fichier source compact (Java 25) `Carte.java` qui affiche exactement la carte suivante :

Sortie console

```
+------------------------------+
|          TERANGASHOP         |
|  Produits locaux, livrés chez |
|  vous à Dakar et environs    |
|  Tél : 33 800 00 00          |
+------------------------------+
```

Chapitre 2 — Variables, types et opérateurs

Objectifs du chapitre

- Déclarer des variables typées, des constantes et utiliser l'inférence de type `var`.
- Connaître les huit types primitifs, leurs limites et les pièges de conversion.
- Savoir pourquoi on ne représente jamais une somme d'argent avec un `double`.
- Maîtriser les opérateurs, la classe `Math`, la saisie au clavier et l'affichage formaté.

2.1 Déclarer une variable

Une **variable** est un emplacement mémoire nommé qui contient une valeur d'un **type** donné. Java est un langage à **typage statique** : le type de chaque variable est connu à la compilation et ne change jamais.

Java

```java
int quantite;            // déclaration : réserve une variable de type int
quantite = 3;            // affectation
int stock = 120;         // déclaration + initialisation
stock = stock - quantite; // réaffectation : stock vaut maintenant 117
String produit = "Bissap 1 L";
double noteClient = 4.5;
boolean enStock = true;
char taille = 'M';
```

Plusieurs variables de même type peuvent être déclarées sur une ligne (`int x = 1, y = 2;`), mais une déclaration par ligne reste plus lisible.

Piège à éviter

Une variable **locale** (déclarée dans une méthode) doit être initialisée avant d'être lue. Le code `int total; System.out.println(total);` ne compile pas : *variable total might not have been initialized*. C'est une protection : en C, vous obtiendriez une valeur aléatoire.

Les constantes : `final`

Le mot-clé `final` interdit toute réaffectation. Par convention, les constantes de classe (`static final`) s'écrivent en majuscules.

Java

```java
final int JOURS_RETOUR = 7;
JOURS_RETOUR = 10;   // ERREUR de compilation : cannot assign a value to final variable
```

L'inférence de type : var

Nouveauté Java 10 — Inférence de type des variables locales

Le mot `var` permet de laisser le compilateur **déduire** le type d'une variable locale à partir de sa valeur d'initialisation. Le typage reste statique : une fois déduit, le type ne change plus.

Java

```java
var prix = 4500;                 // int
var libelle = "Thiakry 500 g";   // String
var clients = new ArrayList<String>();  // ArrayList<String>
prix = "gratuit";   // ERREUR : prix est un int, pour toujours
```

Règles d'usage de `var` :

- uniquement pour les **variables locales** (pas pour les attributs, les paramètres ou les types de retour) ;
- l'initialisation est obligatoire (`var x;` ne compile pas), et `var x = null;` est interdit (type impossible à déduire) ;
- à utiliser quand le type est **évident** à la lecture (`var liste = new ArrayList<Produit>()`), à éviter quand il ne l'est pas (`var r = service.traiter();`).

Identifiants valides

Un nom de variable commence par une lettre, `_` ou `$`, puis contient des lettres, des chiffres, `_` ou `$`. Il est sensible à la casse (`prix` et `Prix` sont deux variables différentes) et ne peut pas être un mot réservé (`class`, `int`, `new`, `return`…). Depuis Java 9, `_` seul n'est plus un identifiant ; depuis Java 22, il sert de *variable anonyme* (chapitre 8).

2.2 Les huit types primitifs

Les types **primitifs** contiennent directement une valeur (et non une référence vers un objet). Leur taille est fixée par la spécification : elle est identique sur toutes les machines, contrairement au C.

Type	Taille	Plage de valeurs	Exemple de littéral
`byte`	8 bits	-128 à 127	`(byte) 100`
`short`	16 bits	-32 768 à 32 767	`(short) 3000`
`int`	32 bits	environ ±2,1 milliards (-2^{31} à $2^{31}-1$)	`42`, `1_000_000`, `0xFF`, `0b1010`
`long`	64 bits	environ ±9,2 × 10^{18}	`9_000_000_000L`
`float`	32 bits	±3,4 × 10^{38}, ~7 chiffres significatifs	`3.14f`
`double`	64 bits	±1,8 × 10^{308}, ~15-16 chiffres significatifs	`3.14`, `1.5e6`
`char`	16 bits	un caractère Unicode (UTF-16), de `'\u0000'` à `'\uffff'`	`'A'`, `'é'`, `'\n'`
`boolean`	—	`true` ou `false`	`true`

Quelques remarques importantes :

- un littéral entier est de type `int` par défaut : pour un `long`, ajoutez le suffixe `L` ;
- un littéral décimal est de type `double` par défaut : pour un `float`, ajoutez `f` ;
- le caractère `_` peut séparer les chiffres pour la lisibilité (`1_500_000`) ;
- en Java, `boolean` n'est **pas** un entier : `if (1)` ne compile pas, contrairement au C.

Limites.java

```java
public class Limites {
    public static void main(String[] args) {
        System.out.println("byte  : " + Byte.MIN_VALUE + " à " + Byte.MAX_VALUE);
        System.out.println("short : " + Short.MIN_VALUE + " à " + Short.MAX_VALUE);
        System.out.println("int   : " + Integer.MIN_VALUE + " à " + Integer.MAX_VALUE);
        System.out.println("long  : " + Long.MIN_VALUE + " à " + Long.MAX_VALUE);
        System.out.println("double max : " + Double.MAX_VALUE);
        char c = 'A';
        System.out.println("Code Unicode de A : " + (int) c);
        System.out.println("Caractère suivant : " + (char) (c + 1));
    }
}
```

Sortie console

```
byte  : -128 à 127
short : -32768 à 32767
int   : -2147483648 à 2147483647
long  : -9223372036854775808 à 9223372036854775807
double max : 1.7976931348623157E308
Code Unicode de A : 65
Caractère suivant : B
```

2.3 Types primitifs et types référence

Tout ce qui n'est pas primitif est un **type référence** : `String`, les tableaux, les classes que vous écrirez, les collections… Une variable de type référence ne contient pas l'objet lui-même mais **l'adresse** de l'objet, qui vit dans une zone mémoire appelée le *tas* (*heap*).

	Type primitif	Type référence
Contenu de la variable	la valeur elle-même	une référence vers un objet
Valeur par défaut (attributs)	`0`, `0.0`, `false`, `'\u0000'`	`null`
Peut valoir `null`	non	oui
Nom	commence par une minuscule (`int`)	commence par une majuscule (`String`)
Comparaison avec `==`	compare les valeurs	compare les **adresses** (voir chapitre 3)

Les classes enveloppes (*wrappers*) et l'autoboxing

À chaque type primitif correspond une classe : `Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`, `Character`, `Boolean`. Elles sont nécessaires car les collections (chapitre 11) ne peuvent contenir que des objets. Java convertit automatiquement dans les deux sens : c'est l'**autoboxing** et l'**unboxing**.

Java

```java
Integer stockObjet = 25;         // autoboxing : int -> Integer
int stock = stockObjet;          // unboxing   : Integer -> int
int quantite = Integer.parseInt("12");       // conversion texte -> int
long montant = Long.parseLong("150000");
double note = Double.parseDouble("4.5");
String texte = String.valueOf(3500);         // int -> texte
```

> **Piège à éviter — == sur des Integer**
>
> Comparer deux objets `Integer` avec `==` compare leurs **références**. Par optimisation, Java met en cache les valeurs de -128 à 127 ; au-delà, deux `Integer` égaux peuvent être deux objets distincts. Utilisez toujours `equals()` ou comparez des `int`.

PiegeInteger.java

```java
public class PiegeInteger {
    public static void main(String[] args) {
        Integer a = 100, b = 100;
        Integer c = 1000, d = 1000;
        System.out.println(a == b);        // true  (valeurs en cache)
        System.out.println(c == d);        // false (deux objets distincts !)
        System.out.println(c.equals(d));   // true  (compare les valeurs)
        Integer inconnu = null;
        // int x = inconnu;  // NullPointerException à l'unboxing
    }
}
```

Sortie console

```
true
false
true
```

2.4 Les conversions de type (*cast*)

Conversion élargissante (automatique)

Passer d'un type « petit » à un type « plus grand » se fait automatiquement, sans perte : `byte → short → int → long → float → double`, et `char → int`.

Java

```java
int quantite = 3;
long grandTotal = quantite;     // automatique
double moyenne = grandTotal;    // automatique : 3.0
```

Conversion rétrécissante (explicite)

Dans l'autre sens, il y a un risque de perte : le programmeur doit l'écrire explicitement avec un **cast** `(type)`.

Conversions.java

```java
public class Conversions {
    public static void main(String[] args) {
        double prixMoyen = 2749.87;
        int arrondiInf = (int) prixMoyen;          // troncature : 2749
        long arrondi = Math.round(prixMoyen);      // arrondi : 2750
        System.out.println(arrondiInf + " / " + arrondi);

        int grand = 130;
        byte petit = (byte) grand;                 // dépasse 127 : -126 !
        System.out.println("130 en byte = " + petit);

        // Division entière : le piège classique
        int total = 7, nbArticles = 2;
        System.out.println("7 / 2       = " + (total / nbArticles));
        System.out.println("7 / 2.0     = " + (total / 2.0));
        System.out.println("(double) 7/2 = " + ((double) total / nbArticles));
        System.out.println("(double)(7/2) = " + (double) (total / nbArticles));
    }
}
```

Sortie console

```
2749 / 2750
130 en byte = -126
7 / 2       = 3
7 / 2.0     = 3.5
(double) 7/2 = 3.5
(double)(7/2) = 3.0
```

La division de deux entiers est une **division entière** : le résultat est tronqué. Il suffit qu'un des deux opérandes soit décimal pour obtenir une division réelle. Dans la dernière ligne, la division entière a lieu **avant** la conversion : le mal est déjà fait.

Le débordement silencieux

Lorsqu'un calcul dépasse la capacité d'un type entier, Java ne signale **aucune erreur** : la valeur « fait le tour » et devient fausse.

Debordement.java

```java
public class Debordement {
    public static void main(String[] args) {
        int ventesJour = 2_000_000_000;           // 2 milliards de FCFA
        int ventesDeuxJours = ventesJour * 2;     // débordement !
        System.out.println("Avec int  : " + ventesDeuxJours);

        long correct = ventesJour * 2L;           // le calcul se fait en long
        System.out.println("Avec long : " + correct);

        try {
            Math.multiplyExact(ventesJour, 2);    // lève une exception au lieu de mentir
        } catch (ArithmeticException e) {
            System.out.println("Détecté : " + e.getMessage());
        }
    }
}
```

Sortie console

```
Avec int  : -294967296
Avec long : 4000000000
Détecté : integer overflow
```

> **Côté backend**
>
> Les montants, les identifiants de base de données et les horodatages en millisecondes dépassent vite 2 milliards : utilisez `long`. Une colonne PostgreSQL `BIGINT` correspond à un `long` Java, une colonne `INTEGER` à un `int`.

2.5 Ne jamais utiliser `double` pour de l'argent

Les types `float` et `double` stockent les nombres en base 2. Beaucoup de décimaux simples, comme 0,1, n'ont pas de représentation exacte en binaire (comme 1/3 n'a pas de représentation exacte en base 10). Les erreurs d'arrondi s'accumulent.

Argent.java

```java
import java.math.BigDecimal;

public class Argent {
    public static void main(String[] args) {
        System.out.println(0.1 + 0.2);                 // pas 0.3 !
        double solde = 1.10;
        solde = solde - 0.20;
        System.out.println("Solde double : " + solde);

        // Solution 1 : des entiers dans la plus petite unité (le FCFA n'a pas de centimes)
        long prixFcfa = 1_500;
        long total = prixFcfa * 3;
        System.out.println("Total : " + total + " FCFA");

        // Solution 2 : BigDecimal pour les devises à décimales (euro, dollar)
        BigDecimal a = new BigDecimal("1.10");
        BigDecimal b = new BigDecimal("0.20");
        System.out.println("Solde BigDecimal : " + a.subtract(b));
    }
}
```

Sortie console

```
0.30000000000000004
Solde double : 0.9000000000000001
Total : 4500 FCFA
Solde BigDecimal : 0.90
```

> **À retenir**
>
> Pour de l'argent : `long` en unités entières (FCFA) ou en centimes (euros), ou bien `BigDecimal` créé **à partir d'une chaîne** (`new BigDecimal("1.10")`, jamais `new BigDecimal(1.10)`). En base de données, on utilise `BIGINT` ou `NUMERIC(12,2)`, jamais `REAL` ou `DOUBLE PRECISION`.

2.6 Les opérateurs

Opérateurs arithmétiques

Opérateur	Rôle	Exemple (a = 17, b = 5)	Résultat
`+`	addition (et concaténation de chaînes)	`a + b`	22
`-`	soustraction	`a - b`	12
`*`	multiplication	`a * b`	85
`/`	division (entière si deux entiers)	`a / b`	3
`%`	reste de la division (modulo)	`a % b`	2
`++` / `--`	incrémentation / décrémentation de 1	`a++`	`a` vaut 18

La différence entre **pré-incrémentation** (`++i`) et **post-incrémentation** (`i++`) n'apparaît que lorsque l'expression est utilisée dans un calcul :

Increment.java

```java
public class Increment {
    public static void main(String[] args) {
        int i = 5;
        int a = i++;   // a reçoit 5, PUIS i passe à 6
        int j = 5;
        int b = ++j;   // j passe à 6, PUIS b reçoit 6
        System.out.println("a=" + a + " i=" + i + " b=" + b + " j=" + j);
        System.out.println(-7 / 2 + " " + -7 % 2);            // troncature vers 0
        System.out.println(Math.floorDiv(-7, 2) + " " + Math.floorMod(-7, 2));
    }
}
```

Sortie console

```
a=5 i=6 b=6 j=6
-3 -1
-4 1
```

> **Astuce**
>
> Pour savoir si un nombre est pair, testez `n % 2 == 0` et non `n % 2 == 1` pour « impair » : avec un nombre négatif, `-3 % 2` vaut `-1`. Écrivez plutôt `n % 2 != 0`.

Opérateurs d'affectation composée

`x += 5` équivaut à `x = x + 5`. Il existe `+=`, `-=`, `*=`, `/=`, `%=`, et des variantes binaires (`&=`, `|=`, `^=`, `<<=`, `>>=`). Attention : l'affectation composée contient un **cast implicite**.

Java

```java
byte b = 10;
b = b + 5;    // ERREUR : b + 5 est un int
b += 5;       // compile : équivaut à b = (byte)(b + 5)
```

Opérateurs de comparaison et logiques

Opérateur	Signification	Opérateur	Signification
`==`	égal à	`&&`	ET logique (court-circuit)
`!=`	différent de	`||`	OU logique (court-circuit)
`<`, `<=`	inférieur (ou égal)	`!`	NON logique
`>`, `>=`	supérieur (ou égal)	`^`	OU exclusif

Les opérateurs `&&` et `||` sont **à court-circuit** : si le résultat est connu après le premier opérande, le second n'est pas évalué. C'est très utile pour se protéger :

Java

```java
if (client != null && client.estFidele()) { ... }  // pas de NullPointerException
if (quantite > 0 && total / quantite > 10_000) { ... } // pas de division par zéro
```

Priorité des opérateurs (de la plus forte à la plus faible)

Niveau	Opérateurs
1	`expr++`, `expr--`
2	`++expr`, `--expr`, `+` et `-` unaires, `!`, `(cast)`
3	`*`, `/`, `%`

Niveau	Opérateurs
4	`+`, `-`
5	`<<`, `>>`, `>>>`
6	`<`, `>`, `<=`, `>=`, `instanceof`
7	`==`, `!=`
8	`&`, puis `^`, puis `|`
9	`&&`
10	`||`
11	`? :` (ternaire)
12	`=`, `+=`, `-=`, etc.

En cas de doute, **ajoutez des parenthèses** : le code est fait pour être relu.

2.7 La classe `Math`

La classe `java.lang.Math` regroupe les fonctions mathématiques usuelles, toutes `static` :

Méthode	Rôle	Exemple	Résultat
`Math.max(a, b)` / `Math.min(a, b)`	maximum / minimum	`Math.max(3, 8)`	`8`
`Math.abs(x)`	valeur absolue	`Math.abs(-5)`	`5`
`Math.pow(a, b)`	puissance (renvoie un `double`)	`Math.pow(2, 10)`	`1024.0`
`Math.sqrt(x)`	racine carrée	`Math.sqrt(81)`	`9.0`
`Math.round(x)`	arrondi au plus proche (`long` pour un `double`)	`Math.round(2.5)`	`3`
`Math.floor(x)` / `Math.ceil(x)`	partie entière inférieure / supérieure	`Math.ceil(2.1)`	`3.0`
`Math.random()`	nombre aléatoire dans [0, 1[	—	—
`Math.addExact`, `Math.multiplyExact`	calcul qui lève une exception en cas de débordement	—	—
`Math.clamp(v, min, max)` *(Java 21)*	borne une valeur dans un intervalle	`Math.clamp(150, 0, 100)`	`100`

2.8 Affichage formaté

`System.out.println` convient pour un affichage simple ; pour aligner des colonnes ou contrôler le nombre de décimales, on utilise `printf` (ou `String.format`, ou la méthode `formatted` des chaînes, voir chapitre 3).

Spécificateur	Rôle	Exemple	Affiche
`%d`	entier	`printf("%d", 42)`	`42`
`%5d` / `%-5d`	entier sur 5 caractères, aligné à droite / à gauche	`printf("[%5d]", 42)`	`[   42]`
`%05d`	complété par des zéros	`printf("%05d", 42)`	`00042`
`%f`, `%.2f`	décimal, avec 2 décimales	`printf("%.2f", 3.14159)`	`3.14`
`%s`, `%-12s`	chaîne, alignée à gauche sur 12 caractères	`printf("%-6s|", "Awa")`	`Awa   |`
`%c`, `%b`	caractère, booléen	`printf("%c %b", 'X', true)`	`X true`

Spécificateur	Rôle	Exemple	Affiche
`%n`	retour à la ligne (portable)		
`%%`	le caractère `%`	`printf("%d%%", 18)`	`18%`

> **Astuce — séparateur décimal et langue du système**
>
> `printf` et `String.format` utilisent la **langue du système** (la *locale*) : sur un poste configuré en français, `printf("%.2f", 3.14159)` affiche `3,14` avec une virgule. Les sorties de ce cours ont été produites avec une configuration neutre (point décimal). Pour imposer un format, précisez la locale : `String.format(Locale.FRANCE, "%.2f", x)` ou `String.format(Locale.ROOT, "%.2f", x)`.

2.9 Lire des données au clavier

Avec Scanner (toutes versions)

SaisieCommande.java

```java
import java.util.Scanner;

public class SaisieCommande {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Nom du client : ");
        String nom = sc.nextLine();
        System.out.print("Quantité : ");
        int quantite = sc.nextInt();
        System.out.print("Prix unitaire (FCFA) : ");
        long prix = sc.nextLong();
        long total = quantite * prix;
        System.out.printf("%n%s doit payer %d FCFA.%n", nom, total);
    }
}
```

Sortie console

```
Nom du client : Quantité : Prix unitaire (FCFA) :
Awa Diop doit payer 4500 FCFA.
```

(Dans un terminal, les valeurs que vous tapez apparaissent après chaque question.)

> **Piège à éviter — `nextInt()` suivi de `nextLine()`**
>
> `nextInt()` lit le nombre mais **laisse le retour à la ligne** dans le tampon. Un `nextLine()` appelé juste après renvoie donc une chaîne vide. Solution : lire toutes les lignes avec `nextLine()` et convertir avec `Integer.parseInt(...)`, ou appeler un `nextLine()` supplémentaire pour « consommer » la fin de ligne.

Avec `IO.readln` (Java 25)

Dans un fichier source compact, `IO.readln(message)` affiche une invite et renvoie la ligne saisie :

Saisie25.java

```java
void main() {
    String nom = IO.readln("Votre prénom : ");
    int nb = Integer.parseInt(IO.readln("Combien de pagnes ? "));
    IO.println("Merci " + nom + ", total : " + (nb * 7_500) + " FCFA");
}
```

Sortie console

```
Votre prénom : Combien de pagnes ? Merci Moussa, total : 15000 FCFA
```

2.10 Fil rouge TerangaShop — le ticket de caisse

Mettons ensemble tout ce que nous avons vu : un client achète 3 bouteilles de bissap à 1 500 FCFA et 2 pots de thiakry à 1 000 FCFA. Une remise de 10 % s'applique au-delà de 5 000 FCFA d'achats, puis la TVA sénégalaise de 18 % est calculée, et enfin les frais de livraison (1 500 FCFA) sont ajoutés.

Ticket.java

```java
public class Ticket {
    static final int TAUX_TVA = 18;              // en pourcentage
    static final int TAUX_REMISE = 10;
    static final long SEUIL_REMISE = 5_000;
    static final long FRAIS_LIVRAISON = 1_500;

    public static void main(String[] args) {
        long prixBissap = 1_500, prixThiakry = 1_000;
        int qteBissap = 3, qteThiakry = 2;

        long sousTotal = prixBissap * qteBissap + prixThiakry * qteThiakry;
        long remise = sousTotal > SEUIL_REMISE ? sousTotal * TAUX_REMISE / 100 : 0;
        long totalHT = sousTotal - remise;
        long tva = Math.round(totalHT * TAUX_TVA / 100.0);
        long totalTTC = totalHT + tva + FRAIS_LIVRAISON;

        System.out.println("========== TERANGASHOP ==========");
        System.out.printf("%-20s %3d x %6d%n", "Bissap 1 L", qteBissap, prixBissap);
        System.out.printf("%-20s %3d x %6d%n", "Thiakry 500 g", qteThiakry, prixThiakry);
        System.out.println("--------------------------------");
        System.out.printf("%-20s %12d%n", "Sous-total", sousTotal);
        System.out.printf("%-20s %12d%n", "Remise " + TAUX_REMISE + "%", -remise);
        System.out.printf("%-20s %12d%n", "Total HT", totalHT);
        System.out.printf("%-20s %12d%n", "TVA " + TAUX_TVA + "%", tva);
        System.out.printf("%-20s %12d%n", "Livraison", FRAIS_LIVRAISON);
        System.out.printf("%-20s %12d FCFA%n", "TOTAL TTC", totalTTC);
    }
}
```

Sortie console

```
========== TERANGASHOP ==========
Bissap 1 L            3 x   1500
Thiakry 500 g         2 x   1000
--------------------------------
Sous-total                  6500
Remise 10%                  -650
Total HT                    5850
TVA 18%                     1053
Livraison                   1500
TOTAL TTC                   8403 FCFA
```

Remarquez l'expression `totalHT * TAUX_TVA / 100.0` : la division par `100.0` (un `double`) évite la troncature de la division entière, puis `Math.round` ramène le résultat à un nombre entier de francs.

Exercices du chapitre 2

Exercice 2.1 ★ — Déclarations valides ou non ?

Pour chaque ligne, dites si elle compile. Si non, expliquez pourquoi et corrigez.

Java

```java
int 2emeClient = 5;
long population = 18_000_000;
float taux = 0.18;
var total;
char initiale = "A";
boolean payé = 1;
long numero = 221771234567;
final double PI = 3.14159;
int class = 3;
var reduction = null;
```

Exercice 2.2 ★ — Prédire l'affichage

Sans exécuter le programme, écrivez ce qu'il affiche. Vérifiez ensuite avec JShell.

Java

```java
int a = 10, b = 4;
System.out.println(a / b);
System.out.println(a % b);
System.out.println((double) a / b);
System.out.println(a / b * 1.0);
System.out.println(a++ + ++a);
System.out.println("Total : " + a + b);
System.out.println("Total : " + (a + b));
System.out.println('A' + 1);
System.out.println((char) ('A' + 1));
```

Exercice 2.3 ★★ — Conversion de durée

Un livreur met un certain nombre de secondes pour effectuer sa tournée. Écrivez un programme qui lit ce nombre (par exemple `9875`) et l'affiche sous la forme `2 h 44 min 35 s`. Utilisez uniquement `/` et `%`.

Exercice 2.4 ★★ — Facture avec TVA

Écrivez un programme qui demande le libellé d'un produit, son prix unitaire HT en FCFA et la quantité, puis affiche le montant HT, la TVA (18 %, arrondie au franc le plus proche) et le montant TTC, alignés en colonnes avec `printf`. Testez avec un prix de 2 750 FCFA et une quantité de 3.

Exercice 2.5 ★★ — Le compteur qui déborde

Un opérateur de paiement mobile traite en moyenne 350 000 transactions par jour, d'un montant moyen de 12 500 FCFA. Écrivez un programme qui calcule le volume total sur 30 jours avec des `int`, puis avec des `long`. Comparez les résultats et expliquez. Réécrivez enfin le calcul `int` avec `Math.multiplyExact` pour que l'erreur soit détectée.

Exercice 2.6 ★★★ — Rendu de monnaie

Le livreur de TerangaShop doit rendre la monnaie en espèces avec le moins de billets et de pièces possible. Les coupures disponibles sont : billets de 10 000, 5 000, 2 000, 1 000 et 500 ; pièces de 250, 100, 50, 25, 10 et 5 FCFA. Écrivez un programme qui lit la somme à rendre (multiple de 5) et affiche le nombre de chaque coupure utilisée, **sans boucle** (seulement avec `/` et `%`). Exemple : pour 18 735 FCFA, on rend 1 × 10 000, 1 × 5 000, 1 × 2 000, 1 × 1 000, 1 × 500, 2 × 100, 1 × 25, 1 × 10.

Chapitre 3 — Chaînes de caractères et text blocks

Objectifs du chapitre

- Manipuler les chaînes avec les méthodes essentielles de la classe `String`.
- Comprendre l'immutabilité des chaînes et comparer correctement deux chaînes.
- Construire efficacement du texte avec `StringBuilder`.
- Écrire des requêtes SQL et des documents JSON lisibles avec les *text blocks*.
- Valider un format simple (téléphone, e-mail) avec une expression régulière.

3.1 La classe `String`

Une chaîne de caractères est un **objet** de la classe `java.lang.String`. Comme elle est omniprésente, Java offre une syntaxe spéciale pour la créer : le littéral entre guillemets doubles.

Java

```
String ville = "Dakar";                 // littéral (forme recommandée)
String vide = "";                        // chaîne vide (longueur 0)
String copie = new String("Dakar");      // crée un NOUVEL objet (à éviter)
```

Une chaîne est une suite de `char` indexés **à partir de 0** : dans `"Dakar"`, le caractère d'indice 0 est `'D'` et celui d'indice 4 est `'r'`.

À retenir — les chaînes sont immuables

Un objet `String` ne peut **jamais** être modifié. Les méthodes comme `toUpperCase()` ou `replace()` ne modifient pas la chaîne : elles renvoient une **nouvelle** chaîne. Si vous n'utilisez pas la valeur renvoyée, l'appel ne sert à rien.

Java

```
String nom = "awa";
nom.toUpperCase();          // ne change rien : le résultat est perdu
nom = nom.toUpperCase();    // correct : nom désigne la nouvelle chaîne "AWA"
```

3.2 Les méthodes essentielles

Méthode	Rôle	Exemple	Résultat
`length()`	nombre de caractères	`"Médina".length()`	`6`
`charAt(i)`	caractère à l'indice `i`	`"Dakar".charAt(1)`	`'a'`
`substring(d)` / `substring(d, f)`	sous-chaîne de `d` (inclus) à `f` (exclu)	`"Diamniadio".substring(0, 5)`	`"Diamn"`
`indexOf(s)` / `lastIndexOf(s)`	position de la première / dernière occurrence, `-1` si absente	`"bissap".indexOf("s")`	`2`
`contains(s)`	contient `s` ?	`"Wave Money".contains("Wave")`	`true`
`startsWith(s)` / `endsWith(s)`	commence / finit par `s` ?	`"TS-001".startsWith("TS")`	`true`
`toUpperCase()` / `toLowerCase()`	majuscules / minuscules	`"Thiès".toUpperCase()`	`"THIÈS"`

Méthode	Rôle	Exemple	Résultat
`strip()` / `trim()`	supprime les espaces au début et à la fin	`"  ok  ".strip()`	`"ok"`
`isEmpty()` / `isBlank()`	longueur nulle / vide ou seulement des espaces	`"  ".isBlank()`	`true`
`replace(a, b)`	remplace toutes les occurrences	`"77-123".replace("-", " ")`	`"77 123"`
`split(regex)`	découpe en tableau	`"a;b;c".split(";")`	`["a","b","c"]`
`String.join(sep, …)`	assemble avec un séparateur	`String.join(", ", "a", "b")`	`"a, b"`
`repeat(n)`	répète `n` fois	`"=".repeat(5)`	`"====="`
`equals(s)` / `equalsIgnoreCase(s)`	égalité de contenu	`"oui".equalsIgnoreCase("OUI")`	`true`
`compareTo(s)`	ordre alphabétique (négatif, 0 ou positif)	`"Awa".compareTo("Binta")`	négatif
`chars()` / `lines()`	flux de caractères / de lignes (chapitre 12)		
`formatted(…)`	formatage façon `printf`	`"%d FCFA".formatted(500)`	`"500 FCFA"`

Nouveautés Java 11 à 15 sur les chaînes

Java 11 a ajouté `isBlank()`, `strip()`, `stripLeading()`, `stripTrailing()`, `repeat(n)` et `lines()`. `strip()` est préférable à `trim()` car il reconnaît tous les espaces Unicode. Java 15 a ajouté `formatted(...)`, `stripIndent()` et les *text blocks*. Java 21 a ajouté `indexOf(ch, début, fin)`, `splitWithDelimiters(...)` et `StringBuilder.repeat(...)`.

MethodesString.java

```java
public class MethodesString {
    public static void main(String[] args) {
        String saisie = "   Sac en cuir de Ngaye Mékhé   ";
        String produit = saisie.strip();
        System.out.println("[" + produit + "] : " + produit.length() + " caractères");
        System.out.println("Majuscules : " + produit.toUpperCase());
        System.out.println("Premier mot : " + produit.substring(0, produit.indexOf(' ')));
        System.out.println("Contient 'cuir' ? " + produit.contains("cuir"));
        System.out.println("Dernier caractère : " + produit.charAt(produit.length() - 1));

        String ligneCsv = "PRD-014;Pagne wax 6 yards;7500;42";
        String[] champs = ligneCsv.split(";");
        System.out.println("Référence = " + champs[0] + ", prix = " + champs[2]);
        int stock = Integer.parseInt(champs[3]);
        System.out.println("Stock après vente de 2 : " + (stock - 2));

        System.out.println(String.join(" | ", "Wave", "Orange Money", "Espèces"));
        System.out.println("-".repeat(30));
        System.out.println("Commande n°%05d : %s".formatted(42, "validée"));
    }
}
```

Sortie console

```
[Sac en cuir de Ngaye Mékhé] : 26 caractères
Majuscules : SAC EN CUIR DE NGAYE MÉKHÉ
Premier mot : Sac
Contient 'cuir' ? true
Dernier caractère : é
Référence = PRD-014, prix = 7500
Stock après vente de 2 : 40
Wave | Orange Money | Espèces
----------------------------
Commande n°00042 : validée
```

3.3 Comparer deux chaînes : == ou equals ?

C'est l'erreur la plus fréquente des débutants en Java.

- `a == b` compare les **références** : les deux variables désignent-elles le **même objet** en mémoire ?
- `a.equals(b)` compare le **contenu** : les deux chaînes ont-elles les mêmes caractères ?

ComparaisonChaines.java

```java
import java.util.Objects;

public class ComparaisonChaines {
    public static void main(String[] args) {
        String a = "Wave";
        String b = "Wave";                  // même littéral : Java réutilise l'objet (pool)
        String c = new String("Wave");      // nouvel objet
        String d = "Wa" + "ve".toLowerCase(); // calculée à l'exécution : nouvel objet

        System.out.println("a == b      : " + (a == b));
        System.out.println("a == c      : " + (a == c));
        System.out.println("a == d      : " + (a == d));
        System.out.println("a.equals(c) : " + a.equals(c));
        System.out.println("a.equals(d) : " + a.equals(d));

        String modePaiement = null;         // valeur absente (ex. champ non rempli)
        System.out.println("Wave".equals(modePaiement));       // false, sans erreur
        System.out.println(Objects.equals(modePaiement, null)); // true
        // modePaiement.equals("Wave") provoquerait une NullPointerException
    }
}
```

Sortie console

```
a == b      : true
a == c      : false
a == d      : false
a.equals(c) : true
a.equals(d) : true
false
true
```

> **Piège à éviter**
>
> Ne comparez **jamais** des chaînes avec `==`. Le code peut sembler fonctionner (grâce au *pool* des littéraux) puis échouer avec des chaînes lues au clavier, dans un fichier ou dans une base de données. Utilisez `equals`, et placez le littéral à gauche (`"Wave".equals(x)`) ou utilisez `Objects.equals(x, y)` pour vous protéger d'une valeur `null`.

Pour trier, on utilise `compareTo`, qui renvoie un entier négatif si la chaîne est avant l'argument dans l'ordre lexicographique, 0 si elles sont égales, positif sinon. Attention : cet ordre est celui des codes Unicode, donc

les majuscules passent avant les minuscules et les lettres accentuées après `z`. Pour un tri alphabétique respectant le français, on utilise un `java.text.Collator` (chapitre 11).

3.4 Construire du texte : `StringBuilder`

Puisque les chaînes sont immuables, chaque concaténation `s = s + x` crée un nouvel objet et **recopie** tous les caractères déjà présents. Dans une boucle de n tours, on recopie donc de l'ordre de n^2 caractères. La classe `StringBuilder` maintient un tampon modifiable et évite ces copies.

Recu.java

```java
public class Recu {
    public static void main(String[] args) {
        String[] articles = {"Bissap 1 L", "Thiakry 500 g", "Café Touba 250 g"};
        long[] prix = {1_500, 1_000, 2_000};

        StringBuilder sb = new StringBuilder();
        sb.append("REÇU TERANGASHOP\n");
        for (int i = 0; i < articles.length; i++) {
            sb.append(i + 1).append(". ")
              .append(articles[i])
              .append(" : ").append(prix[i]).append(" FCFA\n");
        }
        sb.append("=".repeat(16));
        System.out.println(sb);                 // println appelle toString()

        StringBuilder code = new StringBuilder("TS2026");
        code.insert(2, '-').append("-0042");
        System.out.println(code + " / inversé : " + new StringBuilder("bissap").reverse());
    }
}
```

Sortie console

```
REÇU TERANGASHOP
1. Bissap 1 L : 1500 FCFA
2. Thiakry 500 g : 1000 FCFA
3. Café Touba 250 g : 2000 FCFA
================
TS-2026-0042 / inversé : passib
```

Les principales méthodes de `StringBuilder` sont `append`, `insert`, `delete(debut, fin)`, `deleteCharAt(i)`, `replace(debut, fin, s)`, `reverse`, `setCharAt(i, c)`, `length` et `toString`. Elles renvoient le `StringBuilder` lui-même, ce qui permet de les **chaîner**.

> **Astuce**
>
> Pour une concaténation simple sur une ligne (`"Total : " + total + " FCFA"`), le compilateur optimise déjà : utilisez `+`, c'est plus lisible. Réservez `StringBuilder` aux constructions dans une boucle.

3.5 Les *text blocks*

> **Nouveauté Java 15 — Text blocks (JEP 378)**
>
> Un *text block* est une chaîne littérale sur plusieurs lignes, délimitée par trois guillemets `"""`. Il évite les `\n` et les `+` en cascade, et rend lisibles les requêtes SQL, le JSON, le HTML ou les messages longs.

Règles d'écriture :

- l'ouverture `"""` doit être suivie d'un **retour à la ligne** ;

- l'indentation commune à toutes les lignes (y compris la ligne du `"""` fermant) est supprimée automatiquement ;
- placer le `"""` fermant sur sa propre ligne ajoute un `\n` final ; le coller à la dernière ligne l'évite ;
- un `\` en fin de ligne supprime le retour à la ligne (utile pour les longues lignes) ; `\s` force un espace qui ne sera pas supprimé ;
- les guillemets simples `"` n'ont pas besoin d'être échappés.

TextBlocks.java

```java
public class TextBlocks {
    public static void main(String[] args) {
        String sql = """
            SELECT p.reference, p.libelle, p.prix
            FROM produit p
            WHERE p.categorie = ?
              AND p.stock > 0
            ORDER BY p.prix DESC
            """;
        System.out.print(sql);

        String client = "Fatou Sow";
        long total = 18_500;
        String json = """
            {
              "client": "%s",
              "total": %d,
              "devise": "XOF",
              "livraison": "Parcelles Assainies"
            }""".formatted(client, total);
        System.out.println(json);

        String message = """
            Bonjour, votre commande est en cours de \
            livraison. Merci de votre confiance !""";
        System.out.println(message);
    }
}
```

Sortie console

```
SELECT p.reference, p.libelle, p.prix
FROM produit p
WHERE p.categorie = ?
  AND p.stock > 0
ORDER BY p.prix DESC
{
  "client": "Fatou Sow",
  "total": 18500,
  "devise": "XOF",
  "livraison": "Parcelles Assainies"
}
Bonjour, votre commande est en cours de livraison. Merci de votre confiance !
```

Côté backend

Les text blocks sont parfaits pour écrire des requêtes SQL dans un DAO JDBC (chapitre 16) ou des corps JSON dans des tests d'API (chapitres 17 et 18). Mais attention : n'utilisez **jamais** `formatted` ou `+` pour insérer une valeur saisie par un utilisateur **dans une requête SQL** : c'est la porte ouverte aux injections SQL. On utilise des paramètres `?` avec un `PreparedStatement`.

3.6 Les caractères : la classe `Character`

La classe `Character` fournit des méthodes utiles pour analyser un caractère : `isDigit`, `isLetter`, `isLetterOrDigit`, `isWhitespace`, `isUpperCase`, `isLowerCase`, `toUpperCase`, `toLowerCase`, `getNumericValue`.

AnalyseMotDePasse.java

```java
public class AnalyseMotDePasse {
    public static void main(String[] args) {
        String mdp = "Teranga2026!";
        int majuscules = 0, minuscules = 0, chiffres = 0, speciaux = 0;
        for (int i = 0; i < mdp.length(); i++) {
            char c = mdp.charAt(i);
            if (Character.isUpperCase(c)) majuscules++;
            else if (Character.isLowerCase(c)) minuscules++;
            else if (Character.isDigit(c)) chiffres++;
            else speciaux++;
        }
        boolean robuste = mdp.length() >= 10 && majuscules > 0 && minuscules > 0
                          && chiffres > 0 && speciaux > 0;
        System.out.printf("Maj=%d, min=%d, chiffres=%d, spéciaux=%d -> robuste : %b%n",
                          majuscules, minuscules, chiffres, speciaux, robuste);
    }
}
```

Sortie console

```
Maj=1, min=6, chiffres=4, spéciaux=1 -> robuste : true
```

3.7 Un premier contact avec les expressions régulières

Une **expression régulière** (*regex*) décrit un motif de texte. La méthode `matches(regex)` teste si **toute** la chaîne respecte le motif ; `replaceAll(regex, remplacement)` remplace toutes les parties qui correspondent.

Motif	Signification
`\d`	un chiffre (en Java, on écrit `"\\d"` car `\` doit être échappé dans une chaîne)
`\s`	un espace, une tabulation ou un retour à la ligne
`[abc]`, `[a-z]`	un caractère parmi une liste ou un intervalle
`x?`, `x*`, `x+`	`x` zéro ou une fois, zéro ou plusieurs fois, au moins une fois
`x{3}`, `x{2,4}`	`x` exactement 3 fois, entre 2 et 4 fois
`(a|b)`	`a` ou `b`
`^`, `$`	début et fin de chaîne

Exemple : un numéro de mobile sénégalais comporte 9 chiffres et commence par 70, 75, 76, 77 ou 78, éventuellement précédé de l'indicatif `+221`.

ValidationTelephone.java

```java
public class ValidationTelephone {
    static final String REGEX_MOBILE_SN = "(\\+221)?7[05678]\\d{7}";

    static boolean estMobileValide(String numero) {
        String nettoye = numero.replaceAll("[\\s.-]", "");   // retire espaces, points, tirets
        return nettoye.matches(REGEX_MOBILE_SN);
    }

    public static void main(String[] args) {
        String[] essais = {"77 123 45 67", "+221 78-000-11-22", "33 820 00 00",
                "76123456", "70.555.44.33"};
        for (String n : essais) {
            System.out.printf("%-20s -> %s%n", n, estMobileValide(n) ? "valide" : "invalide");
        }
    }
}
```

Sortie console

```
77 123 45 67         -> valide
+221 78-000-11-22    -> valide
33 820 00 00         -> invalide
76123456             -> invalide
70.555.44.33         -> valide
```

Piège à éviter

`split(".")` ne découpe pas sur les points ! L'argument de `split` est une expression régulière, et `.` signifie « n'importe quel caractère ». Écrivez `split("\\.")`. Même remarque pour `|`, `+`, `*`, `?`, `(`, `)`.

3.8 Fil rouge TerangaShop — normaliser les données clients

Les données saisies par les clients sont rarement propres. Avant de les enregistrer en base, un backend les **normalise**.

NormalisationClient.java

```java
public class NormalisationClient {

    /** "  aWA   diOP " -> "Awa Diop" */
    static String normaliserNom(String saisie) {
        String[] mots = saisie.strip().toLowerCase().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (String mot : mots) {
            if (!sb.isEmpty()) sb.append(' ');
            sb.append(Character.toUpperCase(mot.charAt(0))).append(mot.substring(1));
        }
        return sb.toString();
    }

    /** "771234567" -> "77 *** ** 67" */
    static String masquerTelephone(String tel) {
        String t = tel.replaceAll("\\D", "");        // garde uniquement les chiffres
        t = t.substring(t.length() - 9);               // 9 derniers chiffres
        return t.substring(0, 2) + " *** ** " + t.substring(7);
    }

    /** "Pagne Wax 6 Yards !" -> "pagne-wax-6-yards" */
    static String slug(String libelle) {
        return libelle.strip().toLowerCase()
                      .replaceAll("[^a-z0-9]+", "-")
                      .replaceAll("(^-|-$)", "");
    }
```

```
    public static void main(String[] args) {
        System.out.println(normaliserNom("  aWA   diOP "));
        System.out.println(normaliserNom("mouhamadou   moustapha  NDIAYE"));
        System.out.println(masquerTelephone("+221 77 123 45 67"));
        System.out.println(slug("Pagne Wax 6 Yards !"));
        System.out.println("TS-%d-%06d".formatted(2026, 123));
    }
}
```

Sortie console

```
Awa Diop
Mouhamadou Moustapha Ndiaye
77 *** ** 67
pagne-wax-6-yards
TS-2026-000123
```

Exercices du chapitre 3

Exercice 3.1 ★ — Méthodes de base

Soit `String s = "  TerangaShop, Dakar - Sénégal  ";`. Écrivez les instructions qui affichent : la chaîne sans les espaces extérieurs ; sa longueur avant et après nettoyage ; la position de la virgule ; le nom de la ville (`Dakar`) extrait avec `substring` et `indexOf` ; la chaîne en majuscules ; le nombre de mots.

Exercice 3.2 ★ — Palindrome

Écrivez une méthode `static boolean estPalindrome(String s)` qui indique si une phrase se lit de la même façon dans les deux sens, **sans tenir compte des espaces ni de la casse**. Testez avec `"kayak"`, `"Engage le jeu que je le gagne"` et `"Dakar"`. Proposez deux versions : l'une avec une boucle et deux indices, l'autre avec `StringBuilder.reverse()`.

Exercice 3.3 ★★ — Statistiques de texte

Écrivez un programme qui lit une ligne de texte et affiche le nombre de voyelles (a, e, i, o, u, y, y compris les voyelles accentuées courantes é, è, ê, à, ù), de consonnes, de chiffres, d'espaces et d'autres caractères.

Exercice 3.4 ★★ — Validation d'e-mail et de référence

1. Écrivez une méthode `estEmailValide(String email)` qui accepte les adresses de la forme `partie@domaine.ext` (lettres, chiffres, points, tirets et `_` dans la partie locale ; extension de 2 à 6 lettres). Testez avec `awa.diop@isep.edu.sn`, `moussa@gmail`, `@wave.com`.
2. Les références produits de TerangaShop ont la forme `PRD-` suivi de exactement 3 chiffres. Écrivez `estReferenceValide(String ref)`.

Exercice 3.5 ★★ — Analyse d'une ligne CSV

Une ligne d'export des ventes a la forme `"2026-09-14;TS-2026-000087;Awa Diop;Médina;WAVE;18500"`. Écrivez un programme qui la découpe et affiche un résumé lisible sur plusieurs lignes, en mettant le quartier en majuscules et en affichant le montant suivi de « FCFA ». Si la ligne ne contient pas exactement 6 champs, le programme affiche « Ligne invalide ».

Exercice 3.6 ★★★ — Générateur de reçu JSON

Écrivez une méthode `static String recuJson(String client, String[] articles, long[] prix, int[] quantites)` qui renvoie un document JSON bien indenté contenant le nom du client, la liste des lignes (libellé, quantité, prix unitaire, sous-total) et le total général. Utilisez un `StringBuilder` pour les lignes et un *text block* avec `formatted` pour l'enveloppe. Attention aux virgules entre les éléments du tableau JSON : il n'y en a pas après le dernier.

Chapitre 4 — Structures de contrôle

> **Objectifs du chapitre**
>
> - Écrire des conditions avec `if`, `else if`, `else` et l'opérateur ternaire.
> - Utiliser le `switch` classique et, surtout, les **switch expressions** modernes.
> - Choisir la bonne boucle (`while`, `do…while`, `for`, `for-each`) et contrôler son déroulement avec `break` et `continue`.

4.1 Blocs et portée des variables

Un **bloc** est une suite d'instructions entre accolades `{ }`. Une variable déclarée dans un bloc n'existe **que dans ce bloc** (sa *portée*) : elle disparaît à l'accolade fermante.

Java

```java
int total = 0;
if (total == 0) {
    int remise = 500;     // remise n'existe que dans ce bloc
    total += remise;
}
System.out.println(remise); // ERREUR : cannot find symbol
```

Contrairement au C, Java interdit de déclarer dans un bloc interne une variable locale portant le même nom qu'une variable locale d'un bloc englobant.

4.2 L'instruction `if`

Java

```java
if (condition) {
    // exécuté si la condition est vraie
} else if (autreCondition) {
    // exécuté si la première est fausse et la seconde vraie
} else {
    // exécuté sinon
}
```

La condition doit être une expression de type `boolean`. Exemple : TerangaShop accorde une remise selon le montant d'achat et le statut du client.

RemiseClient.java

```java
public class RemiseClient {
    public static void main(String[] args) {
        long[] montants = {8_000, 25_000, 60_000, 60_000};
        boolean[] fideles = {false, true, false, true};

        for (int i = 0; i < montants.length; i++) {
            long montant = montants[i];
            boolean fidele = fideles[i];
            int taux;
            if (montant >= 50_000) {
                taux = fidele ? 15 : 10;
            } else if (montant >= 20_000) {
                taux = 5;
            } else {
                taux = 0;
            }
            long remise = montant * taux / 100;
            System.out.printf("Montant %6d, fidèle=%-5b -> remise %2d%% = %5d FCFA%n",
                              montant, fidele, taux, remise);
        }
    }
}
```

Sortie console

```
Montant   8000, fidèle=false -> remise  0% =     0 FCFA
Montant  25000, fidèle=true  -> remise  5% =  1250 FCFA
Montant  60000, fidèle=false -> remise 10% =  6000 FCFA
Montant  60000, fidèle=true  -> remise 15% =  9000 FCFA
```

Pièges classiques avec `if`

- `if (x = 5)` ne compile pas en Java (la condition doit être un booléen) : bonne nouvelle, cette erreur fréquente en C est détectée.
- `if (stock > 0);` suivi d'un bloc : le point-virgule termine le `if` ; le bloc s'exécute **toujours**.
- Sans accolades, seule la **première** instruction dépend du `if`. Mettez toujours des accolades, même pour une seule ligne.

L'opérateur ternaire

L'expression `condition ? valeurSiVrai : valeurSiFaux` produit une valeur. Elle remplace avantageusement un `if/else` qui ne sert qu'à choisir entre deux valeurs :

Java

```java
String statut = stock > 0 ? "En stock" : "Rupture";
long frais = total >= 50_000 ? 0 : 1_500;
```

N'imbriquez pas plusieurs ternaires : le code devient illisible.

4.3 Le `switch` classique

Le `switch` compare une valeur à une liste de constantes. Il accepte les types `int` (et `byte`, `short`, `char`), `String`, les `enum` et, depuis Java 21, n'importe quel type avec le *pattern matching* (chapitre 8).

JourClassique.java

```java
public class JourClassique {
    public static void main(String[] args) {
        int jour = 5;
        String nom;
```

```
        switch (jour) {
            case 1:
                nom = "Lundi";
                break;
            case 2:
                nom = "Mardi";
                break;
            case 3:
                nom = "Mercredi";
                break;
            case 4:
                nom = "Jeudi";
                break;
            case 5:
                nom = "Vendredi";
                break;
            case 6:
            case 7:
                nom = "Week-end";
                break;
            default:
                nom = "Jour invalide";
        }
        System.out.println(nom);
    }
}
```

Sortie console

```
Vendredi
```

Piège à éviter — l'oubli du break

Dans un `switch` classique, sans `break`, l'exécution **continue** dans le `case` suivant (*fall-through*). C'est une source de bogues très courante. Les switch expressions ci-dessous suppriment ce problème.

4.4 Les switch expressions

Nouveauté Java 14 — Switch expressions (JEP 361)

Avec la syntaxe « flèche » `case X -> …`, il n'y a plus de *fall-through* ni de `break`. Plusieurs valeurs peuvent partager un même `case`, séparées par des virgules. Surtout, le `switch` devient une **expression** qui produit une valeur, et le compilateur vérifie qu'elle est **exhaustive**.

JourModerne.java

```
public class JourModerne {
    public static void main(String[] args) {
        for (int jour = 1; jour <= 7; jour++) {
            String nomFrancais = switch (jour) {
                case 1 -> "Lundi";
                case 2 -> "Mardi";
                case 3 -> "Mercredi";
                case 4 -> "Jeudi";
                case 5 -> "Vendredi";
                case 6 -> "Samedi";
                case 7 -> "Dimanche";
                default -> throw new IllegalArgumentException("Jour invalide : " + jour);
            };
            String nomWolof = switch (jour) {
                case 1 -> "Altine";
                case 2 -> "Talaata";
                case 3 -> "Àllarba";
                case 4 -> "Alxamis";
                case 5 -> "Àjjuma";
```

```
                case 6 -> "Gaawu";
                case 7 -> "Dibéer";
                default -> "?";
            };
            String type = switch (jour) {
                case 6, 7 -> "week-end";
                default -> "semaine";
            };
            System.out.printf("%d : %-9s %-8s (%s)%n", jour, nomFrancais, nomWolof, type);
        }
    }
}
```

Sortie console

```
1 : Lundi     Altine   (semaine)
2 : Mardi     Talaata  (semaine)
3 : Mercredi  Àllarba  (semaine)
4 : Jeudi     Alxamis  (semaine)
5 : Vendredi  Àjjuma   (semaine)
6 : Samedi    Gaawu    (week-end)
7 : Dimanche  Dibéer   (week-end)
```

Plusieurs instructions dans un `case` : `yield`

Lorsqu'une branche doit exécuter plusieurs instructions avant de produire sa valeur, on ouvre un bloc et on renvoie la valeur avec `yield`.

FraisLivraison.java

```
public class FraisLivraison {

    static long frais(String quartier, double poidsKg) {
        return switch (quartier.toUpperCase()) {
            case "PLATEAU", "MEDINA", "FANN" -> 1_000;
            case "PARCELLES ASSAINIES", "GRAND YOFF", "OUAKAM" -> 1_500;
            case "RUFISQUE", "DIAMNIADIO" -> {
                long base = 2_500;
                long supplement = poidsKg > 5 ? 1_000 : 0;
                yield base + supplement;
            }
            default -> throw new IllegalArgumentException("Zone non desservie : " + quartier);
        };
    }

    public static void main(String[] args) {
        System.out.println("Plateau      : " + frais("Plateau", 1.2));
        System.out.println("Ouakam       : " + frais("ouakam", 3));
        System.out.println("Diamniadio   : " + frais("Diamniadio", 2));
        System.out.println("Diamniadio 8kg: " + frais("Diamniadio", 8));
        try {
            frais("Saint-Louis", 1);
        } catch (IllegalArgumentException e) {
            System.out.println("Erreur : " + e.getMessage());
        }
    }
}
```

Sortie console

```
Plateau      : 1000
Ouakam       : 1500
Diamniadio   : 2500
Diamniadio 8kg: 3500
Erreur : Zone non desservie : Saint-Louis
```

> **Astuce**
>
> La syntaxe flèche s'utilise aussi pour un `switch` **instruction** (qui ne produit pas de valeur) : `switch (choix) { case 1 -> ajouter(); case 2 -> afficher(); default -> System.out.println("?"); }`. Préférez-la systématiquement à l'ancienne syntaxe dans du code neuf.

4.5 Les boucles

La boucle `while`

`while` répète un bloc **tant que** la condition est vraie. La condition est testée **avant** chaque tour : le bloc peut ne jamais s'exécuter.

Epargne.java

```java
public class Epargne {
    public static void main(String[] args) {
        long objectif = 250_000;    // achat d'un ordinateur portable
        long versementMensuel = 30_000;
        long epargne = 0;
        int mois = 0;
        while (epargne < objectif) {
            epargne += versementMensuel;
            mois++;
        }
        System.out.println("Objectif atteint en " + mois + " mois (épargne : "
                + epargne + " FCFA)");
    }
}
```

Sortie console

```
Objectif atteint en 9 mois (épargne : 270000 FCFA)
```

La boucle `do … while`

La condition est testée **après** le bloc : celui-ci s'exécute **au moins une fois**. C'est la boucle idéale pour un menu ou une saisie à contrôler.

MenuCaisse.java

```java
import java.util.Scanner;

public class MenuCaisse {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long total = 0;
        int choix;
        do {
            System.out.println("1. Ajouter un article  2. Afficher le total  0. Quitter");
            choix = Integer.parseInt(sc.nextLine());
            switch (choix) {
                case 1 -> {
                    System.out.print("Prix : ");
                    total += Long.parseLong(sc.nextLine());
                }
                case 2 -> System.out.println("Total actuel : " + total + " FCFA");
                case 0 -> System.out.println("Au revoir !");
                default -> System.out.println("Choix invalide");
            }
        } while (choix != 0);
    }
}
```

Sortie console

```
1. Ajouter un article  2. Afficher le total  0. Quitter
Prix : 1. Ajouter un article  2. Afficher le total  0. Quitter
Prix : 1. Ajouter un article  2. Afficher le total  0. Quitter
Total actuel : 3500 FCFA
1. Ajouter un article  2. Afficher le total  0. Quitter
Au revoir !
```

La boucle `for`

Quand le nombre de tours est connu, `for` regroupe sur une ligne l'initialisation, la condition et l'incrémentation :

Java

```java
for (initialisation; condition; mise à jour) {
    // corps
}
```

TableMultiplication.java

```java
public class TableMultiplication {
    public static void main(String[] args) {
        // Tableau des prix pour 1 à 5 articles à 1 500 FCFA, avec remise de 5 % à partir de 4
        for (int qte = 1; qte <= 5; qte++) {
            long brut = qte * 1_500L;
            long net = qte >= 4 ? brut * 95 / 100 : brut;
            System.out.printf("%d article(s) : %5d FCFA%n", qte, net);
        }
        // Compte à rebours, pas de 2
        for (int i = 10; i > 0; i -= 2) {
            System.out.print(i + " ");
        }
        System.out.println("Livraison !");
    }
}
```

Sortie console

```
1 article(s) :  1500 FCFA
2 article(s) :  3000 FCFA
3 article(s) :  4500 FCFA
4 article(s) :  5700 FCFA
5 article(s) :  7125 FCFA
10 8 6 4 2 Livraison !
```

La boucle *for-each*

Pour parcourir **tous** les éléments d'un tableau ou d'une collection, sans avoir besoin de l'indice, on utilise la boucle *for-each* :

Java

```java
String[] zones = {"Plateau", "Médina", "Rufisque"};
for (String zone : zones) {        // « pour chaque zone dans zones »
    System.out.println(zone);
}
```

Elle est plus lisible et évite les erreurs d'indice. En revanche, elle ne donne pas accès à l'indice et ne permet pas de modifier le tableau.

`break`, `continue` et les étiquettes

- `break` quitte immédiatement la boucle ;
- `continue` passe directement au tour suivant ;

- une **étiquette** (`nom:`) placée devant une boucle permet de sortir de plusieurs boucles imbriquées d'un coup avec `break nom;`.

RechercheStock.java

```java
public class RechercheStock {
    public static void main(String[] args) {
        // stocks[entrepôt][produit]
        int[][] stocks = {
            {12, 0, 7},
            {0, 3, 0},
            {5, 0, 9}
        };
        String[] entrepots = {"Pikine", "Thiaroye", "Diamniadio"};

        // 1) Afficher uniquement les stocks non nuls (continue)
        for (int e = 0; e < stocks.length; e++) {
            for (int p = 0; p < stocks[e].length; p++) {
                if (stocks[e][p] == 0) continue;
                System.out.println(entrepots[e] + " / produit " + p + " : " + stocks[e][p]);
            }
        }

        // 2) Trouver le premier entrepôt qui a le produit 1 (break avec étiquette)
        String trouve = "aucun";
        recherche:
        for (int e = 0; e < stocks.length; e++) {
            for (int p = 0; p < stocks[e].length; p++) {
                if (p == 1 && stocks[e][p] > 0) {
                    trouve = entrepots[e];
                    break recherche;
                }
            }
        }
        System.out.println("Produit 1 disponible à : " + trouve);
    }
}
```

Sortie console

```
Pikine / produit 0 : 12
Pikine / produit 2 : 7
Thiaroye / produit 1 : 3
Diamniadio / produit 0 : 5
Diamniadio / produit 2 : 9
Produit 1 disponible à : Thiaroye
```

Quelle boucle choisir ?

Situation	Boucle conseillée
Parcourir tous les éléments d'un tableau ou d'une collection	*for-each*
Nombre de tours connu, besoin de l'indice	`for`
Répéter tant qu'une condition est vraie, nombre de tours inconnu	`while`
Exécuter au moins une fois (menu, saisie contrôlée)	`do … while`
Transformer ou filtrer une collection	un *Stream* (chapitre 12)

Piège à éviter — la boucle infinie

Vérifiez toujours que la condition d'un `while` finit par devenir fausse : la variable testée doit être modifiée dans le corps de la boucle. Si votre programme « ne rend pas la main », appuyez sur `Ctrl + C` dans le terminal pour l'arrêter.

Exercices du chapitre 4

Exercice 4.1 ★ — Mention d'examen

Écrivez une méthode `static String mention(double note)` qui renvoie « Ajourné » (note < 10), « Passable » (10 à < 12), « Assez bien » (12 à < 14), « Bien » (14 à < 16) ou « Très bien » (≥ 16). Rejetez les notes hors de [0, 20] avec une `IllegalArgumentException`. Écrivez ensuite une seconde version avec une switch expression sur `(int) note / 2`.

Exercice 4.2 ★ — Année bissextile

Une année est bissextile si elle est divisible par 4 mais pas par 100, ou si elle est divisible par 400. Écrivez un programme qui affiche toutes les années bissextiles entre 1960 (indépendance du Sénégal) et 2030, dix par ligne.

Exercice 4.3 ★★ — Devine le prix

Le programme choisit un prix aléatoire entre 500 et 20 000 FCFA (multiple de 500) avec `new java.util.Random().nextInt(...)`. L'utilisateur propose des prix ; le programme répond « C'est plus » ou « C'est moins » jusqu'à ce qu'il trouve, puis affiche le nombre d'essais. Utilisez une boucle `do … while`.

Exercice 4.4 ★★ — Motifs

Avec des boucles imbriquées, affichez pour `n = 5` : un triangle rectangle d'étoiles, une pyramide centrée, puis un losange.

Sortie console

```
    *
   ***
  *****
 *******
*********
```

Exercice 4.5 ★★ — Suite de Syracuse

Partant d'un entier `n > 0`, on applique : si `n` est pair, `n = n / 2`, sinon `n = 3n + 1`, jusqu'à obtenir 1. Écrivez un programme qui, pour une valeur de départ lue au clavier, affiche la suite, sa longueur (le « temps de vol ») et la valeur maximale atteinte (« l'altitude maximale »). Pour `n = 27`, le temps de vol est 111 et l'altitude maximale 9 232.

Exercice 4.6 ★★★ — Caisse enregistreuse complète

Améliorez `MenuCaisse` : le menu propose d'ajouter un article (libellé, prix, quantité), d'afficher le ticket détaillé, d'appliquer un code promo (`TERANGA10` : −10 %, `TABASKI` : −15 % au-delà de 30 000 FCFA ; un seul code utilisable), de choisir le mode de paiement (Wave, Orange Money, espèces) et de valider la commande. Les frais de livraison sont gratuits au-delà de 50 000 FCFA. Utilisez des switch expressions et prévoyez des messages d'erreur pour les saisies invalides. Limitez-vous à 20 articles (tableaux de taille fixe).

Chapitre 5 — Tableaux, méthodes et récursivité

Objectifs du chapitre

- Créer, parcourir, copier et trier des tableaux à une ou deux dimensions.
- Utiliser la classe utilitaire `java.util.Arrays`.
- Décomposer un programme en méthodes : paramètres, valeur de retour, surcharge, *varargs*.
- Comprendre que Java passe **toujours** les arguments par valeur.
- Écrire des méthodes récursives.

5.1 Les tableaux

Un **tableau** stocke un nombre **fixe** d'éléments du même type, accessibles par un indice qui va de `0` à `longueur - 1`. En Java, un tableau est un **objet** : la variable contient une référence vers lui.

Java

```java
long[] prix = new long[4];                    // 4 cases initialisées à 0
String[] zones = {"Plateau", "Médina", "Rufisque"};   // initialisation directe
int[] notes = new int[]{12, 15, 9};           // forme équivalente
prix[0] = 1_500;                              // écriture
System.out.println(zones[1]);                 // lecture : Médina
System.out.println(zones.length);             // longueur : 3 (attribut, pas de parenthèses)
```

À la création avec `new`, les cases reçoivent la valeur par défaut du type : `0` pour les nombres, `false` pour `boolean`, `'\u0000'` pour `char` et `null` pour les références (`String`, objets).

Piège à éviter — `ArrayIndexOutOfBoundsException`

Accéder à `zones[3]` dans un tableau de longueur 3 provoque une exception à l'exécution. La boucle correcte est `for (int i = 0; i < t.length; i++)` : **strictement** inférieur.

Un tableau est une référence

Affecter un tableau à une autre variable ne le copie pas : les deux variables désignent **le même** tableau.

ReferenceTableau.java

```java
import java.util.Arrays;

public class ReferenceTableau {
    public static void main(String[] args) {
        int[] stockLundi = {10, 20, 30};
        int[] alias = stockLundi;              // même tableau !
        alias[0] = 0;
        System.out.println(Arrays.toString(stockLundi));   // [0, 20, 30]

        int[] copie = Arrays.copyOf(stockLundi, stockLundi.length); // vraie copie
        copie[1] = 99;
        System.out.println(Arrays.toString(stockLundi) + " / " + Arrays.toString(copie));

        int[] agrandi = Arrays.copyOf(stockLundi, 5);      // copie + agrandissement
        System.out.println(Arrays.toString(agrandi));

        System.out.println(stockLundi == copie);                // false : objets différents
        // true : même contenu
        System.out.println(Arrays.equals(stockLundi, new int[]{0, 20, 30}));
    }
}
```

Sortie console

```
[0, 20, 30]
[0, 20, 30] / [0, 99, 30]
[0, 20, 30, 0, 0]
false
true
```

La classe utilitaire Arrays

Méthode	Rôle
`Arrays.toString(t)`	représentation lisible `[a, b, c]` (afficher `t` directement donne une adresse comme `[I@1b6d3586`)
`Arrays.deepToString(m)`	idem pour un tableau à plusieurs dimensions
`Arrays.sort(t)`	tri croissant (sur place)
`Arrays.binarySearch(t, x)`	recherche dichotomique (le tableau doit être trié)
`Arrays.fill(t, v)`	remplit toutes les cases avec `v`
`Arrays.copyOf(t, n)` / `Arrays.copyOfRange(t, d, f)`	copie (éventuellement tronquée ou agrandie)
`Arrays.equals(a, b)`	compare le contenu
`Arrays.asList(…)` / `List.of(…)`	passage vers une liste (chapitre 11)
`Arrays.stream(t)`	flux pour sommes, moyennes, filtres (chapitre 12)

Tableaux à deux dimensions

Un tableau à deux dimensions est un **tableau de tableaux**. Les lignes peuvent même avoir des longueurs différentes.

VentesSemaine.java

```java
import java.util.Arrays;

public class VentesSemaine {
    public static void main(String[] args) {
        String[] produits = {"Bissap", "Thiakry", "Pagne"};
        // ventes[jour][produit] en nombre d'unités
```

```
        int[][] ventes = {
            {12, 5, 2},
            {8, 9, 1},
            {15, 4, 3}
        };
        System.out.println(Arrays.deepToString(ventes));
        for (int p = 0; p < produits.length; p++) {
            int total = 0;
            for (int j = 0; j < ventes.length; j++) {
                total += ventes[j][p];
            }
            System.out.printf("%-8s : %d unités%n", produits[p], total);
        }
        int[][] triangle = new int[3][];        // lignes de longueurs différentes
        for (int i = 0; i < triangle.length; i++) {
            triangle[i] = new int[i + 1];
            Arrays.fill(triangle[i], i + 1);
        }
        System.out.println(Arrays.deepToString(triangle));
    }
}
```

Sortie console

```
[[12, 5, 2], [8, 9, 1], [15, 4, 3]]
Bissap   : 35 unités
Thiakry  : 18 unités
Pagne    : 6 unités
[[1], [2, 2], [3, 3, 3]]
```

> **Côté backend**
>
> Dans une application réelle, on utilise rarement des tableaux « bruts » : leur taille est fixe. On leur préfère les **collections** (`List`, `Map`…) du chapitre 11. Les tableaux restent cependant utiles pour les performances, pour les paramètres `String[] args` et `varargs`, et on les rencontre dans de nombreuses API (octets d'un fichier, `split`, etc.).

5.2 Les méthodes

Une **méthode** est un bloc de code nommé, qui reçoit éventuellement des **paramètres** et renvoie éventuellement une **valeur**. Découper un programme en méthodes courtes, chacune ayant **une seule responsabilité**, le rend lisible, testable et réutilisable.

Java

```
static long calculerTva(long montantHt, int taux) {   // signature
    return Math.round(montantHt * taux / 100.0);       // corps
}
```

Élément	Rôle
`static`	méthode de classe, appelable sans objet (le chapitre 6 présente les méthodes d'instance)
`long`	type de la valeur renvoyée ; `void` si la méthode ne renvoie rien
`calculerTva`	nom : un verbe en `camelCase`
`(long montantHt, int taux)`	paramètres formels, avec leur type
`return`	renvoie la valeur et termine immédiatement la méthode

Java passe toujours par valeur

Lors d'un appel, **la valeur de chaque argument est copiée** dans le paramètre.

- Pour un type primitif, la méthode reçoit une copie du nombre : la modifier n'a aucun effet sur la variable de l'appelant.
- Pour un type référence, la méthode reçoit une copie **de la référence** : elle peut donc modifier **l'objet désigné** (par exemple le contenu d'un tableau), mais pas faire pointer la variable de l'appelant vers un autre objet.

PassageParValeur.java

```java
import java.util.Arrays;

public class PassageParValeur {
    static void augmenter(int stock) {
        stock += 10;                        // modifie la copie locale seulement
    }
    static void augmenterTous(int[] stocks) {
        for (int i = 0; i < stocks.length; i++) {
            stocks[i] += 10;                // modifie le tableau partagé
        }
    }
    static void remplacer(int[] stocks) {
        stocks = new int[]{0, 0, 0};        // change la copie de la référence seulement
    }

    public static void main(String[] args) {
        int s = 5;
        augmenter(s);
        System.out.println("s = " + s);

        int[] t = {1, 2, 3};
        augmenterTous(t);
        System.out.println("t = " + Arrays.toString(t));
        remplacer(t);
        System.out.println("t = " + Arrays.toString(t));
    }
}
```

Sortie console

```
s = 5
t = [11, 12, 13]
t = [11, 12, 13]
```

La surcharge (*overloading*)

Plusieurs méthodes d'une même classe peuvent porter le **même nom** si leurs **listes de paramètres** diffèrent (par le nombre ou le type des paramètres). Le compilateur choisit la bonne version selon les arguments. Le type de retour seul ne suffit pas à distinguer deux méthodes.

Les paramètres variables (*varargs*)

La notation `type... nom` permet de passer un nombre quelconque d'arguments ; dans la méthode, le paramètre est un tableau. Il doit être le dernier paramètre.

Tarification.java

```java
public class Tarification {
    static long prix(long unitaire, int quantite) {
        return unitaire * quantite;
    }
    static long prix(long unitaire, int quantite, int remisePourcent) {   // surcharge
        return prix(unitaire, quantite) * (100 - remisePourcent) / 100;
    }
    static long total(long... montants) {                                  // varargs
        long somme = 0;
        for (long m : montants) somme += m;
        return somme;
    }

    public static void main(String[] args) {
        System.out.println(prix(1_500, 3));
        System.out.println(prix(1_500, 3, 10));
        System.out.println(total());
        System.out.println(total(4_500, 2_000, 7_500));
        long[] panier = {1_000, 1_000, 3_000};
        System.out.println(total(panier));       // on peut aussi passer un tableau
    }
}
```

Sortie console

```
4500
4050
0
14000
5000
```

Documenter une méthode

Une méthode publique se documente avec un commentaire **Javadoc**, que l'outil `javadoc` transforme en pages HTML et que les IDE affichent au survol.

Java

```java
/**
 * Calcule le montant de TVA arrondi au franc le plus proche.
 *
 * @param montantHt montant hors taxes, en FCFA (doit être positif)
 * @param taux      taux de TVA en pourcentage (18 au Sénégal)
 * @return le montant de la TVA en FCFA
 * @throws IllegalArgumentException si le montant est négatif
 */
static long calculerTva(long montantHt, int taux) { ... }
```

> **Nouveauté Java 23 — Commentaires de documentation en Markdown (JEP 467)**
>
> On peut désormais écrire la Javadoc en Markdown, avec des lignes commençant par `///` : `/// Calcule la TVA. Renvoie **0** si le montant est nul.` Les balises `@param`, `@return` et `@throws` restent utilisables.

5.3 La récursivité

Une méthode est **récursive** lorsqu'elle s'appelle elle-même. Toute méthode récursive comporte :

1. un ou plusieurs **cas de base**, qui se résolvent sans appel récursif ;
2. un **cas général**, qui ramène le problème à un problème plus petit.

Recursivite.java

```java
public class Recursivite {
    static long factorielle(int n) {
        if (n <= 1) return 1;                 // cas de base
        return n * factorielle(n - 1);        // cas général
    }

    static int pgcd(int a, int b) {           // algorithme d'Euclide
        return b == 0 ? a : pgcd(b, a % b);
    }

    /** Recherche dichotomique récursive dans un tableau trié : indice ou -1. */
    static int rechercher(long[] t, long x, int debut, int fin) {
        if (debut > fin) return -1;
        int milieu = (debut + fin) >>> 1;     // évite le débordement de (debut + fin) / 2
        if (t[milieu] == x) return milieu;
        return t[milieu] < x ? rechercher(t, x, milieu + 1, fin)
                             : rechercher(t, x, debut, milieu - 1);
    }

    static long fibonacciNaif(int n) {
        return n < 2 ? n : fibonacciNaif(n - 1) + fibonacciNaif(n - 2);
    }

    static long fibonacci(int n) {            // version itérative, linéaire
        long a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            long c = a + b; a = b; b = c;
        }
        return a;
    }

    public static void main(String[] args) {
        System.out.println("10! = " + factorielle(10));
        System.out.println("pgcd(84, 36) = " + pgcd(84, 36));
        long[] prix = {500, 1_000, 1_500, 2_500, 7_500, 12_000};
        System.out.println("7500 à l'indice " + rechercher(prix, 7_500, 0, prix.length - 1));
        System.out.println("3000 à l'indice " + rechercher(prix, 3_000, 0, prix.length - 1));
        System.out.println("fib(30) = " + fibonacciNaif(30) + " = " + fibonacci(30));
        System.out.println("fib(90) = " + fibonacci(90));
    }
}
```

Sortie console

```
10! = 3628800
pgcd(84, 36) = 12
7500 à l'indice 4
3000 à l'indice -1
fib(30) = 832040 = 832040
fib(90) = 2880067194370816120
```

Piège à éviter

- Sans cas de base (ou si le cas de base n'est jamais atteint), la récursion ne s'arrête pas et la JVM lève une `StackOverflowError`.
- La version récursive naïve de Fibonacci recalcule sans cesse les mêmes valeurs : son temps d'exécution est **exponentiel** (essayez `fibonacciNaif(50)`). Une récursion n'est pas toujours la meilleure solution.

5.4 Fil rouge TerangaShop — statistiques du mois

StatsMois.java

```java
import java.util.Arrays;
```

```java
public class StatsMois {
    static long somme(long[] t) {
        long s = 0;
        for (long v : t) s += v;
        return s;
    }
    static int indiceMax(long[] t) {
        int iMax = 0;
        for (int i = 1; i < t.length; i++) {
            if (t[i] > t[iMax]) iMax = i;
        }
        return iMax;
    }
    static long mediane(long[] t) {
        long[] trie = Arrays.copyOf(t, t.length);   // on ne modifie pas l'original
        Arrays.sort(trie);
        int n = trie.length;
        return n % 2 == 1 ? trie[n / 2] : (trie[n / 2 - 1] + trie[n / 2]) / 2;
    }

    public static void main(String[] args) {
        // chiffre d'affaires quotidien (FCFA) sur 10 jours
        long[] ca = {85_000, 120_500, 64_000, 210_000, 99_500, 45_000, 180_000,
                132_000, 77_500, 150_000};
        long total = somme(ca);
        System.out.println("Total       : " + total + " FCFA");
        System.out.println("Moyenne     : " + total / ca.length + " FCFA/jour");
        System.out.println("Médiane     : " + mediane(ca) + " FCFA");
        int j = indiceMax(ca);
        System.out.println("Meilleur jour : jour " + (j + 1) + " avec " + ca[j] + " FCFA");
        int joursObjectif = 0;
        for (long v : ca) if (v >= 100_000) joursObjectif++;
        System.out.println("Jours >= 100 000 FCFA : " + joursObjectif);
    }
}
```

Sortie console

```
Total       : 1163500 FCFA
Moyenne     : 116350 FCFA/jour
Médiane     : 110000 FCFA
Meilleur jour : jour 4 avec 210000 FCFA
Jours >= 100 000 FCFA : 5
```

Exercices du chapitre 5

Exercice 5.1 ★ — Minimum, maximum, moyenne

Écrivez un programme qui lit `n` entiers au clavier, les range dans un tableau, puis affiche le minimum, le maximum, la moyenne (avec deux décimales) et le nombre de valeurs supérieures à la moyenne. Structurez-le en méthodes `min`, `max`, `moyenne`.

Exercice 5.2 ★ — Inverser un tableau

Écrivez `static void inverser(int[] t)` qui inverse **sur place** l'ordre des éléments (sans tableau auxiliaire), puis `static int[] inverse(int[] t)` qui renvoie un **nouveau** tableau inversé sans modifier l'original. Expliquez la différence du point de vue de l'appelant.

Exercice 5.3 ★★ — Fusion de deux tableaux triés

Écrivez `static int[] fusionner(int[] a, int[] b)` qui reçoit deux tableaux triés par ordre croissant et renvoie un tableau trié contenant tous leurs éléments, en un seul parcours (sans appeler `Arrays.sort`). Exemple : `[1, 4, 9]` et `[2, 3, 10, 12]` donnent `[1, 2, 3, 4, 9, 10, 12]`.

Exercice 5.4 ★★ — Tri par sélection

Implémentez le tri par sélection sur un tableau de `long`, en comptant le nombre de comparaisons effectuées. Comparez le résultat avec `Arrays.sort` sur le même tableau (utilisez `Arrays.copyOf` pour trier deux copies). Combien de comparaisons pour un tableau de 10, 100, 1 000 éléments ? Quelle est la complexité ?

Exercice 5.5 ★★ — Récursivité

Écrivez les méthodes récursives suivantes : `sommeChiffres(int n)` (ex. 2026 → 10) ; `puissance(long x, int n)` en utilisant l'exponentiation rapide (`x^n = (x^(n/2))²` si `n` est pair) ; `estPalindrome(String s)` ; `afficherBinaire(int n)` qui affiche l'écriture binaire de `n`.

Exercice 5.6 ★★★ — Tableau de bord des ventes

TerangaShop vend 4 produits. Les ventes (en unités) des 7 jours de la semaine sont dans un tableau `int[7][4]`, et les prix unitaires dans un tableau `long[4]`. Écrivez des méthodes qui calculent : le chiffre d'affaires de chaque jour, le nombre d'unités vendues de chaque produit, le produit qui a rapporté le plus sur la semaine, et le jour record. Affichez le tout sous forme d'un tableau aligné avec `printf`, avec les jours nommés en français (réutilisez la switch expression du chapitre 4).

PARTIE II — Programmation orientée objet moderne

- Chapitre 6 — Classes, objets et encapsulation
- Chapitre 7 — Héritage, polymorphisme, abstraction et interfaces
- Chapitre 8 — Records, enums et pattern matching
- Chapitre 9 — Exceptions et gestion des erreurs

Chapitre 6 — Classes, objets et encapsulation

Objectifs du chapitre

- Comprendre les notions d'objet, de classe, d'état et de comportement.
- Écrire une classe complète : attributs, constructeurs, méthodes, `toString`, `equals`, `hashCode`.
- Protéger l'état d'un objet grâce à l'encapsulation.
- Distinguer membres d'instance et membres `static`.
- Organiser le code en packages et maîtriser les modificateurs d'accès.
- Concevoir des objets immuables.

6.1 Penser « objet »

En programmation procédurale (comme en C), on sépare les **données** (des structures) et les **traitements** (des fonctions qui reçoivent ces structures). En programmation orientée objet, on regroupe données et traitements dans une même entité : l'**objet**.

Un objet possède :

- un **état** : les valeurs de ses attributs (le prix d'un produit, son stock) ;
- un **comportement** : les méthodes qu'on peut lui appliquer (approvisionner, vendre) ;
- une **identité** : deux produits ayant le même libellé et le même prix restent deux objets distincts en mémoire.

Une **classe** est le modèle (le « moule ») qui décrit l'état et le comportement communs à tous les objets d'un même type. Un objet créé à partir d'une classe est appelé une **instance** de cette classe.

Concept	Analogie	TerangaShop
Classe	le plan d'une maison	`Produit`
Objet (instance)	une maison construite à partir du plan	le produit « Bissap 1 L » à 1 500 FCFA
Attribut	le nombre de pièces, la couleur	`prix`, `stock`
Méthode	ouvrir une porte	`approvisionner(50)`

On représente une classe par un **diagramme de classes UML** : trois compartiments (nom, attributs, méthodes) ; `-` signifie privé et `+` public.

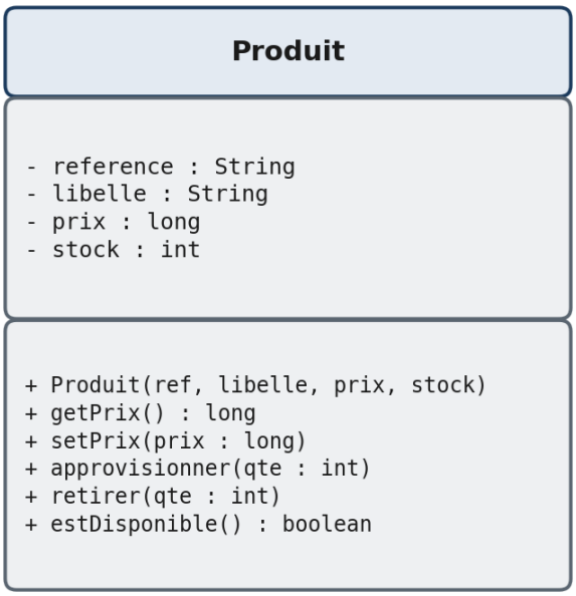


Figure 6.1 — La classe Produit en notation UML

6.2 Une première classe

Produit.java

```java
public class Produit {
    // Attributs (état de chaque objet)
    String libelle;
    long prix;
    int stock;


    // Méthodes (comportement)
    void approvisionner(int quantite) {
        stock += quantite;
    }

    boolean estDisponible() {
        return stock > 0;
    }

    String description() {
        return libelle + " - " + prix + " FCFA (stock : " + stock + ")";
    }
}
```

DemoProduit.java

```java
public class DemoProduit {
    public static void main(String[] args) {
        Produit bissap = new Produit();      // création d'un objet (instanciation)
        bissap.libelle = "Bissap 1 L";
        bissap.prix = 1_500;
        bissap.approvisionner(40);

        Produit pagne = new Produit();       // un second objet, indépendant
        pagne.libelle = "Pagne wax";
        pagne.prix = 7_500;

        System.out.println(bissap.description());
        System.out.println(pagne.description() + " disponible ? " + pagne.estDisponible());

        Produit memeProduit = bissap;        // deux références vers le MÊME objet
        memeProduit.prix = 1_750;
        System.out.println(bissap.prix);
    }
}
```

Sortie console

```
Bissap 1 L - 1500 FCFA (stock : 40)
Pagne wax - 7500 FCFA (stock : 0) disponible ? false
1750
```

Le mot-clé new réserve la mémoire de l'objet dans le **tas**, initialise ses attributs à leur valeur par défaut, exécute le constructeur, puis renvoie une **référence** vers l'objet. La variable `bissap` ne contient pas l'objet mais cette référence.

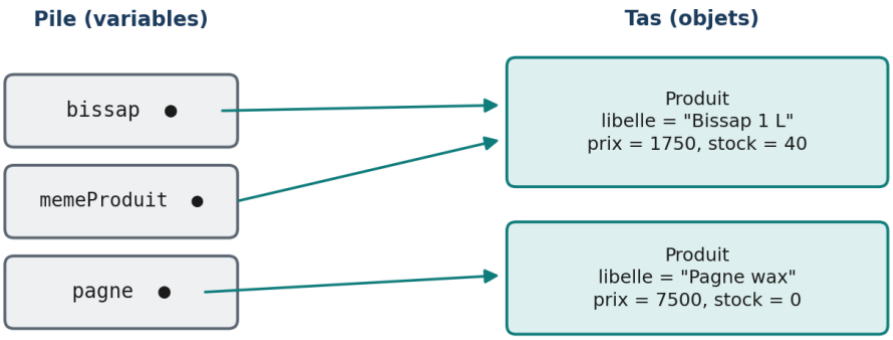


Figure 6.2 — Deux variables peuvent référencer le même objet

Lorsqu'un objet n'est plus référencé par aucune variable, il devient inaccessible ; le **ramasse-miettes** (*Garbage Collector*) récupérera sa mémoire automatiquement. Il n'y a ni `free` ni destructeur en Java.

> **Piège à éviter — `NullPointerException`**
>
> Une variable de type objet qui ne référence rien vaut `null`. Appeler une méthode dessus (`p.description()` avec `p == null`) provoque une `NullPointerException`, l'erreur d'exécution la plus répandue en Java. Depuis Java 14, le message précise quelle variable était nulle (*helpful NullPointerException messages*).

Cette première classe a un défaut grave : n'importe quel code peut écrire `bissap.prix = -500;` ou `bissap.stock = -3;`. Rien ne garantit que l'objet reste **cohérent**. C'est le rôle des constructeurs et de l'encapsulation.

6.3 Les constructeurs

Un **constructeur** est une méthode spéciale, appelée par `new`, qui initialise l'objet. Il porte **exactement le nom de la classe** et n'a **pas de type de retour** (même pas `void`).

- Si vous n'écrivez aucun constructeur, le compilateur fournit un **constructeur par défaut** sans paramètre.
- Dès que vous en écrivez un, ce constructeur par défaut disparaît.
- Une classe peut avoir plusieurs constructeurs (surcharge).

Le mot-clé **`this`** désigne l'objet courant. Il sert à lever l'ambiguïté quand un paramètre a le même nom qu'un attribut (`this.prix = prix;`), et, sous la forme `this(...)`, à appeler un autre constructeur de la même classe.

Java

```java
public Produit(String reference, String libelle, long prix, int stock) {
    this.reference = reference;   // this.reference : l'attribut ; reference : le paramètre
    this.libelle = libelle;
    this.prix = prix;
    this.stock = stock;
}

public Produit(String reference, String libelle, long prix) {
    this(reference, libelle, prix, 0);   // délègue au constructeur complet
}
```

> **Nouveauté Java 25 — Corps de constructeur flexibles (JEP 513)**
>
> Jusqu'à Java 24, l'appel `this(...)` ou `super(...)` devait être la **toute première** instruction d'un constructeur. Depuis Java 25, on peut placer des instructions **avant** cet appel (le « prologue ») : valider des arguments, préparer des valeurs dans des variables locales… Seule restriction : le prologue ne peut pas utiliser l'objet en cours de construction (`this`), puisqu'il n'est pas encore initialisé.

Java

```java
/** Construit un produit à partir d'une ligne CSV "ref;libellé;prix;stock". */
public Produit(String ligneCsv) {
    String[] c = ligneCsv.split(";");              // prologue : autorisé en Java 25
    if (c.length != 4) {
        throw new IllegalArgumentException("Ligne CSV invalide : " + ligneCsv);
    }
    this(c[0], c[1], Long.parseLong(c[2]), Integer.parseInt(c[3]));
}
```

6.4 L'encapsulation

L'**encapsulation** consiste à rendre les attributs **privés** (`private`) et à ne permettre leur lecture ou leur modification qu'à travers des **méthodes publiques** qui contrôlent les règles métier. On garantit ainsi les **invariants** de l'objet : des propriétés qui doivent toujours être vraies (« le prix est strictement positif », « le stock n'est jamais négatif »).

- Un **accesseur** (*getter*) renvoie la valeur d'un attribut : `getPrix()`, `isActif()` pour un booléen.
- Un **mutateur** (*setter*) modifie un attribut **après vérification** : `setPrix(long prix)`.
- On ne crée **pas** systématiquement un setter par attribut : on n'expose que les opérations qui ont un sens métier. Un produit n'a pas de `setStock` mais des méthodes `approvisionner` et `retirer`.

> **À retenir**
>
> Encapsuler, ce n'est pas « ajouter des getters et des setters partout ». C'est **décider quelles opérations** ont le droit de modifier l'état de l'objet, et y placer les vérifications. Un objet bien encapsulé ne peut pas être mis dans un état invalide.

6.5 Membres d'instance et membres `static`

Un attribut ou une méthode **`static`** appartient à la **classe** elle-même et non à chaque objet : il est partagé par toutes les instances.

	Membre d'instance	Membre `static` (de classe)
Existe	une fois **par objet**	une seule fois **pour la classe**
Accès	`objet.membre`	`NomClasse.membre`
Exemple	le prix d'**un** produit	le nombre total de produits créés, `Math.sqrt`, une constante `TAUX_TVA`
Peut utiliser `this`	oui	non (il n'y a pas d'objet courant)

Une méthode `static` ne peut pas accéder directement aux attributs d'instance : elle ne saurait pas de quel objet il s'agit.

6.6 Packages et modificateurs d'accès

Les packages

Un **package** regroupe des classes liées, évite les conflits de noms et contrôle la visibilité. Par convention, le nom d'un package commence par le nom de domaine de l'organisation **inversé** : `sn.isep.terangashop`. La déclaration `package` est la première ligne du fichier, et l'arborescence des dossiers doit correspondre au nom du package.

```
src/
└── sn/isep/terangashop/
    ├── App.java                    (package sn.isep.terangashop;)
    └── model/
        ├── Produit.java            (package sn.isep.terangashop.model;)
        ├── Client.java
        ├── LigneCommande.java
        └── Commande.java
```

Pour utiliser une classe d'un autre package, on l'importe : `import sn.isep.terangashop.model.Produit;` ou `import sn.isep.terangashop.model.*;`. Les classes de `java.lang` (`String`, `Math`, `System`…) sont importées automatiquement.

Terminal

```
javac -d out $(find src -name "*.java")     # compile tout dans le dossier out/
java -cp out sn.isep.terangashop.App         # nom complet de la classe principale
```

> **Nouveauté Java 25 — Déclarations d'import de module (JEP 511)**
>
> `import module java.base;` importe d'un coup tous les packages exportés par un module (`java.util`, `java.time`, `java.io`…). C'est pratique pour les scripts et l'apprentissage ; dans un projet, on préfère des imports explicites, plus lisibles.

Les modificateurs d'accès

Modificateur	Même classe	Même package	Sous-classe (autre package)	Partout
`public`	oui	oui	oui	oui
`protected`	oui	oui	oui	non
(aucun : « package-private »)	oui	oui	non	non
`private`	oui	non	non	non

Règle pratique : **attributs `private`**, méthodes `public` seulement si elles font partie du « contrat » de la classe, le reste `private` ou *package-private*. Commencez toujours par la visibilité la plus restreinte.

6.7 `toString`, `equals` et `hashCode`

Toutes les classes héritent de la classe `Object` (chapitre 7), qui fournit trois méthodes qu'on redéfinit presque toujours :

- **`toString()`** renvoie une représentation textuelle de l'objet ; elle est appelée automatiquement par `println` et par la concaténation. Par défaut, elle affiche un texte peu utile comme `Produit@1b6d3586`.
- **`equals(Object o)`** indique si deux objets sont **égaux en valeur**. Par défaut, elle compare les références (comme `==`).
- **`hashCode()`** renvoie un entier utilisé par les tables de hachage (`HashMap`, `HashSet`, chapitre 11).

> **Le contrat `equals` / `hashCode`**
> Si vous redéfinissez `equals`, vous **devez** redéfinir `hashCode` de façon cohérente : deux objets égaux au sens d'`equals` doivent avoir le même `hashCode`. Sinon, un `HashSet` pourra contenir deux « doublons » et une `HashMap` ne retrouvera pas vos clés. Utilisez `Objects.equals` et `Objects.hash` pour les écrire simplement, ou laissez l'IDE les générer.

Pour une **entité** métier (un produit, un client, une commande), l'égalité repose en général sur son **identifiant** : deux objets `Produit` ayant la même référence `PRD-001` représentent le même produit, même si l'un a un prix mis à jour.

6.8 Fil rouge TerangaShop — le modèle objet

Voici le modèle complet, organisé en packages. Lisez attentivement les commentaires : chaque choix de conception est justifié.

sn/isep/terangashop/model/Produit.java

```
package sn.isep.terangashop.model;

import java.util.Objects;

public class Produit {
    private static int nbProduitsCrees = 0;      // partagé par toutes les instances

    private final String reference;              // final : ne change jamais
    private String libelle;
    private long prix;                            // en FCFA
    private int stock;
```

```
    public Produit(String reference, String libelle, long prix, int stock) {
        if (reference == null || !reference.matches("PRD-\\d{3}")) {
            throw new IllegalArgumentException("Référence invalide : " + reference);
        }
        if (libelle == null || libelle.isBlank()) {
            throw new IllegalArgumentException("Le libellé est obligatoire");
        }
        if (stock < 0) {
            throw new IllegalArgumentException("Stock négatif : " + stock);
        }
        this.reference = reference;
        this.libelle = libelle.strip();
        setPrix(prix);                          // réutilise la validation du prix
        this.stock = stock;
        nbProduitsCrees++;
    }

    public Produit(String reference, String libelle, long prix) {
        this(reference, libelle, prix, 0);
    }

    /** Construit un produit à partir d'une ligne CSV "ref;libellé;prix;stock". */
    public Produit(String ligneCsv) {
        String[] c = ligneCsv.split(";");               // prologue (Java 25)
        if (c.length != 4) {
            throw new IllegalArgumentException("Ligne CSV invalide : " + ligneCsv);
        }
        this(c[0], c[1], Long.parseLong(c[2]), Integer.parseInt(c[3]));
    }

    // --- Accesseurs ---
    public String getReference() { return reference; }
    public String getLibelle()   { return libelle; }
    public long getPrix()        { return prix; }
    public int getStock()        { return stock; }
    public static int getNbProduitsCrees() { return nbProduitsCrees; }

    // --- Opérations métier (pas de setStock !) ---
    public void setPrix(long prix) {
        if (prix <= 0) {
            throw new IllegalArgumentException("Le prix doit être positif : " + prix);
        }
        this.prix = prix;
    }

    public void approvisionner(int quantite) {
        if (quantite <= 0)
                throw new IllegalArgumentException("Quantité invalide : " + quantite);
        stock += quantite;
    }

    public void retirer(int quantite) {
        if (quantite <= 0)
                throw new IllegalArgumentException("Quantité invalide : " + quantite);
        if (quantite > stock) {
            throw new IllegalStateException("Stock insuffisant pour " + reference
                    + " : demandé " + quantite + ", disponible " + stock);
        }
        stock -= quantite;
    }

    public boolean estDisponible() { return stock > 0; }

    // --- Méthodes héritées de Object ---
    @Override
    public String toString() {
        return "%s | %-18s | %6d FCFA | stock %3d".formatted(reference, libelle, prix, stock);
    }
```

```
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Produit autre)) return false;   // pattern matching (chapitre 8)
        return reference.equals(autre.reference);          // égalité sur l'identifiant
    }

    @Override
    public int hashCode() {
        return Objects.hash(reference);
    }
}
```

sn/isep/terangashop/model/Client.java

```
package sn.isep.terangashop.model;

public class Client {
    private static long prochainId = 1;

    private final long id;
    private final String nom;
    private String telephone;
    private String quartier;

    public Client(String nom, String telephone, String quartier) {
        if (nom == null || nom.isBlank()) throw new IllegalArgumentException("Nom obligatoire");
        this.id = prochainId++;                  // identifiant automatique
        this.nom = nom;
        setTelephone(telephone);
        this.quartier = quartier;
    }

    public void setTelephone(String telephone) {
        String t = telephone.replaceAll("\\s", "");
        if (!t.matches("7[05678]\\d{7}")) {
            throw new IllegalArgumentException("Téléphone invalide : " + telephone);
        }
        this.telephone = t;
    }

    public long getId() { return id; }
    public String getNom() { return nom; }
    public String getTelephone() { return telephone; }
    public String getQuartier() { return quartier; }
    public void demenager(String nouveauQuartier) { this.quartier = nouveauQuartier; }

    @Override
    public String toString() {
        return "Client#" + id + " " + nom + " (" + quartier + ")";
    }
}
```

La classe `LigneCommande` est **immuable** : tous ses attributs sont `final` et elle n'a aucune méthode de modification. Elle mémorise le prix unitaire **au moment de la commande** : si le prix du produit change plus tard, les commandes passées ne doivent pas être modifiées.

sn/isep/terangashop/model/LigneCommande.java

```
package sn.isep.terangashop.model;

public final class LigneCommande {          // final : la classe ne peut pas être étendue
    private final Produit produit;
    private final int quantite;
    private final long prixUnitaire;        // photographie du prix au moment de l'achat
```

```
    public LigneCommande(Produit produit, int quantite) {
        if (quantite <= 0) throw new IllegalArgumentException("Quantité invalide");
        this.produit = produit;
        this.quantite = quantite;
        this.prixUnitaire = produit.getPrix();
    }

    public Produit getProduit() { return produit; }
    public int getQuantite() { return quantite; }
    public long getPrixUnitaire() { return prixUnitaire; }
    public long sousTotal() { return prixUnitaire * quantite; }

    @Override
    public String toString() {
        return "%-18s %3d x %6d = %8d".formatted(produit.getLibelle(), quantite,
                prixUnitaire, sousTotal());
    }
}
```

La classe `Commande` utilise une `ArrayList`, une liste dont la taille s'adapte automatiquement (elle sera étudiée en détail au chapitre 11). Notez que la méthode `getLignes()` ne renvoie pas la liste interne, mais une **copie non modifiable** : sinon, un code extérieur pourrait ajouter des lignes sans passer par `ajouter` et contourner la vérification du stock.

sn/isep/terangashop/model/Commande.java

```
package sn.isep.terangashop.model;

import java.util.ArrayList;
import java.util.List;

public class Commande {
    private static int compteur = 0;
    public static final long SEUIL_LIVRAISON_GRATUITE = 50_000;
    public static final long FRAIS_LIVRAISON = 1_500;

    private final String numero;
    private final Client client;
    private final List<LigneCommande> lignes = new ArrayList<>();
    private boolean validee = false;

    public Commande(Client client) {
        this.client = client;
        compteur++;
        this.numero = "TS-2026-%06d".formatted(compteur);
    }

    public void ajouter(Produit produit, int quantite) {
        if (validee) throw new IllegalStateException("Commande déjà validée : " + numero);
        produit.retirer(quantite);                // lève une exception si stock insuffisant
        lignes.add(new LigneCommande(produit, quantite));
    }

    public long sousTotal() {
        long total = 0;
        for (LigneCommande l : lignes) total += l.sousTotal();
        return total;
    }

    public long fraisLivraison() {
        return sousTotal() >= SEUIL_LIVRAISON_GRATUITE ? 0 : FRAIS_LIVRAISON;
    }

    public long total() { return sousTotal() + fraisLivraison(); }
```

```
    public void valider() {
        if (lignes.isEmpty()) throw new IllegalStateException("Commande vide");
        validee = true;
    }

    public String getNumero() { return numero; }
    public Client getClient() { return client; }
    public boolean isValidee() { return validee; }
    public List<LigneCommande> getLignes() { return List.copyOf(lignes); }   // copie défensive

    public String ticket() {
        StringBuilder sb = new StringBuilder();
        sb.append("Commande ").append(numero).append(" - ").append(client).append('\n');
        for (LigneCommande l : lignes) sb.append("  ").append(l).append('\n');
        sb.append("  Livraison : ").append(fraisLivraison()).append(" FCFA\n");
        sb.append("  TOTAL     : ").append(total()).append(" FCFA");
        return sb.toString();
    }
}
```

sn/isep/terangashop/App.java

```
package sn.isep.terangashop;

import sn.isep.terangashop.model.*;

public class App {
    public static void main(String[] args) {
        Produit bissap = new Produit("PRD-001", "Bissap 1 L", 1_500, 50);
        Produit pagne = new Produit("PRD-014;Pagne wax 6 yards;7500;12");   // constructeur CSV
        Produit sac = new Produit("PRD-020", "Sac cuir Ngaye", 25_000);
        sac.approvisionner(3);

        Client awa = new Client("Awa Diop", "77 123 45 67", "Médina");
        Commande c1 = new Commande(awa);
        c1.ajouter(bissap, 4);
        c1.ajouter(pagne, 2);
        c1.valider();
        System.out.println(c1.ticket());

        Commande c2 = new Commande(new Client("Moussa Ndiaye", "78 000 11 22", "Rufisque"));
        c2.ajouter(sac, 2);
        c2.ajouter(bissap, 1);
        System.out.println(c2.ticket());

        try {
            c2.ajouter(sac, 5);
        } catch (IllegalStateException e) {
            System.out.println("Refusé : " + e.getMessage());
        }
        try {
            bissap.setPrix(-100);
        } catch (IllegalArgumentException e) {
            System.out.println("Refusé : " + e.getMessage());
        }

        System.out.println(bissap);
        System.out.println("Produits créés : " + Produit.getNbProduitsCrees());
        System.out.println("Même produit ? " + bissap.equals(new Produit("PRD-001",
                "Bissap", 1_600)));
    }
}
```

Sortie console

```
Commande TS-2026-000001 - Client#1 Awa Diop (Médina)
  Bissap 1 L           4 x   1500 =     6000
  Pagne wax 6 yards    2 x   7500 =    15000
  Livraison : 1500 FCFA
  TOTAL     : 22500 FCFA
Commande TS-2026-000002 - Client#2 Moussa Ndiaye (Rufisque)
  Sac cuir Ngaye       2 x  25000 =    50000
  Bissap 1 L           1 x   1500 =     1500
  Livraison : 0 FCFA
  TOTAL     : 51500 FCFA
Refusé : Stock insuffisant pour PRD-020 : demandé 5, disponible 1
Refusé : Le prix doit être positif : -100
PRD-001 | Bissap 1 L         |   1500 FCFA | stock  45
Produits créés : 3
Même produit ? true
```

6.9 Les objets immuables

Un objet **immuable** ne change jamais d'état après sa construction. `String`, `Integer`, `LocalDate` et notre `LigneCommande` sont immuables. Pour écrire une classe immuable :

1. déclarer tous les attributs `private final` ;
2. ne fournir aucun mutateur ;
3. déclarer la classe `final` (pour qu'aucune sous-classe ne puisse ajouter d'état modifiable) ;
4. faire des **copies défensives** des objets modifiables reçus ou renvoyés (listes, tableaux) ;
5. pour « modifier », renvoyer un **nouvel** objet (méthodes `avecPrix(...)`, `plus(...)`).

Avantages : un objet immuable est simple à raisonner, peut être partagé sans risque entre plusieurs parties du programme et entre plusieurs threads (chapitre 14), et fait une excellente clé de `HashMap`. Les **records** du chapitre 8 permettent d'écrire des classes immuables en une seule ligne.

> **Côté backend**
>
> Dans une application Spring Boot, on distingue les **entités** (objets mutables, avec un identifiant, persistés en base) et les **objets valeur** ou **DTO** (*Data Transfer Objects*, immuables, qui transportent des données entre couches ou vers le client JSON). Cette distinction, que vous venez d'appliquer avec `Produit` et `LigneCommande`, est au cœur de l'architecture backend.

Exercices du chapitre 6

Exercice 6.1 ★ — Compte Wave

Écrivez une classe `CompteMobile` avec un numéro de téléphone (non modifiable), un titulaire et un solde en FCFA. Elle offre : `deposer(long montant)`, `retirer(long montant)` (refusé si le solde est insuffisant), `transferer(CompteMobile destinataire, long montant)` (des frais de 1 % s'appliquent, plafonnés à 5 000 FCFA, et payés par l'émetteur) et `toString()`. Toutes les méthodes refusent les montants négatifs ou nuls. Testez dans un `main`.

Exercice 6.2 ★ — Étudiant

Écrivez une classe `Etudiant` (matricule attribué automatiquement sous la forme `ISEP-2026-001`, nom, prénom, tableau de 5 notes). Ajoutez les méthodes `ajouterNote(int indice, double note)` (note

entre 0 et 20), `moyenne()`, `mention()` (réutilisez l'exercice 4.1) et `toString()`. Ajoutez une méthode `static` qui renvoie le nombre d'étudiants créés.

Exercice 6.3 ★★ — Fraction immuable

Écrivez une classe **immuable** `Fraction` (numérateur, dénominateur non nul). Le constructeur simplifie la fraction (à l'aide du PGCD) et place le signe au numérateur. Ajoutez `plus`, `moins`, `fois`, `divisePar` (qui renvoient une **nouvelle** fraction), `valeur()` (en `double`), `toString()` (`3/4`, ou `2` si le dénominateur vaut 1), `equals` et `hashCode`. Vérifiez que `new Fraction(2, 4).equals(new Fraction(-1, -2))` est vrai.

Exercice 6.4 ★★ — Le contrat equals/hashCode

1. Écrivez une classe `Point` (x, y) qui redéfinit `equals` mais **pas** `hashCode`. Ajoutez deux points égaux dans un `java.util.HashSet<Point>` et affichez la taille de l'ensemble. Expliquez le résultat.
2. Corrigez la classe et recommencez.
3. Que se passe-t-il si l'on modifie `x` d'un point **après** l'avoir ajouté au `HashSet`, puis qu'on appelle `contains` ? Qu'en concluez-vous sur l'intérêt des objets immuables comme clés ?

Exercice 6.5 ★★ — Encapsulation défaillante

La classe suivante est mal encapsulée. Listez tous les moyens par lesquels un code extérieur peut la mettre dans un état incohérent, puis corrigez-la.

Java

```java
public class Panier {
    public String[] articles = new String[10];
    public long[] prix = new long[10];
    public int nb;
    public long total;

    public long[] getPrix() { return prix; }
    public void ajouter(String a, long p) { articles[nb] = a; prix[nb] = p; nb++; total += p; }
}
```

Exercice 6.6 ★★★ — Bibliothèque de l'ISEP

Modélisez la gestion des emprunts de la bibliothèque : classes `Livre` (ISBN, titre, auteur, nombre d'exemplaires), `Adherent` (numéro, nom, liste de ses emprunts en cours ; maximum 3) et `Emprunt` (livre, adhérent, jour d'emprunt sous forme d'entier, jour de retour prévu = emprunt + 14). Écrivez une classe `Bibliotheque` avec les méthodes `emprunter(adherent, livre, jour)` et `rendre(adherent, livre, jour)` qui renvoie la pénalité de retard (100 FCFA par jour). Toutes les règles métier doivent être vérifiées dans les classes du modèle et signalées par des exceptions. Organisez le code en deux packages : `sn.isep.biblio.model` et `sn.isep.biblio`.

Chapitre 7 — Héritage, polymorphisme, abstraction et interfaces

Objectifs du chapitre

- Créer une hiérarchie de classes avec `extends` et appeler le constructeur parent avec `super`.
- Redéfinir des méthodes et exploiter le polymorphisme (liaison dynamique).
- Utiliser des classes abstraites et des interfaces, et savoir choisir entre les deux.
- Connaître les classes internes, anonymes et imbriquées (patron *Builder*).
- Préférer la composition à l'héritage quand c'est pertinent.
- Restreindre une hiérarchie avec les classes scellées (`sealed`).

7.1 L'héritage

L'**héritage** permet de définir une nouvelle classe (la **sous-classe**, ou classe fille) à partir d'une classe existante (la **super-classe**, ou classe mère). La sous-classe **hérite** des attributs et des méthodes de sa mère, peut en **ajouter** et peut **redéfinir** certaines méthodes.

L'héritage exprime une relation « **est un** » : un paiement Wave *est un* paiement ; un livreur *est un* employé. Si la phrase « X est un Y » sonne faux, l'héritage est probablement une mauvaise idée.

Java

```java
public class PaiementWave extends Paiement { ... }
```

Règles essentielles :

- Java n'autorise que l'**héritage simple** : une classe a **une seule** super-classe directe.
- Toute classe hérite, directement ou non, de **`java.lang.Object`**.
- Les constructeurs ne sont **pas** hérités. Le constructeur de la sous-classe doit appeler un constructeur de la super-classe avec `super(...)` ; si vous ne l'écrivez pas, le compilateur insère `super()` (sans argument), qui doit alors exister.
- Les membres `private` de la mère existent dans l'objet fille mais ne sont **pas accessibles** directement ; les membres `protected` le sont.

7.2 Redéfinition de méthodes

Une sous-classe **redéfinit** (*override*) une méthode héritée en déclarant une méthode de **même signature**. L'annotation `@Override` demande au compilateur de vérifier qu'il s'agit bien d'une redéfinition : utilisez-la **toujours**, elle détecte les fautes de frappe (`tostring()` au lieu de `toString()`).

	Surcharge (*overloading*)	Redéfinition (*overriding*)
Où ?	dans la même classe (ou une sous-classe)	dans une sous-classe
Signature	même nom, **paramètres différents**	même nom, **mêmes paramètres**
Choix de la méthode	à la **compilation**, selon le type des arguments	à l'**exécution**, selon le type réel de l'objet
Annotation	aucune	`@Override`

Dans une méthode redéfinie, `super.methode(...)` appelle la version de la super-classe. On ne peut pas réduire la visibilité d'une méthode redéfinie (une méthode `public` reste `public`). Une méthode déclarée `final` ne peut pas être redéfinie ; une classe `final` ne peut pas être étendue (`String` est `final`).

7.3 Le polymorphisme

Le **polymorphisme** (« plusieurs formes ») est la possibilité de manipuler des objets de types différents à travers un **type commun**, tout en obtenant pour chacun le comportement qui lui est propre.

- **Surclassement** (*upcasting*) : une référence de type super-classe peut désigner un objet d'une sous-classe : `Paiement p = new PaiementWave(...)`.
- **Liaison dynamique** : lorsqu'on appelle `p.frais()`, c'est la méthode du **type réel** de l'objet (`PaiementWave`) qui est exécutée, et non celle du type déclaré de la variable (`Paiement`).

C'est ce mécanisme qui permet d'écrire du code **générique** : une méthode qui reçoit un `Paiement` fonctionnera avec tous les modes de paiement, y compris ceux qui seront créés dans deux ans.

7.4 Les classes abstraites

Une **classe abstraite** (`abstract class`) représente un concept trop général pour être instancié directement : on ne crée pas « un paiement » mais un paiement Wave, Orange Money ou en espèces. Elle peut contenir :

- des attributs et des méthodes concrètes, partagés par toutes les sous-classes ;
- des **méthodes abstraites**, déclarées sans corps (`abstract long frais();`), que chaque sous-classe concrète **doit** implémenter.

On ne peut pas écrire `new Paiement(...)` si `Paiement` est abstraite. Une classe abstraite sert souvent à mettre en œuvre le patron de conception **Méthode patron** (*Template Method*) : l'algorithme général est fixé dans la classe mère, et les étapes variables sont déléguées aux sous-classes.

7.5 Les interfaces

Une **interface** définit un **contrat** : un ensemble de méthodes qu'une classe s'engage à fournir, sans imposer comment. Une classe peut **implémenter plusieurs interfaces** (`implements A, B`), ce qui compense l'absence d'héritage multiple de classes.

Java

```java
public interface Notificateur {
    void envoyer(String destinataire, String message);      // abstraite (implicitement public)

    default void envoyerConfirmation(String destinataire, String numeroCommande) {  // Java 8
        envoyer(destinataire, "TerangaShop : votre commande " + numeroCommande
                + " est confirmée.");
    }

    static String masquer(String destinataire) {             // Java 8
        return destinataire.substring(0, 2) + "****";
    }
}
```

Évolution des interfaces (Java 8 et 9)

Depuis Java 8, une interface peut contenir des méthodes **`default`** (avec un corps, héritées par les classes qui l'implémentent) et des méthodes **`static`**. Depuis Java 9, elle peut aussi contenir des méthodes **`private`** pour factoriser le code des méthodes `default`. Une interface ne peut toujours pas avoir d'attributs d'instance : ses seuls attributs sont des constantes (`public static final` implicites).

Classe abstraite ou interface ?

Critère	Classe abstraite	Interface
Relation exprimée	« est un » (famille d'objets)	« sait faire » (capacité, contrat)
Héritage multiple	non : une seule super-classe	oui : plusieurs interfaces
État (attributs d'instance)	oui	non (seulement des constantes)
Constructeur	oui	non
Méthodes avec corps	oui	oui (`default`, `static`, `private`)
Usage typique en backend	base commune d'une hiérarchie d'entités	contrats entre couches : `ProduitRepository`, `PaiementService`, `Notificateur`

Côté backend

En architecture backend, on programme **contre des interfaces**. Le service de commande dépend de l'interface `Notificateur`, pas de la classe `NotificateurSms`. On peut ainsi changer de fournisseur SMS, envoyer par WhatsApp, ou utiliser une fausse implémentation dans les tests, sans modifier le service. C'est le principe d'**inversion des dépendances**, sur lequel repose l'injection de dépendances de Spring (chapitre 18).

7.6 Fil rouge TerangaShop — les modes de paiement

Mettons en œuvre héritage, classe abstraite, polymorphisme et interface. Les frais indiqués sont fictifs.

Paiement.java

```
public abstract class Paiement {
    private static int compteur = 0;

    private final String reference;
    protected final long montant;          // protected : accessible aux sous-classes
    private boolean effectue;

    protected Paiement(long montant) {     // protected : seules les sous-classes l'appellent
        if (montant <= 0) throw new IllegalArgumentException("Montant invalide : " + montant);
        this.montant = montant;
        this.reference = "PAY-" + (++compteur);
    }

    // Méthodes abstraites : chaque mode de paiement DOIT les définir
    public abstract String libelleMode();
    protected abstract long frais();

    // Méthode patron : l'algorithme est fixé ici, final pour qu'il ne soit pas modifié
    public final String traiter() {
        if (effectue) throw new IllegalStateException("Paiement déjà effectué : " + reference);
        verifier();                        // étape redéfinissable
        effectue = true;
        return "%s | %-13s | montant %6d | frais %4d | débité %6d"
                .formatted(reference, libelleMode(), montant, frais(), montant + frais());
```

```
    }

    // Étape optionnelle : par défaut, rien à vérifier
    protected void verifier() { }

    public long getMontant() { return montant; }
    public boolean isEffectue() { return effectue; }
    public String getReference() { return reference; }
}
```

PaiementMobile.java

```
/** Classe intermédiaire, elle aussi abstraite : tous les paiements par téléphone. */
public abstract class PaiementMobile extends Paiement {
    protected final String telephone;

    protected PaiementMobile(long montant, String telephone) {
        String t = telephone.replaceAll("\\s", "");       // prologue avant super(...) : Java 25
        if (!t.matches("7[05678]\\d{7}")) {
            throw new IllegalArgumentException("Numéro invalide : " + telephone);
        }
        super(montant);
        this.telephone = t;
    }

    @Override
    protected void verifier() {
        System.out.println("  -> code de confirmation envoyé au " + telephone);
    }
}
```

PaiementWave.java

```
public class PaiementWave extends PaiementMobile {
    public PaiementWave(long montant, String telephone) {
        super(montant, telephone);
    }
    @Override public String libelleMode() { return "Wave"; }
    @Override protected long frais() { return montant / 100; }               // 1 %
}
```

PaiementOrangeMoney.java

```
public class PaiementOrangeMoney extends PaiementMobile {
    public PaiementOrangeMoney(long montant, String telephone) {
        super(montant, telephone);
    }
    @Override public String libelleMode() { return "Orange Money"; }
    // 1,5 %, min 100
    @Override protected long frais() { return Math.max(100, montant * 15 / 1000); }
}
```

PaiementEspeces.java

```java
public class PaiementEspeces extends Paiement {
    private final long montantRecu;

    public PaiementEspeces(long montant, long montantRecu) {
        super(montant);
        this.montantRecu = montantRecu;
    }
    @Override public String libelleMode() { return "Espèces"; }
    @Override protected long frais() { return 0; }

    @Override
    protected void verifier() {
        super.verifier();                              // appel de la version parente
        if (montantRecu < montant) {
            throw new IllegalStateException("Montant reçu insuffisant : " + montantRecu);
        }
        System.out.println("  -> monnaie à rendre : " + (montantRecu - montant) + " FCFA");
    }
}
```

Notificateur.java

```java
public interface Notificateur {
    void envoyer(String destinataire, String message);

    default void confirmerPaiement(String destinataire, Paiement p) {
        envoyer(destinataire, "Paiement " + p.getReference() + " de "
                + p.getMontant() + " FCFA reçu. Merci !");
    }
}
```

NotificateurSms.java

```java
public class NotificateurSms implements Notificateur {
    @Override
    public void envoyer(String destinataire, String message) {
        System.out.println("  [SMS -> " + destinataire + "] " + message);
    }
}
```

DemoPaiements.java

```java
import java.util.List;

public class DemoPaiements {
    public static void main(String[] args) {
        // polymorphisme : type commun Paiement
        List<Paiement> paiements = List.of(
            new PaiementWave(18_500, "77 123 45 67"),
            new PaiementOrangeMoney(5_000, "78 000 11 22"),
            new PaiementEspeces(12_000, 15_000),
            new PaiementEspeces(9_000, 5_000)
        );

        Notificateur sms = new NotificateurSms();
        long totalEncaisse = 0;
        for (Paiement p : paiements) {
            try {
                System.out.println(p.traiter());          // liaison dynamique
                totalEncaisse += p.getMontant();
                if (p instanceof PaiementMobile pm) {     // test de type + transtypage
                    sms.confirmerPaiement(pm.telephone, p);
                }
            } catch (IllegalStateException e) {
                System.out.println("ÉCHEC " + p.getReference() + " : " + e.getMessage());
            }
        }
        System.out.println("Total encaissé : " + totalEncaisse + " FCFA");
```

```
        // Classe anonyme : une implémentation "jetable" de l'interface
        Notificateur console = new Notificateur() {
            @Override
            public void envoyer(String destinataire, String message) {
                System.out.println("  [LOG] " + destinataire + " : " + message);
            }
        };
        console.envoyer("admin", "Fin du traitement");

        // Lambda (chapitre 12) : Notificateur n'a qu'une méthode abstraite
        Notificateur whatsapp = (dest, msg) -> System.out.println("  [WhatsApp -> "
                + dest + "] " + msg);
        whatsapp.envoyer("771234567", "Votre colis arrive !");
    }
}
```

Sortie console

```
  -> code de confirmation envoyé au 771234567
PAY-1 | Wave          | montant  18500 | frais  185 | débité  18685
  [SMS -> 771234567] Paiement PAY-1 de 18500 FCFA reçu. Merci !
  -> code de confirmation envoyé au 780001122
PAY-2 | Orange Money  | montant   5000 | frais  100 | débité   5100
  [SMS -> 780001122] Paiement PAY-2 de 5000 FCFA reçu. Merci !
  -> monnaie à rendre : 3000 FCFA
PAY-3 | Espèces       | montant  12000 | frais    0 | débité  12000
ÉCHEC PAY-4 : Montant reçu insuffisant : 5000
Total encaissé : 35500 FCFA
  [LOG] admin : Fin du traitement
  [WhatsApp -> 771234567] Votre colis arrive !
```

Observez :

- la boucle `for` ne connaît que le type `Paiement`, mais chaque objet exécute **ses propres** méthodes `frais()`, `libelleMode()` et `verifier()` ;
- `traiter()` est `final` : les sous-classes personnalisent les étapes, jamais l'algorithme ;
- `PaiementMobile` valide le numéro **avant** d'appeler `super(montant)`, grâce aux corps de constructeur flexibles de Java 25 ;
- `p instanceof PaiementMobile pm` teste le type réel **et** déclare une variable `pm` du bon type (*pattern matching*, détaillé au chapitre 8).

> **Piège à éviter — le transtypage descendant**
>
> Pour appeler une méthode propre à une sous-classe à partir d'une référence de la super-classe, il faut **transtyper** (*downcasting*). Un transtypage vers un type incorrect lève une `ClassCastException`. Testez toujours avec `instanceof` avant, ou mieux : utilisez le *pattern matching*. Si votre code est rempli de `instanceof`, c'est souvent le signe qu'une méthode polymorphe manque.

7.7 Classes internes et imbriquées

Une classe peut être déclarée à l'intérieur d'une autre classe :

Sorte	Déclaration	Particularité	Usage typique
Classe imbriquée statique	`static class Builder` dans `Produit`	ne dépend d'aucune instance de la classe englobante	*Builder*, petites classes utilitaires liées

Sorte	Déclaration	Particularité	Usage typique
Classe interne (membre)	`class Iterateur` dans `Liste`	chaque instance est liée à une instance englobante et accède à ses attributs	itérateurs, vues
Classe locale	déclarée dans une méthode	visible uniquement dans cette méthode	rare
Classe anonyme	`new Interface() { ... }`	déclarée et instanciée en une expression	remplacée le plus souvent par une lambda

Le patron *Builder* avec une classe imbriquée statique

Quand un objet a beaucoup de paramètres, dont plusieurs optionnels, un constructeur à rallonge devient illisible (`new Livraison("Awa", "Médina", null, true, 2, false, …)`). Le patron **Builder** résout ce problème :

Livraison.java

```
public class Livraison {
    private final String destinataire;
    private final String quartier;
    private final String instructions;   // optionnel
    private final boolean express;       // optionnel
    private final int creneau;           // optionnel : 1 = matin, 2 = après-midi

    private Livraison(Builder b) {       // constructeur privé
        this.destinataire = b.destinataire;
        this.quartier = b.quartier;
        this.instructions = b.instructions;
        this.express = b.express;
        this.creneau = b.creneau;
    }

    public static Builder pour(String destinataire, String quartier) {
        return new Builder(destinataire, quartier);
    }

    public static class Builder {
        private final String destinataire;
        private final String quartier;
        private String instructions = "";
        private boolean express = false;
        private int creneau = 1;

        private Builder(String destinataire, String quartier) {
            this.destinataire = destinataire;
            this.quartier = quartier;
        }
        public Builder instructions(String i) { this.instructions = i; return this; }
        public Builder express()              { this.express = true; return this; }
        public Builder apresMidi()            { this.creneau = 2; return this; }
        public Livraison construire()         { return new Livraison(this); }
    }

    @Override
    public String toString() {
        return destinataire + " @ " + quartier + (express ? " [EXPRESS]" : "")
               + (creneau == 2 ? " après-midi" : " matin")
               + (instructions.isEmpty() ? "" : " - " + instructions);
    }

    public static void main(String[] args) {
        Livraison l1 = Livraison.pour("Awa Diop", "Médina").construire();
        Livraison l2 = Livraison.pour("Ibrahima Fall", "Ouakam")
                                .express()
                                .apresMidi()
                                .instructions("Appeler en arrivant devant la mosquée")
```

```
                                .construire();
        System.out.println(l1);
        System.out.println(l2);
    }
}
```

Sortie console

```
Awa Diop @ Médina matin
Ibrahima Fall @ Ouakam [EXPRESS] après-midi - Appeler en arrivant devant la mosquée
```

7.8 Composition plutôt qu'héritage

L'héritage crée un couplage fort : toute modification de la classe mère peut casser les classes filles, et une classe ne peut hériter que d'une seule autre. La **composition** consiste à **contenir** un objet d'une autre classe et à lui **déléguer** une partie du travail (relation « **a un** »).

Exemple classique d'héritage abusif : faire hériter `Panier` d'`ArrayList<LigneCommande>`. Le panier « hérite » alors de dizaines de méthodes (`clear`, `set`, `remove`, `add(int, …)`…) qui permettent de contourner toutes les règles métier. La bonne solution est celle de notre classe `Commande` : elle **contient** une liste privée et n'expose que les opérations autorisées.

> **À retenir**
>
> Utilisez l'héritage pour une vraie relation « est un », stable, entre classes que vous maîtrisez. Dans les autres cas, préférez la composition et les interfaces. Ce principe (« *favor composition over inheritance* ») est l'une des règles d'or de la conception objet.

7.9 Les classes scellées

> **Nouveauté Java 17 — Classes et interfaces scellées (JEP 409)**
>
> Une classe ou interface **scellée** (`sealed`) déclare explicitement la liste fermée de ses sous-types autorisés avec `permits`. Chaque sous-type doit être `final` (plus de sous-classes), `sealed` (à son tour restreint) ou `non-sealed` (réouvert à l'héritage). Le compilateur connaît alors **tous** les cas possibles.

Pourquoi restreindre l'héritage ? Parce que certains domaines sont **fermés** par nature : un statut de commande, un résultat de paiement (accepté, refusé, en attente), une forme de livraison. Avec une hiérarchie scellée, le compilateur peut vérifier qu'un `switch` traite **tous** les cas (chapitre 8), et personne ne peut ajouter un sous-type imprévu.

ModeLivraison.java

```
public sealed interface ModeLivraison permits Domicile, PointRelais, RetraitBoutique {
    static void main(String[] args) {
        ModeLivraison[] modes = {
            new Domicile("Parcelles Assainies"),
            new PointRelais("Station Total Liberté 6"),
            new RetraitBoutique()
        };
        for (ModeLivraison m : modes) {
            System.out.println(m.description() + " -> " + m.cout() + " FCFA");
        }
    }
    long cout();
    String description();
}
```

```java
final class Domicile implements ModeLivraison {
    private final String quartier;
    Domicile(String quartier) { this.quartier = quartier; }
    public long cout() { return 1_500; }
    public String description() { return "Livraison à domicile (" + quartier + ")"; }
}

final class PointRelais implements ModeLivraison {
    private final String nom;
    PointRelais(String nom) { this.nom = nom; }
    public long cout() { return 500; }
    public String description() { return "Point relais " + nom; }
}

non-sealed class RetraitBoutique implements ModeLivraison {   // réouvert à l'héritage
    public long cout() { return 0; }
    public String description() { return "Retrait à la boutique de Diamniadio"; }
}
```

Sortie console

```
Livraison à domicile (Parcelles Assainies) -> 1500 FCFA
Point relais Station Total Liberté 6 -> 500 FCFA
Retrait à la boutique de Diamniadio -> 0 FCFA
```

Si une autre classe tente d'écrire `class Drone implements ModeLivraison`, le compilateur refuse : *class is not allowed to extend sealed class: ModeLivraison*.

Exercices du chapitre 7

Exercice 7.1 ★ — Les employés de TerangaShop

Créez une classe abstraite `Employe` (matricule, nom, salaire de base) avec une méthode abstraite `long salaireMensuel()`. Créez trois sous-classes : `Vendeur` (base + 3 % de son chiffre d'affaires du mois), `Livreur` (base + 500 FCFA par livraison effectuée) et `Gerant` (base + prime fixe de 100 000 FCFA). Dans un `main`, rangez plusieurs employés dans un tableau `Employe[]` et calculez la masse salariale totale. Redéfinissez `toString` en réutilisant `super.toString()`.

Exercice 7.2 ★ — Formes géométriques

Écrivez une classe abstraite `Forme` avec les méthodes abstraites `aire()` et `perimetre()`, et une méthode concrète `decrire()` qui les utilise. Implémentez `Cercle`, `Rectangle` et `Triangle` (vérifiez l'inégalité triangulaire dans le constructeur). Question de réflexion : faut-il faire hériter `Carre` de `Rectangle` si `Rectangle` possède des méthodes `setLargeur` et `setHauteur` ? (Recherchez le « principe de substitution de Liskov ».)

Exercice 7.3 ★★ — Interfaces multiples et méthodes par défaut

Créez deux interfaces `Taxable` (méthode `long montantHt()`, méthode par défaut `long tva()` à 18 %) et `Remisable` (méthode `int tauxRemise()`, méthode par défaut `long remise(long montant)`). Faites implémenter les deux à une classe `ArticleSolde`. Ajoutez ensuite une méthode `default String description()` dans **les deux** interfaces : que se passe-t-il à la compilation ? Résolvez le conflit en appelant `Taxable.super.description()`.

Exercice 7.4 ★★ — Prédire l'affichage

Sans l'exécuter, indiquez ce qu'affiche ce programme. Expliquez chaque ligne (attributs non polymorphes, méthode appelée dans un constructeur, méthode `static`).

Java

```java
class A {
    String nom = "A";
    A() { afficher(); }
    void afficher() { System.out.println("A.afficher " + nom); }
    static void statique() { System.out.println("A.statique"); }
}
class B extends A {
    String nom = "B";
    B() { super(); afficher(); }
    @Override void afficher() { System.out.println("B.afficher " + nom); }
    static void statique() { System.out.println("B.statique"); }
}
public class Quiz {
    public static void main(String[] args) {
        A a = new B();
        System.out.println(a.nom);
        a.statique();
        ((B) a).statique();
    }
}
```

Exercice 7.5 ★★ — Builder pour Produit

Réécrivez la création de `Produit` (chapitre 6) avec un *Builder* : la référence et le libellé sont obligatoires, le prix, le stock, la catégorie et la description sont optionnels (valeurs par défaut raisonnables). La validation complète doit avoir lieu dans `construire()`.

Exercice 7.6 ★★★ — Notifications multicanal

Définissez l'interface `Notificateur` et trois implémentations : `NotificateurSms` (message tronqué à 160 caractères), `NotificateurEmail` (ajoute un objet et une signature) et `NotificateurWhatsApp`. Écrivez une classe `NotificateurMultiple` qui implémente elle aussi `Notificateur` et **contient** une liste de notificateurs auxquels elle délègue (patron *Composite*). Enfin, écrivez un `ServiceCommande` qui reçoit un `Notificateur` dans son constructeur et l'utilise lors de la validation d'une commande. Montrez qu'on peut changer de canal sans modifier `ServiceCommande`.

Chapitre 8 — Records, enums et pattern matching

Objectifs du chapitre

- Déclarer des **records** pour représenter des données immuables, avec validation.
- Utiliser des **enums** riches (attributs, méthodes, comportement par constante).
- Maîtriser le **pattern matching** : `instanceof`, `switch` sur les types, *record patterns*, gardes `when`, variables anonymes `_`.
- Combiner records, interfaces scellées et `switch` : la programmation orientée données.

8.1 Les records

Nouveauté Java 16 — Records (JEP 395)

Un **record** est une classe spécialisée dans le transport de données **immuables**. En une ligne, le compilateur génère les attributs `private final`, le constructeur, les accesseurs, `equals`, `hashCode` et `toString`.

Java

```java
public record Coordonnees(double latitude, double longitude) { }
```

Cette unique ligne équivaut à une classe d'une quarantaine de lignes :

Élément généré	Pour `Coordonnees(double latitude, double longitude)`
Attributs	`private final double latitude;` et `private final double longitude;`
Constructeur canonique	`Coordonnees(double latitude, double longitude)`
Accesseurs	`latitude()` et `longitude()` (sans préfixe `get`)
`equals` / `hashCode`	fondés sur **toutes** les composantes
`toString`	`Coordonnees[latitude=14.69, longitude=-17.44]`

Un record est implicitement `final` (il ne peut pas être étendu) et ne peut pas hériter d'une classe (il hérite déjà de `java.lang.Record`), mais il **peut implémenter des interfaces**. On peut lui ajouter des méthodes, des méthodes et attributs `static`, et des constructeurs supplémentaires ; en revanche, on ne peut pas lui ajouter d'attributs d'instance.

Le constructeur compact : valider et normaliser

Le **constructeur compact** s'écrit sans parenthèses ni paramètres. Il s'exécute avant l'affectation automatique des attributs et permet de **valider** et de **normaliser** les valeurs.

DemoRecords.java

```java
import java.util.List;

public class DemoRecords {

    record Coordonnees(double latitude, double longitude) {
        Coordonnees {                                   // constructeur compact
            if (latitude < -90 || latitude > 90) {
                throw new IllegalArgumentException("Latitude invalide : " + latitude);
            }
            if (longitude < -180 || longitude > 180) {
```

```
                throw new IllegalArgumentException("Longitude invalide : " + longitude);
            }
        }

        /** Distance en km (formule de haversine). */
        double distanceKm(Coordonnees autre) {
            double r = 6371;
            double dLat = Math.toRadians(autre.latitude - latitude);
            double dLon = Math.toRadians(autre.longitude - longitude);
            double a = Math.pow(Math.sin(dLat / 2), 2)
                     + Math.cos(Math.toRadians(latitude)) * Math.cos(Math.toRadians(autre.latitude))
                     * Math.pow(Math.sin(dLon / 2), 2);
            return 2 * r * Math.asin(Math.sqrt(a));
        }
    }

    record Adresse(String quartier, String ville, String indications) {
        Adresse {
            if (quartier == null || quartier.isBlank())
                    throw new IllegalArgumentException("Quartier requis");
            quartier = quartier.strip();              // normalisation
            ville = ville == null ? "Dakar" : ville.strip();
            indications = indications == null ? "" : indications;
        }
        Adresse(String quartier) {                    // constructeur supplémentaire
            this(quartier, "Dakar", null);
        }
        Adresse avecIndications(String nouvelles) {   // "wither" : renvoie une copie modifiée
            return new Adresse(quartier, ville, nouvelles);
        }
    }

    record Panier(String client, List<String> articles) {
        Panier {
            articles = List.copyOf(articles);         // copie défensive : vraiment immuable
        }
    }

    public static void main(String[] args) {
        var plateau = new Coordonnees(14.6700, -17.4300);
        var diamniadio = new Coordonnees(14.7200, -17.1800);
        System.out.println(plateau);
        System.out.printf("Plateau -> Diamniadio : %.1f km%n", plateau.distanceKm(diamniadio));

        var a1 = new Adresse("  Médina ");
        var a2 = a1.avecIndications("Rue 11, en face de la boulangerie");
        System.out.println(a1);
        System.out.println(a2.quartier() + " / " + a2.indications());
        System.out.println(a1.equals(new Adresse("Médina", "Dakar", "")));

        var liste = new java.util.ArrayList<>(List.of("Bissap"));
        var panier = new Panier("Awa", liste);
        liste.add("Thiakry");                         // n'affecte pas le record
        System.out.println(panier);

        try {
            new Coordonnees(120, 0);
        } catch (IllegalArgumentException e) {
            System.out.println("Erreur : " + e.getMessage());
        }
    }
}
```

Sortie console

```
Coordonnees[latitude=14.67, longitude=-17.43]
Plateau -> Diamniadio : 27.5 km
Adresse[quartier=Médina, ville=Dakar, indications=]
Médina / Rue 11, en face de la boulangerie
true
Panier[client=Awa, articles=[Bissap]]
Erreur : Latitude invalide : 120.0
```

Côté backend

Les records sont l'outil idéal pour les **DTO** (*Data Transfer Objects*) : le corps JSON d'une requête (`CreerCommandeRequete`), la réponse d'une API (`ProduitReponse`), le résultat d'une requête SQL, un événement, une clé composite de `Map`. Spring Boot et Jackson savent convertir directement un record en JSON et inversement. En revanche, une **entité JPA** mutable ne peut pas être un record.

Piège à éviter — l'immuabilité « superficielle »

Un record est immuable au sens où ses **références** ne changent pas. Si une composante est une liste modifiable, la liste elle-même peut changer. Faites une copie défensive avec `List.copyOf(...)` dans le constructeur compact, comme dans `Panier`.

8.2 Les énumérations (enum)

Une **énumération** définit un type qui possède un ensemble **fini et connu** de valeurs : les modes de paiement, les statuts d'une commande, les jours de la semaine. Chaque valeur est un objet unique (un *singleton*), ce qui permet de les comparer avec `==`.

Java

```java
public enum StatutSimple { EN_ATTENTE, PAYEE, EXPEDIEE, LIVREE, ANNULEE }

StatutSimple s = StatutSimple.PAYEE;
s.name();                          // "PAYEE"
s.ordinal();                       // 1 (position, à ne pas utiliser pour la logique métier)
StatutSimple.valueOf("LIVREE");    // StatutSimple.LIVREE (IllegalArgumentException si inconnu)
StatutSimple.values();             // tableau de toutes les valeurs, dans l'ordre
```

Des enums riches

En Java, un enum est une vraie classe : il peut avoir des **attributs**, un **constructeur** (toujours privé), des **méthodes**, et même un comportement **différent pour chaque constante**.

ModePaiement.java

```java
public enum ModePaiement {
    WAVE("Wave", 1.0, true),
    ORANGE_MONEY("Orange Money", 1.5, true),
    ESPECES("Espèces à la livraison", 0, false) {
        @Override
        public String instructions() {                // comportement propre à cette constante
            return "Préparez l'appoint si possible.";
        }
    };

    private final String libelle;
    private final double tauxFrais;                   // en pourcentage
    private final boolean enLigne;

    ModePaiement(String libelle, double tauxFrais, boolean enLigne) {   // implicitement privé
```

```
        this.libelle = libelle;
        this.tauxFrais = tauxFrais;
        this.enLigne = enLigne;
    }

    public long frais(long montant) { return Math.round(montant * tauxFrais / 100); }
    public String libelle() { return libelle; }
    public boolean enLigne() { return enLigne; }
    public String instructions() { return "Validez le paiement sur votre téléphone."; }

    /** Conversion tolérante depuis une saisie utilisateur ou un JSON. */
    public static ModePaiement depuis(String texte) {
        for (ModePaiement m : values()) {
            if (m.name().equalsIgnoreCase(texte.strip().replace(' ', '_'))) return m;
        }
        throw new IllegalArgumentException("Mode de paiement inconnu : " + texte);
    }

    public static void main(String[] args) {
        for (ModePaiement m : ModePaiement.values()) {
            System.out.printf("%-12s %-23s frais sur 20000 : %3d  %s%n",
                    m, m.libelle(), m.frais(20_000), m.instructions());
        }
        ModePaiement choisi = ModePaiement.depuis("orange money");
        String message = switch (choisi) {            // switch exhaustif : pas de default
            case WAVE, ORANGE_MONEY -> "Paiement mobile (" + choisi.libelle() + ")";
            case ESPECES -> "Paiement à la livraison";
        };
        System.out.println(message);
        System.out.println(choisi == ModePaiement.ORANGE_MONEY);
    }
}
```

Sortie console

```
WAVE         Wave                   frais sur 20000 : 200  Validez le paiement sur votre téléphone.
ORANGE_MONEY Orange Money           frais sur 20000 : 300  Validez le paiement sur votre téléphone.
ESPECES      Espèces à la livraison  frais sur 20000 :   0  Préparez l'appoint si possible.
Paiement mobile (Orange Money)
true
```

Un enum comme machine à états

Le cycle de vie d'une commande est un bel exemple : toutes les transitions ne sont pas permises (on ne livre pas une commande annulée).

StatutCommande.java

```
import java.util.Set;

public enum StatutCommande {
    EN_ATTENTE, PAYEE, EXPEDIEE, LIVREE, ANNULEE;

    public Set<StatutCommande> suivantsPossibles() {
        return switch (this) {
            case EN_ATTENTE -> Set.of(PAYEE, ANNULEE);
            case PAYEE -> Set.of(EXPEDIEE, ANNULEE);
            case EXPEDIEE -> Set.of(LIVREE);
            case LIVREE, ANNULEE -> Set.of();          // états finaux
        };
    }

    public StatutCommande passerA(StatutCommande suivant) {
        if (!suivantsPossibles().contains(suivant)) {
            throw new IllegalStateException("Transition interdite : " + this
                    + " -> " + suivant);
        }
        return suivant;
```

```
    }

    public static void main(String[] args) {
        StatutCommande s = EN_ATTENTE;
        s = s.passerA(PAYEE);
        s = s.passerA(EXPEDIEE);
        System.out.println("Statut : " + s);
        try {
            s.passerA(ANNULEE);
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Sortie console

```
Statut : EXPEDIEE
Transition interdite : EXPEDIEE -> ANNULEE
```

Astuce

Pour stocker un enum en base de données, enregistrez son **nom** (`name()`, colonne `VARCHAR`) et non son **ordinal** : si quelqu'un insère une nouvelle constante au milieu de l'enum, les ordinaux changent et toutes les données existantes deviennent fausses. Pour des ensembles ou des tables indexés par un enum, utilisez `EnumSet` et `EnumMap`, très performants.

8.3 Le pattern matching

Le *pattern matching* (« filtrage par motif ») permet de **tester** la forme d'une valeur et d'en **extraire** des éléments en une seule opération.

`instanceof` avec motif

Nouveauté Java 16 — Pattern matching pour instanceof (JEP 394)

`if (obj instanceof Produit p)` teste le type **et** déclare une variable `p` déjà transtypée, utilisable là où le test est vrai.

Java

```
// Avant Java 16
if (obj instanceof Produit) {
    Produit p = (Produit) obj;
    System.out.println(p.getLibelle());
}
// Depuis Java 16
if (obj instanceof Produit p && p.getPrix() > 10_000) {
    System.out.println(p.getLibelle());
}
```

`switch` sur les types, gardes et `null`

Nouveauté Java 21 — Pattern matching pour switch (JEP 441) et record patterns (JEP 440)

Un `switch` peut maintenant tester le **type** de son sélecteur (`case Integer i ->`), ajouter une **garde** (`case Integer i when i > 100 ->`), traiter explicitement `case null`, et **déconstruire** un record (`case Point(var x, var y) ->`). Sur une hiérarchie scellée, le compilateur vérifie l'exhaustivité.

DescriptionValeur.java

```java
public class DescriptionValeur {
    record Produit(String libelle, long prix) { }

    static String decrire(Object o) {
        return switch (o) {
            case null -> "valeur absente";
            case Integer i when i < 0 -> "entier négatif " + i;
            case Integer i -> "entier " + i;
            case Long l -> "montant de " + l + " FCFA";
            case String s when s.isBlank() -> "chaîne vide";
            case String s -> "texte de " + s.length() + " caractères";
            case Produit(String libelle, long prix) when prix > 10_000 ->
                    "produit haut de gamme : " + libelle;
            case Produit(var libelle, var prix) -> "produit " + libelle + " à " + prix;
            case int[] t -> "tableau de " + t.length + " entiers";
            default -> "objet de type " + o.getClass().getSimpleName();
        };
    }

    public static void main(String[] args) {
        Object[] valeurs = {42, -7, 15_000L, "  ", "Teranga", new Produit("Sac cuir", 25_000),
                            new Produit("Bissap", 1_500), new int[]{1, 2, 3}, 3.14, null};
        for (Object v : valeurs) {
            System.out.println(decrire(v));
        }
    }
}
```

Sortie console

```
entier 42
entier négatif -7
montant de 15000 FCFA
chaîne vide
texte de 7 caractères
produit haut de gamme : Sac cuir
produit Bissap à 1500
tableau de 3 entiers
objet de type Double
valeur absente
```

L'**ordre des `case`** compte : Java choisit le premier motif qui correspond. Un motif plus général placé avant un motif plus spécifique le rendrait inaccessible ; le compilateur signale alors une erreur (*this case label is dominated by a preceding case label*).

Variables anonymes _

Nouveauté Java 22 — Variables et motifs anonymes (JEP 456)

Le caractère _ désigne une variable ou une composante dont on n'a pas besoin : `case Produit(var libelle, _) ->`, `catch (NumberFormatException _)`, `for (var _ : liste)`. Le code exprime clairement l'intention et le compilateur n'avertit plus d'une variable inutilisée.

En preview (Java 23 à 27) — Types primitifs dans les motifs (JEP 532)

Cette fonctionnalité, en cinquième *preview* dans Java 27, autorise les types primitifs dans les motifs : `case int i when i > 100`, `if (x instanceof byte b)` (vrai si la valeur tient dans un `byte` sans perte). À tester avec `--enable-preview`, mais pas encore pour la production.

8.4 La programmation orientée données

En combinant **interfaces scellées**, **records** et **switch avec motifs**, on obtient un style de programmation très expressif pour modéliser des données dont la forme est connue à l'avance. On parle de *programmation orientée données* (*data-oriented programming*).

Exemple : le résultat d'une tentative de paiement auprès d'un opérateur peut être **accepté**, **refusé** ou **en attente**. Chaque cas transporte des informations différentes.

ResultatsPaiement.java

```java
import java.util.List;

public class ResultatsPaiement {

    sealed interface Resultat permits Accepte, Refuse, EnAttente { }
    record Accepte(String transactionId, long montant) implements Resultat { }
    record Refuse(String code, String motif) implements Resultat { }
    record EnAttente(String transactionId, int secondesAvantRelance) implements Resultat { }

    static String traiter(Resultat r) {
        return switch (r) {                                    // exhaustif : pas de default !
            case Accepte(var id, var montant) -> "OK " + id + " : " + montant
                    + " FCFA encaissés";
            case Refuse(var code, var motif) when code.equals("SOLDE") ->
                    "Refusé : solde insuffisant, proposez un autre mode";
            case Refuse(var code, var motif) -> "Refusé (" + code + ") : " + motif;
            case EnAttente(var id, var delai) -> "En attente, nouvelle vérification de " + id
                    + " dans " + delai + " s";
        };
    }

    public static void main(String[] args) {
        List<Resultat> resultats = List.of(
            new Accepte("WAV-88921", 18_500),
            new Refuse("SOLDE", "Balance too low"),
            new Refuse("TIMEOUT", "L'opérateur ne répond pas"),
            new EnAttente("OM-55120", 30)
        );
        resultats.forEach(r -> System.out.println(traiter(r)));

        long totalAccepte = 0;
        for (Resultat r : resultats) {
            if (r instanceof Accepte(_, long montant)) {       // record pattern + motif anonyme
                totalAccepte += montant;
            }
        }
        System.out.println("Total accepté : " + totalAccepte);
    }
}
```

Sortie console

```
OK WAV-88921 : 18500 FCFA encaissés
Refusé : solde insuffisant, proposez un autre mode
Refusé (TIMEOUT) : L'opérateur ne répond pas
En attente, nouvelle vérification de OM-55120 dans 30 s
Total accepté : 18500
```

Le grand avantage est la **sécurité à l'évolution** : si l'on ajoute un quatrième cas `record Rembourse(...) implements Resultat` à la liste `permits`, **tous** les `switch` qui ne le traitent pas cessent de compiler. Le compilateur vous montre exactement ce qu'il faut mettre à jour, là où une hiérarchie classique avec `default` aurait silencieusement ignoré le nouveau cas.

> **Orienté objet ou orienté données ?**
> Les deux approches sont complémentaires. Le **polymorphisme** (chapitre 7) est préférable quand on ajoute souvent de **nouveaux types** et que chaque type porte son propre comportement. Le **pattern matching sur une hiérarchie scellée** est préférable quand l'ensemble des types est **stable** et qu'on ajoute souvent de **nouvelles opérations** sur ces types (affichage, conversion JSON, statistiques…), sans vouloir les disperser dans chaque classe.

8.5 Fil rouge TerangaShop — le journal des événements

Un backend moderne enregistre souvent ce qui se passe sous forme d'**événements** immuables. Modélisons-les avec une hiérarchie scellée de records, puis calculons un tableau de bord.

JournalEvenements.java

```
import java.util.List;

public class JournalEvenements {

    sealed interface Evenement permits CommandeCreee, PaiementRecu,
            CommandeLivree, CommandeAnnulee {
        String numeroCommande();
    }
    record CommandeCreee(String numeroCommande, String client,
            long montant) implements Evenement { }
    record PaiementRecu(String numeroCommande, ModePaiement mode,
            long montant) implements Evenement { }
    record CommandeLivree(String numeroCommande, String quartier) implements Evenement { }
    record CommandeAnnulee(String numeroCommande, String motif) implements Evenement { }

    enum ModePaiement { WAVE, ORANGE_MONEY, ESPECES }

    static String journaliser(Evenement e) {
        return switch (e) {
            case CommandeCreee c -> "[CRÉÉE]    %s pour %s (%d FCFA)"
                    .formatted(c.numeroCommande(), c.client(), c.montant());
            case PaiementRecu(var num, var mode, var montant) ->
                    "[PAYÉE]    %s via %s".formatted(num, mode);
            case CommandeLivree(var num, var quartier) ->
                    "[LIVRÉE]   %s à %s".formatted(num, quartier);
            case CommandeAnnulee(var num, var motif) ->
                    "[ANNULÉE]  %s : %s".formatted(num, motif);
        };
    }

    public static void main(String[] args) {
        List<Evenement> journal = List.of(
            new CommandeCreee("TS-001", "Awa Diop", 22_500),
            new CommandeCreee("TS-002", "Moussa Ndiaye", 51_500),
            new PaiementRecu("TS-001", ModePaiement.WAVE, 22_500),
            new CommandeCreee("TS-003", "Fatou Sow", 8_000),
            new CommandeAnnulee("TS-003", "Client injoignable"),
            new PaiementRecu("TS-002", ModePaiement.ESPECES, 51_500),
            new CommandeLivree("TS-001", "Médina"),
            new CommandeLivree("TS-002", "Rufisque")
        );
        journal.forEach(e -> System.out.println(journaliser(e)));

        long encaisse = 0, paiementsMobiles = 0;
        int annulations = 0;
        for (Evenement e : journal) {
            switch (e) {
                case PaiementRecu(_, var mode, var montant) -> {
                    encaisse += montant;
                    if (mode != ModePaiement.ESPECES) paiementsMobiles++;
```

```
                }
                case CommandeAnnulee _ -> annulations++;
                default -> { }                      // les autres événements sont ignorés ici
            }
        }
        System.out.println("Encaissé : " + encaisse + " FCFA, paiements mobiles : "
                           + paiementsMobiles + ", annulations : " + annulations);
    }
}
```

Sortie console

```
[CRÉÉE]    TS-001 pour Awa Diop (22500 FCFA)
[CRÉÉE]    TS-002 pour Moussa Ndiaye (51500 FCFA)
[PAYÉE]    TS-001 via WAVE
[CRÉÉE]    TS-003 pour Fatou Sow (8000 FCFA)
[ANNULÉE]  TS-003 : Client injoignable
[PAYÉE]    TS-002 via ESPECES
[LIVRÉE]   TS-001 à Médina
[LIVRÉE]   TS-002 à Rufisque
Encaissé : 74000 FCFA, paiements mobiles : 1, annulations : 1
```

Exercices du chapitre 8

Exercice 8.1 ★ — Records de base

1. Écrivez un record `Montant(long valeur, String devise)` dont le constructeur compact refuse une valeur négative et normalise la devise en majuscules (`"xof"` devient `"XOF"`). Ajoutez les méthodes `plus(Montant autre)` (refusée si les devises diffèrent) et `fois(int n)`.
2. Écrivez un record `Periode(int anneeDebut, int anneeFin)` qui refuse `anneeFin < anneeDebut` et offre `duree()` et `contient(int annee)`.
3. Vérifiez sur des exemples que `equals`, `hashCode` et `toString` sont bien générés.

Exercice 8.2 ★ — Enum Categorie

Créez un enum `Categorie` (`ALIMENTAIRE`, `TEXTILE`, `ARTISANAT`, `COSMETIQUE`) avec un libellé et un taux de TVA (alimentaire de base : 0 % ; autres : 18 %). Ajoutez une méthode `long prixTtc(long prixHt)`. Utilisez `EnumMap<Categorie, Long>` pour cumuler le chiffre d'affaires par catégorie d'une liste de ventes.

Exercice 8.3 ★★ — Refactoriser avec le pattern matching

Réécrivez la méthode suivante avec un `switch` à motifs, des gardes et des *record patterns*. Déclarez les types nécessaires sous forme d'une interface scellée et de records.

Java

```java
static double aire(Object forme) {
    if (forme instanceof Cercle) {
        Cercle c = (Cercle) forme;
        return Math.PI * c.rayon() * c.rayon();
    } else if (forme instanceof Rectangle) {
        Rectangle r = (Rectangle) forme;
        if (r.largeur() == r.hauteur()) System.out.println("(c'est un carré)");
        return r.largeur() * r.hauteur();
    } else if (forme instanceof Triangle) {
        Triangle t = (Triangle) forme;
        return t.base() * t.hauteur() / 2;
    } else {
        throw new IllegalArgumentException("Forme inconnue");
    }
}
```

Exercice 8.4 ★★ — Machine à états des livraisons

Créez un enum `EtapeLivraison` (`PREPARATION`, `EN_ROUTE`, `LIVREE`, `ECHEC`, `RETOUR_ENTREPOT`) avec ses transitions autorisées (un échec peut conduire à une nouvelle tentative `EN_ROUTE` ou au retour à l'entrepôt). Écrivez une classe `SuiviLivraison` qui mémorise l'historique des étapes et refuse les transitions interdites. Au-delà de 2 échecs, seul le retour à l'entrepôt est possible.

Exercice 8.5 ★★ — Évaluateur d'expressions

Modélisez des expressions arithmétiques avec une interface scellée `Expr` et les records `Nombre(double v)`, `Addition(Expr g, Expr d)`, `Multiplication(Expr g, Expr d)`, `Negation(Expr e)`. Écrivez, avec des `switch` à motifs : `evaluer(Expr e)`, `afficher(Expr e)` (avec parenthèses) et `simplifier(Expr e)` (règles : `x + 0 = x`, `x * 1 = x`, `x * 0 = 0`, `-(-x) = x`). Testez avec `(2 + 3) * -(-4)`.

Exercice 8.6 ★★★ — Moteur de promotions

TerangaShop propose plusieurs types de promotions : `PourcentageSurTotal(int taux, long minimumAchat)`, `RemiseFixe(long montant, long minimumAchat)`, `ArticleOffert(String reference, int quantiteMinimum)` (un article offert si on en achète au moins N) et `LivraisonOfferte(String quartier)`. Modélisez-les avec une interface scellée de records. Un `Panier` est un record contenant le quartier de livraison et une liste de lignes (`record Ligne(String reference, long prixUnitaire, int quantite)`). Écrivez une méthode `long economie(Promotion p, Panier panier)` utilisant un `switch` exhaustif, puis une méthode qui choisit la **meilleure** promotion applicable parmi une liste. Les frais de livraison sont de 1 500 FCFA.

Chapitre 9 — Exceptions et gestion des erreurs

Objectifs du chapitre

- Comprendre la hiérarchie des exceptions et la différence entre exceptions vérifiées et non vérifiées.
- Intercepter, lever, propager et déclarer des exceptions.
- Libérer les ressources correctement avec *try-with-resources*.
- Concevoir une hiérarchie d'exceptions métier pour un backend.
- Appliquer les bonnes pratiques de gestion des erreurs.

9.1 Qu'est-ce qu'une exception ?

Une **exception** est un événement anormal qui survient pendant l'exécution et interrompt le déroulement normal du programme : division entière par zéro, fichier introuvable, connexion à la base de données perdue, stock insuffisant… En Java, une exception est un **objet** qui décrit l'erreur. Lorsqu'elle est **levée** (*thrown*), l'exécution de la méthode s'arrête et l'exception **remonte** la pile des appels jusqu'à ce qu'un bloc `catch` l'intercepte. Si personne ne l'intercepte, le programme s'arrête et affiche la **trace de la pile** (*stack trace*).

SansGestion.java

```
public class SansGestion {
    static int prixMoyen(int total, int nbArticles) {
        return total / nbArticles;
    }
    public static void main(String[] args) {
        System.out.println("Début");
        System.out.println(prixMoyen(9_000, 0));
        System.out.println("Fin");       // jamais exécuté
    }
}
```

Sortie console

```
Début
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at SansGestion.prixMoyen(SansGestion.java:3)
    at SansGestion.main(SansGestion.java:7)
```

La trace se lit **de haut en bas** : le type de l'exception et son message, puis la méthode où elle s'est produite (`prixMoyen`, ligne 3), puis la méthode appelante (`main`, ligne 7). Apprendre à lire une trace est une compétence essentielle du développeur backend : c'est la première chose que l'on regarde dans les journaux d'un serveur.

9.2 La hiérarchie des exceptions

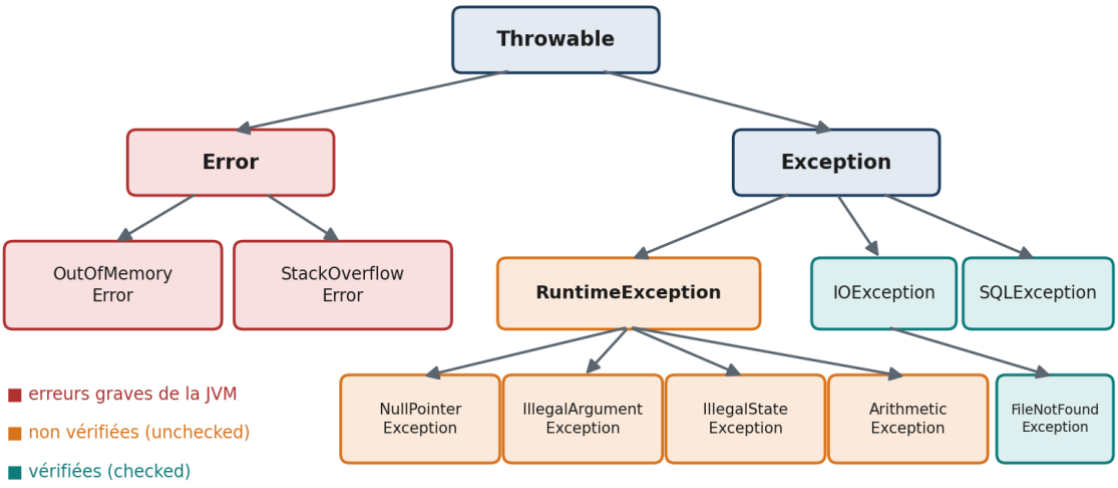


Figure 9.1 — La hiérarchie des classes d'exceptions

- **`Throwable`** est la racine de tout ce qui peut être levé.
- **`Error`** représente des problèmes graves de la JVM (`OutOfMemoryError`, `StackOverflowError`). On ne les intercepte pas.
- **`Exception`** représente les problèmes que l'application peut traiter.
- **`RuntimeException`** et ses sous-classes sont les exceptions **non vérifiées** (*unchecked*) : erreurs de programmation ou violations de règles détectées à l'exécution.
- Les autres sous-classes d'`Exception` sont **vérifiées** (*checked*) : le compilateur **oblige** à les intercepter ou à les déclarer avec `throws`.

Exception	Type	Cause typique
`NullPointerException`	non vérifiée	appel de méthode sur une référence `null`
`ArithmeticException`	non vérifiée	division entière par zéro
`ArrayIndexOutOfBoundsException`	non vérifiée	indice hors du tableau
`NumberFormatException`	non vérifiée	`Integer.parseInt("douze")`
`IllegalArgumentException`	non vérifiée	argument invalide passé à une méthode
`IllegalStateException`	non vérifiée	opération impossible dans l'état actuel de l'objet
`ClassCastException`	non vérifiée	transtypage incorrect
`UnsupportedOperationException`	non vérifiée	modification d'une liste non modifiable (`List.of`)
`IOException`, `FileNotFoundException`	**vérifiée**	erreur d'entrée/sortie, fichier absent
`SQLException`	**vérifiée**	erreur d'accès à la base de données
`InterruptedException`	**vérifiée**	thread interrompu pendant une attente

9.3 Intercepter une exception : `try` / `catch` / `finally`

Java

```java
try {
    // code susceptible de lever une exception
} catch (TypeException1 e) {
    // traitement si TypeException1 (ou une sous-classe) est levée
} catch (TypeException2 | TypeException3 e) {      // multi-catch (Java 7)
    // traitement commun à deux types
} finally {
    // exécuté DANS TOUS LES CAS : succès, exception interceptée ou non
}
```

- Les blocs `catch` sont examinés **dans l'ordre** : le premier dont le type correspond est exécuté. Placez donc les exceptions les plus **spécifiques** d'abord ; le compilateur refuse un `catch` inaccessible.
- Le bloc `finally` sert à libérer des ressources ; depuis Java 7, on lui préfère *try-with-resources* (section 9.6).
- La variable `e` donne accès à `e.getMessage()`, `e.getCause()` et `e.printStackTrace()`.

SaisieRobuste.java

```java
public class SaisieRobuste {
    static int lireQuantite(String saisie) {
        try {
            int q = Integer.parseInt(saisie.strip());
            if (q <= 0 || q > 100) {
                throw new IllegalArgumentException("Quantité hors limites (1 à 100) : " + q);
            }
            return q;
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("« " + saisie + " » n'est pas un nombre", e);
        } finally {
            System.out.println("  (saisie analysée : " + saisie + ")");
        }
    }

    public static void main(String[] args) {
        String[] saisies = {"3", " 12 ", "douze", "0", "250"};
        for (String s : saisies) {
            try {
                System.out.println("Quantité acceptée : " + lireQuantite(s));
            } catch (IllegalArgumentException e) {
                System.out.println("Erreur : " + e.getMessage());
            }
        }
    }
}
```

Sortie console

```
  (saisie analysée : 3)
Quantité acceptée : 3
  (saisie analysée :  12 )
Quantité acceptée : 12
  (saisie analysée : douze)
Erreur : « douze » n'est pas un nombre
  (saisie analysée : 0)
Erreur : Quantité hors limites (1 à 100) : 0
  (saisie analysée : 250)
Erreur : Quantité hors limites (1 à 100) : 250
```

Remarquez que le bloc `finally` s'exécute même lorsque la méthode se termine par `return` ou par une exception. Dans le `catch` de `NumberFormatException`, la nouvelle exception reçoit l'exception d'origine en second argument : c'est le **chaînage** des exceptions, qui préserve la cause pour le diagnostic.

9.4 Lever une exception : `throw`

Le mot-clé `throw` lève une exception. On l'utilise pour signaler qu'une **règle** n'est pas respectée, le plus tôt possible (principe *fail fast*) :

Java

```java
public void retirer(int quantite) {
    if (quantite <= 0) {
        throw new IllegalArgumentException("Quantité invalide : " + quantite);
    }
    if (quantite > stock) {
        throw new IllegalStateException("Stock insuffisant");
    }
    stock -= quantite;
}
```

Pour les vérifications de nullité, la classe `java.util.Objects` offre des raccourcis :

Java

```java
this.client = Objects.requireNonNull(client, "Le client est obligatoire");
int index = Objects.checkIndex(i, lignes.size());   // IndexOutOfBoundsException si invalide
```

9.5 Déclarer une exception : `throws`, et la propagation

Une méthode qui peut laisser échapper une exception **vérifiée** doit le déclarer dans sa signature avec `throws`. L'appelant a alors deux choix : **intercepter** l'exception, ou la **déclarer** à son tour pour la laisser remonter.

Propagation.java

```java
import java.io.IOException;

public class Propagation {
    static String lireCatalogue(String fichier) throws IOException {       // déclare
        if (!fichier.endsWith(".csv")) {
            throw new IOException("Format non supporté : " + fichier);
        }
        return "PRD-001;Bissap 1 L;1500;50";
    }

    static int compterProduits(String fichier) throws IOException {      // propage
        String contenu = lireCatalogue(fichier);
        return contenu.split("\n").length;
    }

    public static void main(String[] args) {                              // intercepte
        for (String f : new String[]{"catalogue.csv", "catalogue.xlsx"}) {
            try {
                System.out.println(f + " : " + compterProduits(f) + " produit(s)");
            } catch (IOException e) {
                System.out.println(f + " : impossible de lire (" + e.getMessage() + ")");
            }
        }
    }
}
```

Sortie console

```
catalogue.csv : 1 produit(s)
catalogue.xlsx : impossible de lire (Format non supporté : catalogue.xlsx)
```

`throw` ou `throws` ?

`throw` (sans s) est une **instruction** qui lève une exception maintenant. `throws` (avec s) est une **déclaration** dans la signature d'une méthode, qui prévient qu'elle peut laisser passer certaines exceptions vérifiées.

9.6 Libérer les ressources : *try-with-resources*

Fichiers, connexions réseau, connexions à la base de données : ces **ressources** doivent être fermées après usage, même en cas d'erreur, sinon le serveur finit par manquer de connexions ou de descripteurs de fichiers. Avant Java 7, on écrivait des blocs `finally` longs et fragiles.

Java 7 — try-with-resources

Toute ressource déclarée entre parenthèses après `try` et implémentant l'interface `AutoCloseable` est **fermée automatiquement** à la fin du bloc, dans l'ordre inverse de son ouverture, qu'une exception ait été levée ou non. Depuis Java 9, on peut aussi y placer une variable existante effectivement finale : `try (connexion) { ... }`.

`Ressources.java`

```java
public class Ressources {
    static class Connexion implements AutoCloseable {
        private final String nom;
        Connexion(String nom) {
            this.nom = nom;
            System.out.println("Ouverture de " + nom);
        }
        void executer(String requete) {
            if (requete.contains("DROP"))
                    throw new IllegalStateException("Requête interdite : " + requete);
            System.out.println("  " + nom + " exécute : " + requete);
        }
        @Override
        public void close() {
            System.out.println("Fermeture de " + nom);
        }
    }

    public static void main(String[] args) {
        try (Connexion bd = new Connexion("PostgreSQL");
             Connexion cache = new Connexion("Cache")) {
            bd.executer("SELECT * FROM produit");
            cache.executer("GET produit:PRD-001");
        }
        System.out.println("---");
        try (Connexion bd = new Connexion("PostgreSQL")) {
            bd.executer("DROP TABLE produit");
            System.out.println("jamais affiché");
        } catch (IllegalStateException e) {
            System.out.println("Erreur interceptée : " + e.getMessage());
        }
    }
}
```

Sortie console

```
Ouverture de PostgreSQL
Ouverture de Cache
  PostgreSQL exécute : SELECT * FROM produit
  Cache exécute : GET produit:PRD-001
Fermeture de Cache
Fermeture de PostgreSQL
---
Ouverture de PostgreSQL
Fermeture de PostgreSQL
Erreur interceptée : Requête interdite : DROP TABLE produit
```

Observez que la ressource est fermée **avant** l'exécution du bloc `catch`. Si `close()` lève elle-même une exception alors qu'une exception est déjà en cours, cette seconde exception n'écrase pas la première : elle lui est attachée comme exception **supprimée** (`e.getSuppressed()`).

9.7 Créer ses propres exceptions

Les exceptions standard (`IllegalArgumentException`, `IllegalStateException`) conviennent aux erreurs de programmation. Pour les **erreurs métier**, on crée une hiérarchie d'exceptions propre à l'application. Elle permet :

- d'intercepter une catégorie précise d'erreurs (`catch (StockInsuffisantException e)`) ;
- de transporter des **informations structurées** (la référence du produit, la quantité disponible) et pas seulement un message ;
- de traduire chaque type d'erreur en une réponse HTTP adaptée (section 9.9).

Pour créer une exception, on étend `RuntimeException` (non vérifiée) ou `Exception` (vérifiée) et on fournit au moins un constructeur qui transmet le message (et éventuellement la cause) à `super`.

TerangaShopException.java

```java
/** Racine de toutes les erreurs métier de l'application. */
public abstract class TerangaShopException extends RuntimeException {
    private final String code;

    protected TerangaShopException(String code, String message) {
        super(message);
        this.code = code;
    }
    protected TerangaShopException(String code, String message, Throwable cause) {
        super(message, cause);
        this.code = code;
    }
    public String getCode() { return code; }
}
```

ProduitIntrouvableException.java

```java
public class ProduitIntrouvableException extends TerangaShopException {
    public ProduitIntrouvableException(String reference) {
        super("PRODUIT_INTROUVABLE", "Aucun produit avec la référence " + reference);
    }
}
```

StockInsuffisantException.java

```java
public class StockInsuffisantException extends TerangaShopException {
    private final String reference;
    private final int demande;
    private final int disponible;

    public StockInsuffisantException(String reference, int demande, int disponible) {
        super("STOCK_INSUFFISANT", "Stock insuffisant pour %s : demandé %d, disponible %d"
                .formatted(reference, demande, disponible));
        this.reference = reference;
        this.demande = demande;
        this.disponible = disponible;
    }
    public String getReference() { return reference; }
    public int getDemande() { return demande; }
    public int getDisponible() { return disponible; }
}
```

PaiementRefuseException.java

```java
public class PaiementRefuseException extends TerangaShopException {
    public PaiementRefuseException(String motif, Throwable cause) {
        super("PAIEMENT_REFUSE", "Paiement refusé : " + motif, cause);
    }
}
```

ServiceCommande.java

```java
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.TimeoutException;

public class ServiceCommande {
    private final Map<String, Integer> stocks = new HashMap<>(Map.of("PRD-001",
            50, "PRD-014", 2));

    public void reserver(String reference, int quantite) {
        Integer stock = stocks.get(reference);
        if (stock == null) throw new ProduitIntrouvableException(reference);
        if (quantite > stock) throw new StockInsuffisantException(reference, quantite, stock);
        stocks.put(reference, stock - quantite);
    }

    public void payer(long montant) {
        try {
            appelerOperateur(montant);                          // API externe simulée
        // exception technique vérifiée...
        } catch (TimeoutException e) {
            // ...traduite en exception métier
            throw new PaiementRefuseException("l'opérateur ne répond pas", e);
        }
    }

    private void appelerOperateur(long montant) throws TimeoutException {
        if (montant > 100_000) throw new TimeoutException("délai de 30 s dépassé");
    }

    public static void main(String[] args) {
        ServiceCommande service = new ServiceCommande();
        Runnable[] operations = {
            () -> service.reserver("PRD-001", 3),
            () -> service.reserver("PRD-999", 1),
            () -> service.reserver("PRD-014", 5),
            () -> service.payer(250_000)
        };
        for (Runnable op : operations) {
            try {
                op.run();
                System.out.println("OK");
```

```
            } catch (StockInsuffisantException e) {
                System.out.println("[" + e.getCode() + "] " + e.getMessage()
                                   + " -> proposer " + e.getDisponible() + " article(s)");
            } catch (TerangaShopException e) {
                System.out.println("[" + e.getCode() + "] " + e.getMessage()
                                   + (e.getCause() != null ? " (cause : " + e.getCause()
                                           .getMessage() + ")" : ""));
            }
        }
    }
}
```

Sortie console

```
OK
[PRODUIT_INTROUVABLE] Aucun produit avec la référence PRD-999
[STOCK_INSUFFISANT] Stock insuffisant pour PRD-014 : demandé 5, disponible 2 -> proposer 2 article(s)
[PAIEMENT_REFUSE] Paiement refusé : l'opérateur ne répond pas (cause : délai de 30 s dépassé)
```

Vérifiée ou non vérifiée ?

Le choix fait débat depuis longtemps. La pratique actuelle, notamment dans l'écosystème Spring, est la suivante :

- **non vérifiée** (`RuntimeException`) pour les erreurs de programmation et pour les **erreurs métier** : elles traversent les couches sans polluer toutes les signatures avec des `throws`, et sont traitées **à un seul endroit** (le gestionnaire global d'erreurs de l'API) ;
- **vérifiée** lorsque l'appelant **peut et doit** réagir immédiatement (réessayer, choisir une autre source), typiquement pour les entrées/sorties dans du code bas niveau ;
- lorsqu'une exception vérifiée technique (`IOException`, `SQLException`) doit franchir une couche, on l'**enveloppe** dans une exception non vérifiée qui a du sens pour la couche supérieure, **en conservant la cause** (comme `PaiementRefuseException` ci-dessus, ou `UncheckedIOException`).

9.8 Bonnes pratiques

À faire	À éviter
Intercepter l'exception **la plus spécifique** possible	`catch (Exception e)` partout, et surtout `catch (Throwable t)`
Écrire des messages avec le **contexte** (identifiants, valeurs)	`throw new RuntimeException("Erreur")`
Conserver la **cause** lors d'un enveloppement : `new XException(msg, e)`	Perdre la trace d'origine : `new XException(e.getMessage())`
Valider les arguments **au début** des méthodes (*fail fast*)	Laisser l'erreur apparaître loin de sa cause
Fermer les ressources avec *try-with-resources*	Des `finally` complexes ou pas de fermeture du tout
Journaliser **une seule fois**, au niveau qui traite l'erreur	Journaliser puis relancer à chaque couche (traces en double)
Utiliser les exceptions pour les situations **exceptionnelles**	S'en servir comme d'un `if` pour le flux normal

> **Piège à éviter — avaler une exception**
>
> Le code suivant est l'un des pires anti-modèles : l'erreur disparaît sans laisser de trace, et le programme continue dans un état incohérent.

Java

```java
try {
    service.payer(montant);
} catch (Exception e) {
    // TODO   <- l'erreur est silencieusement ignorée !
}
```

9.9 Côté backend : des exceptions aux codes HTTP

Dans une API REST, chaque type d'erreur doit produire une **réponse HTTP** précise, avec un corps JSON exploitable par le client (application mobile, site web). Une hiérarchie d'exceptions métier rend cette correspondance simple et centralisée.

Exception	Code HTTP	Signification
Données invalides (`IllegalArgumentException`, validation)	400 Bad Request	la requête du client est incorrecte
Non authentifié	401 Unauthorized	le client doit s'identifier
Accès interdit	403 Forbidden	identifié, mais pas autorisé
`ProduitIntrouvableException`	404 Not Found	la ressource n'existe pas
`StockInsuffisantException`, transition de statut interdite	409 Conflict	la requête entre en conflit avec l'état actuel
`PaiementRefuseException` (opérateur indisponible)	502 Bad Gateway / 503	un service externe a échoué
Toute autre exception inattendue	500 Internal Server Error	bogue côté serveur : ne jamais exposer la trace au client

Au chapitre 18, nous verrons comment Spring Boot centralise cette traduction avec `@RestControllerAdvice` et le format standard `ProblemDetail` (RFC 9457).

Exercices du chapitre 9

Exercice 9.1 ★ — Suivre le flux d'exécution

Indiquez, pour chaque appel `test(0)`, `test(1)` et `test(2)`, ce qui est affiché et la valeur renvoyée. Vérifiez ensuite en exécutant le code. Que penser d'un `return` dans un bloc `finally` ?

Java

```java
static int test(int cas) {
    try {
        System.out.println("try");
        if (cas == 1) throw new IllegalStateException("cas 1");
        if (cas == 2) return 20;
        return 10;
    } catch (IllegalStateException e) {
        System.out.println("catch " + e.getMessage());
        return 30;
    } finally {
        System.out.println("finally");
        if (cas == 2) return 99;
    }
}
```

Exercice 9.2 ★ — Saisie contrôlée

Écrivez une méthode `static int lireEntier(Scanner sc, String invite, int min, int max)` qui redemande la saisie tant que l'utilisateur ne tape pas un entier compris entre `min` et `max`, en affichant un message adapté à chaque erreur (pas un nombre / hors limites). Utilisez-la dans le menu de caisse du chapitre 4.

Exercice 9.3 ★★ — Exceptions du compte mobile

Reprenez la classe `CompteMobile` de l'exercice 6.1. Créez une exception `SoldeInsuffisantException` qui mémorise le solde et le montant demandé. Écrivez une première version où elle est **vérifiée**, puis une seconde où elle est **non vérifiée**. Comparez l'impact sur le code des méthodes `retirer`, `transferer` et du `main`. Laquelle préférez-vous pour ce cas, et pourquoi ?

Exercice 9.4 ★★ — Ressources et exceptions supprimées

Écrivez une classe `FichierSimule implements AutoCloseable` dont la méthode `close()` lève une `IllegalStateException("fermeture impossible")`. Dans un *try-with-resources*, levez une autre exception dans le corps du `try`. Interceptez-la et affichez son message, puis les messages de ses exceptions supprimées (`getSuppressed()`). Que se passerait-il avec un `try/finally` classique ?

Exercice 9.5 ★★ — Import tolérant aux erreurs

On reçoit les lignes suivantes d'un fichier de commandes. Écrivez un programme qui analyse chaque ligne (`numéro;client;quantité;prixUnitaire`), calcule le total des lignes valides et produit un **rapport d'erreurs** indiquant, pour chaque ligne invalide, son numéro et la raison (nombre de champs incorrect, quantité non numérique, prix négatif…). Le traitement ne doit **pas** s'arrêter à la première erreur.

```
TS-101;Awa Diop;2;7500
TS-102;Moussa Ndiaye;trois;1500
TS-103;Fatou Sow;1
TS-104;Ibrahima Fall;4;-200
TS-105;Aminata Ba;1;25000
```

Exercice 9.6 ★★★ — Gestionnaire global d'erreurs

Définissez un record `ErreurApi(int statut, String code, String message)`. Écrivez une classe `GestionnaireErreurs` avec une méthode `ErreurApi traduire(Throwable t)` qui utilise un `switch` à motifs pour convertir : les exceptions de la hiérarchie `TerangaShopException` (404 pour un produit introuvable, 409 pour un stock insuffisant, 502 pour un paiement refusé), `IllegalArgumentException` (400) et toute autre exception (500, avec un message générique **qui ne révèle pas** le détail technique). Simulez ensuite un « contrôleur » qui exécute des opérations et affiche la réponse JSON correspondante (avec un *text block*).

PARTIE III — La bibliothèque standard du développeur backend

- Chapitre 10 — Les génériques
- Chapitre 11 — Les collections
- Chapitre 12 — Lambdas, Optional et Streams
- Chapitre 13 — Dates, fichiers, configuration et journalisation
- Chapitre 14 — Concurrence moderne et threads virtuels

Chapitre 10 — Les génériques

Objectifs du chapitre

- Comprendre le problème que résolvent les génériques.
- Écrire des classes, records, interfaces et méthodes génériques.
- Utiliser les bornes (`extends`) et les jokers (`? extends`, `? super`).
- Connaître les limites dues à l'effacement de type.
- Écrire un dépôt (*repository*) générique, brique de base de tout backend.

10.1 Pourquoi les génériques ?

Avant Java 5, une liste contenait des `Object`. On pouvait y mettre n'importe quoi, et il fallait transtyper à la lecture, au risque d'une erreur **à l'exécution** :

Java

```
List liste = new ArrayList();            // type « brut » (raw type) : à ne plus jamais écrire
liste.add("Bissap");
liste.add(1500);                         // accepté : aucune vérification
String s = (String) liste.get(1);        // ClassCastException à l'exécution !
```

Les **génériques** permettent de **paramétrer un type par un autre type**. `List<String>` est une liste de chaînes : le compilateur refuse d'y ajouter autre chose, et la lecture ne nécessite aucun transtypage. Les erreurs sont détectées **à la compilation**, ce qui est toujours préférable.

Java

```
// <> : l'« opérateur diamant » (Java 7) déduit le type
List<String> produits = new ArrayList<>();
produits.add("Bissap");
produits.add(1500);                          // ERREUR de compilation
String s = produits.get(0);                  // pas de cast
```

10.2 Classes et records génériques

On déclare un ou plusieurs **paramètres de type** entre chevrons après le nom. Par convention, ce sont des lettres majuscules : `T` (type), `E` (élément), `K` et `V` (clé et valeur), `ID` (identifiant), `R` (résultat).

DemoGeneriques.java

```
import java.util.List;

public class DemoGeneriques {

    /** Une paire de deux valeurs de types quelconques. */
    record Paire<A, B>(A premier, B second) {
        <C> Paire<A, C> avecSecond(C nouveau) {      // méthode générique dans un type générique
            return new Paire<>(premier, nouveau);
        }
    }

    /** Une page de résultats, comme celles renvoyées par une API paginée. */
    record Page<T>(List<T> contenu, int numero, int taille, long totalElements) {
        Page {
            contenu = List.copyOf(contenu);
        }
```

```
        int totalPages() {
            // division arrondie au supérieur
            return (int) ((totalElements + taille - 1) / taille);
        }
        boolean estDerniere() {
            return numero >= totalPages() - 1;
        }
    }

    /** Méthode générique : T est déduit à chaque appel. */
    static <T> Page<T> paginer(List<T> tout, int numero, int taille) {
        int debut = Math.min(numero * taille, tout.size());
        int fin = Math.min(debut + taille, tout.size());
        return new Page<>(tout.subList(debut, fin), numero, taille, tout.size());
    }

    public static void main(String[] args) {
        Paire<String, Long> p = new Paire<>("Bissap 1 L", 1_500L);
        Paire<String, Integer> stock = p.avecSecond(48);
        System.out.println(p + " / " + stock);

        List<String> produits = List.of("Bissap", "Thiakry", "Pagne", "Sac cuir", "Café Touba",
                                         "Savon karité", "Beurre de karité");
        for (int n = 0; n < 3; n++) {
            Page<String> page = paginer(produits, n, 3);
            System.out.printf("Page %d/%d %s dernière=%b%n",
                    page.numero() + 1, page.totalPages(), page.contenu(), page.estDerniere());
        }
        Page<Integer> pageNombres = paginer(List.of(10, 20, 30), 0, 2);   // T = Integer
        System.out.println(pageNombres);
    }
}
```

Sortie console

```
Paire[premier=Bissap 1 L, second=1500] / Paire[premier=Bissap 1 L, second=48]
Page 1/3 [Bissap, Thiakry, Pagne] dernière=false
Page 2/3 [Sac cuir, Café Touba, Savon karité] dernière=false
Page 3/3 [Beurre de karité] dernière=true
Page[contenu=[10, 20], numero=0, taille=2, totalElements=3]
```

Piège à éviter — pas de types primitifs

Un paramètre de type ne peut pas être un type primitif : `List<int>` ne compile pas. On utilise la classe enveloppe correspondante, `List<Integer>`, et l'autoboxing fait le reste (au prix d'un léger coût en mémoire).

10.3 Méthodes génériques et bornes

Une méthode générique déclare ses paramètres de type **avant** son type de retour : `static <T> T premier(List<T> liste)`. Souvent, on a besoin d'imposer une contrainte sur `T` : c'est la **borne supérieure** `T extends Type`, qui signifie « `T` est `Type` ou un de ses sous-types » (le mot `extends` vaut ici aussi pour les interfaces).

Bornes.java

```
import java.util.List;

public class Bornes {
    /** Renvoie le plus grand élément : T doit savoir se comparer à lui-même. */
    static <T extends Comparable<T>> T max(List<T> elements) {
        if (elements.isEmpty()) throw new IllegalArgumentException("Liste vide");
        T meilleur = elements.get(0);
        for (T e : elements) {
```

```
            if (e.compareTo(meilleur) > 0) meilleur = e;
        }
        return meilleur;
    }

    /** Somme de nombres de n'importe quel type numérique. */
    static <N extends Number> double somme(List<N> nombres) {
        double s = 0;
        for (N n : nombres) s += n.doubleValue();   // doubleValue() existe car N est un Number
        return s;
    }

    public static void main(String[] args) {
        System.out.println(max(List.of(1_500, 7_500, 2_000)));
        System.out.println(max(List.of("Médina", "Fann", "Yoff")));
        System.out.println(somme(List.of(1, 2, 3)));
        System.out.println(somme(List.of(2.5, 0.5)));
        // max(List.of(new Object()));   // ERREUR : Object n'implémente pas Comparable
    }
}
```

Sortie console

```
7500
Yoff
6.0
3.0
```

10.4 L'invariance et les jokers (*wildcards*)

Un point surprend souvent : bien que `Integer` soit un sous-type de `Number`, **`List<Integer>` n'est pas un sous-type de `List<Number>`**. Les génériques sont **invariants**. Si c'était permis, on pourrait écrire :

Java

```
List<Integer> entiers = new ArrayList<>();
List<Number> nombres = entiers;       // ERREUR de compilation (heureusement !)
nombres.add(3.14);                    // on aurait glissé un Double dans une liste d'Integer
```

Pour écrire des méthodes plus souples, on utilise des **jokers** :

Joker	Lecture	Signification	On peut…
`List<?>`	« liste de quelque chose »	type inconnu	lire des `Object`, rien ajouter (sauf `null`)
`List<? extends Number>`	« liste d'un sous-type de Number »	**producteur** de `Number`	**lire** des `Number` ; pas ajouter
`List<? super Integer>`	« liste d'un super-type d'Integer »	**consommateur** d'`Integer`	**ajouter** des `Integer` ; lire seulement des `Object`

La règle mnémotechnique **PECS** (*Producer Extends, Consumer Super*) résume l'usage : si la structure **produit** des valeurs que vous lisez, utilisez `extends` ; si elle **consomme** des valeurs que vous y écrivez, utilisez `super`. C'est exactement la signature de `Collections.copy(List<? super T> dest, List<? extends T> src)`.

Java

```
// accepte List<Integer>, List<Long>…
static double total(List<? extends Number> montants) { ... }
// accepte List<Integer>, List<Number>, List<Object>
static void remplirAvecZeros(List<? super Integer> cible, int n) {
    for (int i = 0; i < n; i++) cible.add(0);
}
```

10.5 L'effacement de type et ses limites

Les génériques sont une vérification **du compilateur**. Dans le bytecode, les paramètres de type sont **effacés** (*type erasure*) et remplacés par leur borne (`Object` le plus souvent). Cela garantit la compatibilité avec le code d'avant Java 5, mais impose quelques limites :

Interdit	Pourquoi	Alternative
`new T()`	le type réel de `T` n'existe plus à l'exécution	passer un `Supplier<T>` ou un `Class<T>`
`new T[10]`	idem	utiliser une `List<T>`, ou `(T[]) new Object[10]` avec prudence
`obj instanceof List<String>`	le paramètre `String` a été effacé	`obj instanceof List<?> l`
attribut `static T`	le membre statique est partagé par toutes les paramétrisations	—
surcharge `m(List<String>)` et `m(List<Integer>)`	même signature après effacement	noms différents

10.6 Fil rouge TerangaShop — un dépôt générique

Dans un backend, la couche d'**accès aux données** est organisée en *repositories* (dépôts) : un par type d'entité, avec toujours les mêmes opérations (enregistrer, chercher par identifiant, lister, supprimer). Plutôt que de les réécrire pour chaque entité, on définit une **interface générique**. C'est exactement ce que fait Spring Data avec `CrudRepository<T, ID>` (chapitre 18).

Identifiable.java

```java
/** Tout objet qui possède un identifiant de type ID. */
public interface Identifiable<ID> {
    ID id();
}
```

Repository.java

```java
import java.util.List;
import java.util.Optional;
import java.util.function.Predicate;

public interface Repository<T extends Identifiable<ID>, ID> {
    T enregistrer(T entite);
    // Optional : la valeur peut être absente (chapitre 12)
    Optional<T> trouverParId(ID id);
    List<T> trouverTous();
    List<T> trouverSi(Predicate<? super T> critere);
    boolean supprimer(ID id);
    long compter();

    default T obtenir(ID id) {                // méthode par défaut réutilisant trouverParId
        return trouverParId(id).orElseThrow(
            () -> new IllegalArgumentException("Introuvable : " + id));
    }
}
```

RepositoryMemoire.java

```java
import java.util.*;
import java.util.function.Predicate;

/** Implémentation en mémoire, utile pour les prototypes et les tests. */
public class RepositoryMemoire<T extends Identifiable<ID>, ID> implements Repository<T, ID> {
```

```
    private final Map<ID, T> donnees = new LinkedHashMap<>();   // conserve l'ordre d'insertion

    @Override
    public T enregistrer(T entite) {
        donnees.put(entite.id(), entite);
        return entite;
    }
    @Override
    public Optional<T> trouverParId(ID id) {
        return Optional.ofNullable(donnees.get(id));
    }
    @Override
    public List<T> trouverTous() {
        return List.copyOf(donnees.values());
    }
    @Override
    public List<T> trouverSi(Predicate<? super T> critere) {
        List<T> resultat = new ArrayList<>();
        for (T t : donnees.values()) {
            if (critere.test(t)) resultat.add(t);
        }
        return resultat;
    }
    @Override
    public boolean supprimer(ID id) {
        return donnees.remove(id) != null;
    }
    @Override
    public long compter() {
        return donnees.size();
    }
}
```

DemoRepository.java

```
public class DemoRepository {
    record Produit(String id, String libelle, long prix) implements Identifiable<String> { }
    record Client(Long id, String nom, String quartier) implements Identifiable<Long> { }

    public static void main(String[] args) {
        Repository<Produit, String> produits = new RepositoryMemoire<>();
        produits.enregistrer(new Produit("PRD-001", "Bissap 1 L", 1_500));
        produits.enregistrer(new Produit("PRD-014", "Pagne wax", 7_500));
        produits.enregistrer(new Produit("PRD-020", "Sac cuir", 25_000));

        Repository<Client, Long> clients = new RepositoryMemoire<>();
        clients.enregistrer(new Client(1L, "Awa Diop", "Médina"));
        clients.enregistrer(new Client(2L, "Moussa Ndiaye", "Rufisque"));

        System.out.println(produits.obtenir("PRD-014"));
        System.out.println(produits.trouverParId("PRD-999").isPresent());
        System.out.println(produits.trouverSi(p -> p.prix() > 5_000));
        System.out.println(clients.trouverSi(c -> c.quartier().equals("Rufisque")));
        produits.supprimer("PRD-001");
        System.out.println(produits.compter() + " produits, " + clients.compter() + " clients");
        // produits.enregistrer(new Client(3L, "Fatou", "Fann"));  // ERREUR de compilation
    }
}
```

Sortie console

```
Produit[id=PRD-014, libelle=Pagne wax, prix=7500]
false
[Produit[id=PRD-014, libelle=Pagne wax, prix=7500], Produit[id=PRD-020, libelle=Sac cuir, prix=25000]]
[Client[id=2, nom=Moussa Ndiaye, quartier=Rufisque]]
2 produits, 2 clients
```

Une seule implémentation sert désormais pour toutes les entités, et le compilateur empêche d'enregistrer un client dans le dépôt des produits.

Exercices du chapitre 10

Exercice 10.1 ★ — Méthodes génériques simples

Écrivez les méthodes génériques suivantes et testez-les avec plusieurs types : `static <T> int compter(T[] tableau, T valeur)` (utilisez `Objects.equals`) ; `static <T> void echanger(T[] tableau, int i, int j)` ; `static <T> List<T> repeter(T valeur, int n)`.

Exercice 10.2 ★ — Minimum et maximum

Écrivez `static <T extends Comparable<? super T>> Paire<T, T> minMax(List<T> liste)` qui renvoie en un seul parcours le minimum et le maximum. Pourquoi la borne `Comparable<? super T>` est-elle plus souple que `Comparable<T>` ? (Indice : pensez à une classe qui hérite d'une classe déjà comparable.)

Exercice 10.3 ★★ — Une pile générique

Implémentez une classe `Pile<E>` avec un tableau interne qui s'agrandit automatiquement (doublement de la capacité). Méthodes : `empiler(E e)`, `E depiler()`, `E sommet()`, `boolean estVide()`, `int taille()`. `depiler` et `sommet` lèvent une `IllegalStateException` si la pile est vide. Utilisez-la pour vérifier qu'une expression est bien parenthésée : `"{[(a+b)*c]}"` est correcte, `"(a+b]"` ne l'est pas.

Exercice 10.4 ★★ — Producteurs et consommateurs

1. Écrivez `static <T> void copier(List<? super T> destination, List<? extends T> source)`.
2. Montrez sur un exemple qu'on peut copier une `List<Integer>` dans une `List<Number>`, puis dans une `List<Object>`.
3. Expliquez pourquoi on ne peut pas ajouter un élément dans une `List<? extends Number>`.

Exercice 10.5 ★★ — Un cache à capacité limitée

Écrivez une classe générique `Cache<K, V>` de capacité fixe. `put(K cle, V valeur)` ajoute ou remplace une entrée ; lorsque la capacité est atteinte, l'entrée **la plus ancienne** est supprimée. `Optional<V> get(K cle)` renvoie la valeur si elle est présente. Ajoutez des compteurs de *hits* et de *miss* et une méthode `double tauxSucces()`. Utilisez-le pour mettre en cache des prix de produits.

Exercice 10.6 ★★★ — Un type Résultat

Plutôt que de lever des exceptions, certaines API renvoient un résultat qui est soit un succès, soit un échec. Définissez `sealed interface Resultat<T> permits Succes, Echec`, avec `record Succes<T>(T valeur)` et `record Echec<T>(String erreur)`. Ajoutez les méthodes par défaut `<R> Resultat<R> map(Function<? super T, ? extends R> f)`, `<R> Resultat<R> flatMap(Function<? super T, Resultat<R>> f)` et `T orElse(T defaut)`, implémentées avec un `switch` à motifs. Utilisez ce type pour enchaîner la validation d'une commande : vérifier le téléphone, puis le quartier, puis le montant minimum, en s'arrêtant à la première erreur.

Chapitre 11 — Les collections

Objectifs du chapitre

- Connaître l'organisation du *Collections Framework* : `List`, `Set`, `Queue`/`Deque`, `Map`.
- Choisir la bonne implémentation selon le besoin et la complexité des opérations.
- Créer des collections immuables et comprendre la différence avec une vue non modifiable.
- Utiliser les *Sequenced Collections* de Java 21.
- Trier avec `Comparable` et `Comparator`.

11.1 Vue d'ensemble

Le *Collections Framework* (package `java.util`) fournit des **interfaces** qui décrivent des structures de données et des **implémentations** qui les réalisent. On déclare toujours une variable avec le type de l'**interface** et on choisit l'implémentation à la création : `List<Produit> catalogue = new ArrayList<>();`. On peut ainsi changer d'implémentation sans toucher au reste du code.

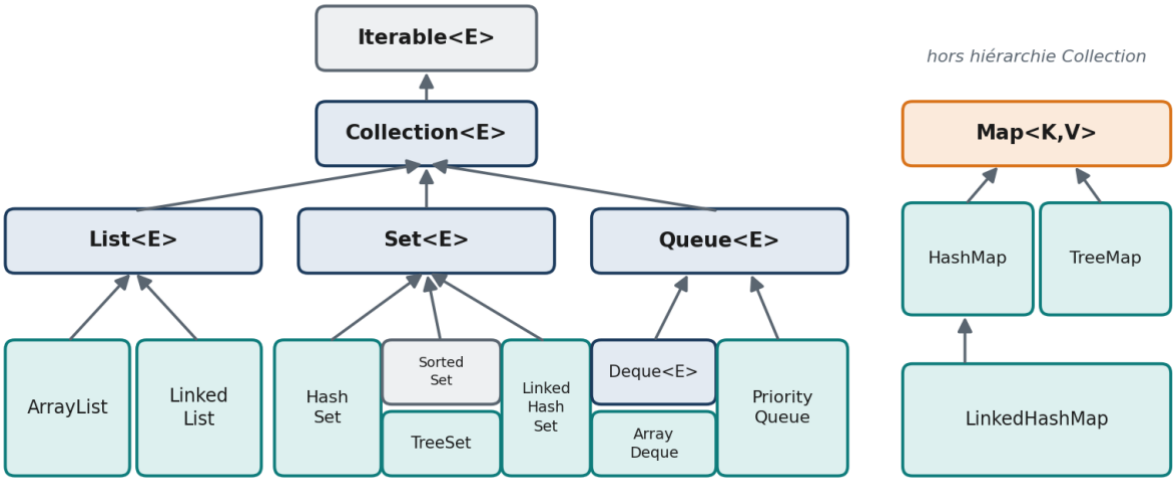


Figure 11.1 — Les principales interfaces et implémentations des collections

Interface	Caractéristique	Implémentations principales
`List<E>`	séquence **ordonnée**, accès par indice, doublons autorisés	`ArrayList`, `LinkedList`
`Set<E>`	**pas de doublons**	`HashSet`, `LinkedHashSet`, `TreeSet` (trié)
`Queue<E>` / `Deque<E>`	file (FIFO), pile (LIFO), file à double entrée	`ArrayDeque`, `PriorityQueue`, `LinkedList`
`Map<K, V>`	associations **clé → valeur**, clés uniques	`HashMap`, `LinkedHashMap`, `TreeMap` (trié par clé)

11.2 Les listes

`ArrayList` est l'implémentation à utiliser **par défaut** : elle stocke les éléments dans un tableau qui s'agrandit automatiquement. `LinkedList` (liste doublement chaînée) n'est plus intéressante que dans de rares cas.

Méthode	Rôle
`add(e)` / `add(i, e)`	ajoute à la fin / à la position `i`
`get(i)` / `set(i, e)`	lit / remplace l'élément d'indice `i`
`remove(i)` / `remove(objet)`	supprime par indice / la première occurrence de l'objet
`size()`, `isEmpty()`, `clear()`	taille, test de vide, vidage
`contains(o)`, `indexOf(o)`	recherche (utilise `equals`)
`subList(d, f)`	vue sur une portion de la liste
`sort(comparateur)`	tri sur place
`removeIf(condition)`	supprime tous les éléments qui vérifient une condition
`replaceAll(fonction)`	transforme chaque élément

DemoListes.java

```java
import java.util.*;

public class DemoListes {
    public static void main(String[] args) {
        List<String> panier = new ArrayList<>();
        panier.add("Bissap");
        panier.add("Thiakry");
        panier.add("Bissap");                       // doublon autorisé
        panier.add(1, "Café Touba");                // insertion à l'indice 1
        System.out.println(panier + " taille=" + panier.size());
        System.out.println("indexOf(Bissap)=" + panier.indexOf("Bissap")
                           + ", contient Pagne ? " + panier.contains("Pagne"));

        panier.set(2, "Thiakry vanille");
        panier.remove("Bissap");                    // retire la PREMIÈRE occurrence
        System.out.println(panier);

        List<Integer> quantites = new ArrayList<>(List.of(5, 10, 15, 20));
        quantites.remove(1);                        // retire l'élément d'INDICE 1 (10)
        quantites.remove(Integer.valueOf(20));      // retire l'OBJET 20
        System.out.println(quantites);

        List<Long> prix = new ArrayList<>(List.of(1_500L, 25_000L, 7_500L, 900L, 12_000L));
        prix.removeIf(p -> p < 1_000);              // lambda : voir chapitre 12
        prix.replaceAll(p -> p * 110 / 100);         // hausse de 10 %
        prix.sort(null);                             // null = ordre naturel
        System.out.println(prix);
    }
}
```

Sortie console

```
[Bissap, Café Touba, Thiakry, Bissap] taille=4
indexOf(Bissap)=0, contient Pagne ? false
[Café Touba, Thiakry vanille, Bissap]
[5, 15]
[1650, 8250, 13200, 27500]
```

Piège à éviter — `remove(int)` ou `remove(Object)` ?

Sur une `List<Integer>`, `liste.remove(1)` supprime l'élément **d'indice 1**, pas la valeur 1. Pour supprimer la valeur, écrivez `liste.remove(Integer.valueOf(1))`.

Parcourir une liste et la modifier

Modifier une liste (ajout ou suppression) **pendant** qu'on la parcourt avec une boucle *for-each* provoque une `ConcurrentModificationException`. Trois solutions correctes : `removeIf`, un `Iterator` explicite avec sa méthode `remove()`, ou construire une nouvelle liste.

ModificationParcours.java

```
import java.util.*;

public class ModificationParcours {
    public static void main(String[] args) {
        List<String> commandes = new ArrayList<>(List.of("TS-1:LIVREE",
                "TS-2:EN_ATTENTE", "TS-3:LIVREE"));
        try {
            for (String c : commandes) {
                if (c.endsWith("LIVREE")) commandes.remove(c);    // interdit !
            }
        } catch (ConcurrentModificationException e) {
            System.out.println("ConcurrentModificationException !");
        }

        Iterator<String> it = commandes.iterator();
        while (it.hasNext()) {
            if (it.next().endsWith("LIVREE")) it.remove();       // correct
        }
        System.out.println(commandes);
    }
}
```

Sortie console

```
ConcurrentModificationException !
[TS-2:EN_ATTENTE]
```

11.3 Collections immuables et vues non modifiables

Java 9 à 16 — Fabriques de collections immuables

`List.of(...)`, `Set.of(...)` et `Map.of(...)` (Java 9) créent des collections **immuables** ; `List.copyOf(...)` (Java 10) en fait une copie immuable ; `stream.toList()` (Java 16) produit une liste immuable. Toute tentative de modification lève une `UnsupportedOperationException`. Elles refusent aussi les éléments `null`.

Création	Modifiable ?	Remarque
`new ArrayList<>(...)`	oui	la liste « normale »
`List.of(a, b, c)`	**non**	immuable, pas de `null`
`List.copyOf(liste)`	**non**	copie indépendante de l'original
`Collections.unmodifiableList(liste)`	non, mais…	**vue** : si l'original change, la vue change aussi
`Arrays.asList(a, b, c)`	partiellement	taille fixe : `set` possible, `add`/`remove` impossibles

Astuce

Renvoyez des collections immuables depuis vos méthodes publiques (`return List.copyOf(lignes);`) : l'appelant ne pourra pas modifier l'état interne de votre objet par accident. C'est ce que fait `Commande.getLignes()` au chapitre 6.

11.4 Les ensembles (Set)

Un `Set` refuse les doublons, au sens de `equals` et `hashCode` : ces deux méthodes doivent donc être correctement redéfinies dans vos classes (ou générées par un record).

Implémentation	Ordre de parcours	add/contains	Usage
`HashSet`	aucun ordre garanti	O(1) en moyenne	le plus rapide, par défaut
`LinkedHashSet`	ordre d'insertion	O(1)	dédoublonner en conservant l'ordre
`TreeSet`	ordre **trié**	O(log n)	ensemble trié, recherches de plages (`headSet`, `ceiling`…)

DemoEnsembles.java

```java
import java.util.*;

public class DemoEnsembles {
    public static void main(String[] args) {
        List<String> quartiersCommandes = List.of("Médina", "Fann", "Médina", "Yoff",
                "Fann", "Almadies");
        Set<String> distincts = new LinkedHashSet<>(quartiersCommandes);
        System.out.println("Quartiers distincts : " + distincts);
        System.out.println("Triés : " + new TreeSet<>(quartiersCommandes));

        Set<String> clientsWave = new HashSet<>(Set.of("Awa", "Moussa", "Fatou", "Ibou"));
        Set<String> clientsOm = new HashSet<>(Set.of("Fatou", "Aminata", "Ibou"));

        Set<String> union = new TreeSet<>(clientsWave);
        union.addAll(clientsOm);
        Set<String> lesDeux = new TreeSet<>(clientsWave);
        lesDeux.retainAll(clientsOm);                    // intersection
        Set<String> waveSeulement = new TreeSet<>(clientsWave);
        waveSeulement.removeAll(clientsOm);              // différence
        System.out.println("Union : " + union);
        System.out.println("Les deux modes : " + lesDeux);
        System.out.println("Wave seulement : " + waveSeulement);
        // false : déjà présent
        System.out.println("add(Awa) une 2e fois : " + clientsWave.add("Awa"));
    }
}
```

Sortie console

```
Quartiers distincts : [Médina, Fann, Yoff, Almadies]
Triés : [Almadies, Fann, Médina, Yoff]
Union : [Aminata, Awa, Fatou, Ibou, Moussa]
Les deux modes : [Fatou, Ibou]
Wave seulement : [Awa, Moussa]
add(Awa) une 2e fois : false
```

11.5 Les tables associatives (Map)

Une `Map<K, V>` associe des **clés** uniques à des **valeurs**. C'est la structure la plus utilisée en backend : index par identifiant, cache, comptage, regroupement.

Méthode	Rôle
`put(k, v)`	associe `v` à `k` (remplace et renvoie l'ancienne valeur éventuelle)
`get(k)`	valeur associée, ou `null` si absente
`getOrDefault(k, defaut)`	valeur associée, ou `defaut` si absente
`containsKey(k)`, `containsValue(v)`	tests de présence

Méthode	Rôle
`remove(k)`	supprime l'association
`putIfAbsent(k, v)`	n'ajoute que si la clé est absente
`merge(k, v, fonction)`	ajoute `v`, ou combine avec la valeur existante (idéal pour compter ou cumuler)
`computeIfAbsent(k, fonction)`	crée la valeur si absente, puis la renvoie (idéal pour regrouper)
`keySet()`, `values()`, `entrySet()`	vues sur les clés, les valeurs, les couples

DemoMaps.java

```java
import java.util.*;

public class DemoMaps {
    record Vente(String quartier, String produit, long montant) { }

    public static void main(String[] args) {
        List<Vente> ventes = List.of(
            new Vente("Médina", "Bissap", 4_500),
            new Vente("Fann", "Pagne", 15_000),
            new Vente("Médina", "Pagne", 7_500),
            new Vente("Yoff", "Bissap", 3_000),
            new Vente("Médina", "Thiakry", 2_000),
            new Vente("Fann", "Bissap", 1_500)
        );

        // 1) Cumuler le chiffre d'affaires par quartier : merge
        Map<String, Long> caParQuartier = new TreeMap<>();      // TreeMap : clés triées
        for (Vente v : ventes) {
            caParQuartier.merge(v.quartier(), v.montant(), Long::sum);
        }
        System.out.println("CA par quartier : " + caParQuartier);

        // 2) Regrouper les produits vendus par quartier : computeIfAbsent
        Map<String, Set<String>> produitsParQuartier = new TreeMap<>();
        for (Vente v : ventes) {
            produitsParQuartier.computeIfAbsent(v.quartier(), q -> new TreeSet<>())
                    .add(v.produit());
        }
        System.out.println("Produits par quartier : " + produitsParQuartier);

        // 3) Parcourir les couples clé/valeur
        for (Map.Entry<String, Long> e : caParQuartier.entrySet()) {
            System.out.printf("  %-7s %6d FCFA%n", e.getKey(), e.getValue());
        }

        // 4) Lecture sûre
        System.out.println("CA Ouakam : " + caParQuartier.getOrDefault("Ouakam", 0L));
        Map<String, Long> fraisLivraison = Map.of("Médina", 1_000L, "Fann",
                1_000L, "Yoff", 1_500L);
        System.out.println("Frais Yoff : " + fraisLivraison.get("Yoff"));
    }
}
```

Sortie console

```
CA par quartier : {Fann=16500, Médina=14000, Yoff=3000}
Produits par quartier : {Fann=[Bissap, Pagne], Médina=[Bissap, Pagne, Thiakry], Yoff=[Bissap]}
  Fann     16500 FCFA
  Médina   14000 FCFA
  Yoff      3000 FCFA
CA Ouakam : 0
Frais Yoff : 1500
```

Piège à éviter — clés mutables

Ne modifiez jamais un objet **après** l'avoir utilisé comme clé d'une `HashMap` ou élément d'un `HashSet` : son `hashCode` change, et la table ne le retrouve plus. Utilisez des clés immuables : `String`, `Long`, enums, records.

11.6 Files et piles : Queue, Deque, PriorityQueue

- `ArrayDeque` est l'implémentation recommandée à la fois pour une **file** FIFO (`offer`, `poll`, `peek`) et pour une **pile** LIFO (`push`, `pop`, `peek`). N'utilisez plus la vieille classe `Stack`.
- `PriorityQueue` sort toujours en premier l'élément **le plus prioritaire** selon un ordre (naturel ou donné par un `Comparator`).

FilesAttente.java

```java
import java.util.*;

public class FilesAttente {
    record Commande(String numero, boolean express, long montant) { }

    public static void main(String[] args) {
        Deque<String> file = new ArrayDeque<>();            // file FIFO
        file.offer("TS-1"); file.offer("TS-2"); file.offer("TS-3");
        System.out.println("Traitée : " + file.poll() + ", reste " + file);

        Deque<String> historique = new ArrayDeque<>();      // pile LIFO (annuler)
        historique.push("ajout Bissap");
        historique.push("ajout Pagne");
        historique.push("remise 10 %");
        System.out.println("Annuler : " + historique.pop() + ", puis " + historique.pop());

        // Les commandes express d'abord, puis les plus gros montants
        Comparator<Commande> priorite = Comparator.comparing(Commande::express).reversed()
                                                  .thenComparing(Commande::montant,
                                                                 Comparator.reverseOrder());
        PriorityQueue<Commande> aPreparer = new PriorityQueue<>(priorite);
        aPreparer.add(new Commande("TS-10", false, 50_000));
        aPreparer.add(new Commande("TS-11", true, 5_000));
        aPreparer.add(new Commande("TS-12", false, 90_000));
        aPreparer.add(new Commande("TS-13", true, 20_000));
        while (!aPreparer.isEmpty()) {
            Commande c = aPreparer.poll();
            System.out.println("Préparer " + c.numero() + (c.express() ? " (express)" : "")
                    + " " + c.montant());
        }
    }
}
```

Sortie console

```
Traitée : TS-1, reste [TS-2, TS-3]
Annuler : remise 10 %, puis ajout Pagne
Préparer TS-13 (express) 20000
Préparer TS-11 (express) 5000
Préparer TS-12 90000
Préparer TS-10 50000
```

11.7 Les *Sequenced Collections*

> **Nouveauté Java 21 — Sequenced Collections (JEP 431)**
>
> Les nouvelles interfaces `SequencedCollection`, `SequencedSet` et `SequencedMap` unifient l'accès au **premier** et au **dernier** élément des collections ordonnées (`List`, `Deque`, `LinkedHashSet`, `TreeSet`, `LinkedHashMap`, `TreeMap`…) : `getFirst()`, `getLast()`, `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, et `reversed()` qui renvoie une **vue** en ordre inverse. Les maps ordonnées gagnent `firstEntry()`, `lastEntry()`, `pollFirstEntry()`, `putFirst()`…

Sequenced.java

```java
import java.util.*;

public class Sequenced {
    public static void main(String[] args) {
        List<String> etapes = new ArrayList<>(List.of("Préparée", "Expédiée", "En route"));
        etapes.addLast("Livrée");
        // avant : get(size() - 1)
        System.out.println(etapes.getFirst() + " -> " + etapes.getLast());
        System.out.println("Ordre inverse : " + etapes.reversed());

        LinkedHashMap<String, Long> dernieresVentes = new LinkedHashMap<>();
        dernieresVentes.put("TS-201", 12_000L);
        dernieresVentes.put("TS-202", 4_500L);
        dernieresVentes.put("TS-203", 30_000L);
        System.out.println("Plus ancienne : " + dernieresVentes.firstEntry());
        System.out.println("Plus récente  : " + dernieresVentes.lastEntry());
        dernieresVentes.pollFirstEntry();
        System.out.println(dernieresVentes);
    }
}
```

Sortie console

```
Préparée -> Livrée
Ordre inverse : [Livrée, En route, Expédiée, Préparée]
Plus ancienne : TS-201=12000
Plus récente  : TS-203=30000
{TS-202=4500, TS-203=30000}
```

11.8 Trier : `Comparable` et `Comparator`

Il existe deux façons de définir l'ordre des objets :

- **`Comparable<T>`** : la classe définit elle-même son **ordre naturel** en implémentant `compareTo`. `String`, `Integer`, `LocalDate` sont `Comparable`. `TreeSet`, `TreeMap`, `Collections.sort(liste)` et `liste.sort(null)` utilisent cet ordre.
- **`Comparator<T>`** : un objet **externe** qui compare deux éléments. On peut en définir autant qu'on veut (par prix, par libellé, par date…), sans modifier la classe.

La méthode `compareTo(autre)` (ou `compare(a, b)`) renvoie un entier **négatif** si le premier élément doit venir avant, **zéro** s'ils sont équivalents, **positif** sinon.

Depuis Java 8, on construit les comparateurs par **composition**, avec des références de méthodes (détaillées au chapitre 12) :

Tris.java

```java
import java.text.Collator;
```

```java
import java.util.*;

public class Tris {
    record Produit(String libelle, String categorie, long prix) implements Comparable<Produit> {
        @Override
        public int compareTo(Produit autre) {              // ordre naturel : par prix
            return Long.compare(this.prix, autre.prix);
        }
    }

    public static void main(String[] args) {
        List<Produit> catalogue = new ArrayList<>(List.of(
            new Produit("Pagne wax", "Textile", 7_500),
            new Produit("Bissap 1 L", "Boisson", 1_500),
            new Produit("Boubou brodé", "Textile", 35_000),
            new Produit("Café Touba", "Boisson", 2_000),
            new Produit("Éventail en paille", "Artisanat", 3_000)
        ));

        Collections.sort(catalogue);                       // ordre naturel
        System.out.println("Par prix : " + libelles(catalogue));

        catalogue.sort(Comparator.comparing(Produit::prix).reversed());
        System.out.println("Prix décroissant : " + libelles(catalogue));

        catalogue.sort(Comparator.comparing(Produit::categorie)
                                 .thenComparing(Produit::prix, Comparator.reverseOrder()));
        System.out.println("Catégorie puis prix décroissant : " + libelles(catalogue));

        catalogue.sort(Comparator.comparing(Produit::libelle));
        System.out.println("Ordre Unicode : " + libelles(catalogue));
        Collator francais = Collator.getInstance(Locale.FRENCH);
        catalogue.sort(Comparator.comparing(Produit::libelle, francais));
        System.out.println("Ordre français : " + libelles(catalogue));
    }

    static List<String> libelles(List<Produit> ps) {
        List<String> r = new ArrayList<>();
        for (Produit p : ps) r.add(p.libelle());
        return r;
    }
}
```

Sortie console

```
Par prix : [Bissap 1 L, Café Touba, Éventail en paille, Pagne wax, Boubou brodé]
Prix décroissant : [Boubou brodé, Pagne wax, Éventail en paille, Café Touba, Bissap 1 L]
Catégorie puis prix décroissant : [Éventail en paille, Café Touba, Bissap 1 L, Boubou brodé, Pagne wax]
Ordre Unicode : [Bissap 1 L, Boubou brodé, Café Touba, Pagne wax, Éventail en paille]
Ordre français : [Bissap 1 L, Boubou brodé, Café Touba, Éventail en paille, Pagne wax]
```

Piège à éviter

N'écrivez pas `return this.prix - autre.prix;` dans un `compareTo` : la soustraction peut déborder et inverser le signe. Utilisez `Long.compare`, `Integer.compare` ou `Comparator.comparing`.

11.9 Choisir la bonne collection

Besoin	Choix	Complexité des opérations clés
Liste d'éléments, accès par indice	`ArrayList`	`get` O(1), `add` en fin O(1) amorti, `contains` O(n)
Test d'appartenance rapide, sans doublons	`HashSet`	`add`/`contains` O(1)

Besoin	Choix	Complexité des opérations clés
Idem en gardant l'ordre d'insertion	`LinkedHashSet`	O(1)
Ensemble trié, plages de valeurs	`TreeSet`	O(log n)
Recherche par clé	`HashMap`	`get`/`put` O(1)
Idem, ordre d'insertion (ex. cache LRU)	`LinkedHashMap`	O(1)
Clés triées, « première clé ≥ x »	`TreeMap`	O(log n)
File FIFO ou pile LIFO	`ArrayDeque`	O(1) aux deux extrémités
Toujours extraire le plus prioritaire	`PriorityQueue`	`poll` O(log n), `peek` O(1)
Accès concurrent depuis plusieurs threads	`ConcurrentHashMap`, `CopyOnWriteArrayList`	voir chapitre 14

Exercices du chapitre 11

Exercice 11.1 ★ — Manipulations de listes

À partir de la liste `["Awa", "moussa", "Fatou", "awa", "Ibrahima", "Fatou", "aminata"]` : normalisez les prénoms (majuscule initiale) ; supprimez les doublons en conservant l'ordre de première apparition ; triez par ordre alphabétique puis par longueur décroissante ; affichez le premier et le dernier élément avec les méthodes de Java 21.

Exercice 11.2 ★ — Fréquence des mots

Écrivez un programme qui compte le nombre d'occurrences de chaque mot d'un texte (en ignorant la casse et la ponctuation) avec une `HashMap` et `merge`. Affichez ensuite les 5 mots les plus fréquents, par fréquence décroissante puis par ordre alphabétique en cas d'égalité.

Exercice 11.3 ★★ — Annuaire des clients par quartier

Écrivez une classe `Annuaire` qui gère une `Map<String, SortedSet<String>>` associant chaque quartier à l'ensemble trié des noms de ses clients. Méthodes : `ajouter(nom, quartier)`, `retirer(nom)` (le client peut être dans n'importe quel quartier ; supprimez le quartier s'il devient vide), `clientsDe(quartier)`, `quartierLePlusActif()` et `toString()` (quartiers triés).

Exercice 11.4 ★★ — Service client

Simulez la file d'attente du service client de TerangaShop. Les demandes (`record Demande(int numero, String client, boolean clientPremium, int minuteArrivee)`) arrivent dans une `PriorityQueue` : les clients premium passent en premier, puis par ordre d'arrivée. Un conseiller traite une demande toutes les 3 minutes. Affichez l'ordre de traitement et le temps d'attente de chaque client, puis le temps d'attente moyen.

Exercice 11.5 ★★ — Chasse aux bogues

Chacun des extraits suivants contient un bogue. Expliquez-le et corrigez-le.

Java

```java
// (a)
List<String> zones = List.of("Plateau", "Médina");
zones.add("Fann");

// (b)
List<Integer> ids = new ArrayList<>(List.of(3, 7, 1));
ids.remove(3);

// (c)
for (String c : clients) { if (c.isBlank()) clients.remove(c); }

// (d)
Map<String, Integer> stock = new HashMap<>();
int s = stock.get("PRD-001");

// (e)
record Point(int x, int y) {}
Set<int[]> dejaVus = new HashSet<>();
dejaVus.add(new int[]{1, 2});
System.out.println(dejaVus.contains(new int[]{1, 2}));   // pourquoi false ?
```

Exercice 11.6 ★★★ — Gestion de stock avec historique

Écrivez une classe `GestionStock` qui enregistre des mouvements (`record Mouvement(String reference, int quantite, String motif)`, quantité positive pour une entrée et négative pour une sortie) dans une `Map<String, Deque<Mouvement>>`. Méthodes : `enregistrer(Mouvement m)` (refuse une sortie qui rendrait le stock négatif) ; `stock(String reference)` ; `annulerDernier(String reference)` (pile) ; `sousLeSeuil(int seuil)` qui renvoie une `SortedMap<String, Integer>` des produits dont le stock est inférieur au seuil ; `historique(String reference)` qui renvoie la liste des mouvements du plus récent au plus ancien (utilisez `reversed()`).

Chapitre 12 — Lambdas, Optional et Streams

Objectifs du chapitre

- Écrire des expressions lambda et des références de méthodes.
- Connaître les interfaces fonctionnelles standard de `java.util.function` et les composer.
- Représenter l'absence de valeur avec `Optional` sans `NullPointerException`.
- Traiter des collections de façon déclarative avec l'API Stream et les collecteurs.
- Utiliser les *Stream Gatherers* de Java 24.

12.1 Des classes anonymes aux lambdas

Au chapitre 7, nous avons passé un comportement (une façon d'envoyer une notification) sous forme d'objet. Avant Java 8, il fallait écrire une **classe anonyme**, très verbeuse. Une **expression lambda** est une fonction anonyme concise, utilisable partout où l'on attend une **interface fonctionnelle**, c'est-à-dire une interface qui ne possède qu'**une seule méthode abstraite**.

Java

```java
// Classe anonyme (avant Java 8)
Comparator<Produit> parPrix = new Comparator<Produit>() {
    @Override
    public int compare(Produit a, Produit b) {
        return Long.compare(a.prix(), b.prix());
    }
};

// Lambda (Java 8)
Comparator<Produit> parPrix = (a, b) -> Long.compare(a.prix(), b.prix());

// Référence de méthode combinée à une fabrique
Comparator<Produit> parPrix = Comparator.comparing(Produit::prix);
```

Syntaxe des lambdas

Forme	Exemple
Sans paramètre	`() -> System.out.println("Bonjour")`
Un paramètre (parenthèses facultatives)	`p -> p.prix() > 5_000`
Plusieurs paramètres	`(a, b) -> a + b`
Types explicites	`(Produit p, int q) -> p.prix() * q`
Corps en bloc	`p -> { long ttc = p.prix() * 118 / 100; return ttc; }`
Paramètre inutilisé (Java 22)	`(_, valeur) -> valeur * 2`

Une lambda peut utiliser les variables locales de la méthode englobante, à condition qu'elles soient **effectivement finales** (jamais réaffectées après leur initialisation).

Java

```java
long seuil = 5_000;
Predicate<Produit> cher = p -> p.prix() > seuil;   // OK : seuil n'est jamais modifié
seuil = 10_000;                                     // rendrait la lambda invalide : ERREUR
```

12.2 Les interfaces fonctionnelles standard

Le package `java.util.function` fournit les interfaces fonctionnelles dont on a besoin dans 95 % des cas. Inutile d'en créer de nouvelles.

Interface	Méthode	Rôle	Exemple
`Supplier<T>`	`T get()`	fournit une valeur	`() -> new ArrayList<>()`
`Consumer<T>`	`void accept(T t)`	consomme une valeur	`p -> System.out.println(p)`
`BiConsumer<T, U>`	`void accept(T t, U u)`	consomme deux valeurs	`(k, v) -> System.out.println(k + "=" + v)`
`Function<T, R>`	`R apply(T t)`	transforme T en R	`p -> p.libelle()`
`BiFunction<T, U, R>`	`R apply(T t, U u)`	combine deux valeurs	`(prix, qte) -> prix * qte`
`UnaryOperator<T>`	`T apply(T t)`	transforme T en T	`s -> s.toUpperCase()`
`BinaryOperator<T>`	`T apply(T a, T b)`	combine deux T	`Long::sum`
`Predicate<T>`	`boolean test(T t)`	condition	`p -> p.stock() > 0`
`BiPredicate<T, U>`	`boolean test(T t, U u)`	condition à deux arguments	`(p, max) -> p.prix() <= max`

Il existe aussi des versions **spécialisées** pour les types primitifs, qui évitent l'autoboxing : `IntPredicate`, `LongFunction<R>`, `ToLongFunction<T>`, `IntUnaryOperator`, `LongBinaryOperator`, etc.

Ces interfaces possèdent des méthodes `default` pour **composer** des fonctions :

Composition.java

```java
import java.util.List;
import java.util.function.*;

public class Composition {
    record Produit(String libelle, String categorie, long prix, int stock) { }

    public static void main(String[] args) {
        Predicate<Produit> enStock = p -> p.stock() > 0;
        Predicate<Produit> abordable = p -> p.prix() <= 5_000;
        Predicate<Produit> textile = p -> p.categorie().equals("Textile");

        Predicate<Produit> suggestion = enStock.and(abordable.or(textile))
                .and(Predicate.not(p -> p.libelle().isBlank()));

        Function<Long, Long> ttc = ht -> ht * 118 / 100;
        // arrondi au 500 supérieur
        Function<Long, Long> arrondi500 = m -> (m + 499) / 500 * 500;
        Function<Long, String> affichage = m -> m + " FCFA";
        Function<Long, String> prixAffiche = ttc.andThen(arrondi500).andThen(affichage);

        List<Produit> catalogue = List.of(
            new Produit("Bissap 1 L", "Boisson", 1_500, 40),
            new Produit("Boubou brodé", "Textile", 35_000, 2),
            new Produit("Sac cuir", "Artisanat", 25_000, 5),
            new Produit("Café Touba", "Boisson", 2_000, 0)
        );
        for (Produit p : catalogue) {
            if (suggestion.test(p)) {
                System.out.println(p.libelle() + " : " + prixAffiche.apply(p.prix()));
            }
        }
        Supplier<List<String>> fabrique = java.util.ArrayList::new;
        List<String> vide = fabrique.get();
        BinaryOperator<Long> somme = Long::sum;
```

```
        System.out.println(vide + " " + somme.apply(1_500L, 2_000L));
    }
}
```

Sortie console

```
Bissap 1 L : 2000 FCFA
Boubou brodé : 41500 FCFA
[] 3500
```

12.3 Les références de méthodes

Lorsqu'une lambda se contente d'appeler une méthode existante, on peut la remplacer par une **référence de méthode** avec l'opérateur `::`.

Sorte	Syntaxe	Lambda équivalente
Méthode `static`	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Méthode d'instance d'un objet donné	`System.out::println`	`x -> System.out.println(x)`
Méthode d'instance du paramètre	`String::toUpperCase`	`s -> s.toUpperCase()`
Méthode d'instance à deux paramètres	`String::compareTo`	`(a, b) -> a.compareTo(b)`
Constructeur	`ArrayList::new`	`() -> new ArrayList<>()`

12.4 `Optional` : représenter une valeur absente

Une méthode comme `trouverClientParTelephone` peut ne rien trouver. Renvoyer `null` oblige chaque appelant à penser à tester la nullité ; un oubli provoque une `NullPointerException`, souvent loin de la cause. `Optional<T>` (Java 8) est une « boîte » qui contient **zéro ou une** valeur, et qui rend l'absence **explicite dans le type de retour**.

Méthode	Rôle
`Optional.of(v)`, `Optional.ofNullable(v)`, `Optional.empty()`	création (`of` refuse `null`)
`isPresent()`, `isEmpty()`	tests
`orElse(defaut)`	valeur, ou `defaut` si vide (toujours évalué)
`orElseGet(() -> …)`	valeur, ou résultat du fournisseur (évalué seulement si vide)
`orElseThrow()`, `orElseThrow(() -> new XException())`	valeur, ou exception si vide
`map(f)`, `flatMap(f)`, `filter(p)`	transformations, sans jamais manipuler `null`
`ifPresent(action)`, `ifPresentOrElse(action, siVide)`	actions
`or(() -> autreOptional)`	alternative si vide (Java 9)

DemoOptional.java

```
import java.util.*;

public class DemoOptional {
    record Client(String nom, String telephone, String quartier) { }

    static final List<Client> CLIENTS = List.of(
        new Client("Awa Diop", "771234567", "Médina"),
        new Client("Moussa Ndiaye", "780001122", null)          // quartier non renseigné
```

```
    );
    static final Map<String, Long> FRAIS = Map.of("Médina", 1_000L, "Rufisque", 2_500L);

    static Optional<Client> trouverParTelephone(String tel) {
        for (Client c : CLIENTS) {
            if (c.telephone().equals(tel)) return Optional.of(c);
        }
        return Optional.empty();
    }

    static long fraisLivraison(String telephone) {
        return trouverParTelephone(telephone)
                // Optional<String> (vide si quartier null)
                .map(Client::quartier)
                // Optional<Long> (vide si zone inconnue)
                .map(FRAIS::get)
                .orElse(3_000L);                              // tarif par défaut
    }

    public static void main(String[] args) {
        System.out.println(fraisLivraison("771234567"));
        System.out.println(fraisLivraison("780001122"));
        System.out.println(fraisLivraison("700000000"));

        trouverParTelephone("771234567")
            .ifPresentOrElse(c -> System.out.println("Bonjour " + c.nom()),
                             () -> System.out.println("Client inconnu"));

        String nom = trouverParTelephone("700000000").map(Client::nom).orElse("invité");
        System.out.println("Nom affiché : " + nom);

        try {
            trouverParTelephone("700000000")
                .orElseThrow(() -> new NoSuchElementException("Aucun client pour 700000000"));
        } catch (NoSuchElementException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Sortie console

```
1000
3000
3000
Bonjour Awa Diop
Nom affiché : invité
Aucun client pour 700000000
```

Bonnes pratiques avec Optional

- Utilisez `Optional` **comme type de retour** d'une méthode qui peut ne rien trouver. Ne l'utilisez pas pour les attributs, les paramètres ou les éléments de collections.
- N'appelez jamais `get()` sans vérifier : c'est reproduire le problème de `null`. Préférez `orElse`, `orElseThrow`, `map`, `ifPresent`.
- Une méthode qui renvoie `Optional` ne doit **jamais** renvoyer `null`.
- Pour une collection, renvoyez une collection **vide**, pas un `Optional<List<…>>`.

12.5 L'API Stream

Un **stream** (flux) est une séquence d'éléments sur laquelle on enchaîne des opérations de façon **déclarative** : on décrit *ce que l'on veut* (« les libellés des produits en stock, triés par prix ») plutôt que *comment l'obtenir* (boucles, variables temporaires, `if`).

Un traitement par stream comporte trois parties :

1. une **source** : `collection.stream()`, `Stream.of(...)`, `Arrays.stream(t)`, `IntStream.range(0, 10)`, `Files.lines(chemin)`… ;
2. zéro ou plusieurs **opérations intermédiaires**, qui renvoient un nouveau stream : `filter`, `map`, `sorted`… ;
3. une **opération terminale**, qui déclenche le calcul et produit un résultat : `toList()`, `count()`, `sum()`, `collect(...)`, `forEach(...)`.



Figure 12.1 — Un pipeline de stream : source, opérations intermédiaires, opération terminale

Caractéristiques importantes :

- un stream **ne modifie pas** sa source ;
- les opérations intermédiaires sont **paresseuses** : rien n'est calculé tant qu'aucune opération terminale n'est appelée, et les éléments sont traités un par un à travers tout le pipeline ;
- un stream ne peut être **consommé qu'une seule fois**.

Java

```java
// Impératif
List<String> resultat = new ArrayList<>();
for (Produit p : catalogue) {
    if (p.stock() > 0) {
        resultat.add(p.libelle().toUpperCase());
    }
}
Collections.sort(resultat);

// Déclaratif avec un stream
List<String> resultat = catalogue.stream()
        .filter(p -> p.stock() > 0)
        .map(p -> p.libelle().toUpperCase())
        .sorted()
        .toList();
```

Opérations intermédiaires

Opération	Rôle
`filter(predicat)`	garde les éléments qui vérifient la condition
`map(fonction)`	transforme chaque élément
`mapToInt`, `mapToLong`, `mapToDouble`	transforme en flux de primitifs (qui offre `sum`, `average`…)

Opération	Rôle
`flatMap(fonction)`	transforme chaque élément en stream et « aplatit » le résultat
`distinct()`	supprime les doublons (`equals`)
`sorted()` / `sorted(comparateur)`	trie
`limit(n)` / `skip(n)`	garde les `n` premiers / ignore les `n` premiers
`takeWhile(p)` / `dropWhile(p)` (Java 9)	prend / ignore les éléments tant que la condition est vraie
`peek(action)`	observe les éléments au passage (débogage)
`gather(gatherer)` (Java 24)	opération intermédiaire personnalisée (section 12.7)

Opérations terminales

Opération	Résultat
`toList()` (Java 16)	liste **immuable**
`collect(collecteur)`	résultat construit par un collecteur (section 12.6)
`forEach(action)`	exécute une action sur chaque élément
`count()`	nombre d'éléments
`sum()`, `average()`, `min()`, `max()`, `summaryStatistics()`	sur les flux primitifs
`min(comp)`, `max(comp)`, `findFirst()`, `findAny()`	un `Optional`
`anyMatch(p)`, `allMatch(p)`, `noneMatch(p)`	un booléen (court-circuit)
`reduce(neutre, operation)`	combine tous les éléments en une valeur

DemoStreams.java

```
import java.util.*;
import java.util.stream.*;

public class DemoStreams {
    record Produit(String libelle, String categorie, long prix, int stock) { }
    record Commande(String numero, String client, List<String> articles) { }

    public static void main(String[] args) {
        List<Produit> catalogue = List.of(
            new Produit("Bissap 1 L", "Boisson", 1_500, 40),
            new Produit("Café Touba", "Boisson", 2_000, 0),
            new Produit("Pagne wax", "Textile", 7_500, 12),
            new Produit("Boubou brodé", "Textile", 35_000, 2),
            new Produit("Sac cuir", "Artisanat", 25_000, 5),
            new Produit("Panier tressé", "Artisanat", 6_000, 8)
        );

        List<String> disponibles = catalogue.stream()
                .filter(p -> p.stock() > 0)
                .sorted(Comparator.comparingLong(Produit::prix))
                .map(Produit::libelle)
                .toList();
        System.out.println("Disponibles par prix : " + disponibles);

        long valeurStock = catalogue.stream().mapToLong(p -> p.prix() * p.stock()).sum();
        System.out.println("Valeur du stock : " + valeurStock + " FCFA");

        OptionalDouble prixMoyenTextile = catalogue.stream()
                .filter(p -> p.categorie().equals("Textile"))
                .mapToLong(Produit::prix).average();
        System.out.println("Prix moyen textile : " + prixMoyenTextile.orElse(0));

        Optional<Produit> plusCher = catalogue.stream()
```

```
                .max(Comparator.comparingLong(Produit::prix));
        System.out.println("Plus cher : " + plusCher.map(Produit::libelle).orElse("-"));

        System.out.println("Rupture ? " + catalogue.stream().anyMatch(p -> p.stock() == 0));
        System.out.println("Tous < 50 000 ? " + catalogue.stream()
                .allMatch(p -> p.prix() < 50_000));

        List<String> categories = catalogue.stream().map(Produit::categorie)
                .distinct().toList();
        System.out.println("Catégories : " + categories);

        // flatMap : de la liste des commandes à la liste de tous les articles vendus
        List<Commande> commandes = List.of(
            new Commande("TS-1", "Awa", List.of("Bissap 1 L", "Pagne wax")),
            new Commande("TS-2", "Moussa", List.of("Sac cuir")),
            new Commande("TS-3", "Fatou", List.of("Bissap 1 L", "Bissap 1 L", "Café Touba"))
        );
        long nbBissap = commandes.stream()
                .flatMap(c -> c.articles().stream())
                .filter("Bissap 1 L"::equals)
                .count();
        System.out.println("Bissap vendus : " + nbBissap);

        // Sources numériques et reduce
        int sommeCarres = IntStream.rangeClosed(1, 5).map(i -> i * i).sum();
        long produit = LongStream.rangeClosed(1, 10).reduce(1, (a, b) -> a * b);
        System.out.println("Somme des carrés = " + sommeCarres + ", 10! = " + produit);
        String codes = Stream.iterate(1, n -> n <= 5, n -> n + 1)
                             .map(n -> "TS-%03d".formatted(n))
                             .collect(Collectors.joining(", ", "[", "]"));
        System.out.println(codes);
    }
}
```

Sortie console

```
Disponibles par prix : [Bissap 1 L, Panier tressé, Pagne wax, Sac cuir, Boubou brodé]
Valeur du stock : 393000 FCFA
Prix moyen textile : 21250.0
Plus cher : Boubou brodé
Rupture ? true
Tous < 50 000 ? true
Catégories : [Boisson, Textile, Artisanat]
Bissap vendus : 3
Somme des carrés = 55, 10! = 3628800
[TS-001, TS-002, TS-003, TS-004, TS-005]
```

12.6 Les collecteurs

La méthode `collect` reçoit un **collecteur** qui décrit comment accumuler les éléments. La classe `Collectors` en fournit de nombreux, qui se **combinent** entre eux.

Collecteur	Résultat
`toList()`, `toSet()`, `toCollection(TreeSet::new)`	une collection
`toMap(cle, valeur)` / `toMap(cle, valeur, fusion)`	une `Map` (la fonction de fusion gère les clés en double)
`joining(sep, prefixe, suffixe)`	une chaîne
`counting()`, `summingLong(f)`, `averagingLong(f)`	un nombre
`summarizingLong(f)`	min, max, somme, moyenne et nombre en une passe
`groupingBy(cle)` / `groupingBy(cle, collecteurAval)`	`Map<cle, List>` ou `Map<cle, résultat du collecteur aval>`

Collecteur	Résultat
`partitioningBy(predicat)`	`Map<Boolean, List>` en deux groupes
`mapping(f, aval)`, `filtering(p, aval)`	transformation ou filtre avant le collecteur aval
`teeing(c1, c2, fusion)` (Java 12)	deux collecteurs en une seule passe, puis fusion

Collecteurs.java

```
import java.util.*;
import java.util.stream.*;

public class Collecteurs {
    enum Mode { WAVE, ORANGE_MONEY, ESPECES }
    record Vente(String client, String quartier, String produit, Mode mode, long montant) { }

    public static void main(String[] args) {
        List<Vente> ventes = List.of(
            new Vente("Awa", "Médina", "Pagne wax", Mode.WAVE, 15_000),
            new Vente("Moussa", "Rufisque", "Sac cuir", Mode.ESPECES, 25_000),
            new Vente("Fatou", "Médina", "Bissap 1 L", Mode.ORANGE_MONEY, 4_500),
            new Vente("Awa", "Médina", "Café Touba", Mode.WAVE, 2_000),
            new Vente("Ibou", "Fann", "Boubou brodé", Mode.WAVE, 35_000),
            new Vente("Fatou", "Médina", "Pagne wax", Mode.WAVE, 7_500),
            new Vente("Moussa", "Rufisque", "Bissap 1 L", Mode.ESPECES, 1_500)
        );

        Map<String, Long> caParQuartier = ventes.stream()
                .collect(Collectors.groupingBy(Vente::quartier, TreeMap::new,
                                               Collectors.summingLong(Vente::montant)));
        System.out.println("CA par quartier : " + caParQuartier);

        Map<Mode, Long> nbParMode = ventes.stream()
                .collect(Collectors.groupingBy(Vente::mode, () -> new EnumMap<>(Mode.class),
                                               Collectors.counting()));
        System.out.println("Ventes par mode : " + nbParMode);

        Map<String, Set<String>> produitsParClient = ventes.stream()
                .collect(Collectors.groupingBy(Vente::client, TreeMap::new,
                         Collectors.mapping(Vente::produit,
                                 Collectors.toCollection(TreeSet::new))));
        System.out.println("Produits par client : " + produitsParClient);

        Map<Boolean, List<String>> grosPaniers = ventes.stream()
                .collect(Collectors.partitioningBy(v -> v.montant() >= 10_000,
                         Collectors.mapping(Vente::client, Collectors.toList())));
        System.out.println("Montant >= 10 000 : " + grosPaniers.get(true));

        LongSummaryStatistics stats = ventes.stream()
                .collect(Collectors.summarizingLong(Vente::montant));
        System.out.printf("Nb=%d, total=%d, moyenne=%.0f, min=%d, max=%d%n",
                stats.getCount(), stats.getSum(), stats.getAverage(),
                        stats.getMin(), stats.getMax());

        Map<String, Long> totalParClient = ventes.stream()
                .collect(Collectors.toMap(Vente::client, Vente::montant, Long::sum,
                        LinkedHashMap::new));
        System.out.println("Total par client : " + totalParClient);

        List<String> top3 = totalParClient.entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
                .limit(3)
                .map(e -> e.getKey() + " (" + e.getValue() + ")")
                .toList();
        System.out.println("Top 3 clients : " + top3);

        double partMobile = ventes.stream().collect(Collectors.teeing(
                Collectors.filtering(v -> v.mode() != Mode.ESPECES,
```

```
                        Collectors.summingLong(Vente::montant)),
                Collectors.summingLong(Vente::montant),
                (mobile, total) -> 100.0 * mobile / total));
        System.out.printf("Part des paiements mobiles : %.1f %%%n", partMobile);
    }
}
```

Sortie console

```
CA par quartier : {Fann=35000, Médina=29000, Rufisque=26500}
Ventes par mode : {WAVE=4, ORANGE_MONEY=1, ESPECES=2}
Produits par client : {Awa=[Café Touba, Pagne wax], Fatou=[Bissap 1 L, Pagne wax], Ibou=[Boubou brodé],
Moussa=[Bissap 1 L, Sac cuir]}
Montant >= 10 000 : [Awa, Moussa, Ibou]
Nb=7, total=90500, moyenne=12929, min=1500, max=35000
Total par client : {Awa=17000, Moussa=26500, Fatou=12000, Ibou=35000}
Top 3 clients : [Ibou (35000), Moussa (26500), Awa (17000)]
Part des paiements mobiles : 70.7 %
```

Côté backend

Les streams sont omniprésents dans la couche service : conversion d'entités en DTO (`entites.stream().map(ProduitReponse::depuis).toList()`), calculs de statistiques, filtrage, regroupement. Mais attention : si les données viennent d'une base, il est presque toujours **bien plus efficace** de filtrer, trier, regrouper et paginer **en SQL** (`WHERE`, `ORDER BY`, `GROUP BY`, `LIMIT`) que de charger toute la table en mémoire pour la traiter avec un stream.

12.7 Les *Stream Gatherers*

Nouveauté Java 24 — Stream Gatherers (JEP 485)

L'opération intermédiaire `gather(...)` permet d'appliquer des transformations que les opérations classiques ne savent pas exprimer, notamment celles qui ont besoin d'un **état** ou de **plusieurs éléments à la fois**. La classe `java.util.stream.Gatherers` en fournit plusieurs, prêts à l'emploi : `windowFixed(n)` (paquets de n éléments), `windowSliding(n)` (fenêtres glissantes), `fold(...)` (accumulation), `scan(...)` (accumulation avec résultats intermédiaires) et `mapConcurrent(n, f)` (transformation concurrente avec des threads virtuels).

DemoGatherers.java

```
import java.util.List;
import java.util.stream.Gatherers;

public class DemoGatherers {
    public static void main(String[] args) {
        List<Long> caJournalier = List.of(85_000L, 120_000L, 64_000L, 210_000L, 99_000L,
                45_000L, 180_000L);

        // Moyenne mobile sur 3 jours
        List<Long> moyenneMobile = caJournalier.stream()
                .gather(Gatherers.windowSliding(3))
                .map(fenetre -> fenetre.stream().mapToLong(Long::longValue)
                        .sum() / fenetre.size())
                .toList();
        System.out.println("Moyenne mobile 3 j : " + moyenneMobile);

        // Cumul progressif du chiffre d'affaires
        List<Long> cumul = caJournalier.stream()
                .gather(Gatherers.scan(() -> 0L, Long::sum))
                .toList();
        System.out.println("Cumul : " + cumul);
```

```
        // Découper une liste de commandes en lots de 3 pour l'envoi aux livreurs
        List<List<String>> lots = List.of("TS-1", "TS-2", "TS-3", "TS-4", "TS-5",
                "TS-6", "TS-7").stream()
                .gather(Gatherers.windowFixed(3))
                .toList();
        System.out.println("Lots : " + lots);
    }
}
```

Sortie console

```
Moyenne mobile 3 j : [89666, 131333, 124333, 118000, 108000]
Cumul : [85000, 205000, 269000, 479000, 578000, 623000, 803000]
Lots : [[TS-1, TS-2, TS-3], [TS-4, TS-5, TS-6], [TS-7]]
```

12.8 Pièges et bonnes pratiques

Pièges à éviter

- **Effets de bord** : une lambda passée à `map` ou `filter` ne doit pas modifier de variable ou de collection extérieure. Pour accumuler, utilisez un collecteur, pas `forEach(x -> liste.add(x))`.
- **Réutilisation** : un stream consommé ne peut pas être réutilisé (`IllegalStateException: stream has already been operated upon or closed`). Recréez-le à partir de la source.
- **Lisibilité** : un pipeline de quinze opérations imbriquées est illisible. Découpez-le, nommez les étapes intermédiaires ou extrayez des méthodes. Une boucle `for` simple reste parfois la meilleure solution.
- **Streams parallèles** : `parallelStream()` n'accélère que les traitements **longs, sans état partagé**, sur de gros volumes. Dans un serveur web qui traite déjà de nombreuses requêtes en parallèle, il est rarement utile et peut même dégrader les performances.

Exercices du chapitre 12

Exercice 12.1 ★ — Des classes anonymes aux lambdas

Réécrivez chacun des extraits suivants d'abord avec une lambda, puis, si possible, avec une référence de méthode.

Java

```
Runnable r = new Runnable() { public void run() { System.out.println("Livraison"); } };
Comparator<String> c = new Comparator<String>() {
    public int compare(String a, String b) { return a.length() - b.length(); } };
Function<String, Integer> f = new Function<String, Integer>() {
    public Integer apply(String s) { return Integer.parseInt(s); } };
Supplier<List<String>> s = new Supplier<List<String>>() {
    public List<String> get() { return new ArrayList<>(); } };
BiFunction<Long, Integer, Long> total = new BiFunction<Long, Integer, Long>() {
    public Long apply(Long prix, Integer qte) { return prix * qte; } };
```

Exercice 12.2 ★ — Filtres combinables

Pour la page « catalogue » de TerangaShop, écrivez des méthodes qui renvoient des `Predicate<Produit>` : `categorie(String c)`, `prixEntre(long min, long max)`, `enStock()`, `libelleContient(String motCle)` (sans tenir compte de la casse). Écrivez ensuite une méthode `List<Produit> rechercher(List<Produit> catalogue, List<Predicate<Produit>> criteres)` qui combine tous les critères avec `reduce(Predicate::and)`.

Exercice 12.3 ★★ — Optional en cascade

Un client possède éventuellement une adresse, qui possède éventuellement des coordonnées GPS. Modélisez `Client`, `Adresse` et `Coordonnees` avec des records dont les accesseurs « facultatifs » renvoient des `Optional` (par exemple `Optional<Adresse> adresse()`). Écrivez `double distanceDepuisEntrepot(Client c)` qui renvoie la distance, ou lève une exception explicite si l'information est manquante, **sans aucun test `!= null`** (utilisez `flatMap`).

Exercice 12.4 ★★ — Statistiques de promotion

Soit une liste d'étudiants (`record Etudiant(String nom, String filiere, double moyenne)`). Avec des streams, calculez : la moyenne générale ; les trois meilleurs étudiants ; le nombre d'étudiants par filière ; la meilleure moyenne de chaque filière (`groupingBy` + `maxBy`) ; la partition admis (moyenne ≥ 10) / ajournés ; une chaîne « Awa (15,5), Moussa (12,0)… » des admis, triés par moyenne décroissante.

Exercice 12.5 ★★ — Fréquence des mots, version stream

Réécrivez l'exercice 11.2 avec un seul pipeline de stream : `Arrays.stream(texte.split(...))`, `groupingBy(..., counting())`, puis tri des entrées et `limit(5)`. Comparez la lisibilité des deux versions.

Exercice 12.6 ★★★ — Rapport de ventes mensuel

À partir d'une liste de ventes (`record Vente(int jour, String quartier, String produit, String categorie, int quantite, long prixUnitaire, String mode)`) d'au moins 20 éléments que vous inventerez, produisez un rapport contenant : le chiffre d'affaires total et moyen par jour ; le chiffre d'affaires par catégorie, trié par ordre décroissant ; le produit le plus vendu (en quantité) de chaque quartier ; le meilleur jour ; la moyenne mobile sur 7 jours (avec `Gatherers.windowSliding`) ; les jours où le chiffre d'affaires a baissé par rapport à la veille (indice : `windowSliding(2)`). Présentez le rapport sous forme de texte aligné.

Chapitre 13 — Dates, fichiers, configuration et journalisation

Objectifs du chapitre

- Manipuler dates, heures, durées et fuseaux horaires avec l'API `java.time`.
- Lire et écrire des fichiers texte avec NIO.2 (`Path`, `Files`).
- Importer et exporter des données au format CSV.
- Externaliser la configuration d'une application.
- Journaliser correctement au lieu d'utiliser `System.out`.

13.1 Dates et heures avec `java.time`

Java 8 — L'API java.time (JSR 310)

Les anciennes classes `java.util.Date` et `Calendar` étaient mutables, mal conçues (les mois commençaient à 0 !) et peu sûres en environnement concurrent. Depuis Java 8, le package `java.time` offre des classes **immuables**, claires et complètes. N'utilisez plus `Date` ni `Calendar` dans du code neuf.

Classe	Représente	Exemple
`LocalDate`	une date sans heure ni fuseau	`2026-09-14` (date d'une commande)
`LocalTime`	une heure sans date	`10:30` (ouverture de la boutique)
`LocalDateTime`	date + heure, sans fuseau	`2026-09-14T10:30`
`ZonedDateTime`	date + heure + fuseau horaire	`2026-09-14T10:30Z[Africa/Dakar]`
`Instant`	un instant précis sur la ligne du temps (UTC)	horodatage d'un événement serveur
`Duration`	une durée en heures, minutes, secondes	`PT48H` (délai de livraison)
`Period`	une durée en années, mois, jours	`P1Y2M` (ancienneté d'un client)
`DayOfWeek`, `Month`, `YearMonth`	jour de la semaine, mois, mois d'une année	`FRIDAY`, `SEPTEMBER`, `2026-09`

DemoDates.java

```
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.time.temporal.ChronoUnit;
import java.time.temporal.TemporalAdjusters;
import java.util.Locale;

public class DemoDates {
    static final DateTimeFormatter FORMAT_FR =
            DateTimeFormatter.ofPattern("EEEE d MMMM yyyy 'à' HH'h'mm", Locale.FRENCH);

    /** Ajoute n jours ouvrés (la boutique ne livre pas le dimanche). */
    static LocalDate ajouterJoursOuvres(LocalDate date, int n) {
        LocalDate d = date;
        while (n > 0) {
            d = d.plusDays(1);
            if (d.getDayOfWeek() != DayOfWeek.SUNDAY) n--;
        }
        return d;
    }

    public static void main(String[] args) {
```

```
        LocalDateTime commande = LocalDateTime.of(2026, Month.SEPTEMBER, 12, 17, 45);
        System.out.println("Commande : " + commande);
        System.out.println("En français : " + commande.format(FORMAT_FR));

        LocalDate livraison = ajouterJoursOuvres(commande.toLocalDate(), 2);
        System.out.println("Livraison prévue : " + livraison + " ("
                + livraison.getDayOfWeek() + ")");

        LocalDate finRetour = livraison.plusDays(7);
        System.out.println("Retour possible jusqu'au : "
                + finRetour.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG)
                        .withLocale(Locale.FRENCH)));

        LocalDate inscription = LocalDate.parse("2024-06-30");
        Period anciennete = Period.between(inscription, commande.toLocalDate());
        System.out.println("Client depuis " + anciennete.getYears() + " an(s) et "
                + anciennete.getMonths() + " mois");
        System.out.println("Soit " + ChronoUnit.DAYS.between(inscription,
                commande.toLocalDate()) + " jours");

        LocalDateTime livreeLe = LocalDateTime.of(2026, 9, 15, 11, 20);
        Duration delai = Duration.between(commande, livreeLe);
        System.out.printf("Délai réel : %d h %02d min (%s)%n", delai.toHours(),
                delai.toMinutesPart(), delai);
        System.out.println("Dans les 48 h ? " + (delai.compareTo(Duration.ofHours(48)) <= 0));

        LocalDate premierVendredi = LocalDate.of(2026, 10, 1)
                .with(TemporalAdjusters.firstInMonth(DayOfWeek.FRIDAY));
        LocalDate finMois = commande.toLocalDate().with(TemporalAdjusters.lastDayOfMonth());
        System.out.println("Premier vendredi d'octobre : " + premierVendredi
                + ", fin du mois : " + finMois);
        System.out.println("2028 bissextile ? " + Year.of(2028).isLeap());

        ZonedDateTime dakar = commande.atZone(ZoneId.of("Africa/Dakar"));
        ZonedDateTime paris = dakar.withZoneSameInstant(ZoneId.of("Europe/Paris"));
        System.out.println("Dakar " + dakar.toLocalTime() + " = Paris " + paris.toLocalTime());
        System.out.println("Instant UTC : " + dakar.toInstant());
    }
}
```

Sortie console

```
Commande : 2026-09-12T17:45
En français : samedi 12 septembre 2026 à 17h45
Livraison prévue : 2026-09-15 (TUESDAY)
Retour possible jusqu'au : 22 septembre 2026
Client depuis 2 an(s) et 2 mois
Soit 804 jours
Délai réel : 65 h 35 min (PT65H35M)
Dans les 48 h ? false
Premier vendredi d'octobre : 2026-10-02, fin du mois : 2026-09-30
2028 bissextile ? true
Dakar 17:45 = Paris 19:45
Instant UTC : 2026-09-12T17:45:00Z
```

Symboles de format les plus utiles : `yyyy` (année), `MM` (mois en chiffres), `MMMM` (mois en toutes lettres), `dd` (jour), `EEEE` (jour de la semaine), `HH` (heure 0-23), `mm` (minutes), `ss` (secondes). Le texte littéral se place entre apostrophes.

Côté backend — dates et fuseaux

Un serveur peut tourner dans un fuseau différent de celui des utilisateurs. Règles d'or : **stockez les instants en UTC** (`Instant` en Java, `TIMESTAMP WITH TIME ZONE` en PostgreSQL), et ne convertissez vers le fuseau de l'utilisateur qu'**à l'affichage**. Utilisez `LocalDate` pour les dates « calendaires » sans heure (date de naissance, date d'échéance). Dans les API JSON, échangez les dates au format ISO 8601 (`2026-09-12T17:45:00Z`). Enfin, pour pouvoir tester du code qui dépend de « maintenant », injectez une `java.time.Clock` au lieu d'appeler directement `LocalDate.now()`.

13.2 Lire et écrire des fichiers avec NIO.2

Depuis Java 7, le package `java.nio.file` (NIO.2) offre une API simple et puissante. On représente un chemin par un objet `Path`, et la classe utilitaire `Files` fournit les opérations.

Opération	Méthode
Construire un chemin	`Path.of("donnees", "catalogue.csv")`, `chemin.resolve("sous-dossier")`
Tester l'existence	`Files.exists(p)`, `Files.isDirectory(p)`, `Files.isReadable(p)`
Créer	`Files.createDirectories(p)`, `Files.createFile(p)`, `Files.createTempDirectory("x")`
Lire tout le fichier	`Files.readString(p)` (Java 11), `Files.readAllLines(p)`
Lire ligne par ligne (gros fichiers)	`Files.lines(p)` (un stream, à fermer !), `Files.newBufferedReader(p)`
Écrire	`Files.writeString(p, texte)`, `Files.write(p, lignes)`, `Files.newBufferedWriter(p)`
Ajouter en fin de fichier	option `StandardOpenOption.APPEND` (avec `CREATE`)
Copier, déplacer, supprimer	`Files.copy`, `Files.move`, `Files.delete`, `Files.deleteIfExists`
Lister, parcourir	`Files.list(dossier)`, `Files.walk(dossier)` (streams, à fermer)
Taille, date de modification	`Files.size(p)`, `Files.getLastModifiedTime(p)`

Toutes ces méthodes utilisent l'encodage **UTF-8** par défaut (et depuis Java 18, UTF-8 est aussi l'encodage par défaut de toute la plate-forme). Elles lèvent des `IOException` **vérifiées**, qu'il faut intercepter ou déclarer.

Fil rouge TerangaShop — import et export CSV

`ImportCatalogue.java`

```
import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.file.*;
import java.util.*;
import java.util.stream.*;

public class ImportCatalogue {
    record Produit(String reference, String libelle, String categorie, long prix, int stock) {
        static Produit depuisCsv(String ligne) {
            String[] c = ligne.split(";", -1);
            if (c.length != 5) throw new IllegalArgumentException("5 champs attendus, "
                    + c.length + " trouvés");
            long prix = Long.parseLong(c[3].strip());
            if (prix <= 0) throw new IllegalArgumentException("prix négatif ou nul");
            return new Produit(c[0].strip(), c[1].strip(), c[2].strip(), prix,
                    Integer.parseInt(c[4].strip()));
        }
    }
```

```
public static void main(String[] args) throws IOException {
    Path dossier = Files.createTempDirectory("terangashop");
    Path catalogue = dossier.resolve("catalogue.csv");

    // 1) Écriture d'un fichier d'exemple (text block)
    Files.writeString(catalogue, """
            reference;libelle;categorie;prix;stock
            PRD-001;Bissap 1 L;Boisson;1500;40
            PRD-002;Café Touba 250 g;Boisson;2000;0
            PRD-014;Pagne wax 6 yards;Textile;7500;12
            PRD-020;Sac en cuir;Artisanat;25000;5
            PRD-021;Panier tressé;Artisanat;-6000;8
            PRD-030;Boubou brodé;Textile;35000
            """);

    // 2) Lecture ligne par ligne avec un stream (fermé par try-with-resources)
    List<Produit> produits = new ArrayList<>();
    List<String> erreurs = new ArrayList<>();
    try (Stream<String> lignes = Files.lines(catalogue)) {
        // saute l'en-tête
        List<String> toutes = lignes.skip(1).filter(l -> !l.isBlank()).toList();
        for (int i = 0; i < toutes.size(); i++) {
            try {
                produits.add(Produit.depuisCsv(toutes.get(i)));
            } catch (IllegalArgumentException e) {        // NumberFormatException en hérite
                erreurs.add("ligne " + (i + 2) + " : " + e.getMessage());
            }
        }
    }
    System.out.println(produits.size() + " produits importés, "
            + erreurs.size() + " erreur(s)");
    erreurs.forEach(e -> System.out.println("  " + e));

    // 3) Export d'un rapport par catégorie
    Map<String, Long> valeurParCategorie = produits.stream()
            .collect(Collectors.groupingBy(Produit::categorie, TreeMap::new,
                     Collectors.summingLong(p -> p.prix() * p.stock())));
    List<String> rapport = new ArrayList<>();
    rapport.add("categorie;valeur_stock_fcfa");
    valeurParCategorie.forEach((cat, val) -> rapport.add(cat + ";" + val));
    Path fichierRapport = dossier.resolve("rapports").resolve("valeur-stock.csv");
    Files.createDirectories(fichierRapport.getParent());
    Files.write(fichierRapport, rapport);

    // 4) Relecture et informations sur le fichier
    System.out.println("Contenu de " + fichierRapport.getFileName() + " :");
    Files.readAllLines(fichierRapport).forEach(l -> System.out.println("  " + l));
    System.out.println("Taille : " + Files.size(fichierRapport) + " octets");

    // 5) Ajout en fin de fichier (journal des ruptures de stock)
    Path journal = dossier.resolve("ruptures.log");
    for (Produit p : produits) {
        if (p.stock() == 0) {
            Files.writeString(journal, p.reference() + " en rupture\n",
                    StandardOpenOption.CREATE, StandardOpenOption.APPEND);
        }
    }
    System.out.print("Journal : " + Files.readString(journal));

    // 6) Fichier absent
    try {
        Files.readString(dossier.resolve("absent.csv"));
    } catch (NoSuchFileException e) {
        System.out.println("Fichier introuvable : " + Path.of(e.getFile()).getFileName());
    }

    // 7) Nettoyage : suppression récursive du dossier temporaire
    try (Stream<Path> tout = Files.walk(dossier)) {
        tout.sorted(Comparator.reverseOrder()).forEach(p -> {
```

```
                try { Files.delete(p); } catch (IOException e)
                        { throw new UncheckedIOException(e); }
            });
        }
        System.out.println("Dossier supprimé : " + !Files.exists(dossier));
    }
}
```

Sortie console

```
4 produits importés, 2 erreur(s)
  ligne 6 : prix négatif ou nul
  ligne 7 : 5 champs attendus, 4 trouvés
Contenu de valeur-stock.csv :
  categorie;valeur_stock_fcfa
  Artisanat;125000
  Boisson;60000
  Textile;90000
Taille : 73 octets
Journal : PRD-002 en rupture
Fichier introuvable : absent.csv
Dossier supprimé : true
```

Pièges à éviter

- `Files.lines` et `Files.walk` ouvrent le fichier ou le dossier : ils doivent être utilisés dans un *try-with-resources*, sinon le descripteur reste ouvert.
- `Files.readAllLines` et `Files.readString` chargent **tout** le fichier en mémoire : ne les utilisez pas pour un fichier de plusieurs centaines de mégaoctets.
- `split(";")` ignore les champs vides en fin de ligne : `"a;b;;".split(";")` donne 2 éléments. Passez `-1` en second argument pour les conserver.
- Un vrai fichier CSV peut contenir des séparateurs entre guillemets (`"Dakar; Sénégal"`). Pour des fichiers réels, utilisez une bibliothèque dédiée comme *Apache Commons CSV* ou *OpenCSV*.

13.3 Externaliser la configuration

Une application ne doit jamais contenir en dur les informations qui changent d'un environnement à l'autre : URL et mot de passe de la base de données, clés d'API des opérateurs de paiement, port du serveur. On les place dans un **fichier de configuration** ou dans des **variables d'environnement** (recommandé en production et avec Docker).

Configuration.java

```
import java.io.IOException;
import java.io.StringReader;
import java.util.Properties;

public class Configuration {
    public static void main(String[] args) throws IOException {
        // En pratique : props.load(Files.newBufferedReader(Path.of("application.properties")))
        String contenu = """
                # Configuration de TerangaShop
                app.nom=TerangaShop
                serveur.port=8080
                bd.url=jdbc:postgresql://localhost:5432/teranga_shop
                bd.utilisateur=teranga
                livraison.frais.defaut=1500
                """;
        Properties props = new Properties();
        props.load(new StringReader(contenu));
```

```
        String nom = props.getProperty("app.nom");
        int port = Integer.parseInt(props.getProperty("serveur.port", "8000"));
        long frais = Long.parseLong(props.getProperty("livraison.frais.defaut"));
        String mode = props.getProperty("app.mode", "developpement");   // valeur par défaut

        // Une variable d'environnement a priorité sur le fichier (utile en production)
        String motDePasse = System.getenv().getOrDefault("BD_MOT_DE_PASSE", "(non défini)");

        System.out.printf("%s démarre sur le port %d en mode %s%n", nom, port, mode);
        System.out.println("Base : " + props.getProperty("bd.url") + " (utilisateur "
                + props.getProperty("bd.utilisateur") + ")");
        System.out.println("Frais de livraison par défaut : " + frais);
        System.out.println("Mot de passe BD : " + motDePasse);
    }
}
```

Sortie console

```
TerangaShop démarre sur le port 8080 en mode developpement
Base : jdbc:postgresql://localhost:5432/teranga_shop (utilisateur teranga)
Frais de livraison par défaut : 1500
Mot de passe BD : (non défini)
```

Sécurité

Ne mettez **jamais** de mot de passe, de clé d'API ou de secret dans le code source ni dans un fichier de configuration versionné avec Git. Utilisez des variables d'environnement ou un gestionnaire de secrets. Ajoutez les fichiers de configuration locaux au `.gitignore`.

13.4 Journaliser au lieu d'afficher

Sur un serveur, personne ne regarde la console. `System.out.println` est donc inadapté : on utilise un **journal** (*log*) qui horodate chaque message, indique sa **sévérité** et peut être dirigé vers un fichier ou un système centralisé.

Niveau	Usage
`ERROR`	une opération a échoué et nécessite une intervention (paiement perdu, base inaccessible)
`WARNING` / `WARN`	situation anormale mais gérée (stock bientôt épuisé, nouvelle tentative)
`INFO`	événements métier importants (commande créée, application démarrée)
`DEBUG` / `TRACE`	détails pour le diagnostic, désactivés en production

Le JDK fournit `System.Logger` (Java 9), une façade simple :

Java

```
private static final System.Logger LOG = System.getLogger(ServiceCommande.class.getName());

LOG.log(System.Logger.Level.INFO, "Commande {0} créée pour {1}", numero, client);
LOG.log(System.Logger.Level.ERROR, "Échec du paiement " + reference, exception);
```

En entreprise, on utilise presque toujours la façade **SLF4J** avec l'implémentation **Logback** (c'est le choix par défaut de Spring Boot) :

Java

```java
private static final Logger log = LoggerFactory.getLogger(ServiceCommande.class);

// {} : pas de concaténation inutile
log.info("Commande {} créée pour {}", numero, client.nom());
log.warn("Stock faible pour {} : {} unités", reference, stock);
// l'exception en dernier argument
log.error("Échec du paiement {}", reference, exception);
```

> **Astuce**
>
> Ne journalisez jamais de données sensibles : mots de passe, numéros complets de carte, codes de confirmation, jetons d'authentification. Masquez les numéros de téléphone (fonction `masquerTelephone` du chapitre 3).

Exercices du chapitre 13

Exercice 13.1 ★ — Anniversaire

Écrivez un programme qui lit une date de naissance au format `jj/MM/aaaa`, puis affiche : l'âge exact (années, mois, jours) ; le jour de la semaine de la naissance, en français ; le nombre de jours avant le prochain anniversaire ; la date du prochain anniversaire en toutes lettres. Gérez une saisie invalide (`DateTimeParseException`).

Exercice 13.2 ★ — Calendrier des tournées

Les livreurs de TerangaShop desservent Rufisque et Diamniadio les mardis et vendredis. Écrivez une méthode `List<LocalDate> prochainesTournees(LocalDate depuis, int n)` qui renvoie les `n` prochaines dates de tournée, en excluant une liste de jours fériés passée en paramètre (par exemple le 4 avril, fête de l'Indépendance). Utilisez `TemporalAdjusters.next(...)` ou un `Stream.iterate`.

Exercice 13.3 ★★ — Respect des délais

À partir d'une liste de livraisons (`record Livraison(String numero, LocalDateTime commandee, LocalDateTime livree)`), calculez avec des streams : le délai moyen (en heures et minutes) ; le pourcentage de livraisons effectuées en moins de 48 h ; la liste des livraisons en retard, triées de la plus en retard à la moins en retard ; le délai moyen par jour de la semaine de commande.

Exercice 13.4 ★★ — Journal de caisse quotidien

Écrivez une classe `JournalCaisse` qui ajoute chaque vente dans un fichier `ventes-AAAA-MM-JJ.csv` (un fichier par jour, créé automatiquement avec une ligne d'en-tête), puis une méthode `long totalDuJour(LocalDate jour)` qui relit le fichier correspondant. Si le fichier n'existe pas, le total vaut 0. Testez sur trois jours différents avec un dossier temporaire.

Exercice 13.5 ★★ — Inventaire d'un dossier

Écrivez un programme qui reçoit un chemin de dossier en argument et affiche, grâce à `Files.walk` : le nombre de fichiers par extension (triés par nombre décroissant), la taille totale en Ko, les 5 fichiers les plus volumineux, et les fichiers modifiés depuis moins de 7 jours.

Exercice 13.6 ★★★ — Import configurable avec rapport d'erreurs

Écrivez un outil d'import de catalogue piloté par un fichier `import.properties` (dossier d'entrée, séparateur, encodage, dossier de sortie). L'outil traite **tous** les fichiers `.csv` du dossier d'entrée, valide chaque ligne (référence au format `PRD-000`, prix positif, stock entier positif ou nul, catégorie parmi une liste autorisée), fusionne les produits (en cas de doublon de référence, la dernière ligne gagne et un avertissement est journalisé), puis écrit `catalogue-consolide.csv` et `erreurs.csv` (fichier, numéro de ligne, message). Utilisez `System.Logger` pour tracer les étapes.

Chapitre 14 — Concurrence moderne et threads virtuels

Objectifs du chapitre

- Comprendre pourquoi un serveur est par nature concurrent.
- Créer des threads et identifier les problèmes de concurrence (conditions de course).
- Protéger les données partagées : `synchronized`, classes atomiques, collections concurrentes.
- Utiliser les exécuteurs, `Future` et `CompletableFuture`.
- Exploiter les **threads virtuels** de Java 21, `ScopedValue` (Java 25) et découvrir la concurrence structurée.

14.1 Pourquoi la concurrence en backend ?

Un serveur web reçoit des centaines de requêtes **en même temps** : Awa consulte le catalogue pendant que Moussa paie sa commande et que le livreur met à jour un statut. Le modèle classique d'un serveur Java est « **un thread par requête** » : chaque requête est traitée par un **thread**, un fil d'exécution indépendant qui partage la mémoire du programme avec les autres.

La plupart du temps, un thread de requête **attend** : la réponse de la base de données, celle de l'API de l'opérateur de paiement, l'écriture d'un fichier. Pendant ces attentes (on parle de traitements *I/O-bound*), le processeur pourrait servir d'autres requêtes. La concurrence permet donc d'augmenter le **débit** du serveur. Mais elle introduit une difficulté majeure : plusieurs threads qui modifient les **mêmes données** peuvent les corrompre.

14.2 Créer des threads

Un thread exécute une tâche décrite par l'interface fonctionnelle `Runnable` (méthode `void run()`). On le démarre avec `start()` (et **non** `run()`, qui exécuterait la tâche dans le thread courant), et on attend sa fin avec `join()`.

PremiersThreads.java

```java
public class PremiersThreads {
    public static void main(String[] args) throws InterruptedException {
        Runnable preparation = () -> {
            String nom = Thread.currentThread().getName();
            for (int i = 1; i <= 3; i++) {
                System.out.println(nom + " prépare le colis " + i);
            }
        };
        Thread t1 = new Thread(preparation, "Préparateur-A");
        // Java 21
        Thread t2 = Thread.ofPlatform().name("Préparateur-B").unstarted(preparation);
        t1.start();
        t2.start();
        t1.join();          // le thread main attend la fin de t1…
        t2.join();          // …puis celle de t2
        System.out.println("Tous les colis sont prêts");
    }
}
```

Exemple de sortie (varie d'une exécution à l'autre)

```
Préparateur-A prépare le colis 1
Préparateur-B prépare le colis 1
Préparateur-A prépare le colis 2
Préparateur-A prépare le colis 3
Préparateur-B prépare le colis 2
Préparateur-B prépare le colis 3
Tous les colis sont prêts
```

L'ordre des lignes des deux préparateurs **change d'une exécution à l'autre** : c'est l'ordonnanceur du système qui décide quand chaque thread s'exécute. Seule la dernière ligne est garantie en dernière position, grâce aux `join()`.

14.3 Les conditions de course

Lorsque plusieurs threads lisent et modifient une même variable sans précaution, le résultat dépend de l'entrelacement de leurs instructions : c'est une **condition de course** (*race condition*). L'instruction `compteur++` semble atomique, mais elle se décompose en trois étapes : lire, ajouter 1, écrire. Deux threads peuvent lire la même valeur et écrire tous les deux la même valeur + 1 : une incrémentation est **perdue**.

ConditionCourse.java

```java
import java.util.concurrent.atomic.AtomicLong;

public class ConditionCourse {
    static long ventesNonProtegees = 0;
    static long ventesSynchronisees = 0;
    static final AtomicLong ventesAtomiques = new AtomicLong();
    static final Object verrou = new Object();

    public static void main(String[] args) throws InterruptedException {
        Runnable caisse = () -> {
            for (int i = 0; i < 100_000; i++) {
                // non atomique : des mises à jour se perdent
                ventesNonProtegees++;
                // un seul thread à la fois dans ce bloc
                synchronized (verrou) {
                    ventesSynchronisees++;
                }
                ventesAtomiques.incrementAndGet();        // opération atomique sans verrou
            }
        };
        Thread[] caisses = new Thread[4];
        for (int i = 0; i < caisses.length; i++) {
            caisses[i] = new Thread(caisse);
            caisses[i].start();
        }
        for (Thread t : caisses) t.join();

        System.out.println("Attendu           : 400000");
        System.out.println("Non protégé       : " + ventesNonProtegees);
        System.out.println("synchronized      : " + ventesSynchronisees);
        System.out.println("AtomicLong        : " + ventesAtomiques.get());
    }
}
```

Exemple de sortie (varie d'une exécution à l'autre)

```
Attendu           : 400000
Non protégé       : 231458
synchronized      : 400000
AtomicLong        : 400000
```

La valeur « non protégée » est fausse et **différente à chaque exécution**. Ce type de bogue est redoutable : il n'apparaît parfois qu'en production, sous forte charge.

Les moyens de protection :

Outil	Principe	Usage
`synchronized` (bloc ou méthode)	exclusion mutuelle : un seul thread à la fois possède le verrou	sections critiques simples
`ReentrantLock`	verrou explicite, avec essai (`tryLock`) et délai	besoins avancés
`AtomicInteger`, `AtomicLong`, `LongAdder`	opérations atomiques matérielles, sans verrou	compteurs, statistiques
`volatile`	garantit la **visibilité** d'une écriture par les autres threads (pas l'atomicité)	drapeaux d'arrêt
collections concurrentes	structures conçues pour l'accès simultané	caches, files, index partagés
objets immuables	rien à protéger : aucun thread ne peut modifier l'objet	records, `String`, `List.of`

> **À retenir**
>
> La meilleure façon d'éviter les problèmes de concurrence est de **ne pas partager d'état modifiable**. Dans un backend bien conçu, les services sont sans état (*stateless*), les données transitent dans des objets immuables (records), et l'état partagé vit dans la base de données, qui gère elle-même la concurrence avec ses transactions (chapitre 16).

> **Piège à éviter — l'interblocage (*deadlock*)**
>
> Si le thread 1 détient le verrou A et attend le verrou B, pendant que le thread 2 détient B et attend A, les deux threads attendent **indéfiniment**. Pour l'éviter, acquérez toujours les verrous dans le **même ordre**, gardez les sections critiques courtes, et n'appelez pas de code inconnu en tenant un verrou.

14.4 Les exécuteurs et les `Future`

Créer et gérer soi-même des threads est fastidieux et coûteux. Un **exécuteur** (`ExecutorService`) gère un ensemble de threads (un *pool*) auxquels on **soumet des tâches**. Une tâche qui renvoie un résultat est un `Callable<T>` ; la soumission renvoie un `Future<T>`, qui permet de récupérer le résultat quand il est prêt.

Executeurs.java

```java
import java.util.*;
import java.util.concurrent.*;

public class Executeurs {
    /** Simule l'appel (lent) à l'API d'un transporteur. */
    static long tarifTransporteur(String transporteur,
            String quartier) throws InterruptedException {
        Thread.sleep(300);                                  // latence réseau simulée
        return switch (transporteur) {
            case "Yobanté Express" -> 1_500;
            case "Jakarta Moto" -> 1_200;
            default -> 2_000;
        } + (quartier.equals("Diamniadio") ? 1_000 : 0);
    }

    public static void main(String[] args) throws Exception {
        List<String> transporteurs = List.of("Yobanté Express",
                "Jakarta Moto", "Livraison Pro");
```

```
        long debut = System.nanoTime();
        // Depuis Java 19, ExecutorService est AutoCloseable : close() attend la fin des tâches
        try (ExecutorService pool = Executors.newFixedThreadPool(3)) {
            Map<String, Future<Long>> resultats = new LinkedHashMap<>();
            for (String t : transporteurs) {
                resultats.put(t, pool.submit(() -> tarifTransporteur(t, "Diamniadio")));
            }
            for (var e : resultats.entrySet()) {
                // get() attend
                System.out.println(e.getKey() + " : " + e.getValue().get() + " FCFA");
            }
        }
        long ms = (System.nanoTime() - debut) / 1_000_000;
        System.out.println("Trois appels de 300 ms en parallèle : moins d'une seconde ? "
                + (ms < 1_000));
    }
}
```

Sortie console

```
Yobanté Express : 2500 FCFA
Jakarta Moto : 2200 FCFA
Livraison Pro : 3000 FCFA
Trois appels de 300 ms en parallèle : moins d'une seconde ? true
```

14.5 Composer des traitements asynchrones : `CompletableFuture`

`Future.get()` bloque le thread appelant. `CompletableFuture` (Java 8) permet de **décrire** une chaîne de traitements asynchrones : transformer un résultat, combiner deux résultats, gérer les erreurs, imposer un délai maximal.

Méthode	Rôle
`supplyAsync(fournisseur, executeur)`	lance un calcul asynchrone qui produit une valeur
`thenApply(f)`	transforme le résultat
`thenCompose(f)`	enchaîne un autre traitement asynchrone
`thenCombine(autre, f)`	combine deux résultats indépendants
`allOf(...)`, `anyOf(...)`	attend tous / le premier
`exceptionally(f)`, `handle(f)`	récupère une erreur
`orTimeout(d, unite)`, `completeOnTimeout(v, d, unite)`	délai maximal (Java 9)
`join()`	attend et renvoie le résultat

Asynchrone.java

```
import java.util.concurrent.*;

public class Asynchrone {
    static void pause(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException e)
                { Thread.currentThread().interrupt(); }
    }
    static long prixProduit(String ref) { pause(200); return 7_500; }
    static long fraisLivraison(String quartier) { pause(300); return 1_500; }
    static int stockDisponible(String ref)
            { pause(100); throw new IllegalStateException("service de stock indisponible"); }

    public static void main(String[] args) {
        try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
            CompletableFuture<Long> prix =
                    CompletableFuture.supplyAsync(() -> prixProduit("PRD-014"), ex);
```

```
            CompletableFuture<Long> frais =
                    CompletableFuture.supplyAsync(() -> fraisLivraison("Médina"), ex);
            CompletableFuture<Integer> stock = CompletableFuture
                    .supplyAsync(() -> stockDisponible("PRD-014"), ex)
                    .exceptionally(erreur -> {
                        System.out.println("Repli : " + erreur.getCause().getMessage());
                        return -1;                                  // valeur de repli
                    });
            CompletableFuture<Long> delaiLent = CompletableFuture
                    .supplyAsync(() -> { pause(5_000); return 48L; }, ex)
                    // valeur par défaut après 500 ms
                    .completeOnTimeout(72L, 500, TimeUnit.MILLISECONDS);

            long total = prix.thenCombine(frais, Long::sum).join();
            System.out.println("Total à payer : " + total + " FCFA");
            System.out.println("Stock : " + (stock.join() < 0 ? "inconnu" : stock.join()));
            System.out.println("Délai annoncé : " + delaiLent.join() + " h");
        }
    }
}
```

Sortie console

```
Repli : service de stock indisponible
Total à payer : 9000 FCFA
Stock : inconnu
Délai annoncé : 72 h
```

14.6 Les threads virtuels

> **Nouveauté Java 21 — Threads virtuels (JEP 444)**
>
> Un thread « classique » (dit *de plateforme*) correspond à un thread du système d'exploitation : il consomme environ un mégaoctet de mémoire et sa création est coûteuse. Un serveur ne peut donc en entretenir que quelques centaines ou milliers. Un **thread virtuel** est géré par la JVM elle-même : il ne coûte que quelques centaines d'octets, et on peut en créer **des millions**. Lorsqu'il se bloque (attente réseau, `sleep`, lecture en base), la JVM le « démonte » de son thread porteur, qui peut aussitôt exécuter un autre thread virtuel.

Conséquence pour le développeur backend : on peut garder le style de programmation simple et lisible « un thread par requête », avec du code **bloquant** ordinaire, tout en atteignant la capacité de montée en charge des frameworks réactifs, beaucoup plus complexes à écrire.

ThreadsVirtuels.java

```
import java.time.Duration;
import java.util.concurrent.*;
import java.util.stream.IntStream;

public class ThreadsVirtuels {
    public static void main(String[] args) {
        int nbRequetes = 10_000;
        long debut = System.nanoTime();
        try (ExecutorService executeur = Executors.newVirtualThreadPerTaskExecutor()) {
            IntStream.range(0, nbRequetes).forEach(i ->
                executeur.submit(() -> {
                    Thread.sleep(Duration.ofSeconds(1));        // attente d'I/O simulée (1 s)
                    return i;
                })
            );
        }   // close() attend la fin des 10 000 tâches
        long ms = (System.nanoTime() - debut) / 1_000_000;
        System.out.println(nbRequetes + " requêtes d'une seconde traitées");
        System.out.println("Durée totale inférieure à 3 s ? " + (ms < 3_000));
```

```
        Thread virtuel = Thread.ofVirtual().name("commande-42").start(() ->
            System.out.println("Je suis " + Thread.currentThread().getName()
                               + ", virtuel ? " + Thread.currentThread().isVirtual()));
        try { virtuel.join(); } catch (InterruptedException e)
                { Thread.currentThread().interrupt(); }
    }
}
```

Sortie console

```
10000 requêtes d'une seconde traitées
Durée totale inférieure à 3 s ? true
Je suis commande-42, virtuel ? true
```

Avec un pool de 200 threads de plateforme, le même traitement durerait environ 10 000 / 200 = 50 secondes.

Règles d'utilisation des threads virtuels :

- ils sont faits pour les traitements qui **attendent** (I/O) ; pour du **calcul intensif** (CPU), ils n'apportent rien : le nombre de cœurs reste la limite ;
- on ne les met **jamais dans un pool** : on en crée un nouveau par tâche (`newVirtualThreadPerTaskExecutor`), c'est leur raison d'être ;
- pour limiter l'accès à une ressource rare (par exemple 20 connexions à la base), on utilise un `Semaphore`, pas un pool de threads ;
- depuis Java 24 (JEP 491), un thread virtuel bloqué dans un bloc `synchronized` ne monopolise plus son thread porteur ; les versions 21 à 23 recommandaient `ReentrantLock` dans ce cas.

> **Côté backend**
> Avec Spring Boot 3.2 et plus, une seule ligne de configuration fait traiter chaque requête HTTP par un thread virtuel : `spring.threads.virtual.enabled=true`. Tomcat, les appels JDBC et les clients HTTP en profitent sans aucune modification du code métier.

14.7 Les collections concurrentes

Les collections classiques (`ArrayList`, `HashMap`) ne sont **pas** conçues pour être modifiées par plusieurs threads à la fois. Le package `java.util.concurrent` fournit des alternatives :

Collection	Usage
`ConcurrentHashMap`	table associative partagée ; `merge`, `compute` et `computeIfAbsent` y sont **atomiques**
`CopyOnWriteArrayList`	liste très souvent lue et rarement modifiée (liste d'écouteurs, configuration)
`ArrayBlockingQueue`, `LinkedBlockingQueue`	files bloquantes pour le modèle **producteur / consommateur**
`ConcurrentLinkedQueue`	file non bloquante

ProducteurConsommateur.java

```
import java.util.Map;
import java.util.concurrent.*;

public class ProducteurConsommateur {
    record Commande(String numero, String quartier) { }
```

```
    // « pilule empoisonnée » : signal d'arrêt
    static final Commande FIN = new Commande("FIN", "");

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<Commande> file = new ArrayBlockingQueue<>(5);
        Map<String, Integer> parQuartier = new ConcurrentHashMap<>();
        String[] quartiers = {"Médina", "Fann", "Médina", "Yoff", "Médina", "Fann",
                "Rufisque", "Yoff"};

        Thread reception = Thread.ofVirtual().start(() -> {
            try {
                for (int i = 0; i < quartiers.length; i++) {
                    // bloque si la file est pleine
                    file.put(new Commande("TS-" + (i + 1), quartiers[i]));
                }
                file.put(FIN);
                file.put(FIN);                                  // une pilule par consommateur
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });

        Runnable preparateur = () -> {
            try {
                while (true) {
                    Commande c = file.take();                   // bloque si la file est vide
                    if (c == FIN) break;
                    parQuartier.merge(c.quartier(), 1, Integer::sum);   // atomique
                }
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        };
        Thread p1 = Thread.ofVirtual().start(preparateur);
        Thread p2 = Thread.ofVirtual().start(preparateur);
        reception.join(); p1.join(); p2.join();

        System.out.println("Commandes préparées par quartier : "
                + new java.util.TreeMap<>(parQuartier));
    }
}
```

Sortie console

```
Commandes préparées par quartier : {Fann=2, Médina=3, Rufisque=1, Yoff=2}
```

14.8 Transmettre un contexte : ScopedValue

Dans un serveur, on a souvent besoin de connaître, au fond d'une méthode, des informations liées à la **requête en cours** : l'identifiant de l'utilisateur connecté, un identifiant de corrélation pour les journaux. Les passer en paramètre à toutes les méthodes est pénible. On utilisait jusqu'ici `ThreadLocal`, une variable propre à chaque thread, mais elle est mutable, facile à oublier de nettoyer, et coûteuse avec des millions de threads virtuels.

> **Nouveauté Java 25 — Scoped Values (JEP 506)**
>
> Une `ScopedValue` est une valeur **immuable**, liée pour la durée d'un appel (`ScopedValue.where(CLE, valeur).run(...)`) et lisible (`CLE.get()`) par toutes les méthodes appelées pendant ce temps, dans ce thread et ses sous-tâches. À la sortie du bloc, la liaison disparaît automatiquement.

ContexteRequete.java

```
public class ContexteRequete {
    static final ScopedValue<String> UTILISATEUR = ScopedValue.newInstance();
    static final ScopedValue<String> ID_REQUETE = ScopedValue.newInstance();

    static void journaliser(String message) {                // aucun paramètre de contexte !
```

```
        System.out.printf("[%s][%s] %s%n", ID_REQUETE.get(),
                UTILISATEUR.isBound() ? UTILISATEUR.get() : "anonyme", message);
    }

    static void validerCommande(String numero) {
        journaliser("validation de la commande " + numero);
        verifierPaiement(numero);
    }

    static void verifierPaiement(String numero) {
        journaliser("paiement vérifié pour " + numero);
    }

    public static void main(String[] args) {
        ScopedValue.where(ID_REQUETE, "req-001").where(UTILISATEUR, "awa.diop")
                   .run(() -> validerCommande("TS-2026-000087"));
        ScopedValue.where(ID_REQUETE, "req-002")
                   .run(() -> journaliser("consultation du catalogue"));
        System.out.println("Hors requête, lié ? " + ID_REQUETE.isBound());
    }
}
```

Sortie console

```
[req-001][awa.diop] validation de la commande TS-2026-000087
[req-001][awa.diop] paiement vérifié pour TS-2026-000087
[req-002][anonyme] consultation du catalogue
Hors requête, lié ? false
```

14.9 En preview : la concurrence structurée

En preview (Java 21 à 27) — Structured Concurrency (JEP 505 en Java 25, JEP 533 en Java 27)

La **concurrence structurée** traite un groupe de sous-tâches concurrentes comme **une seule unité de travail**. Toutes les sous-tâches sont lancées dans une portée (`StructuredTaskScope`) délimitée par un bloc `try` : si l'une échoue, les autres sont annulées automatiquement, et aucune ne peut « s'échapper » du bloc. L'API est encore en *preview* ; elle nécessite `--enable-preview` et peut changer.

ConcurrenceStructuree.java

```
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.StructuredTaskScope.Subtask;

public class ConcurrenceStructuree {
    record Fiche(String client, long solde, int commandesEnCours) { }

    static String client(long id) throws InterruptedException {
        Thread.sleep(200);
        return "Awa Diop";
    }
    static long solde(long id) throws InterruptedException {
        Thread.sleep(300);
        return 42_000;
    }
    static int commandes(long id) throws InterruptedException {
        Thread.sleep(100);
        return 2;
    }

    static Fiche fiche(long id) throws InterruptedException {
        // par défaut : échec si une sous-tâche échoue
        try (var portee = StructuredTaskScope.open()) {
            Subtask<String> nom = portee.fork(() -> client(id));
            Subtask<Long> solde = portee.fork(() -> solde(id));
            Subtask<Integer> nb = portee.fork(() -> commandes(id));
```

```
            portee.join();                                          // attend les trois sous-tâches
            return new Fiche(nom.get(), solde.get(), nb.get());
        }
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println(fiche(1));
    }
}
```

Sortie console

```
Fiche[client=Awa Diop, solde=42000, commandesEnCours=2]
```

Terminal

```
javac --release 25 --enable-preview ConcurrenceStructuree.java
java --enable-preview ConcurrenceStructuree
```

Exercices du chapitre 14

Exercice 14.1 ★ — Observer l'ordonnancement

Créez trois threads nommés « Wave », « Orange Money » et « Espèces », qui affichent chacun cinq fois leur nom suivi d'un numéro, avec une pause aléatoire de 0 à 100 ms entre deux affichages. Exécutez le programme plusieurs fois. Que constatez-vous ? Modifiez-le pour que le message « Clôture de la caisse » s'affiche **toujours** en dernier.

Exercice 14.2 ★★ — Le compte partagé

Deux threads effectuent chacun 1 000 retraits de 100 FCFA sur un même `CompteMobile` crédité de 150 000 FCFA ; la méthode `retirer` vérifie d'abord le solde, puis le diminue. Montrez qu'on peut obtenir un solde final incorrect, voire négatif si l'on ajoute une petite pause entre la vérification et le retrait. Corrigez avec `synchronized`, puis proposez une seconde version avec `AtomicLong` et `compareAndSet` (ou `updateAndGet`).

Exercice 14.3 ★★ — Somme parallèle

Remplissez un tableau de 20 millions de `long` aléatoires. Calculez sa somme : (1) avec une boucle ; (2) en découpant le tableau en 8 morceaux soumis comme `Callable<Long>` à un `newFixedThreadPool(8)` ; (3) avec `Arrays.stream(t).parallel().sum()`. Mesurez les durées avec `System.nanoTime()` et commentez. Pourquoi les threads virtuels n'apporteraient-ils rien ici ?

Exercice 14.4 ★★ — Comparateur de transporteurs

Trois transporteurs répondent en un temps aléatoire compris entre 100 et 1 500 ms, et l'un d'eux échoue une fois sur quatre. Avec `CompletableFuture`, interrogez-les en parallèle et affichez l'offre la moins chère parmi celles qui ont répondu **en moins d'une seconde**. Si aucun n'a répondu, appliquez un tarif par défaut de 2 000 FCFA.

Exercice 14.5 ★★ — Chaîne de préparation

Mettez en place un système producteur / consommateur à deux étages : un thread de réception dépose des commandes dans une première `BlockingQueue` ; deux préparateurs les prennent, simulent la préparation (200 ms) et les déposent dans une seconde file ; un livreur les récupère et les « livre ». Utilisez

des threads virtuels et une pilule empoisonnée pour arrêter proprement chaque étage. Affichez le temps total pour 20 commandes.

Exercice 14.6 ★★★ — Banc d'essai : plateforme contre virtuels

Écrivez un programme qui simule 5 000 requêtes, chacune effectuant trois « appels » bloquants de 50 ms (base de données, service de stock, opérateur de paiement). Comparez la durée totale avec : un pool de 100 threads de plateforme ; un exécuteur de threads virtuels. Chaque requête doit journaliser son identifiant grâce à une `ScopedValue`. Enfin, limitez à 50 le nombre d'« appels base de données » simultanés avec un `Semaphore` et mesurez l'impact.

PARTIE IV — Java côté serveur

- Chapitre 15 — Organiser un projet : Maven et tests JUnit
- Chapitre 16 — Accéder à PostgreSQL avec JDBC
- Chapitre 17 — HTTP et API REST en Java pur
- Chapitre 18 — Introduction à Spring Boot 4

Chapitre 15 — Organiser un projet : Maven et tests JUnit

> **Objectifs du chapitre**
>
> - Structurer un projet Java professionnel avec Maven.
> - Comprendre le fichier `pom.xml`, les dépendances, leurs portées et le cycle de vie.
> - Écrire des tests unitaires avec JUnit (assertions, tests paramétrés, cycle de vie).
> - Isoler le code testé avec des doublures de test.
> - Découvrir le développement piloté par les tests (TDD).

15.1 Pourquoi un outil de construction ?

Jusqu'ici, nous avons compilé à la main avec `javac`. Dans un vrai projet, cela ne suffit plus :

- l'application dépend de **bibliothèques externes** (pilote PostgreSQL, pool de connexions, Jackson, Spring…), qui dépendent elles-mêmes d'autres bibliothèques ;
- il faut compiler, exécuter les tests, produire un fichier `.jar`, parfois une image Docker, **de la même façon** sur le poste de chaque développeur et sur le serveur d'intégration continue ;
- toute l'équipe doit partager la même structure de dossiers et les mêmes versions.

Un **outil de construction** (*build tool*) automatise tout cela à partir d'un fichier de description du projet. Les deux outils standard de l'écosystème Java sont **Maven** (fichier `pom.xml`, en XML, le plus répandu en entreprise) et **Gradle** (fichier `build.gradle.kts`, en Kotlin, utilisé notamment par Android). Nous utilisons Maven ; les concepts sont identiques avec Gradle.

15.2 La structure standard d'un projet Maven

Maven impose une structure de dossiers conventionnelle. Tous les développeurs Java la reconnaissent, et tous les IDE la comprennent.

```
terangashop/
├── pom.xml                          description du projet
├── mvnw, mvnw.cmd, .mvn/            Maven Wrapper (Maven sans installation)
├── src/
│   ├── main/
│   │   ├── java/                    code de l'application
│   │   │   └── sn/isep/terangashop/...
│   │   └── resources/               fichiers de configuration, scripts SQL…
│   │       └── application.properties
│   └── test/
│       ├── java/                    code des tests (même arborescence de packages)
│       │   └── sn/isep/terangashop/...
│       └── resources/
└── target/                          fichiers générés (classes, jar, rapports) — jamais versionné
```

15.3 Le fichier `pom.xml`

Le **POM** (*Project Object Model*) décrit le projet : son identité, sa version de Java, ses dépendances et ses plugins.

pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- Identité du projet : les « coordonnées » Maven (GAV) -->
    <groupId>sn.isep</groupId>
    <artifactId>terangashop</artifactId>
    <version>1.0.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.release>25</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <junit.version>6.1.0</junit.version>
    </properties>

    <dependencies>
        <!-- Pilote JDBC PostgreSQL : nécessaire seulement à l'exécution -->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <version>42.7.12</version>
            <scope>runtime</scope>
        </dependency>
        <!-- Pool de connexions -->
        <dependency>
            <groupId>com.zaxxer</groupId>
            <artifactId>HikariCP</artifactId>
            <version>7.1.0</version>
        </dependency>
        <!-- Tests unitaires : uniquement pour compiler et exécuter les tests -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.4</version>
            </plugin>
        </plugins>
    </build>
</project>
```

Chaque bibliothèque est identifiée par ses **coordonnées** : `groupId` (l'organisation), `artifactId` (le nom du module) et `version`. Maven télécharge automatiquement les dépendances depuis le dépôt central (*Maven Central*, consultable sur `central.sonatype.com`), ainsi que leurs propres dépendances (**dépendances transitives**), et les range dans un cache local (`~/.m2/repository`).

Une version se terminant par `-SNAPSHOT` désigne une version **en cours de développement** ; une version sans suffixe (`1.0.0`) est une version publiée, qui ne doit plus jamais changer.

Les portées (*scopes*) des dépendances

Portée	Disponible pour…	Exemple
`compile` (par défaut)	compiler, tester, exécuter ; incluse dans le livrable	HikariCP, Jackson, Spring
`runtime`	tester et exécuter, mais pas compiler	pilote JDBC (le code n'utilise que l'API `java.sql`)
`test`	compiler et exécuter les tests uniquement	JUnit, Mockito, AssertJ
`provided`	compiler ; fournie par l'environnement d'exécution	API Servlet dans un serveur d'applications

15.4 Les commandes Maven

Maven organise la construction en **phases** successives : exécuter une phase exécute aussi toutes les phases précédentes.

Commande	Effet
`mvn compile`	compile le code de `src/main/java` dans `target/classes`
`mvn test`	compile, puis exécute tous les tests ; échoue si un test échoue
`mvn package`	compile, teste, puis produit `target/terangashop-1.0.0-SNAPSHOT.jar`
`mvn verify`	ajoute les vérifications d'intégration (qualité, couverture…)
`mvn install`	copie le livrable dans le dépôt local `~/.m2`
`mvn clean`	supprime le dossier `target`
`mvn dependency:tree`	affiche l'arbre complet des dépendances (utile en cas de conflit de versions)
`mvn test -Dtest=CalculateurLivraisonTest`	exécute une seule classe de test

Terminal

```
# Créer un nouveau projet à partir d'un modèle (archétype)
mvn archetype:generate -DgroupId=sn.isep -DartifactId=terangashop \
    -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

# Enchaînement le plus courant
mvn clean package
java -cp target/terangashop-1.0.0-SNAPSHOT.jar sn.isep.terangashop.App

# Avec le Maven Wrapper, inutile d'installer Maven : le script le télécharge
./mvnw clean test          # Linux / macOS
mvnw.cmd clean test        # Windows
```

> **Astuce**
>
> Versionnez toujours le **Maven Wrapper** (`mvnw`, `mvnw.cmd` et le dossier `.mvn`) avec votre projet : chaque membre de l'équipe et le serveur d'intégration continue utilisent ainsi exactement la même version de Maven. Les projets générés par `start.spring.io` (chapitre 18) l'incluent d'office.

15.5 Pourquoi tester ?

Un **test automatisé** est un programme qui vérifie qu'une partie de votre code se comporte comme prévu. Écrire des tests coûte du temps au départ, mais :

- ils détectent les **régressions** : une modification qui casse un comportement existant est repérée immédiatement, et non par un client ;
- ils permettent de **refactoriser** (restructurer le code) sans crainte ;
- ils **documentent** précisément le comportement attendu, avec des exemples exécutables ;
- ils poussent à écrire du code **mieux découpé**, car un code difficile à tester est souvent un code mal conçu.

On distingue les **tests unitaires** (une classe ou une méthode isolée, très rapides, très nombreux), les **tests d'intégration** (plusieurs composants ensemble, par exemple le DAO et une vraie base PostgreSQL) et les **tests de bout en bout** (l'API complète). C'est la « pyramide des tests » : beaucoup de tests unitaires à la base, moins de tests d'intégration, peu de tests de bout en bout.

15.6 Écrire des tests avec JUnit

JUnit est le cadre de test standard en Java. Sa version actuelle est **JUnit 6** (septembre 2025, Java 17 minimum), qui reprend l'API « Jupiter » de JUnit 5 : les annotations et assertions présentées ici sont identiques dans les deux versions.

Voici la classe à tester : un calculateur de frais de livraison.

CalculateurLivraison.java

```java
import java.util.Map;

public class CalculateurLivraison {
    static final long SEUIL_GRATUITE = 50_000;
    static final long SUPPLEMENT_EXPRESS = 1_000;
    private static final Map<String, Long> TARIFS = Map.of(
        "PLATEAU", 1_000L, "MEDINA", 1_000L, "FANN", 1_000L,
        "OUAKAM", 1_500L, "PARCELLES ASSAINIES", 1_500L,
        "RUFISQUE", 2_500L, "DIAMNIADIO", 2_500L
    );

    public long frais(String quartier, long montantPanier, double poidsKg, boolean express) {
        if (montantPanier <= 0)
                throw new IllegalArgumentException("Montant invalide : " + montantPanier);
        if (poidsKg <= 0) throw new IllegalArgumentException("Poids invalide : " + poidsKg);
        Long base = TARIFS.get(quartier.strip().toUpperCase());
        if (base == null)
                throw new IllegalArgumentException("Zone non desservie : " + quartier);

        long frais = montantPanier >= SEUIL_GRATUITE ? 0 : base;
        // 500 FCFA par kg au-delà de 10 kg
        if (poidsKg > 10) frais += 500 * (long) Math.ceil(poidsKg - 10);
        if (express) frais += SUPPLEMENT_EXPRESS;
        return frais;
    }
}
```

Et sa classe de test, placée dans `src/test/java`, dans le **même package** :

CalculateurLivraisonTest.java

```java
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Calcul des frais de livraison")
```

```
class CalculateurLivraisonTest {

    private CalculateurLivraison calculateur;

    @BeforeEach                                    // exécuté avant CHAQUE test : isolation
    void setUp() {
        calculateur = new CalculateurLivraison();
    }

    @Test
    @DisplayName("Tarif de base pour une zone proche")
    void tarifDeBase() {
        // Arrange (préparer) — Act (agir) — Assert (vérifier)
        long frais = calculateur.frais("Plateau", 10_000, 2, false);
        assertEquals(1_000, frais);
    }

    @Test
    void livraisonGratuiteAuDelaDuSeuil() {
        assertEquals(0, calculateur.frais("Rufisque", 50_000, 3, false));
        assertEquals(2_500, calculateur.frais("Rufisque", 49_999, 3, false),
                     "juste sous le seuil, le tarif normal s'applique");
    }

    @Test
    void supplementsCumules() {
        long frais = calculateur.frais("Ouakam", 60_000, 12.5, true);
        // vérifie tout, puis signale TOUS les échecs
        assertAll("gratuité + poids + express",
            () -> assertTrue(frais > 0),
            () -> assertEquals(500 * 3 + 1_000, frais)
        );
    }

    @ParameterizedTest(name = "{0} -> {1} FCFA")
    @CsvSource({
        "Plateau,             1000",
        "'  fann ',           1000",
        "Parcelles Assainies, 1500",
        "DIAMNIADIO,          2500"
    })
    void tarifsParZone(String quartier, long attendu) {
        assertEquals(attendu, calculateur.frais(quartier, 5_000, 1, false));
    }

    @ParameterizedTest
    @ValueSource(strings = {"Saint-Louis", "Thiès", ""})
    void zoneNonDesservie(String quartier) {
        var e = assertThrows(IllegalArgumentException.class,
                             () -> calculateur.frais(quartier, 5_000, 1, false));
        assertTrue(e.getMessage().startsWith("Zone non desservie"));
    }

    @Nested
    @DisplayName("Paramètres invalides")
    class ParametresInvalides {
        @Test
        void montantNegatif() {
            assertThrows(IllegalArgumentException.class,
                    () -> calculateur.frais("Fann", -1, 1, false));
        }
        @Test
        void poidsNul() {
            assertThrows(IllegalArgumentException.class, () -> calculateur.frais("Fann",
                    1_000, 0, false));
        }
    }
}
```

Terminal

```
mvn test
```

Sortie console

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running Calcul des frais de livraison
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

L'essentiel de l'API JUnit

Élément	Rôle
`@Test`	marque une méthode de test (pas besoin qu'elle soit `public`)
`@DisplayName("…")`	nom lisible affiché dans les rapports
`@BeforeEach` / `@AfterEach`	exécutée avant / après **chaque** test
`@BeforeAll` / `@AfterAll`	exécutée une fois avant / après tous les tests de la classe (méthode `static`)
`@Nested`	regroupe des tests dans une classe interne
`@ParameterizedTest` + `@ValueSource`, `@CsvSource`, `@MethodSource`	même test exécuté avec plusieurs jeux de données
`@Disabled("raison")`	désactive temporairement un test
`assertEquals(attendu, obtenu)`	égalité (attention à l'ordre des arguments)
`assertTrue`, `assertFalse`, `assertNull`, `assertNotNull`	conditions simples
`assertThrows(Type.class, () -> …)`	vérifie qu'une exception est levée, et la renvoie
`assertAll(…)`	regroupe plusieurs assertions et rapporte tous les échecs
`assertTimeout(Duration, () -> …)`	vérifie une durée maximale

> **Les qualités d'un bon test (FIRST)**
>
> **F**ast (rapide), **I**ndependent (indépendant des autres tests et de leur ordre), **R**epeatable (même résultat à chaque exécution, sans dépendre de l'heure, du réseau ou d'un fichier local), **S**elf-validating (réussite ou échec sans interprétation humaine), **T**imely (écrit en même temps que le code). Un test vérifie **un comportement**, et son nom le décrit.

15.7 Isoler le code testé : les doublures de test

Un test unitaire ne doit pas envoyer de vrais SMS, appeler la vraie API Wave ou écrire dans la vraie base. Si le code dépend d'**interfaces** (chapitre 7), on peut lui fournir, pendant le test, une implémentation simplifiée : une **doublure de test** (*test double*). Une doublure qui mémorise les appels reçus pour qu'on puisse les vérifier s'appelle un **espion** (*spy*) ou un **mock**.

Notificateur.java

```
public interface Notificateur {
    void envoyer(String telephone, String message);
}
```

ServiceFidelite.java

```java
/** Attribue des points de fidélité et prévient le client à chaque palier de 100 points. */
public class ServiceFidelite {
    private final Notificateur notificateur;
    private final java.util.Map<String, Integer> points = new java.util.HashMap<>();

    public ServiceFidelite(Notificateur notificateur) {    // la dépendance est injectée
        this.notificateur = notificateur;
    }

    public int crediter(String telephone, long montantAchat) {
        int gagnes = (int) (montantAchat / 1_000);          // 1 point par tranche de 1 000 FCFA
        int avant = points.getOrDefault(telephone, 0);
        int apres = avant + gagnes;
        points.put(telephone, apres);
        if (apres / 100 > avant / 100) {
            notificateur.envoyer(telephone, "Félicitations ! Vous avez atteint "
                    + (apres / 100 * 100) + " points.");
        }
        return apres;
    }
}
```

ServiceFideliteTest.java

```java
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

class ServiceFideliteTest {

    /** Espion : une fausse implémentation qui enregistre les messages au lieu de les envoyer. */
    static class NotificateurEspion implements Notificateur {
        final List<String> messages = new ArrayList<>();
        @Override
        public void envoyer(String telephone, String message) {
            messages.add(telephone + " : " + message);
        }
    }

    @Test
    void pasDeNotificationSousLePalier() {
        var espion = new NotificateurEspion();
        var service = new ServiceFidelite(espion);
        assertEquals(45, service.crediter("771234567", 45_000));
        assertTrue(espion.messages.isEmpty());
    }

    @Test
    void notificationQuandOnFranchitUnPalier() {
        var espion = new NotificateurEspion();
        var service = new ServiceFidelite(espion);
        service.crediter("771234567", 80_000);
        // 80 + 30 = 110 points : palier de 100 franchi
        service.crediter("771234567", 30_000);
        assertEquals(List.of("771234567 : Félicitations ! Vous avez atteint 100 points."),
                espion.messages);
    }
}
```

> **Côté backend**
>
> En entreprise, on crée rarement ces doublures à la main : la bibliothèque **Mockito** les génère (`Notificateur mock = mock(Notificateur.class); … verify(mock).envoyer(eq("771234567"), anyString());`). **AssertJ** offre des assertions plus lisibles (`assertThat(frais).isEqualTo(1_000)`), et **Testcontainers** démarre une vraie base PostgreSQL dans Docker pour les tests d'intégration. Spring Boot intègre JUnit, Mockito et AssertJ dans sa dépendance `spring-boot-starter-test`.

15.8 Le développement piloté par les tests (TDD)

Le **TDD** (*Test-Driven Development*) inverse l'ordre habituel : on écrit d'abord un test qui échoue, puis le code minimal pour le faire passer, puis on améliore le code. C'est le cycle **Rouge → Vert → Refactoriser** :

1. **Rouge** : écrire un petit test pour le prochain comportement attendu. Il échoue (ou ne compile même pas).
2. **Vert** : écrire le code **le plus simple** qui fait passer ce test, sans anticiper.
3. **Refactoriser** : améliorer la lisibilité et la structure du code (et des tests), en gardant tous les tests au vert.

On répète ce cycle toutes les quelques minutes. Le code obtenu est testé par construction, et ses fonctionnalités sont exactement celles qui ont été demandées.

Exercices du chapitre 15

Exercice 15.1 ★ — Premier projet Maven

Créez un projet Maven `sn.isep:outils-caisse` avec l'archétype *quickstart*. Modifiez le `pom.xml` pour utiliser Java 25 et JUnit 6. Déplacez-y la classe `Tarification` du chapitre 5. Exécutez `mvn package`, puis lancez le programme avec `java -cp target/…jar`. Affichez l'arbre des dépendances avec `mvn dependency:tree` et expliquez d'où vient chaque bibliothèque.

Exercice 15.2 ★ — Tester une classe existante

Écrivez une classe de test complète pour la classe `Fraction` de l'exercice 6.3 : simplification, signe, opérations, égalité, dénominateur nul. Utilisez au moins un test paramétré avec `@CsvSource`.

Exercice 15.3 ★★ — Tests paramétrés et cas limites

Écrivez les tests de la méthode `estMobileValide` (chapitre 3) : au moins 6 numéros valides et 6 invalides avec `@ValueSource`, en pensant aux cas limites (indicatif `+221`, espaces, points, 8 ou 10 chiffres, préfixe `33` de ligne fixe, chaîne vide, `null`). Un de vos tests échoue-t-il ? Corrigez alors la méthode.

Exercice 15.4 ★★ — TDD : le calcul des points de fidélité

En suivant strictement le cycle Rouge / Vert / Refactoriser, développez une classe `CalculPoints` avec les règles suivantes, introduites **une par une** : 1 point par tranche complète de 1 000 FCFA ; points doublés le vendredi ; bonus de 50 points pour la première commande ; plafond de 500 points par commande. Conservez l'historique de vos étapes (par exemple avec des commits Git) et présentez-le.

Exercice 15.5 ★★★ — Tester un service avec des doublures

Reprenez le `ServiceCommande` de l'exercice 7.6 (ou écrivez-en un) qui dépend de trois interfaces : `CatalogueRepository` (trouver un produit, décrémenter le stock), `PasserellePaiement` (débiter un téléphone, peut échouer) et `Notificateur`. Écrivez à la main des doublures (un faux dépôt en mémoire, une passerelle configurable pour réussir ou échouer, un espion de notifications) et testez : la commande réussie ; le stock insuffisant (aucun paiement ne doit être tenté) ; le paiement refusé (le stock doit être restauré et le client prévenu).

Chapitre 16 — Accéder à PostgreSQL avec JDBC

Objectifs du chapitre

- Comprendre l'architecture de JDBC et le rôle du pilote.
- Se connecter à PostgreSQL et exécuter des requêtes avec `PreparedStatement`.
- Se protéger des injections SQL.
- Lire des résultats et les convertir en objets Java (records).
- Gérer les transactions, les traitements par lots et les clés générées.
- Organiser l'accès aux données avec le patron DAO et un pool de connexions HikariCP.

16.1 L'architecture de JDBC

JDBC (*Java Database Connectivity*) est l'API standard de Java pour dialoguer avec une base de données relationnelle. Elle est composée d'**interfaces** du package `java.sql` (`Connection`, `PreparedStatement`, `ResultSet`…). Chaque éditeur de base de données fournit un **pilote** (*driver*) qui implémente ces interfaces pour sa base. Votre code n'utilise que les interfaces : changer de base (PostgreSQL, MySQL, Oracle…) ne demande en principe que de changer de pilote et d'URL de connexion.

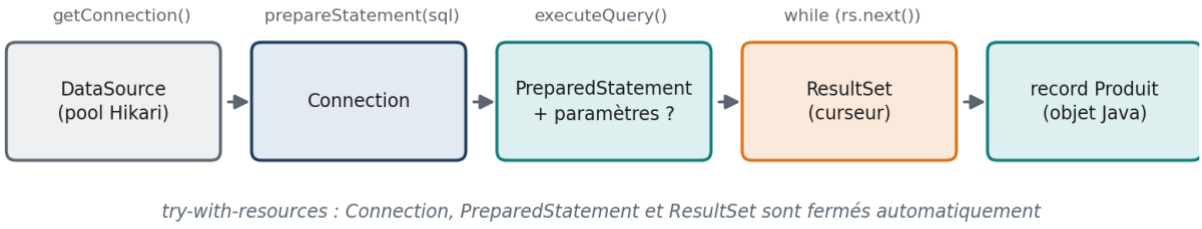


Figure 16.1 — Les objets JDBC, de la source de données jusqu'aux objets Java

Les types de pilotes : un peu d'histoire

Les anciens supports décrivaient quatre types de pilotes : la passerelle JDBC-ODBC (type 1), les pilotes à API native (type 2), les pilotes réseau via un intergiciel (type 3) et les pilotes « légers » 100 % Java (type 4). Aujourd'hui, **seul le type 4 est utilisé** : le pilote est un simple fichier `.jar` qui parle directement le protocole réseau de la base. La passerelle JDBC-ODBC a été supprimée du JDK en Java 8. De même, l'appel `Class.forName("org.postgresql.Driver")` est **inutile depuis JDBC 4.0** (Java 6) : les pilotes présents dans le *classpath* sont chargés automatiquement.

Interface ou classe	Rôle
`DriverManager`	fabrique de connexions simple (pour les petits programmes et les tests)
`DataSource`	fabrique de connexions configurable, le plus souvent avec un **pool** (usage en production)
`Connection`	une session avec la base ; gère les transactions
`Statement`	exécute une requête SQL **fixe** (à réserver au DDL)
`PreparedStatement`	exécute une requête **paramétrée**, précompilée et sûre
`ResultSet`	curseur sur les lignes renvoyées par une requête `SELECT`
`SQLException`	exception vérifiée levée en cas d'erreur, avec un code `SQLState` normalisé

16.2 Préparer la base

Installez PostgreSQL (version 14 ou plus), puis créez la base et l'utilisateur de l'application avec `psql` :

SQL

```sql
CREATE ROLE teranga LOGIN PASSWORD 'teranga';
CREATE DATABASE teranga_shop OWNER teranga;
```

Ajoutez ensuite le pilote à votre projet Maven (la version à jour se trouve sur Maven Central) :

XML

```xml
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.12</version>
</dependency>
```

Voici le schéma de TerangaShop. Observez les **contraintes** : ce sont elles qui garantissent la cohérence des données, quelle que soit l'application qui y accède.

schema.sql

```sql
CREATE TABLE client (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nom         VARCHAR(100) NOT NULL,
    telephone   VARCHAR(9)   NOT NULL UNIQUE,
    quartier    VARCHAR(60)  NOT NULL,
    cree_le     TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE produit (
    reference   VARCHAR(7)   PRIMARY KEY CHECK (reference ~ '^PRD-[0-9]{3}$'),
    libelle     VARCHAR(100) NOT NULL,
    categorie   VARCHAR(30)  NOT NULL,
    prix        BIGINT       NOT NULL CHECK (prix > 0),
    stock       INTEGER      NOT NULL CHECK (stock >= 0)
);

CREATE TABLE commande (
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    client_id      BIGINT      NOT NULL REFERENCES client(id),
    statut         VARCHAR(20) NOT NULL DEFAULT 'EN_ATTENTE',
    mode_paiement  VARCHAR(20) NOT NULL,
    total          BIGINT      NOT NULL CHECK (total >= 0),
    cree_le        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE ligne_commande (
    commande_id    BIGINT     NOT NULL REFERENCES commande(id) ON DELETE CASCADE,
    reference      VARCHAR(7) NOT NULL REFERENCES produit(reference),
    quantite       INTEGER    NOT NULL CHECK (quantite > 0),
    prix_unitaire  BIGINT     NOT NULL,
    PRIMARY KEY (commande_id, reference)
);
```

16.3 Se connecter

Une URL JDBC a la forme `jdbc:<sous-protocole>://<hôte>:<port>/<base>`. Pour notre base locale : `jdbc:postgresql://localhost:5432/teranga_shop`. Nous regroupons les paramètres de connexion dans une petite classe utilitaire. Conformément au chapitre 13, ils sont lus dans des **variables d'environnement**, avec des valeurs par défaut pour le poste de développement.

Bd.java

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/** Paramètres de connexion lus dans l'environnement (avec des valeurs par défaut). */
public final class Bd {
    static final String URL = env("BD_URL", "jdbc:postgresql://localhost:5432/teranga_shop");
    static final String UTILISATEUR = env("BD_UTILISATEUR", "teranga");
    static final String MOT_DE_PASSE = env("BD_MOT_DE_PASSE", "teranga");

    private Bd() { }                                  // classe utilitaire : pas d'instance

    static Connection connexion() throws SQLException {
        return DriverManager.getConnection(URL, UTILISATEUR, MOT_DE_PASSE);
    }

    private static String env(String cle, String defaut) {
        return System.getenv().getOrDefault(cle, defaut);
    }
}
```

Produit.java

```java
public record Produit(String reference, String libelle, String categorie, long prix, int stock)
        { }
```

Le premier programme crée le schéma (avec un `Statement`, qui convient pour du DDL sans paramètre) puis interroge les **métadonnées** de la base :

InitBase.java

```java
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class InitBase {
    static final String SCHEMA = """
        DROP TABLE IF EXISTS ligne_commande, commande, produit, client CASCADE;
        CREATE TABLE client (
            id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            nom VARCHAR(100) NOT NULL,
            telephone VARCHAR(9) NOT NULL UNIQUE,
            quartier VARCHAR(60) NOT NULL,
            cree_le TIMESTAMPTZ NOT NULL DEFAULT now());
        CREATE TABLE produit (
            reference VARCHAR(7) PRIMARY KEY CHECK (reference ~ '^PRD-[0-9]{3}$'),
            libelle VARCHAR(100) NOT NULL,
            categorie VARCHAR(30) NOT NULL,
            prix BIGINT NOT NULL CHECK (prix > 0),
            stock INTEGER NOT NULL CHECK (stock >= 0));
        CREATE TABLE commande (
            id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            client_id BIGINT NOT NULL REFERENCES client(id),
            statut VARCHAR(20) NOT NULL DEFAULT 'EN_ATTENTE',
            mode_paiement VARCHAR(20) NOT NULL,
            total BIGINT NOT NULL CHECK (total >= 0),
            cree_le TIMESTAMPTZ NOT NULL DEFAULT now());
        CREATE TABLE ligne_commande (
            commande_id BIGINT NOT NULL REFERENCES commande(id) ON DELETE CASCADE,
            reference VARCHAR(7) NOT NULL REFERENCES produit(reference),
            quantite INTEGER NOT NULL CHECK (quantite > 0),
            prix_unitaire BIGINT NOT NULL,
            PRIMARY KEY (commande_id, reference));
        """;

    public static void main(String[] args) throws SQLException {
        try (Connection cx = Bd.connexion();
```

```
             Statement st = cx.createStatement()) {
            st.execute(SCHEMA);

            DatabaseMetaData meta = cx.getMetaData();
            System.out.println("Base : " + meta.getDatabaseProductName()
                               + " / pilote : " + meta.getDriverName());
            List<String> tables = new ArrayList<>();
            try (ResultSet rs = meta.getTables(null, "public", "%", new String[]{"TABLE"})) {
                while (rs.next()) tables.add(rs.getString("TABLE_NAME"));
            }
            System.out.println("Tables créées : " + tables);
        }
    }
}
```

Sortie console

```
Base : PostgreSQL / pilote : PostgreSQL JDBC Driver
Tables créées : [client, commande, ligne_commande, produit]
```

Toujours fermer les ressources JDBC

`Connection`, `Statement`, `PreparedStatement` et `ResultSet` détiennent des ressources côté client **et** côté serveur. Déclarez-les **toujours** dans un *try-with-resources* (chapitre 9). Une connexion non fermée finit par épuiser le nombre maximal de connexions de PostgreSQL (100 par défaut), et l'application entière se bloque.

16.4 Insérer des données : `PreparedStatement` et traitement par lots

Un `PreparedStatement` contient une requête dont les valeurs variables sont remplacées par des **paramètres** `?`, numérotés **à partir de 1**. On leur donne une valeur avec les méthodes `setString`, `setLong`, `setInt`, `setObject`… puis on exécute :

- `executeUpdate()` pour `INSERT`, `UPDATE`, `DELETE` : renvoie le nombre de lignes modifiées ;
- `executeQuery()` pour `SELECT` : renvoie un `ResultSet`.

Pour insérer de nombreuses lignes, on accumule les exécutions avec `addBatch()` et on les envoie en **un seul aller-retour** réseau avec `executeBatch()` : c'est beaucoup plus rapide qu'une requête par ligne.

`ImportProduits.java`

```
import java.sql.*;
import java.util.List;

public class ImportProduits {
    public static void main(String[] args) throws SQLException {
        List<Produit> catalogue = List.of(
            new Produit("PRD-001", "Bissap 1 L", "Boisson", 1_500, 40),
            new Produit("PRD-002", "Café Touba 250 g", "Boisson", 2_000, 25),
            new Produit("PRD-003", "Thiakry 500 g", "Alimentaire", 1_000, 30),
            new Produit("PRD-014", "Pagne wax 6 yards", "Textile", 7_500, 12),
            new Produit("PRD-015", "Boubou brodé", "Textile", 35_000, 3),
            new Produit("PRD-020", "Sac en cuir de Ngaye", "Artisanat", 25_000, 5),
            new Produit("PRD-021", "Panier tressé", "Artisanat", 6_000, 8),
            new Produit("PRD-030", "Beurre de karité", "Cosmétique", 3_500, 2)
        );
        String sql = "INSERT INTO produit (reference, libelle, categorie, prix, stock) VALUES "
                + "(?, ?, ?, ?, ?)";
        try (Connection cx = Bd.connexion();
             PreparedStatement ps = cx.prepareStatement(sql)) {
            for (Produit p : catalogue) {
                ps.setString(1, p.reference());
```

```
                ps.setString(2, p.libelle());
                ps.setString(3, p.categorie());
                ps.setLong(4, p.prix());
                ps.setInt(5, p.stock());
                ps.addBatch();                          // mis en attente
            }
            int[] resultats = ps.executeBatch();        // un seul envoi
            System.out.println(resultats.length + " produits insérés");

            try (PreparedStatement maj = cx.prepareStatement(
                    "UPDATE produit SET prix = prix * (100 + ?) / 100 WHERE categorie = ?")) {
                maj.setInt(1, 10);                      // +10 %
                maj.setString(2, "Textile");
                int n = maj.executeUpdate();
                System.out.println(n + " produits textiles augmentés de 10 %");
            }
        }
    }
}
```

Sortie console

```
8 produits insérés
2 produits textiles augmentés de 10 %
```

16.5 Lire des données : `ResultSet`

Un `ResultSet` est un **curseur** positionné *avant* la première ligne. Chaque appel à `next()` avance d'une ligne et renvoie `false` quand il n'y en a plus. Les méthodes `getString("colonne")`, `getLong(...)`, `getInt(...)`, `getObject(..., LocalDate.class)` lisent les colonnes de la ligne courante. Utilisez de préférence le **nom** de la colonne plutôt que son numéro : c'est plus lisible et plus robuste.

La bonne pratique est de convertir chaque ligne en **objet Java** (ici un record) dès la lecture : le reste de l'application ne manipule plus jamais de `ResultSet`.

LireProduits.java

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class LireProduits {

    static Produit versProduit(ResultSet rs) throws SQLException {
        return new Produit(rs.getString("reference"), rs.getString("libelle"),
                           rs.getString("categorie"), rs.getLong("prix"), rs.getInt("stock"));
    }

    static List<Produit> parCategorie(Connection cx, String categorie) throws SQLException {
        String sql = """
            SELECT reference, libelle, categorie, prix, stock
            FROM produit
            WHERE categorie = ?
            ORDER BY prix DESC
            """;
        try (PreparedStatement ps = cx.prepareStatement(sql)) {
            ps.setString(1, categorie);
            try (ResultSet rs = ps.executeQuery()) {
                List<Produit> resultat = new ArrayList<>();
                while (rs.next()) resultat.add(versProduit(rs));
                return resultat;
            }
        }
    }
```

```
    public static void main(String[] args) throws SQLException {
        try (Connection cx = Bd.connexion()) {
            for (Produit p : parCategorie(cx, "Textile")) {
                System.out.printf("%s  %-20s %7d FCFA  stock %d%n", p.reference(), p.libelle(),
                        p.prix(), p.stock());
            }
            // Requête d'agrégation : le calcul est fait par la base, pas en Java
            String stats = """
                SELECT categorie, COUNT(*) AS nb, SUM(prix * stock) AS valeur
                FROM produit GROUP BY categorie ORDER BY valeur DESC
                """;
            try (PreparedStatement ps = cx.prepareStatement(stats);
                 ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    System.out.printf("%-12s %d produit(s), valeur du stock : %d FCFA%n",
                            rs.getString("categorie"), rs.getInt("nb"), rs.getLong("valeur"));
                }
            }
        }
    }
}
```

Sortie console

```
PRD-015  Boubou brodé           38500 FCFA  stock 3
PRD-014  Pagne wax 6 yards       8250 FCFA  stock 12
Textile      2 produit(s), valeur du stock : 214500 FCFA
Artisanat    2 produit(s), valeur du stock : 173000 FCFA
Boisson      2 produit(s), valeur du stock : 110000 FCFA
Alimentaire  1 produit(s), valeur du stock : 30000 FCFA
Cosmétique   1 produit(s), valeur du stock : 7000 FCFA
```

Piège à éviter — les valeurs NULL

Pour une colonne SQL `NULL`, `getLong` et `getInt` renvoient **0**, ce qui est trompeur. Utilisez `rs.getObject("colonne", Long.class)`, qui renvoie `null`, ou testez `rs.wasNull()` juste après la lecture.

Correspondance des types SQL et Java

Type PostgreSQL	Type Java	Méthode de lecture
`BIGINT`, `BIGSERIAL`	`long` / `Long`	`getLong`, `getObject(c, Long.class)`
`INTEGER`	`int` / `Integer`	`getInt`
`NUMERIC(p, s)`	`BigDecimal`	`getBigDecimal`
`VARCHAR`, `TEXT`	`String`	`getString`
`BOOLEAN`	`boolean`	`getBoolean`
`DATE`	`LocalDate`	`getObject(c, LocalDate.class)`
`TIMESTAMP`	`LocalDateTime`	`getObject(c, LocalDateTime.class)`
`TIMESTAMPTZ`	`OffsetDateTime` / `Instant`	`getObject(c, OffsetDateTime.class)`
`UUID`	`java.util.UUID`	`getObject(c, UUID.class)`

16.6 Les injections SQL

Une **injection SQL** se produit lorsqu'une valeur fournie par l'utilisateur est **concaténée** dans le texte d'une requête : l'utilisateur peut alors modifier la requête elle-même. C'est l'une des failles de sécurité les plus exploitées au monde (elle figure en bonne place dans le classement OWASP Top 10).

InjectionSql.java

```java
import java.sql.*;

public class InjectionSql {
    /** VULNÉRABLE : ne jamais écrire ce code ! */
    static int compterDangereux(Connection cx, String categorie) throws SQLException {
        String sql = "SELECT COUNT(*) FROM produit WHERE categorie = '" + categorie + "'";
        try (Statement st = cx.createStatement(); ResultSet rs = st.executeQuery(sql)) {
            rs.next();
            return rs.getInt(1);
        }
    }

    /** SÛR : la valeur est transmise séparément, comme une donnée, jamais comme du SQL. */
    static int compterSur(Connection cx, String categorie) throws SQLException {
        try (PreparedStatement ps = cx.prepareStatement("SELECT COUNT(*) FROM produit WHERE "
                + "categorie = ?")) {
            ps.setString(1, categorie);
            try (ResultSet rs = ps.executeQuery()) {
                rs.next();
                return rs.getInt(1);
            }
        }
    }

    public static void main(String[] args) throws SQLException {
        String saisieNormale = "Textile";
        String saisieMalveillante = "x' OR '1'='1";
        try (Connection cx = Bd.connexion()) {
            System.out.println("Dangereux, saisie normale      : "
                    + compterDangereux(cx, saisieNormale));
            System.out.println("Dangereux, saisie malveillante : " + compterDangereux(cx,
                    saisieMalveillante));
            System.out.println("Sûr,       saisie malveillante : " + compterSur(cx,
                    saisieMalveillante));
        }
    }
}
```

Sortie console

```
Dangereux, saisie normale      : 2
Dangereux, saisie malveillante : 8
Sûr,       saisie malveillante : 0
```

Avec la saisie malveillante, la requête dangereuse devient `... WHERE categorie = 'x' OR '1'='1'`, une condition toujours vraie : l'attaquant accède à **toutes** les lignes. Avec d'autres saisies, il pourrait lire la table des utilisateurs ou supprimer des données. Avec un `PreparedStatement`, la saisie est simplement comparée, caractère par caractère, aux valeurs de la colonne.

> **Règle absolue**
>
> **Toute** valeur qui provient de l'extérieur (formulaire, paramètre d'URL, corps JSON, fichier importé) doit passer par un paramètre `?` de `PreparedStatement`. Les noms de tables ou de colonnes ne peuvent pas être des paramètres : s'ils doivent varier (par exemple pour un tri choisi par l'utilisateur), comparez-les à une **liste blanche** de valeurs autorisées.

16.7 Clés générées et codes d'erreur

Lorsque la base génère l'identifiant (`GENERATED ALWAYS AS IDENTITY`), on le récupère avec l'option `Statement.RETURN_GENERATED_KEYS` et la méthode `getGeneratedKeys()`. Avec PostgreSQL, on peut aussi utiliser la clause `RETURNING id` et `executeQuery()`.

Chaque `SQLException` porte un code **`SQLState`** normalisé qui permet de réagir précisément : `23505` pour la violation d'une contrainte d'unicité, `23503` pour une clé étrangère, `23514` pour une contrainte `CHECK`, `40001` pour un conflit de sérialisation…

CreerClients.java

```java
import java.sql.*;

public class CreerClients {
    static long creer(Connection cx, String nom, String telephone,
            String quartier) throws SQLException {
        String sql = "INSERT INTO client (nom, telephone, quartier) VALUES (?, ?, ?)";
        try (PreparedStatement ps = cx.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, nom);
            ps.setString(2, telephone);
            ps.setString(3, quartier);
            ps.executeUpdate();
            try (ResultSet cles = ps.getGeneratedKeys()) {
                cles.next();
                return cles.getLong("id");
            }
        }
    }

    public static void main(String[] args) throws SQLException {
        try (Connection cx = Bd.connexion()) {
            System.out.println("Awa Diop      -> id " + creer(cx, "Awa Diop",
                    "771234567", "Médina"));
            System.out.println("Moussa Ndiaye -> id " + creer(cx, "Moussa Ndiaye",
                    "780001122", "Rufisque"));
            try {
                creer(cx, "Awa D.", "771234567", "Fann");          // même téléphone
            } catch (SQLException e) {
                if ("23505".equals(e.getSQLState())) {
                    System.out.println("Refusé : ce numéro de téléphone est déjà utilisé");
                } else {
                    throw e;
                }
            }
        }
    }
}
```

Sortie console

```
Awa Diop      -> id 1
Moussa Ndiaye -> id 2
Refusé : ce numéro de téléphone est déjà utilisé
```

16.8 Les transactions

Une **transaction** regroupe plusieurs opérations qui doivent réussir **toutes ensemble** ou **échouer toutes ensemble**. Passer une commande consiste à : décrémenter le stock de chaque produit, créer la commande, créer ses lignes. Si le deuxième produit est en rupture après que le stock du premier a été décrémenté, il faut **tout annuler**.

Les transactions garantissent les propriétés **ACID** : **A**tomicité (tout ou rien), **C**ohérence (les contraintes sont respectées), **I**solation (les transactions concurrentes ne se perturbent pas), **D**urabilité (une fois validées, les données survivent à une panne).

Par défaut, une connexion JDBC est en mode **auto-commit** : chaque requête est validée immédiatement. Pour une transaction, on désactive ce mode, puis on termine par `commit()` (valider) ou `rollback()` (annuler).

PasserCommande.java

```java
import java.sql.*;
import java.util.LinkedHashMap;
import java.util.Map;

public class PasserCommande {

    static class StockInsuffisantException extends RuntimeException {
        StockInsuffisantException(String ref) { super("Stock insuffisant pour " + ref); }
    }

    /** Crée une commande complète dans UNE transaction ; renvoie son identifiant. */
    static long passerCommande(Connection cx, long clientId, String mode, Map<String,
            Integer> articles)
            throws SQLException {
        boolean autoCommitInitial = cx.getAutoCommit();
        cx.setAutoCommit(false);                                    // début de la transaction
        try {
            Map<String, Long> prix = new LinkedHashMap<>();
            // 1) Décrément atomique : la condition "stock >= ?" évite toute vente à découvert,
            //    même si deux clients commandent le même produit au même instant.
            String sqlStock = "UPDATE produit SET stock = stock - ? WHERE reference = ? AND "
                    + "stock >= ? RETURNING prix";
            try (PreparedStatement ps = cx.prepareStatement(sqlStock)) {
                for (var a : articles.entrySet()) {
                    ps.setInt(1, a.getValue());
                    ps.setString(2, a.getKey());
                    ps.setInt(3, a.getValue());
                    try (ResultSet rs = ps.executeQuery()) {
                        if (!rs.next()) throw new StockInsuffisantException(a.getKey());
                        prix.put(a.getKey(), rs.getLong("prix"));
                    }
                }
            }
            long total = 0;
            for (var a : articles.entrySet()) total += prix.get(a.getKey()) * a.getValue();

            // 2) La commande
            long commandeId;
            try (PreparedStatement ps = cx.prepareStatement(
                    "INSERT INTO commande (client_id, mode_paiement, total) VALUES (?, ?, ?) "
                            + "RETURNING id")) {
                ps.setLong(1, clientId);
                ps.setString(2, mode);
                ps.setLong(3, total);
                try (ResultSet rs = ps.executeQuery())
                        { rs.next(); commandeId = rs.getLong(1); }
            }
            // 3) Les lignes, par lot
            try (PreparedStatement ps = cx.prepareStatement(
                    "INSERT INTO ligne_commande (commande_id, reference, quantite, "
                            + "prix_unitaire) VALUES (?, ?, ?, ?)")) {
                for (var a : articles.entrySet()) {
                    ps.setLong(1, commandeId);
                    ps.setString(2, a.getKey());
                    ps.setInt(3, a.getValue());
                    ps.setLong(4, prix.get(a.getKey()));
                    ps.addBatch();
                }
                ps.executeBatch();
            }
            cx.commit();                                            // tout est validé
            return commandeId;
        } catch (SQLException | RuntimeException e) {
            cx.rollback();                                          // tout est annulé
            throw e;
        } finally {
            cx.setAutoCommit(autoCommitInitial);
        }
    }
```

```
    static String stocks(Connection cx) throws SQLException {
        try (PreparedStatement ps = cx.prepareStatement(
                "SELECT reference, stock FROM produit WHERE reference IN ('PRD-001', "
                        + "'PRD-030') ORDER BY reference");
             ResultSet rs = ps.executeQuery()) {
            StringBuilder sb = new StringBuilder();
            while (rs.next()) sb.append(rs.getString(1)).append('=')
                    .append(rs.getInt(2)).append(' ');
            return sb.toString().strip();
        }
    }

    public static void main(String[] args) throws SQLException {
        try (Connection cx = Bd.connexion()) {
            System.out.println("Stocks avant         : " + stocks(cx));

            var panier1 = new LinkedHashMap<String, Integer>();
            panier1.put("PRD-001", 4);
            panier1.put("PRD-030", 1);
            long id = passerCommande(cx, 1, "WAVE", panier1);
            System.out.println("Commande " + id + " validée   : " + stocks(cx));

            var panier2 = new LinkedHashMap<String, Integer>();
            panier2.put("PRD-001", 2);                          // disponible...
            panier2.put("PRD-030", 5);                          // ...mais pas celui-ci
            try {
                passerCommande(cx, 2, "ESPECES", panier2);
            } catch (StockInsuffisantException e) {
                System.out.println("Échec (" + e.getMessage() + ")");
            }
            System.out.println("Après annulation     : " + stocks(cx));

            try (PreparedStatement ps = cx.prepareStatement("""
                    SELECT c.id, cl.nom, c.total, COUNT(l.reference) AS nb_lignes
                    FROM commande c
                    JOIN client cl ON cl.id = c.client_id
                    JOIN ligne_commande l ON l.commande_id = c.id
                    GROUP BY c.id, cl.nom, c.total
                    """);
                 ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    System.out.printf("Commande %d de %s : %d FCFA, %d ligne(s)%n",
                            rs.getLong("id"), rs.getString("nom"), rs.getLong("total"),
                                    rs.getInt("nb_lignes"));
                }
            }
        }
    }
}
```

Sortie console

```
Stocks avant         : PRD-001=40 PRD-030=2
Commande 1 validée   : PRD-001=36 PRD-030=1
Échec (Stock insuffisant pour PRD-030)
Après annulation     : PRD-001=36 PRD-030=1
Commande 1 de Awa Diop : 9500 FCFA, 2 ligne(s)
```

Lors de la seconde commande, le stock de `PRD-001` avait bien été décrémenté de 2 avant que l'on découvre la rupture de `PRD-030`. Le `rollback()` a annulé cette modification : le stock est revenu à 36.

Côté backend — concurrence et niveaux d'isolation

Deux clients peuvent commander le dernier boubou au même instant. L'instruction `UPDATE … WHERE stock >= ?` est **atomique** dans PostgreSQL : une seule des deux transactions réussira. C'est bien plus sûr que de lire le stock avec un `SELECT`, de le tester en Java, puis de l'écrire avec un `UPDATE` : entre les deux, un autre thread peut avoir modifié la valeur (condition de course, chapitre 14). Les niveaux d'isolation (`cx.setTransactionIsolation(...)` : `READ_COMMITTED` par défaut dans PostgreSQL, `REPEATABLE_READ`, `SERIALIZABLE`) permettent de régler finement ce comportement.

16.9 Le patron DAO et le pool de connexions

Le patron DAO

Écrire du JDBC dans le code métier mélange les responsabilités. Le patron **DAO** (*Data Access Object*), aussi appelé *Repository*, isole tout l'accès aux données derrière une **interface** : le service métier ne sait pas s'il parle à PostgreSQL, à une base en mémoire ou à une doublure de test (chapitre 15). Le DAO convertit aussi les `SQLException`, vérifiées et techniques, en une exception **non vérifiée** propre à l'application.

Le pool de connexions

Ouvrir une connexion à PostgreSQL est **coûteux** (connexion réseau, authentification, création d'un processus serveur) : plusieurs dizaines de millisecondes. Un serveur qui ouvrirait une connexion par requête HTTP serait lent et saturerait vite la base. Un **pool de connexions** ouvre un petit nombre de connexions au démarrage et les **prête** aux threads qui en ont besoin ; `close()` ne ferme alors pas vraiment la connexion, il la **rend** au pool. **HikariCP** est le pool de référence (c'est celui de Spring Boot). On l'utilise à travers l'interface standard `javax.sql.DataSource`.

XML

```xml
<dependency>
    <groupId>com.zaxxer</groupId>
    <artifactId>HikariCP</artifactId>
    <version>7.1.0</version>
</dependency>
```

AccesDonneesException.java

```java
/** Erreur technique d'accès aux données, non vérifiée. */
public class AccesDonneesException extends RuntimeException {
    public AccesDonneesException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

ProduitDao.java

```java
import java.util.List;
import java.util.Optional;

public interface ProduitDao {
    Optional<Produit> trouverParReference(String reference);
    List<Produit> trouverTous();
    List<Produit> trouverSousLeSeuil(int seuil);
    void enregistrer(Produit produit);            // insertion ou mise à jour
    boolean supprimer(String reference);
}
```

JdbcProduitDao.java

```java
import javax.sql.DataSource;
```

```java
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class JdbcProduitDao implements ProduitDao {
    private static final String COLONNES = "reference, libelle, categorie, prix, stock";
    private final DataSource dataSource;

    public JdbcProduitDao(DataSource dataSource) {          // la source de données est injectée
        this.dataSource = dataSource;
    }

    @Override
    public Optional<Produit> trouverParReference(String reference) {
        List<Produit> r = requete("SELECT " + COLONNES
                + " FROM produit WHERE reference = ?", reference);
        return r.stream().findFirst();
    }

    @Override
    public List<Produit> trouverTous() {
        return requete("SELECT " + COLONNES + " FROM produit ORDER BY reference");
    }

    @Override
    public List<Produit> trouverSousLeSeuil(int seuil) {
        return requete("SELECT " + COLONNES + " FROM produit WHERE stock < ? ORDER BY stock",
                seuil);
    }

    @Override
    public void enregistrer(Produit p) {
        String sql = """
            INSERT INTO produit (reference, libelle, categorie, prix, stock) VALUES (?, ?, ?, ?, ?)
            ON CONFLICT (reference) DO UPDATE
               SET libelle = EXCLUDED.libelle, categorie = EXCLUDED.categorie,
                   prix = EXCLUDED.prix, stock = EXCLUDED.stock
            """;
        miseAJour(sql, p.reference(), p.libelle(), p.categorie(), p.prix(), p.stock());
    }

    @Override
    public boolean supprimer(String reference) {
        return miseAJour("DELETE FROM produit WHERE reference = ?", reference) == 1;
    }

    // ------------------------------------------------------------------ outils privés
    private List<Produit> requete(String sql, Object... parametres) {
        try (Connection cx = dataSource.getConnection();              // emprunt au pool
             PreparedStatement ps = preparer(cx, sql, parametres);
             ResultSet rs = ps.executeQuery()) {
            List<Produit> resultat = new ArrayList<>();
            while (rs.next()) {
                resultat.add(new Produit(rs.getString("reference"), rs.getString("libelle"),
                        rs.getString("categorie"), rs.getLong("prix"), rs.getInt("stock")));
            }
            return resultat;
        } catch (SQLException e) {
            throw new AccesDonneesException("Échec de la requête : " + sql, e);
        // close() rend la connexion au pool
        }
    }

    private int miseAJour(String sql, Object... parametres) {
        try (Connection cx = dataSource.getConnection();
             PreparedStatement ps = preparer(cx, sql, parametres)) {
            return ps.executeUpdate();
        } catch (SQLException e) {
```

```
            throw new AccesDonneesException("Échec de la mise à jour : " + sql, e);
        }
    }

    private static PreparedStatement preparer(Connection cx, String sql,
            Object... parametres) throws SQLException {
        PreparedStatement ps = cx.prepareStatement(sql);
        for (int i = 0; i < parametres.length; i++) {
            ps.setObject(i + 1, parametres[i]);
        }
        return ps;
    }
}
```

DemoDao.java

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;

public class DemoDao {
    public static void main(String[] args) {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl(Bd.URL);
        config.setUsername(Bd.UTILISATEUR);
        config.setPassword(Bd.MOT_DE_PASSE);
        config.setMaximumPoolSize(10);                // au plus 10 connexions ouvertes
        config.setPoolName("terangashop-pool");

        try (HikariDataSource dataSource = new HikariDataSource(config)) {
            ProduitDao dao = new JdbcProduitDao(dataSource);

            // insertion
            dao.enregistrer(new Produit("PRD-040", "Savon au karité", "Cosmétique", 1_200, 50));
            // mise à jour
            dao.enregistrer(new Produit("PRD-030", "Beurre de karité",
                    "Cosmétique", 3_000, 20));
            System.out.println(dao.trouverParReference("PRD-030").orElseThrow());
            System.out.println("PRD-999 existe ? "
                    + dao.trouverParReference("PRD-999").isPresent());
            System.out.println("Sous le seuil de 10 :");
            dao.trouverSousLeSeuil(10).forEach(p -> System.out.println("  " + p.reference()
                    + " " + p.stock()));
            System.out.println("Supprimé : " + dao.supprimer("PRD-040") + ", total : "
                    + dao.trouverTous().size());
            try {
                // viole la contrainte CHECK
                dao.enregistrer(new Produit("MAUVAISE", "?", "?", 1, 1));
            } catch (AccesDonneesException e) {
                var cause = (java.sql.SQLException) e.getCause();
                System.out.println("Erreur d'accès aux données [SQLState "
                        + cause.getSQLState() + "]");
            }
        }
    }
}
```

Sortie console

```
Produit[reference=PRD-030, libelle=Beurre de karité, categorie=Cosmétique, prix=3000, stock=20]
PRD-999 existe ? false
Sous le seuil de 10 :
  PRD-015 3
  PRD-020 5
  PRD-021 8
Supprimé : true, total : 8
Erreur d'accès aux données [SQLState 22001]
```

Dans cette dernière ligne, la référence `MAUVAISE` (8 caractères) dépasse la taille de la colonne `VARCHAR(7)` : PostgreSQL la rejette avec le code `22001` (*string data right truncation*) avant même de vérifier la contrainte `CHECK`. La cause technique est conservée dans l'exception, ce qui permet de la journaliser.

> **Côté backend — dimensionner le pool**
>
> Un pool plus grand n'est pas plus rapide : au-delà du nombre de cœurs du serveur de base de données, les connexions se concurrencent. Une valeur de départ raisonnable est d'une dizaine de connexions par instance d'application. Avec les threads virtuels (chapitre 14), des milliers de requêtes peuvent attendre une connexion : c'est le pool qui protège la base.

16.10 Au-delà de JDBC

JDBC est la fondation de tout accès aux bases relationnelles en Java, mais on l'utilise rarement « à nu » dans une grande application. Les outils suivants s'appuient tous sur lui :

Outil	Principe
Spring `JdbcClient`	JDBC simplifié : plus de gestion manuelle des ressources ni des exceptions
JPA / Hibernate	*ORM* (*Object-Relational Mapping*) : les classes Java annotées sont automatiquement associées aux tables ; les requêtes simples sont générées
Spring Data JPA	génère l'implémentation des *repositories* à partir d'une simple interface (chapitre 18)
jOOQ	construction de requêtes SQL typées en Java, vérifiées à la compilation
MyBatis	requêtes SQL écrites dans des fichiers XML ou des annotations, correspondance automatique vers les objets
Flyway, **Liquibase**	versionnement et application automatique des scripts de schéma (*migrations*)

Exercices du chapitre 16

Exercice 16.1 ★ — Premières requêtes

Écrivez un programme qui affiche, pour la base TerangaShop : la liste des clients triés par nom ; le nombre de produits par catégorie ; le produit le plus cher de chaque catégorie (utilisez une requête SQL avec `DISTINCT ON` ou une sous-requête) ; les produits dont le libellé contient un mot saisi au clavier (`ILIKE ?` avec la valeur `"%" + mot + "%"`).

Exercice 16.2 ★ — Mise à jour sécurisée

Écrivez une méthode `int appliquerPromotion(Connection cx, String categorie, int pourcentage)` qui diminue les prix d'une catégorie. Refusez un pourcentage hors de [1, 50]. Le prix ne doit jamais descendre sous 100 FCFA (utilisez `GREATEST` en SQL). Affichez le nombre de produits modifiés.

Exercice 16.3 ★★ — DAO des clients

Écrivez l'interface `ClientDao` et son implémentation JDBC : `creer(Client)` (renvoie le client avec son identifiant généré), `trouverParId`, `trouverParTelephone`, `trouverParQuartier`, `changerQuartier(id, quartier)`. Traduisez la violation d'unicité du téléphone (`SQLState 23505`) en une exception métier `TelephoneDejaUtiliseException`. Testez avec une base de test.

Exercice 16.4 ★★ — Import massif

Générez un fichier CSV de 10 000 produits fictifs (références `PRD-000` à `PRD-999` réparties sur plusieurs fichiers ou élargissez la contrainte de la colonne). Importez-le de trois façons : une requête `INSERT` par ligne en auto-commit ; les mêmes requêtes dans une seule transaction ; un traitement par lots de 500 lignes. Mesurez et comparez les durées. Que concluez-vous ?

Exercice 16.5 ★★ — Annulation de commande

Écrivez une méthode transactionnelle `annulerCommande(Connection cx, long commandeId)` qui : vérifie que la commande existe et est `EN_ATTENTE` ou `PAYEE` (sinon exception) ; remet en stock les quantités de chaque ligne ; passe le statut à `ANNULEE`. Vérifiez qu'en cas d'erreur au milieu du traitement, aucune modification n'est conservée.

Exercice 16.6 ★★★ — Tableau de bord SQL

Écrivez une classe `StatistiquesDao` qui renvoie des records construits à partir de requêtes d'agrégation : chiffre d'affaires par jour sur les 30 derniers jours (`date_trunc`) ; top 5 des produits les plus vendus (en quantité et en montant) ; panier moyen par mode de paiement ; clients n'ayant passé aucune commande (`LEFT JOIN … WHERE … IS NULL`) ; taux d'annulation par quartier. Insérez d'abord un jeu de données réaliste d'au moins 50 commandes à l'aide d'un programme Java utilisant des traitements par lots.

Chapitre 17 — HTTP et API REST en Java pur

Objectifs du chapitre

- Rappeler le fonctionnement de HTTP et les principes d'une API REST.
- Appeler une API avec le client HTTP du JDK (`java.net.http.HttpClient`).
- Convertir des objets Java en JSON et inversement avec Jackson.
- Construire une petite API REST **sans framework**, organisée en couches, avec threads virtuels.
- Comprendre ce qu'apporte un framework comme Spring Boot.

17.1 HTTP en bref

HTTP (*HyperText Transfer Protocol*) est le protocole d'échange du Web. Un **client** (navigateur, application mobile, autre serveur) envoie une **requête** ; le **serveur** renvoie une **réponse**.

```
POST /api/commandes HTTP/1.1                    ← méthode, chemin, version
Host: api.terangashop.sn                        ← en-têtes
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...

{"clientId": 1, "articles": [{"reference": "PRD-001", "quantite": 4}]}   ← corps

HTTP/1.1 201 Created                            ← code de statut
Location: /api/commandes/87
Content-Type: application/json

{"id": 87, "statut": "EN_ATTENTE", "total": 7500}
```

Les méthodes

Méthode	Usage REST	Sûre (sans effet)	Idempotente
`GET`	lire une ressource ou une collection	oui	oui
`POST`	créer une ressource (ou déclencher une action)	non	non
`PUT`	remplacer entièrement une ressource	non	oui
`PATCH`	modifier partiellement une ressource	non	non (en général)
`DELETE`	supprimer une ressource	non	oui

Une méthode est **idempotente** si l'exécuter plusieurs fois a le même effet que l'exécuter une seule fois. C'est important quand le réseau est instable : un client peut renvoyer sans risque un `PUT` ou un `DELETE` dont il n'a pas reçu la réponse, mais pas un `POST` (il créerait deux commandes).

Les codes de statut

Famille	Codes courants
2xx Succès	`200 OK`, `201 Created` (avec l'en-tête `Location`), `204 No Content`
3xx Redirection	`301 Moved Permanently`, `304 Not Modified`
4xx Erreur du client	`400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`, `405 Method Not Allowed`, `409 Conflict`, `422 Unprocessable Content`, `429 Too Many Requests`

Famille	Codes courants
5xx Erreur du serveur	`500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`

Les principes REST

Une API **REST** (*Representational State Transfer*) organise l'application autour de **ressources** identifiées par des URI, manipulées avec les méthodes HTTP standard, et échangées dans un format de représentation (JSON le plus souvent).

Opération	Requête
Lister les produits (avec filtre et pagination)	`GET /api/produits?categorie=Textile&page=0&taille=20`
Lire un produit	`GET /api/produits/PRD-014`
Créer un produit	`POST /api/produits` avec le produit en JSON dans le corps
Remplacer un produit	`PUT /api/produits/PRD-014`
Modifier le stock	`PATCH /api/produits/PRD-014` avec `{"stock": 30}`
Supprimer un produit	`DELETE /api/produits/PRD-014`
Les commandes d'un client	`GET /api/clients/12/commandes`

Bonnes pratiques : des **noms** au pluriel (pas de verbes : `/api/produits`, pas `/api/getProduits`) ; des codes de statut exacts ; des erreurs au format JSON cohérent ; aucune session côté serveur (**sans état**, chaque requête porte toutes les informations nécessaires, par exemple un jeton d'authentification) ; une version d'API (`/api/v1/...`) lorsque des clients externes en dépendent.

17.2 JSON et Jackson

JSON (*JavaScript Object Notation*) est le format d'échange standard des API : objets `{ }`, tableaux `[ ]`, chaînes, nombres, booléens et `null`. Le JDK ne contient pas (encore) de bibliothèque JSON ; la référence est **Jackson**, utilisée par défaut par Spring Boot. Elle convertit automatiquement les objets Java, et en particulier les **records**, en JSON (*sérialisation*) et inversement (*désérialisation*).

XML

```xml
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.20.0</version>
</dependency>
```

DemoJson.java

```java
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import java.util.List;

public class DemoJson {
    record Ligne(String reference, int quantite) { }
    record CommandeRequete(long clientId, String modePaiement, List<Ligne> articles) { }

    public static void main(String[] args) throws Exception {
        ObjectMapper json = new ObjectMapper();

        // Java -> JSON
        var commande = new CommandeRequete(1, "WAVE", List.of(new Ligne("PRD-001", 4),
                new Ligne("PRD-030", 1)));
        System.out.println(json.writeValueAsString(commande));
```

```
        // JSON -> Java (le JSON reçu dans le corps d'une requête POST)
        String recu = """
            {"clientId": 2, "modePaiement": "ORANGE_MONEY",
             "articles": [{"reference": "PRD-014", "quantite": 2}]}
            """;
        CommandeRequete lue = json.readValue(recu, CommandeRequete.class);
        System.out.println(lue);
        System.out.println("Premier article : " + lue.articles().getFirst().reference());

        // Affichage indenté, pratique pour le débogage
        System.out.println(json.enable(SerializationFeature.INDENT_OUTPUT)
                .writeValueAsString(lue.articles()));
    }
}
```

Sortie console

```
{"clientId":1,"modePaiement":"WAVE","articles":[{"reference":"PRD-001","quantite":4},{"reference":"PRD-030","quantite":1}]}
CommandeRequete[clientId=2, modePaiement=ORANGE_MONEY, articles=[Ligne[reference=PRD-014, quantite=2]]]
Premier article : PRD-014
[ {
  "reference" : "PRD-014",
  "quantite" : 2
} ]
```

Jackson 3

Jackson 3 (octobre 2025), utilisé par défaut par Spring Boot 4, change le nom des packages (`tools.jackson.databind` au lieu de `com.fasterxml.jackson.databind`) et rend ses exceptions **non vérifiées**. Le principe d'utilisation reste le même : `ObjectMapper`/`JsonMapper`, `writeValueAsString` et `readValue`.

17.3 Appeler une API : `HttpClient`

Java 11 — Le client HTTP standard (JEP 321)

Le package `java.net.http` fournit un client HTTP moderne : API fluide, HTTP/2 par défaut, appels **synchrones** (`send`) ou **asynchrones** (`sendAsync`, qui renvoie un `CompletableFuture`), WebSocket. Java 26 y a ajouté la prise en charge de HTTP/3 (JEP 517).

Java

```java
HttpClient client = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(5))
        .build();

// GET synchrone
HttpRequest get = HttpRequest.newBuilder(URI.create("https://api.exemple.sn/taux?devise=EUR"))
        .header("Accept", "application/json")
        .timeout(Duration.ofSeconds(10))
        .GET()
        .build();
HttpResponse<String> reponse = client.send(get, HttpResponse.BodyHandlers.ofString());
System.out.println(reponse.statusCode() + " " + reponse.body());

// POST avec un corps JSON
HttpRequest post = HttpRequest.newBuilder(URI.create("https://api.exemple.sn/paiements"))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(json.writeValueAsString(demande)))
        .build();

// Appel asynchrone
client.sendAsync(get, HttpResponse.BodyHandlers.ofString())
      .thenApply(HttpResponse::body)
      .thenAccept(System.out::println);
```

Côté backend

Un backend appelle souvent d'autres API : opérateurs de paiement (Wave, Orange Money), envoi de SMS, géolocalisation, services internes. Fixez **toujours** des délais maximaux (`connectTimeout`, `timeout`), prévoyez des nouvelles tentatives limitées pour les erreurs temporaires (503), journalisez les échecs, et ne laissez jamais une panne d'un service externe bloquer toute votre application (délai, valeur de repli, disjoncteur).

17.4 Une API REST sans framework

Pour bien comprendre ce que font les frameworks, construisons une petite API de gestion du catalogue avec les seuls outils du JDK : le serveur HTTP intégré `com.sun.net.httpserver.HttpServer` (module `jdk.httpserver`), des threads virtuels, et Jackson pour le JSON. Le code est organisé en **couches** (chapitre 16) : un contrôleur qui gère HTTP, un service qui porte les règles métier, un dépôt qui stocke les données.

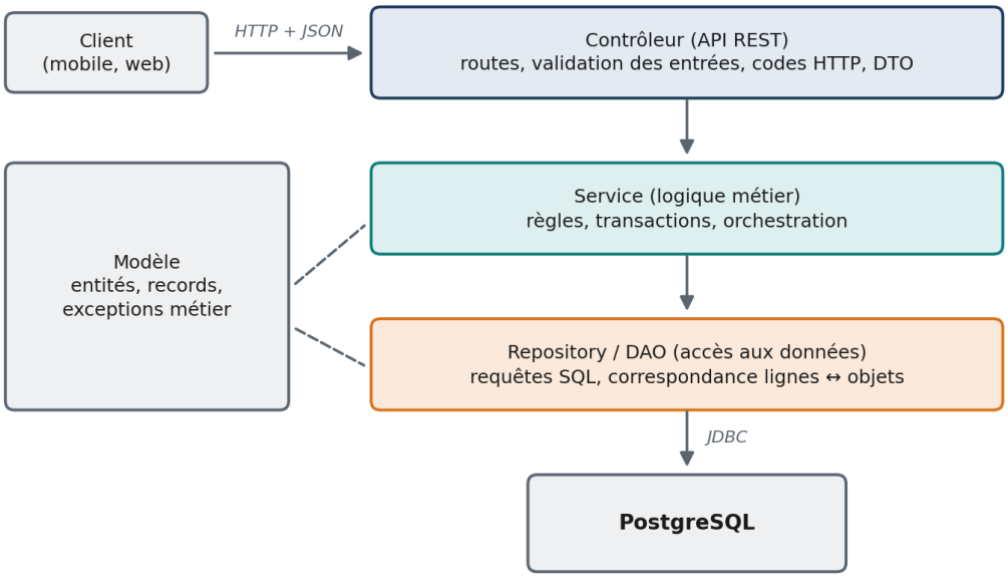


Figure 17.1 — L'architecture en couches d'un backend

MiniApi.java

```java
import com.fasterxml.jackson.databind.ObjectMapper;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.*;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;

public class MiniApi {
    // ---------------------------------------------------------------- modèle et erreurs
    record Produit(String reference, String libelle, long prix, int stock) { }
    record ErreurApi(int statut, String code, String message) { }

    static class ApiException extends RuntimeException {
        final int statut; final String code;
        ApiException(int statut, String code, String message)
                { super(message); this.statut = statut; this.code = code; }
    }

    // ---------------------------------------------------------------- couche service
    static class ProduitService {
        // dépôt en mémoire
        private final Map<String, Produit> depot = new ConcurrentHashMap<>();

        List<Produit> lister(Long prixMax) {
            return depot.values().stream()
                    .filter(p -> prixMax == null || p.prix() <= prixMax)
                    .sorted(Comparator.comparing(Produit::reference))
                    .toList();
        }
        Produit obtenir(String ref) {
            Produit p = depot.get(ref);
            if (p == null) throw new ApiException(404, "PRODUIT_INTROUVABLE",
                    "Aucun produit " + ref);
            return p;
        }
        Produit creer(Produit p) {
            if (p.reference() == null || !p.reference().matches("PRD-\\d{3}"))
                throw new ApiException(400, "REFERENCE_INVALIDE",
                        "Référence invalide : " + p.reference());
```

```
            if (p.prix() <= 0) throw new ApiException(400, "PRIX_INVALIDE",
                    "Le prix doit être positif");
            if (depot.putIfAbsent(p.reference(), p) != null)                        // atomique
                throw new ApiException(409, "PRODUIT_EXISTANT", "Le produit " + p.reference()
                        + " existe déjà");
            return p;
        }
        void supprimer(String ref) {
            if (depot.remove(ref) == null) throw new ApiException(404, "PRODUIT_INTROUVABLE",
                    "Aucun produit " + ref);
        }
    }

    // ------------------------------------------------------------------ couche contrôleur (HTTP)
    static class ProduitControleur {
        private final ProduitService service;
        private final ObjectMapper json = new ObjectMapper();
        ProduitControleur(ProduitService service) { this.service = service; }

        void traiter(HttpExchange ex) throws IOException {
            try {
                // ["", "api", "produits", "PRD-001"]
                String[] segments = ex.getRequestURI().getPath().split("/");
                String route = ex.getRequestMethod() + " " + segments.length;
                switch (route) {
                    case "GET 3" -> envoyer(ex, 200, service.lister(parametre(ex, "prixMax")
                            .map(Long::valueOf).orElse(null)));
                    case "GET 4" -> envoyer(ex, 200, service.obtenir(segments[3]));
                    case "POST 3" -> {
                        Produit cree = service.creer(json.readValue(ex.getRequestBody(),
                                Produit.class));
                        ex.getResponseHeaders().add("Location", "/api/produits/"
                                + cree.reference());
                        envoyer(ex, 201, cree);
                    }
                    case "DELETE 4" -> {
                        service.supprimer(segments[3]);
                        envoyer(ex, 204, null);
                    }
                    default -> throw new ApiException(405, "METHODE_NON_AUTORISEE",
                            "Route inconnue : " + route);
                }
            } catch (ApiException e) {
                envoyer(ex, e.statut, new ErreurApi(e.statut, e.code, e.getMessage()));
            // JSON mal formé
            } catch (IOException e) {
                envoyer(ex, 400, new ErreurApi(400, "JSON_INVALIDE",
                        "Corps de requête illisible"));
            // bogue : ne rien divulguer
            } catch (RuntimeException e) {
                envoyer(ex, 500, new ErreurApi(500, "ERREUR_INTERNE",
                        "Une erreur inattendue est survenue"));
            }
        }

        private void envoyer(HttpExchange ex, int statut, Object corps) throws IOException {
            if (corps == null) {
                // -1 : pas de corps
                ex.sendResponseHeaders(statut, -1);
            } else {
                byte[] octets = json.writeValueAsBytes(corps);
                ex.getResponseHeaders().add("Content-Type", "application/json; charset=utf-8");
                ex.sendResponseHeaders(statut, octets.length);
                try (var out = ex.getResponseBody()) { out.write(octets); }
            }
            ex.close();
        }

        private static Optional<String> parametre(HttpExchange ex, String nom) {
            String q = ex.getRequestURI().getQuery();
            if (q == null) return Optional.empty();
            return Arrays.stream(q.split("&"))
```

```
                    .map(kv -> kv.split("=", 2))
                    .filter(kv -> kv.length == 2 && kv[0].equals(nom))
                    .map(kv -> kv[1])
                    .findFirst();
        }
    }

    // ---------------------------------------------------------- démarrage + démonstration
    public static void main(String[] args) throws Exception {
        var controleur = new ProduitControleur(new ProduitService());
        // port libre
        HttpServer serveur = HttpServer.create(new InetSocketAddress("localhost", 0), 0);
        serveur.createContext("/api/produits", controleur::traiter);
        // un thread virtuel par requête
        serveur.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        serveur.start();
        String base = "http://localhost:" + serveur.getAddress().getPort();

        HttpClient client = HttpClient.newHttpClient();
        appeler(client, "POST", base + "/api/produits", """
                {"reference":"PRD-001","libelle":"Bissap 1 L","prix":1500,"stock":40}""");
        appeler(client, "POST", base + "/api/produits", """
                {"reference":"PRD-014","libelle":"Pagne wax","prix":7500,"stock":12}""");
        appeler(client, "POST", base + "/api/produits", """
                {"reference":"PRD-001","libelle":"Doublon","prix":100,"stock":1}""");
        appeler(client, "POST", base + "/api/produits", """
                {"reference":"X-1","libelle":"?","prix":100,"stock":1}""");
        appeler(client, "POST", base + "/api/produits", "{pas du json");
        appeler(client, "GET", base + "/api/produits?prixMax=5000", null);
        appeler(client, "GET", base + "/api/produits/PRD-014", null);
        appeler(client, "DELETE", base + "/api/produits/PRD-014", null);
        appeler(client, "GET", base + "/api/produits/PRD-014", null);
        appeler(client, "PUT", base + "/api/produits", null);
        serveur.stop(0);
    }

    static void appeler(HttpClient client, String methode, String url,
            String corps) throws Exception {
        HttpRequest requete = HttpRequest.newBuilder(URI.create(url))
                .header("Content-Type", "application/json")
                .method(methode, corps == null ? HttpRequest.BodyPublishers.noBody()
                                               : HttpRequest.BodyPublishers.ofString(corps))
                .build();
        HttpResponse<String> r = client.send(requete, HttpResponse.BodyHandlers.ofString());
        String chemin = URI.create(url).getRawPath() + (URI.create(url).getQuery() == null ? ""
                : "?" + URI.create(url).getQuery());
        System.out.printf("%-6s %-28s -> %d %s%n", methode, chemin, r.statusCode(), r.body());
    }
}
```

Sortie console

```
POST   /api/produits               -> 201 {"reference":"PRD-001","libelle":"Bissap 1
L","prix":1500,"stock":40}
POST   /api/produits               -> 201 {"reference":"PRD-014","libelle":"Pagne
wax","prix":7500,"stock":12}
POST   /api/produits               -> 409 {"statut":409,"code":"PRODUIT_EXISTANT","message":"Le produit
PRD-001 existe déjà"}
POST   /api/produits               -> 400 {"statut":400,"code":"REFERENCE_INVALIDE","message":"Référence
invalide : X-1"}
POST   /api/produits               -> 400 {"statut":400,"code":"JSON_INVALIDE","message":"Corps de
requête illisible"}
GET    /api/produits?prixMax=5000  -> 200 [{"reference":"PRD-001","libelle":"Bissap 1
L","prix":1500,"stock":40}]
GET    /api/produits/PRD-014       -> 200 {"reference":"PRD-014","libelle":"Pagne
wax","prix":7500,"stock":12}
DELETE /api/produits/PRD-014       -> 204
GET    /api/produits/PRD-014       -> 404 {"statut":404,"code":"PRODUIT_INTROUVABLE","message":"Aucun
produit PRD-014"}
PUT    /api/produits               -> 405 {"statut":405,"code":"METHODE_NON_AUTORISEE","message":"Route
inconnue : PUT 3"}
```

Vous pouvez aussi tester cette API depuis un terminal, avec l'outil `curl` (en remplaçant le port 0 par un port fixe, 8080 par exemple) :

Terminal

```
curl -i http://localhost:8080/api/produits
curl -i -X POST http://localhost:8080/api/produits \
     -H "Content-Type: application/json" \
     -d '{"reference":"PRD-020","libelle":"Sac en cuir","prix":25000,"stock":5}'
curl -i -X DELETE http://localhost:8080/api/produits/PRD-020
```

> **Astuce**
>
> Des outils graphiques comme **Bruno**, **Insomnia** ou **Postman** permettent de construire, d'enregistrer et de partager des collections de requêtes HTTP. Les IDE (IntelliJ IDEA, VS Code avec l'extension REST Client) savent aussi exécuter des fichiers `.http` contenant des requêtes.

17.5 Ce qu'il reste à faire… et pourquoi utiliser un framework

Notre mini-API fonctionne, mais une API de production demande beaucoup plus :

- un **routage** expressif (`/api/clients/{id}/commandes/{numero}`), avec extraction et conversion automatiques des paramètres ;
- la **validation** déclarative des données reçues, avec des messages d'erreur par champ ;
- l'**authentification** (jetons JWT, OAuth 2) et les **autorisations** par rôle ;
- la **gestion des transactions** et l'accès aux données sans écrire de JDBC à la main ;
- l'**injection de dépendances** pour assembler contrôleurs, services et dépôts ;
- la **configuration** par profil (développement, test, production), la journalisation, la surveillance (*health checks*, métriques) ;
- la **documentation** de l'API (OpenAPI / Swagger) ;
- des outils de **test** adaptés à chaque couche.

Réécrire tout cela pour chaque projet serait long et risqué. C'est le rôle des **frameworks** backend : **Spring Boot** (le plus répandu), **Quarkus** et **Micronaut** (orientés démarrage rapide et conteneurs), **Helidon** ou **Javalin** (plus légers). Le chapitre suivant présente Spring Boot. Grâce à ce chapitre, vous savez maintenant ce qu'il fait « sous le capot ».

Exercices du chapitre 17

Exercice 17.1 ★ — Concevoir des routes REST

Proposez les routes (méthode + URI + codes de statut possibles) d'une API de gestion des livraisons de TerangaShop : lister les livraisons du jour d'un livreur ; affecter une commande à un livreur ; signaler une livraison effectuée ; signaler un échec avec un motif ; consulter l'historique des statuts d'une livraison. Justifiez le choix de chaque méthode HTTP et indiquez lesquelles sont idempotentes.

Exercice 17.2 ★ — Client d'API publique

Avec `HttpClient`, écrivez un programme qui interroge une API publique de votre choix renvoyant du JSON (par exemple une API de taux de change ou de météo pour Dakar), puis désérialisez la réponse dans un

record avec Jackson (utilisez `@JsonIgnoreProperties(ignoreUnknown = true)` pour ignorer les champs inutiles). Gérez les erreurs : délai dépassé, code de statut différent de 200, JSON inattendu.

Exercice 17.3 ★★ — Compléter la mini-API

Ajoutez à `MiniApi` : la route `PATCH /api/produits/{ref}` qui modifie le prix et/ou le stock (seuls les champs présents dans le JSON sont modifiés ; utilisez `ObjectMapper.readTree`) ; la pagination de `GET /api/produits?page=0&taille=2` avec le record `Page<T>` du chapitre 10 ; un en-tête `X-Total-Count` ; et un journal (`System.Logger`) de chaque requête avec sa méthode, son chemin, son code de statut et sa durée.

Exercice 17.4 ★★ — Brancher la base de données

Remplacez le dépôt en mémoire de `MiniApi` par le `JdbcProduitDao` du chapitre 16, avec un pool HikariCP. Le service ne doit dépendre que de l'interface `ProduitDao`. Vérifiez que les codes 404 et 409 fonctionnent toujours, et traduisez une `AccesDonneesException` en réponse 503.

Exercice 17.5 ★★★ — API de commandes et test de charge

Ajoutez une ressource `/api/commandes` : `POST` crée une commande (corps JSON avec le client, le mode de paiement et les articles) en réutilisant la méthode transactionnelle `passerCommande` du chapitre 16, et renvoie `201` avec l'en-tête `Location`, ou `409` si le stock est insuffisant ; `GET /api/commandes/{id}` renvoie la commande et ses lignes. Écrivez ensuite un petit programme de test de charge qui envoie 1 000 commandes en parallèle avec `HttpClient.sendAsync` et des threads virtuels, puis vérifie que le stock final est exact (aucune vente à découvert).

Chapitre 18 — Introduction à Spring Boot 4

Objectifs du chapitre

- Comprendre ce que sont Spring et Spring Boot : conteneur, injection de dépendances, auto-configuration.
- Créer un projet Spring Boot et le configurer.
- Écrire une API REST complète en couches : entité JPA, *repository* Spring Data, service transactionnel, DTO validés, contrôleur.
- Centraliser la gestion des erreurs avec `ProblemDetail`.
- Tester un service et un contrôleur.

Note sur ce chapitre

Ce chapitre est une **introduction** : Spring Boot mérite un cours complet à lui seul. L'objectif est de relier tout ce que vous avez appris (interfaces, records, exceptions, génériques, JDBC, HTTP, tests) à la façon dont on écrit réellement un backend Java en entreprise. Le code cible **Spring Boot 4.1** (juin 2026), qui repose sur Spring Framework 7, Jakarta EE 11, Hibernate 7 et Jackson 3, et exige Java 17 au minimum (Java 25 est recommandé).

18.1 Spring et Spring Boot

Spring Framework est né en 2003 pour simplifier le développement d'applications Java d'entreprise. Son cœur est un **conteneur d'inversion de contrôle** (IoC) : au lieu que chaque objet crée lui-même ses dépendances (`new JdbcProduitDao(...)`), c'est le conteneur qui **crée les objets** (appelés *beans*), les **assemble** en injectant leurs dépendances, et gère leur cycle de vie. C'est exactement ce que nous faisions à la main dans `DemoDao` et `MiniApi`, mais automatisé.

Spring Boot (2014) ajoute par-dessus Spring :

- des **starters** : une seule dépendance Maven apporte un ensemble cohérent de bibliothèques compatibles (`spring-boot-starter-webmvc` apporte Spring MVC, Jackson, un serveur Tomcat intégré…) ;
- l'**auto-configuration** : Spring Boot détecte ce qui est présent dans le *classpath* et configure les composants avec des valeurs par défaut raisonnables (un pilote PostgreSQL et une URL ? il crée un pool HikariCP et un `DataSource`) ;
- un **serveur intégré** : l'application est un simple `.jar` exécutable avec `java -jar`, sans serveur d'applications à installer ;
- des fonctions de **production** : configuration externalisée, profils, surveillance avec *Actuator*, journalisation.

18.2 Créer le projet

Le site **start.spring.io** (Spring Initializr), également intégré à IntelliJ IDEA et à VS Code, génère un projet prêt à l'emploi. Choisissez : projet **Maven**, langage **Java**, Spring Boot **4.1.x**, groupe `sn.isep`, artefact

`terangashop-api`, Java **25**, et les dépendances **Spring Web**, **Spring Data JPA**, **PostgreSQL Driver**, **Validation** et **Spring Boot Actuator**.

pom.xml

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>          <!-- fixe les versions de TOUTES les dépendances -->
</parent>

<groupId>sn.isep</groupId>
<artifactId>terangashop-api</artifactId>
<version>1.0.0-SNAPSHOT</version>

<properties>
    <java.version>25</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>  <!-- REST, Tomcat, Jackson -->
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>    <!-- JPA, Hibernate, HikariCP -->
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>  <!-- Bean Validation -->
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>    <!-- santé, métriques -->
    </dependency>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>        <!-- version fournie par le parent -->
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <!-- JUnit, Mockito, AssertJ, MockMvc -->
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Spring Boot 4 — des starters plus modulaires

Spring Boot 4 a découpé son code en modules plus fins. Certains starters ont été renommés : `spring-boot-starter-web` devient `spring-boot-starter-webmvc`, et chaque technologie a son starter de test (`spring-boot-starter-webmvc-test`, `spring-boot-starter-data-jpa-test`…). Dans les tutoriels écrits pour Spring Boot 3, vous rencontrerez encore les anciens noms.

La classe principale générée démarre toute l'application :

TerangaShopApplication.java

```java
package sn.isep.terangashop;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

// active l'auto-configuration et la détection des composants du package
@SpringBootApplication
public class TerangaShopApplication {
    public static void main(String[] args) {
        SpringApplication.run(TerangaShopApplication.class, args);
    }
}
```

Nous organisons le code **par fonctionnalité** (un package `produit`, un package `commande`…), chaque package contenant ses différentes couches :

```
src/main/java/sn/isep/terangashop/
├── TerangaShopApplication.java
├── commun/
│   └── GestionnaireErreurs.java          (@RestControllerAdvice)
└── produit/
    ├── Produit.java                       (@Entity)
    ├── ProduitRepository.java             (interface Spring Data)
    ├── ProduitService.java                (@Service, @Transactional)
    ├── ProduitControleur.java             (@RestController)
    ├── ProduitDto.java                    (records de requête et de réponse)
    └── ProduitIntrouvableException.java
src/main/resources/
└── application.properties
```

18.3 La configuration

application.properties

```properties
spring.application.name=terangashop-api
server.port=8080

# Base de données (le mot de passe vient d'une variable d'environnement)
spring.datasource.url=jdbc:postgresql://localhost:5432/teranga_shop
spring.datasource.username=teranga
spring.datasource.password=${BD_MOT_DE_PASSE:teranga}

# JPA : vérifie que les entités correspondent au schéma, sans le modifier
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false

# Un thread virtuel par requête HTTP (chapitre 14)
spring.threads.virtual.enabled=true

# Actuator : exposer l'état de santé
management.endpoints.web.exposure.include=health,info
```

Toute propriété peut être redéfinie **sans modifier le code** : par une variable d'environnement (`SPRING_DATASOURCE_URL` remplace `spring.datasource.url`), par un argument (`java -jar app.jar --server.port=9090`) ou par un **profil** : le fichier `application-prod.properties` est chargé en plus lorsque l'on active le profil `prod` (`SPRING_PROFILES_ACTIVE=prod`).

18.4 L'injection de dépendances

Les classes annotées `@Component` ou une de ses spécialisations (`@Service`, `@Repository`, `@RestController`, `@Configuration`) sont détectées au démarrage et deviennent des **beans** gérés par le conteneur (un seul exemplaire par défaut : portée *singleton*). Lorsqu'un bean a besoin d'un autre, il le déclare simplement comme **paramètre de son constructeur** : Spring le lui fournit.

Java

```java
@Service
public class ProduitService {
    private final ProduitRepository repository;     // dépendance

    public ProduitService(ProduitRepository repository) {   // injection par constructeur
        this.repository = repository;
    }
}
```

> **Astuce — toujours l'injection par constructeur**
>
> Préférez l'injection par constructeur à l'annotation `@Autowired` sur un attribut : les dépendances sont `final`, visibles dans la signature, et la classe reste testable **sans Spring**, en lui passant de simples doublures (chapitre 15). Lorsqu'une classe n'a qu'un constructeur, `@Autowired` est inutile.

18.5 L'entité JPA

JPA (*Jakarta Persistence*) est la norme Java d'**ORM** (*Object-Relational Mapping*) ; **Hibernate** en est l'implémentation utilisée par Spring Boot. Une **entité** est une classe dont les instances correspondent aux lignes d'une table. Hibernate génère les requêtes SQL de base à votre place.

Produit.java

```java
package sn.isep.terangashop.produit;

import jakarta.persistence.*;

@Entity
@Table(name = "produit")
public class Produit {

    @Id
    @Column(length = 7)
    private String reference;

    @Column(nullable = false, length = 100)
    private String libelle;

    @Column(nullable = false, length = 30)
    private String categorie;

    @Column(nullable = false)
    private long prix;

    @Column(nullable = false)
    private int stock;

    protected Produit() { }                       // exigé par JPA, inutilisable ailleurs

    public Produit(String reference, String libelle, String categorie, long prix, int stock) {
        this.reference = reference;
        this.libelle = libelle;
```

```
        this.categorie = categorie;
        this.prix = prix;
        this.stock = stock;
    }

    public void retirer(int quantite) {           // la règle métier reste dans l'entité
        if (quantite > stock) throw new StockInsuffisantException(reference, quantite, stock);
        stock -= quantite;
    }

    public void modifierPrix(long nouveauPrix) {
        if (nouveauPrix <= 0) throw new IllegalArgumentException("Prix invalide");
        this.prix = nouveauPrix;
    }

    public String getReference() { return reference; }
    public String getLibelle() { return libelle; }
    public String getCategorie() { return categorie; }
    public long getPrix() { return prix; }
    public int getStock() { return stock; }
}
```

Annotation	Rôle
`@Entity`, `@Table(name = …)`	déclare une entité et sa table
`@Id`	clé primaire
`@GeneratedValue(strategy = GenerationType.IDENTITY)`	clé générée par la base (`GENERATED … AS IDENTITY`)
`@Column(nullable, length, name)`	détails de la colonne
`@Enumerated(EnumType.STRING)`	stocke un enum par son nom (chapitre 8)
`@ManyToOne`, `@OneToMany(mappedBy = …)`	relations entre entités (commande → client, commande → lignes)
`@Version`	verrouillage optimiste : détecte deux modifications concurrentes

18.6 Le *repository* Spring Data

Avec **Spring Data JPA**, il suffit de déclarer une **interface** qui étend `JpaRepository<Entité, TypeDeLaClé>` : Spring en génère l'implémentation au démarrage. Vous obtenez gratuitement `save`, `findById`, `findAll`, `deleteById`, `count`, la pagination… Mieux : Spring déduit la requête du **nom des méthodes** que vous déclarez. C'est l'aboutissement du dépôt générique du chapitre 10.

ProduitRepository.java

```java
package sn.isep.terangashop.produit;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.util.List;

public interface ProduitRepository extends JpaRepository<Produit, String> {

    // SELECT … FROM produit WHERE categorie = ? ORDER BY prix DESC
    List<Produit> findByCategorieOrderByPrixDesc(String categorie);

    // SELECT … FROM produit WHERE stock < ?
    List<Produit> findByStockLessThan(int seuil);

    List<Produit> findByLibelleContainingIgnoreCase(String motCle);

    // Requête explicite en JPQL (sur les entités, pas sur les tables)
    @Query("select p from Produit p where p.prix between :min and :max order by p.prix")
    List<Produit> dansLaFourchette(long min, long max);
}
```

18.7 Les DTO et la validation

On n'expose **jamais** directement une entité dans l'API : on utilise des **DTO** sous forme de **records** (chapitre 8). Cela découple le format de l'API du schéma de la base, et évite d'exposer des champs internes. Les annotations de **Jakarta Bean Validation** décrivent les contraintes que doivent respecter les données reçues.

ProduitDto.java

```java
package sn.isep.terangashop.produit;

import jakarta.validation.constraints.*;

public final class ProduitDto {
    private ProduitDto() { }

    /** Corps JSON attendu pour créer un produit. */
    public record CreerProduit(
            @NotBlank @Pattern(regexp = "PRD-\\d{3}",
                    message = "format attendu : PRD-000") String reference,
            @NotBlank @Size(max = 100) String libelle,
            @NotBlank String categorie,
            @Positive long prix,
            @PositiveOrZero int stock) { }

    /** Représentation renvoyée au client. */
    public record ProduitReponse(String reference, String libelle, String categorie, long prix,
            boolean disponible) {
        static ProduitReponse depuis(Produit p) {
            return new ProduitReponse(p.getReference(), p.getLibelle(), p.getCategorie(),
                    p.getPrix(), p.getStock() > 0);
        }
    }
}
```

18.8 Le service

Le service porte la logique métier. L'annotation `@Transactional` fait exécuter chaque méthode publique dans une transaction : **commit** si la méthode se termine normalement, **rollback** si elle lève une exception

non vérifiée. Tout le code de gestion de transaction du chapitre 16 (auto-commit, commit, rollback, finally) disparaît.

ProduitService.java

```java
package sn.isep.terangashop.produit;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import sn.isep.terangashop.produit.ProduitDto.CreerProduit;
import java.util.List;

@Service
@Transactional
public class ProduitService {
    private final ProduitRepository repository;

    public ProduitService(ProduitRepository repository) {
        this.repository = repository;
    }

    @Transactional(readOnly = true)
    public List<Produit> lister(String categorie) {
        return categorie == null ? repository.findAll()
                                 : repository.findByCategorieOrderByPrixDesc(categorie);
    }

    @Transactional(readOnly = true)
    public Produit obtenir(String reference) {
        return repository.findById(reference)
                .orElseThrow(() -> new ProduitIntrouvableException(reference));
    }

    public Produit creer(CreerProduit demande) {
        if (repository.existsById(demande.reference())) {
            throw new ProduitExistantException(demande.reference());
        }
        return repository.save(new Produit(demande.reference(), demande.libelle(),
                demande.categorie(), demande.prix(), demande.stock()));
    }

    public Produit changerPrix(String reference, long nouveauPrix) {
        Produit p = obtenir(reference);
        p.modifierPrix(nouveauPrix);      // pas de save() : l'entité est suivie, Hibernate
        // écrit la modification au commit (« dirty checking »)
        return p;
    }

    public void supprimer(String reference) {
        repository.delete(obtenir(reference));
    }
}
```

Les exceptions métier sont de simples `RuntimeException`, comme au chapitre 9 ; `ProduitExistantException` et `StockInsuffisantException` s'écrivent sur le même modèle :

ProduitIntrouvableException.java

```java
package sn.isep.terangashop.produit;

public class ProduitIntrouvableException extends RuntimeException {
    public ProduitIntrouvableException(String reference) {
        super("Aucun produit avec la référence " + reference);
    }
}
```

18.9 Le contrôleur REST

ProduitControleur.java

```java
package sn.isep.terangashop.produit;

import jakarta.validation.Valid;
import jakarta.validation.constraints.Positive;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import sn.isep.terangashop.produit.ProduitDto.CreerProduit;
import sn.isep.terangashop.produit.ProduitDto.ProduitReponse;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/produits")
public class ProduitControleur {
    private final ProduitService service;

    public ProduitControleur(ProduitService service) {
        this.service = service;
    }

    // GET /api/produits?categorie=Textile
    @GetMapping
    public List<ProduitReponse> lister(@RequestParam(required = false) String categorie) {
        return service.lister(categorie).stream().map(ProduitReponse::depuis).toList();
    }

    @GetMapping("/{reference}")                          // GET /api/produits/PRD-014
    public ProduitReponse obtenir(@PathVariable String reference) {
        return ProduitReponse.depuis(service.obtenir(reference));
    }

    @PostMapping                                         // POST /api/produits
    public ResponseEntity<ProduitReponse> creer(@Valid @RequestBody CreerProduit demande) {
        Produit cree = service.creer(demande);
        return ResponseEntity.created(URI.create("/api/produits/" + cree.getReference()))
                             .body(ProduitReponse.depuis(cree));
    }

    // PATCH /api/produits/PRD-014/prix?valeur=8000
    @PatchMapping("/{reference}/prix")
    public ProduitReponse changerPrix(@PathVariable String reference,
            @RequestParam @Positive long valeur) {
        return ProduitReponse.depuis(service.changerPrix(reference, valeur));
    }

    @DeleteMapping("/{reference}")                       // DELETE /api/produits/PRD-014
    public ResponseEntity<Void> supprimer(@PathVariable String reference) {
        service.supprimer(reference);
        return ResponseEntity.noContent().build();       // 204
    }
}
```

Comparez avec `MiniApi` (chapitre 17) : le routage, l'extraction des paramètres, la conversion JSON, les codes de statut et la validation sont désormais **déclaratifs**.

Annotation	Rôle
`@RestController`	contrôleur dont les valeurs de retour sont converties en JSON
`@RequestMapping("/api/…")`	préfixe commun des routes
`@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`	route pour une méthode HTTP

Annotation	Rôle
`@PathVariable`	segment variable de l'URI (`/{reference}`)
`@RequestParam`	paramètre de requête (`?categorie=…`)
`@RequestBody`	corps de la requête, converti depuis JSON
`@Valid`	déclenche la validation du DTO (sinon erreur 400)
`ResponseEntity<T>`	contrôle complet de la réponse : statut, en-têtes, corps

18.10 La gestion centralisée des erreurs

Une classe annotée **`@RestControllerAdvice`** intercepte les exceptions levées par **tous** les contrôleurs et les traduit en réponses HTTP (chapitre 9, section 9.9). Spring fournit le type **`ProblemDetail`**, qui implémente la norme **RFC 9457** (*Problem Details for HTTP APIs*) : un format JSON d'erreur standard compris par de nombreux outils.

GestionnaireErreurs.java

```java
package sn.isep.terangashop.commun;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import sn.isep.terangashop.produit.ProduitExistantException;
import sn.isep.terangashop.produit.ProduitIntrouvableException;
import sn.isep.terangashop.produit.StockInsuffisantException;

import java.util.Map;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GestionnaireErreurs {

    @ExceptionHandler(ProduitIntrouvableException.class)
    public ProblemDetail introuvable(ProduitIntrouvableException e) {
        return probleme(HttpStatus.NOT_FOUND, "Produit introuvable", e.getMessage());
    }

    @ExceptionHandler({ProduitExistantException.class, StockInsuffisantException.class})
    public ProblemDetail conflit(RuntimeException e) {
        return probleme(HttpStatus.CONFLICT, "Conflit", e.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail invalide(MethodArgumentNotValidException e) {
        Map<String, String> erreurs = e.getBindingResult().getFieldErrors().stream()
                .collect(Collectors.toMap(f -> f.getField(),
                        f -> String.valueOf(f.getDefaultMessage()), (a, b) -> a));
        ProblemDetail pd = probleme(HttpStatus.BAD_REQUEST, "Données invalides",
                "La requête contient des champs invalides");
        pd.setProperty("erreurs", erreurs);
        return pd;
    }

    private static ProblemDetail probleme(HttpStatus statut, String titre, String detail) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(statut, detail);
        pd.setTitle(titre);
        return pd;
    }
}
```

Exemple de réponse à une requête `POST /api/produits` contenant `{"reference": "X1", "libelle": "", "categorie": "Textile", "prix": -5, "stock": 3}` :

JSON

```
{
  "type": "about:blank",
  "title": "Données invalides",
  "status": 400,
  "detail": "La requête contient des champs invalides",
  "instance": "/api/produits",
  "erreurs": {
    "reference": "format attendu : PRD-000",
    "libelle": "must not be blank",
    "prix": "must be greater than 0"
  }
}
```

(Les messages par défaut de Bean Validation existent en plusieurs langues, dont le français : la langue utilisée dépend de la configuration du serveur et de l'en-tête `Accept-Language` de la requête.)

18.11 Démarrer et tester l'application

Terminal

```
./mvnw spring-boot:run                          # démarrage en mode développement

curl http://localhost:8080/api/produits?categorie=Textile
curl -i -X POST http://localhost:8080/api/produits -H "Content-Type: application/json" \
     -d '{"reference":"PRD-050","libelle":"Tapis de prière",
          "categorie":"Artisanat","prix":12000,"stock":6}'
curl http://localhost:8080/actuator/health      # {"status":"UP"}

./mvnw clean package              # produit target/terangashop-api-1.0.0-SNAPSHOT.jar
java -jar target/terangashop-api-1.0.0-SNAPSHOT.jar --spring.profiles.active=prod
```

Tester le service sans Spring

Grâce à l'injection par constructeur, le service se teste comme n'importe quelle classe, avec une doublure du *repository* créée par **Mockito** :

ProduitServiceTest.java

```
package sn.isep.terangashop.produit;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class ProduitServiceTest {

    @Mock ProduitRepository repository;          // doublure générée par Mockito
    @InjectMocks ProduitService service;         // service construit avec la doublure

    @Test
    void obtenirUnProduitInconnuLeveUneException() {
        when(repository.findById("PRD-999")).thenReturn(Optional.empty());
```

```
        assertThatThrownBy(() -> service.obtenir("PRD-999"))
                .isInstanceOf(ProduitIntrouvableException.class)
                .hasMessageContaining("PRD-999");
    }

    @Test
    void changerLePrixModifieLEntite() {
        var bissap = new Produit("PRD-001", "Bissap 1 L", "Boisson", 1_500, 40);
        when(repository.findById("PRD-001")).thenReturn(Optional.of(bissap));

        service.changerPrix("PRD-001", 1_750);

        assertThat(bissap.getPrix()).isEqualTo(1_750);
        verify(repository, never()).save(any());   // aucune sauvegarde explicite nécessaire
    }
}
```

Tester le contrôleur

Un test « en tranche » (*slice test*) annoté `@WebMvcTest` ne démarre que la couche web : le service y est remplacé par une doublure déclarée avec `@MockitoBean`, et `MockMvcTester` simule des requêtes HTTP sans démarrer de vrai serveur.

ProduitControleurTest.java

```
package sn.isep.terangashop.produit;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.http.HttpStatus;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.assertj.MockMvcTester;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.Mockito.when;

@WebMvcTest(ProduitControleur.class)
class ProduitControleurTest {

    @Autowired MockMvcTester mvc;
    @MockitoBean ProduitService service;

    @Test
    void produitInconnuDonne404AuFormatProblemDetail() {
        when(service.obtenir("PRD-999")).thenThrow(new ProduitIntrouvableException("PRD-999"));

        assertThat(mvc.get().uri("/api/produits/PRD-999"))
                .hasStatus(HttpStatus.NOT_FOUND)
                .bodyJson().extractingPath("$.title").isEqualTo("Produit introuvable");
    }
}
```

18.12 Pour aller plus loin

Sujet	Outil
Authentification et autorisations (JWT, OAuth 2, rôles)	Spring Security
Documentation interactive de l'API (Swagger UI)	springdoc-openapi
Versionnement du schéma de la base	Flyway ou Liquibase

Sujet	Outil
Tests d'intégration avec une vraie base PostgreSQL	Testcontainers
Conteneurisation et déploiement	Docker, `./mvnw spring-boot:build-image`
Appels à d'autres API	`RestClient` (Spring 6.1+), clients HTTP déclaratifs `@HttpExchange`
Surveillance en production	Actuator, Micrometer, Prometheus, Grafana

Exercices du chapitre 18

Exercice 18.1 ★ — Premier contrôleur

Générez un projet avec start.spring.io (dépendance Spring Web uniquement). Écrivez un contrôleur qui répond à `GET /api/bonjour?nom=Awa` par le JSON `{"message": "Bonjour Awa, bienvenue sur TerangaShop", "date": "2026-09-27"}` (un record avec un `LocalDate`). Si le paramètre est absent, utilisez « visiteur ». Testez avec `curl` puis avec un test `@WebMvcTest`.

Exercice 18.2 ★★ — CRUD des clients

Ajoutez la gestion des clients (table `client` du chapitre 16) : entité avec identifiant généré, repository avec les méthodes `findByTelephone` et `findByQuartierOrderByNom`, DTO de création validé (nom obligatoire, téléphone au format sénégalais avec `@Pattern`, quartier obligatoire), service transactionnel, contrôleur avec les routes `GET /api/clients`, `GET /api/clients/{id}`, `POST /api/clients`, `PATCH /api/clients/{id}/quartier`. Un téléphone déjà utilisé doit produire une réponse 409 au format `ProblemDetail`.

Exercice 18.3 ★★ — Pagination et tri

Modifiez `GET /api/produits` pour accepter un paramètre `Pageable` (`?page=0&size=5&sort=prix,desc`) et renvoyer une page de `ProduitReponse` (`repository.findAll(pageable).map(ProduitReponse::depuis)`). Ajoutez un filtre optionnel par fourchette de prix. Limitez la taille maximale de page à 50.

Exercice 18.4 ★★ — Tests du service et du contrôleur

Écrivez les tests unitaires de `ProduitService` avec Mockito : création d'un produit déjà existant, suppression d'un produit inconnu, liste filtrée par catégorie. Écrivez ensuite les tests `@WebMvcTest` du contrôleur : création valide (201 et en-tête `Location`), création invalide (400 avec la liste des champs en erreur), suppression (204).

Exercice 18.5 ★★★ — Passer une commande avec Spring

Créez les entités `Commande` (relation `@ManyToOne` vers `Client`, statut sous forme d'enum stocké par son nom) et `LigneCommande` (`@ManyToOne` vers `Produit`), leurs repositories, et un service `CommandeService.passer(PasserCommandeRequete)` annoté `@Transactional` qui décrémente les stocks (via la méthode `retirer` de l'entité), calcule le total et enregistre la commande. Vérifiez par un test d'intégration (`@SpringBootTest` avec une base de test) qu'une rupture de stock sur le dernier article annule **toutes** les modifications. Ajoutez un champ `@Version` à `Produit` et expliquez ce qui se passe si deux commandes simultanées modifient le même produit.

PARTIE V — Projet et annexes

- Chapitre 19 — Projet intégrateur : l'API TerangaShop
- Annexe A — Chronologie des nouveautés de Java 8 à Java 27
- Annexe B — Aide-mémoire
- Annexe C — Corrigés des exercices

Chapitre 19 — Projet intégrateur : l'API TerangaShop

> **Objectifs du projet**
>
> - Réaliser, en équipe de 3 ou 4, le backend complet d'une boutique en ligne.
> - Mobiliser l'ensemble du cours : modélisation objet, records, enums, exceptions, collections, streams, dates, concurrence, JDBC, HTTP, tests, Maven.
> - Travailler comme en entreprise : dépôt Git, itérations, revue de code, documentation.

19.1 Le contexte

TerangaShop souhaite remplacer ses fichiers Excel par une véritable application. Une équipe « mobile » développera l'application Android et une équipe « web » le site ; **votre équipe est chargée de l'API REST** qu'elles utiliseront toutes les deux. Le client attend une API fiable, documentée, testée, et capable de supporter les pics de commandes des veilles de fêtes (Tabaski, Korité, fin d'année).

Deux variantes sont possibles, au choix de l'enseignant :

- **Variante A — Java pur** : serveur HTTP du JDK, Jackson, JDBC et HikariCP (chapitres 16 et 17). Elle fait travailler tous les mécanismes en profondeur.
- **Variante B — Spring Boot** : Spring Web, Spring Data JPA, Validation (chapitre 18). Elle correspond aux pratiques de l'industrie.

19.2 Les fonctionnalités attendues

Lot	Fonctionnalité	Priorité
1. Catalogue	CRUD des produits et des catégories ; recherche par mot-clé, catégorie, fourchette de prix ; pagination et tri	indispensable
1. Catalogue	Import d'un catalogue au format CSV avec rapport d'erreurs (chapitre 13)	indispensable
2. Clients	Inscription (téléphone unique, format sénégalais), consultation, modification du quartier	indispensable
3. Commandes	Création d'une commande avec plusieurs articles, dans une transaction ; frais de livraison selon le quartier ; gratuité au-delà de 50 000 FCFA	indispensable
3. Commandes	Cycle de vie : `EN_ATTENTE` → `PAYEE` → `EXPEDIEE` → `LIVREE`, ou `ANNULEE` (avec remise en stock) ; transitions contrôlées (chapitre 8)	indispensable
4. Paiements	Enregistrement d'un paiement Wave, Orange Money ou espèces ; simulation de l'appel à l'opérateur avec délai et échec possibles (chapitres 14 et 17)	indispensable
5. Statistiques	Chiffre d'affaires par jour, par catégorie, par quartier ; meilleurs produits ; panier moyen (en SQL ou avec les Streams)	importante
6. Promotions	Codes promo (pourcentage, montant fixe, livraison offerte) avec dates de validité (chapitres 8 et 13)	bonus
7. Notifications	Envoi (simulé) d'un SMS à chaque changement de statut, derrière une interface `Notificateur` (chapitre 7)	bonus
8. Sécurité	Authentification par jeton, rôles `CLIENT` et `GERANT`	bonus

19.3 Les routes minimales de l'API

Méthode	Route	Rôle	Réponses
GET	/api/produits?categorie=&motCle=&prixMin=&prixMax=&page=&taille=	rechercher des produits	200
GET	/api/produits/{reference}	détail d'un produit	200, 404
POST	/api/produits	créer un produit	201, 400, 409
PUT	/api/produits/{reference}	remplacer un produit	200, 400, 404
DELETE	/api/produits/{reference}	supprimer un produit jamais commandé	204, 404, 409
POST	/api/produits/import	importer un CSV	200 (rapport)
POST	/api/clients	inscrire un client	201, 400, 409
GET	/api/clients/{id}/commandes	historique d'un client	200, 404
POST	/api/commandes	passer une commande	201, 400, 404, 409
GET	/api/commandes/{id}	détail d'une commande et de ses lignes	200, 404
POST	/api/commandes/{id}/paiements	payer une commande	201, 409, 502
PATCH	/api/commandes/{id}/statut	faire évoluer le statut	200, 409
GET	/api/statistiques/ventes?du=&au=	tableau de bord	200, 400

Toutes les erreurs sont renvoyées au format `ProblemDetail` (RFC 9457), avec un code d'erreur métier.

19.4 Les exigences techniques

- **Java 25**, projet **Maven** avec Maven Wrapper, dépôt **Git** avec un historique propre (un commit par étape logique, messages explicites).
- Base **PostgreSQL** ; le schéma est livré sous forme de scripts SQL versionnés (ou de migrations Flyway pour la variante B).
- Architecture en **couches** (contrôleur, service, accès aux données) et organisation **par fonctionnalité**.
- **Records** pour tous les DTO, **enums** pour les statuts et modes de paiement, **exceptions métier** non vérifiées traduites en codes HTTP à un seul endroit.
- Aucun montant en `double` : `long` en FCFA.
- Aucune requête SQL construite par concaténation de données externes.
- Threads virtuels pour le traitement des requêtes.

- **Tests** : au moins 30 tests JUnit, dont des tests unitaires des services (avec doublures) et au moins 5 tests d'intégration avec une vraie base.
- Un **README** expliquant l'installation, la configuration (variables d'environnement), le lancement et l'utilisation de l'API (exemples `curl` ou collection Bruno/Postman).

19.5 Le déroulement

Itération	Durée	Contenu	Livrable
0	1 semaine	Constitution de l'équipe, dépôt Git, squelette Maven, schéma SQL, choix de la variante	Dépôt initialisé, schéma relu par l'enseignant
1	2 semaines	Lots 1 et 2 (catalogue, clients) avec leurs tests	Démonstration du CRUD complet
2	2 semaines	Lots 3 et 4 (commandes, paiements, transactions)	Démonstration d'un scénario de commande complet
3	2 semaines	Lot 5 et un bonus ; test de charge ; documentation	Version 1.0 étiquetée dans Git, soutenance

À chaque fin d'itération, l'équipe présente une démonstration de 10 minutes et chaque membre relit (*revue de code*) au moins une contribution d'un autre membre.

19.6 La grille d'évaluation

Critère	Points
Fonctionnalités indispensables opérationnelles	6
Qualité de la conception : couches, responsabilités, records, enums, interfaces	3
Robustesse : validation des entrées, gestion des erreurs, transactions, absence d'injection SQL	3
Tests : pertinence, couverture des cas d'erreur, tests d'intégration	3
Lisibilité du code : nommage, méthodes courtes, absence de duplication	2
Documentation (README, exemples d'appels) et historique Git	1
Soutenance : démonstration, réponses aux questions individuelles	2
Bonus (promotions, notifications, sécurité, Docker, OpenAPI)	jusqu'à +2
Total	**20**

> **Conseils**
>
> Commencez par le **schéma de la base** et par les **records** de l'API : ils structurent tout le reste. Écrivez les tests des règles métier en même temps que le code. Intégrez souvent (plusieurs fois par jour) sur la branche principale plutôt que de fusionner d'énormes branches la veille de la démonstration. Et gardez toujours une version qui fonctionne.

Annexe A — Chronologie des nouveautés de Java 8 à Java 27

Les fonctionnalités marquées *(preview)* ou *(incubator)* ne sont pas définitives dans la version indiquée ; la version où elles sont devenues définitives est précisée lorsqu'elle existe.

Version	Date	Nouveautés principales pour le développeur
8 (LTS)	mars 2014	Lambdas et références de méthodes ; interfaces fonctionnelles ; API Stream ; `Optional` ; méthodes `default` et `static` dans les interfaces ; API `java.time` ; `CompletableFuture`
9	sept. 2017	Système de modules (JPMS) ; JShell ; `List.of`, `Set.of`, `Map.of` ; méthodes `private` dans les interfaces ; `takeWhile`/`dropWhile` ; `Optional.or`, `ifPresentOrElse` ; `System.Logger`
10	mars 2018	`var` pour les variables locales ; `List.copyOf` ; `Collectors.toUnmodifiableList` ; début du rythme semestriel
11 (LTS)	sept. 2018	Client HTTP standard (`java.net.http`) ; lancement direct d'un fichier `.java` ; `String.isBlank`, `strip`, `lines`, `repeat` ; `Files.readString`/`writeString` ; `var` dans les lambdas
12	mars 2019	`Collectors.teeing` ; `String.indent`, `transform` ; switch expressions *(preview)*
13	sept. 2019	Text blocks *(preview)*
14	mars 2020	**Switch expressions** ; messages d'erreur `NullPointerException` explicites ; records *(preview)*
15	sept. 2020	**Text blocks** ; `String.formatted` ; classes scellées *(preview)* ; ramasse-miettes ZGC en production
16	mars 2021	**Records** ; **pattern matching pour `instanceof`** ; `Stream.toList()` ; `mapMulti`
17 (LTS)	sept. 2021	**Classes scellées** ; générateurs aléatoires unifiés (`RandomGenerator`) ; pattern matching pour `switch` *(preview)*
18	mars 2022	**UTF-8 par défaut** ; serveur web simple `jwebserver` ; balise `@snippet` dans la Javadoc
19	sept. 2022	Threads virtuels *(preview)* ; record patterns *(preview)* ; concurrence structurée *(incubator)*
20	mars 2023	Scoped values *(incubator)* ; nouvelles previews des threads virtuels et du pattern matching
21 (LTS)	sept. 2023	**Threads virtuels** ; **pattern matching pour `switch`** ; **record patterns** ; **Sequenced Collections** ; `Math.clamp` ; `StringBuilder.repeat` ; méthodes `main` d'instance *(preview)*
22	mars 2024	**Variables et motifs anonymes `_`** ; **API d'accès à la mémoire native (FFM)** ; lancement de programmes source multi-fichiers ; Stream Gatherers *(preview)*
23	sept. 2024	**Commentaires de documentation en Markdown** ; types primitifs dans les motifs *(preview)* ; import de modules *(preview)*
24	mars 2025	**Stream Gatherers** ; **API Class-File** ; threads virtuels sans « épinglage » dans `synchronized` ; cryptographie post-quantique (ML-KEM, ML-DSA) ; cache AOT pour un démarrage plus rapide
25 (LTS)	sept. 2025	**Fichiers source compacts et `main` d'instance** (classe `IO`) ; **import de modules** ; **corps de constructeur flexibles** ; **Scoped Values** ; en-têtes d'objets compacts (option) ; profilage JFR amélioré ; concurrence structurée *(preview)* ; *stable values (preview)*
26	mars 2026	**HTTP/3** dans le client HTTP ; suppression de l'API Applet ; préparation pour que `final` signifie vraiment « non modifiable » ; cache AOT avec tous les ramasse-miettes ; *lazy constants (preview)*
27	sept. 2026	**G1 ramasse-miettes par défaut dans tous les environnements** ; **en-têtes d'objets compacts par défaut** ; échange de clés hybride post-quantique pour TLS 1.3 ; concurrence structurée, types primitifs dans les motifs, *lazy constants (preview)*

Prochaine version LTS attendue : **Java 29**, en septembre 2027.

Annexe B — Aide-mémoire

B.1 Commandes du JDK

Commande	Rôle
`java -version`, `javac -version`	versions installées
`javac -d out Fichier.java`	compiler vers le dossier `out`
`javac --release 25 --enable-preview …`	compiler avec les fonctionnalités *preview*
`java -cp out paquet.Classe arg1 arg2`	exécuter une classe principale
`java Fichier.java`	compiler en mémoire et exécuter (fichiers source)
`java --enable-preview Fichier.java`	exécuter avec les *previews*
`jshell`	interpréteur interactif
`jar --create --file app.jar --main-class paquet.App -C out .`	créer un `.jar` exécutable
`java -jar app.jar`	exécuter un `.jar`
`javadoc -d doc $(find src -name "*.java")`	générer la documentation
`jwebserver -p 8000`	servir le dossier courant en HTTP (Java 18)
`jcmd`, `jstack <pid>`, `jfr`	diagnostic d'une JVM en cours d'exécution

B.2 Commandes Maven

Commande	Rôle
`./mvnw compile`	compiler
`./mvnw test`	compiler et tester
`./mvnw package`	produire le `.jar`
`./mvnw clean verify`	reconstruction complète avec toutes les vérifications
`./mvnw test -Dtest=NomTest#methode`	exécuter un seul test
`./mvnw dependency:tree`	arbre des dépendances
`./mvnw spring-boot:run`	lancer une application Spring Boot

B.3 Syntaxe moderne en un coup d'œil

Besoin	Syntaxe
Inférence de type locale	`var liste = new ArrayList<String>();`
Texte sur plusieurs lignes	`String sql = """ … """;`
Switch expression	`var frais = switch (zone) { case "A", "B" -> 1000; default -> 2500; };`
Classe de données immuable	`record Point(int x, int y) { }`
Validation dans un record	`record Prix(long v) { Prix { if (v < 0) throw new IllegalArgumentException(); } }`
Hiérarchie fermée	`sealed interface Forme permits Cercle, Carre { }`
Test de type avec variable	`if (o instanceof Produit p) { … }`

Besoin	Syntaxe
Switch sur les types	`switch (o) { case Integer i when i > 0 -> …; case String s -> …; default -> …; }`
Déconstruction	`if (r instanceof Accepte(var id, var montant)) { … }`
Composante ignorée	`case Refuse(_, var motif) -> …`
Collection immuable	`List.of(…)`, `Set.of(…)`, `Map.of(k, v, …)`, `List.copyOf(l)`
Premier / dernier élément	`liste.getFirst()`, `liste.getLast()`, `liste.reversed()`
Stream vers liste	`stream.filter(…).map(…).toList()`
Thread virtuel	`Thread.ofVirtual().start(() -> …)`, `Executors.newVirtualThreadPerTaskExecutor()`
Programme minimal	`void main() { IO.println("Bonjour"); }`

Annexe C — Corrigés des exercices

Les corrigés proposent **une** solution possible, pas la seule. Pour les exercices ★★★, qui sont de véritables petits projets, le corrigé donne la structure de la solution et le code des parties les plus délicates. Tous les programmes complets ont été compilés et exécutés avec le JDK 25.

Corrigés du chapitre 1

Corrigé 1.1 — Mon premier programme

Presentation.java

```java
public class Presentation {
    public static void main(String[] args) {
        System.out.println("Awa Diop");
        System.out.println("Licence professionnelle - Développement backend");
        System.out.println("J'apprends Java 25");
    }
}
```

Sortie console

```
Awa Diop
Licence professionnelle - Développement backend
J'apprends Java 25
```

Après `javac Presentation.java`, le dossier contient `Presentation.java` **et** `Presentation.class` (le bytecode). La commande `java Presentation` exécute ce fichier `.class`. En revanche, `java Presentation.java` compile en mémoire et ne crée **aucun** fichier `.class` : si vous supprimez `Presentation.class` puis relancez `java Presentation.java`, le programme fonctionne toujours.

Corrigé 1.2 — Questions de cours

1. La **JVM** est le moteur qui exécute le bytecode. Le **JDK** est le kit complet du développeur : il contient une JVM, les bibliothèques standard et les outils (`javac`, `jshell`, `jar`…).
2. Un fichier `.class` contient le **bytecode** d'une classe. Oui, il s'exécute sur n'importe quel système disposant d'une JVM de version suffisante : c'est la portabilité de Java.
3. Java 25 est une version **LTS** : elle recevra des correctifs de sécurité pendant plusieurs années, alors que Java 27 n'est maintenue que six mois. En production, on privilégie la stabilité et la durée du support.
4. Une fonctionnalité *preview* est complète mais pas encore définitive : elle doit être activée avec `--enable-preview` et peut encore changer (ou disparaître) dans les versions suivantes.

Corrigé 1.3 — Chasse aux erreurs

Les cinq erreurs : `string` au lieu de `String` (Java distingue majuscules et minuscules) ; `printn` au lieu de `println` ; le point-virgule manquant après le premier appel ; le guillemet fermant manquant dans `"Livraison à Diamniadio"` ; l'accolade fermante de la classe manquante.

Boutique.java

```java
public class Boutique {
    public static void main(String[] args) {
        System.out.println("TerangaShop ouvre ses portes !");
        System.out.println("Livraison à Diamniadio");
    }
}
```

Sortie console

```
TerangaShop ouvre ses portes !
Livraison à Diamniadio
```

Corrigé 1.4 — Explorer JShell

Expression	Résultat	Type
`7 / 2`	`3`	`int` (division entière)
`7 / 2.0`	`3.5`	`double`
`7 % 2`	`1`	`int`
`"Dakar".length()`	`5`	`int`
`"Dakar" + 221`	`"Dakar221"`	`String`
`221 + 1 + "Dakar"`	`"222Dakar"`	`String`
`"Dakar" + 221 + 1`	`"Dakar2211"`	`String`

L'opérateur `+` est évalué **de gauche à droite**. Dans `221 + 1 + "Dakar"`, la première opération additionne deux entiers (222), puis concatène le résultat à la chaîne. Dans `"Dakar" + 221 + 1`, la première opération concatène déjà une chaîne et un entier : tout le reste devient concaténation. Pour forcer l'addition, il faut des parenthèses : `"Dakar" + (221 + 1)`.

Corrigé 1.5 — Carte de visite TerangaShop

Carte.java

```java
void main() {
    IO.println("+------------------------------+");
    IO.println("|         TERANGASHOP          |");
    IO.println("|  Produits locaux, livrés chez |");
    IO.println("|  vous à Dakar et environs     |");
    IO.println("|  Tél : 33 800 00 00           |");
    IO.println("+------------------------------+");
}
```

Sortie console

```
+------------------------------+
|         TERANGASHOP          |
|  Produits locaux, livrés chez |
|  vous à Dakar et environs     |
|  Tél : 33 800 00 00           |
+------------------------------+
```

Corrigés du chapitre 2

Corrigé 2.1 — Déclarations valides ou non ?

Déclaration	Compile ?	Correction
`int 2emeClient = 5;`	non : un identifiant ne commence pas par un chiffre	`int deuxiemeClient = 5;`
`long population = 18_000_000;`	oui	—
`float taux = 0.18;`	non : `0.18` est un `double`	`float taux = 0.18f;`
`var total;`	non : `var` exige une initialisation	`var total = 0;`
`char initiale = "A";`	non : `"A"` est une `String`	`char initiale = 'A';`
`boolean payé = 1;`	non : `1` n'est pas un booléen (le nom `payé` est, lui, autorisé)	`boolean paye = true;`
`long numero = 221771234567;`	non : littéral `int` trop grand	`long numero = 221771234567L;`
`final double PI = 3.14159;`	oui (préférez `Math.PI`)	—
`int class = 3;`	non : `class` est un mot réservé	`int classe = 3;`
`var reduction = null;`	non : type impossible à déduire	`Long reduction = null;`

Corrigé 2.2 — Prédire l'affichage

Prediction.java

```java
public class Prediction {
    public static void main(String[] args) {
        int a = 10, b = 4;
        System.out.println(a / b);
        System.out.println(a % b);
        System.out.println((double) a / b);
        System.out.println(a / b * 1.0);
        System.out.println(a++ + ++a);
        System.out.println("Total : " + a + b);
        System.out.println("Total : " + (a + b));
        System.out.println('A' + 1);
        System.out.println((char) ('A' + 1));
    }
}
```

Sortie console

```
2
2
2.5
2.0
22
Total : 124
Total : 16
66
B
```

Explications : `a / b * 1.0` effectue d'abord la division entière (2), puis la multiplication (2.0). Dans `a++ + ++a`, `a++` vaut 10 (puis `a` passe à 11), puis `++a` fait passer `a` à 12 et vaut 12 : 10 + 12 = 22. Ensuite `a` vaut 12, d'où `"Total : " + 12 + 4` = `"Total : 124"`. Enfin `'A' + 1` est un calcul sur des entiers (65 + 1).

Corrigé 2.3 — Conversion de durée

Duree.java

```java
import java.util.Scanner;

public class Duree {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Durée en secondes : ");
        int total = sc.nextInt();
        int heures = total / 3600;
        int minutes = total % 3600 / 60;
        int secondes = total % 60;
        System.out.println(heures + " h " + minutes + " min " + secondes + " s");
    }
}
```

Sortie console

```
Durée en secondes : 2 h 44 min 35 s
```

Corrigé 2.4 — Facture avec TVA

Facture.java

```java
import java.util.Scanner;

public class Facture {
    static final int TAUX_TVA = 18;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Libellé : ");
        String libelle = sc.nextLine();
        System.out.print("Prix unitaire HT : ");
        long prix = Long.parseLong(sc.nextLine().strip());
        System.out.print("Quantité : ");
        int quantite = Integer.parseInt(sc.nextLine().strip());

        long ht = prix * quantite;
        long tva = Math.round(ht * TAUX_TVA / 100.0);
        long ttc = ht + tva;
        System.out.println();
        System.out.printf("%-12s %3d x %6d%n", libelle, quantite, prix);
        System.out.printf("%-12s %15d FCFA%n", "Montant HT", ht);
        System.out.printf("%-12s %15d FCFA%n", "TVA 18 %", tva);
        System.out.printf("%-12s %15d FCFA%n", "Montant TTC", ttc);
    }
}
```

Sortie console

```
Libellé : Prix unitaire HT : Quantité :
Pagne wax      3 x   2750
Montant HT              8250 FCFA
TVA 18 %                1485 FCFA
Montant TTC             9735 FCFA
```

Corrigé 2.5 — Le compteur qui déborde

VolumeMobile.java

```java
public class VolumeMobile {
    public static void main(String[] args) {
        int transactionsParJour = 350_000;
        int montantMoyen = 12_500;
        int jours = 30;

        int volumeInt = transactionsParJour * montantMoyen * jours;
        long volumeLong = (long) transactionsParJour * montantMoyen * jours;
        System.out.println("Avec int  : " + volumeInt);
        System.out.println("Avec long : " + volumeLong);
        try {
            int v = Math.multiplyExact(Math.multiplyExact(transactionsParJour,
                    montantMoyen), jours);
            System.out.println(v);
        } catch (ArithmeticException e) {
            System.out.println("Débordement détecté : " + e.getMessage());
        }
    }
}
```

Sortie console

```
Avec int  : -1893986176
Avec long : 131250000000
Débordement détecté : integer overflow
```

Le volume d'**une seule journée** (4,375 milliards de FCFA) dépasse déjà la capacité d'un `int` (environ 2,1 milliards) : le résultat en `int` est faux et n'a aucun sens. Il suffit de convertir le **premier** opérande en `long` pour que tout le calcul se fasse en `long`. Attention : écrire `(long) (transactionsParJour * montantMoyen * jours)` ne suffirait pas, car le débordement aurait lieu avant la conversion.

Corrigé 2.6 — Rendu de monnaie

RenduMonnaie.java

```java
import java.util.Scanner;

public class RenduMonnaie {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Somme à rendre (FCFA) : ");
        int reste = sc.nextInt();
        System.out.println();

        int n10000 = reste / 10_000; reste %= 10_000;
        int n5000 = reste / 5_000;   reste %= 5_000;
        int n2000 = reste / 2_000;   reste %= 2_000;
        int n1000 = reste / 1_000;   reste %= 1_000;
        int n500 = reste / 500;      reste %= 500;
        int n250 = reste / 250;      reste %= 250;
        int n100 = reste / 100;      reste %= 100;
        int n50 = reste / 50;        reste %= 50;
        int n25 = reste / 25;        reste %= 25;
        int n10 = reste / 10;        reste %= 10;
        int n5 = reste / 5;

        System.out.println("Billets de 10000 : " + n10000);
        System.out.println("Billets de 5000  : " + n5000);
        System.out.println("Billets de 2000  : " + n2000);
        System.out.println("Billets de 1000  : " + n1000);
        System.out.println("Billets de 500   : " + n500);
        System.out.println("Pièces de 250    : " + n250);
        System.out.println("Pièces de 100    : " + n100);
        System.out.println("Pièces de 50     : " + n50);
```

```
        System.out.println("Pièces de 25     : " + n25);
        System.out.println("Pièces de 10     : " + n10);
        System.out.println("Pièces de 5      : " + n5);
    }
}
```

Sortie console

```
Somme à rendre (FCFA) :
Billets de 10000 : 1
Billets de 5000  : 1
Billets de 2000  : 1
Billets de 1000  : 1
Billets de 500   : 1
Pièces de 250    : 0
Pièces de 100    : 2
Pièces de 50     : 0
Pièces de 25     : 1
Pièces de 10     : 1
Pièces de 5      : 0
```

Au chapitre 5, avec un tableau des coupures et une boucle, ce programme tiendra en quelques lignes.

Corrigés du chapitre 3

Corrigé 3.1 — Méthodes de base

MethodesBase.java

```java
public class MethodesBase {
    public static void main(String[] args) {
        String s = "  TerangaShop, Dakar - Sénégal  ";
        String propre = s.strip();
        System.out.println("[" + propre + "]");
        System.out.println("Longueurs : " + s.length() + " / " + propre.length());
        int virgule = propre.indexOf(',');
        System.out.println("Position de la virgule : " + virgule);
        String ville = propre.substring(virgule + 2, propre.indexOf(" -"));
        System.out.println("Ville : " + ville);
        System.out.println(propre.toUpperCase());
        System.out.println("Nombre de mots : " + propre.split("\\s+").length);
    }
}
```

Sortie console

```
[TerangaShop, Dakar - Sénégal]
Longueurs : 32 / 28
Position de la virgule : 11
Ville : Dakar
TERANGASHOP, DAKAR - SÉNÉGAL
Nombre de mots : 4
```

Remarque : le tiret `-` est compté comme un « mot » par `split("\\s+")`. Pour ne compter que les mots formés de lettres, on peut filtrer avec une expression régulière (chapitre 12).

Corrigé 3.2 — Palindrome

Palindrome.java

```java
public class Palindrome {
    static String nettoyer(String s) {
        return s.replace(" ", "").toLowerCase();
    }

    /** Version 1 : deux indices qui se rapprochent. */
```

```
    static boolean estPalindrome(String s) {
        String t = nettoyer(s);
        int i = 0, j = t.length() - 1;
        while (i < j) {
            if (t.charAt(i) != t.charAt(j)) return false;
            i++;
            j--;
        }
        return true;
    }

    /** Version 2 : comparaison avec la chaîne inversée. */
    static boolean estPalindromeV2(String s) {
        String t = nettoyer(s);
        return t.equals(new StringBuilder(t).reverse().toString());
    }

    public static void main(String[] args) {
        for (String s : new String[]{"kayak", "Engage le jeu que je le gagne", "Dakar"}) {
            System.out.println(s + " -> " + estPalindrome(s) + " / " + estPalindromeV2(s));
        }
    }
}
```

Sortie console

```
kayak -> true / true
Engage le jeu que je le gagne -> true / true
Dakar -> false / false
```

La première version est plus efficace (elle s'arrête à la première différence et ne crée pas de seconde chaîne) ; la seconde est plus courte. Pour ignorer aussi les accents, on pourrait normaliser la chaîne avec `java.text.Normalizer`.

Corrigé 3.3 — Statistiques de texte

StatsTexte.java

```
public class StatsTexte {
    public static void main(String[] args) {
        String texte = "Livraison à Médina : 3 colis, prévue à 14h30 !";
        int voyelles = 0, consonnes = 0, chiffres = 0, espaces = 0, autres = 0;
        for (char c : texte.toLowerCase().toCharArray()) {
            if ("aeiouyéèêàù".indexOf(c) >= 0) voyelles++;
            else if (Character.isLetter(c)) consonnes++;
            else if (Character.isDigit(c)) chiffres++;
            else if (Character.isWhitespace(c)) espaces++;
            else autres++;
        }
        System.out.printf("voyelles=%d consonnes=%d chiffres=%d espaces=%d autres=%d%n",
                          voyelles, consonnes, chiffres, espaces, autres);
    }
}
```

Sortie console

```
voyelles=14 consonnes=15 chiffres=5 espaces=9 autres=3
```

La méthode `toCharArray()` permet d'utiliser une boucle *for-each* sur les caractères d'une chaîne ; le test `"aeiouy…".indexOf(c) >= 0` vérifie qu'un caractère appartient à une liste.

Corrigé 3.4 — Validation d'e-mail et de référence

Validations.java

```java
public class Validations {
    static boolean estEmailValide(String email) {
        return email != null && email.matches("[\\w.-]+@[\\w-]+(\\.[\\w-]+)*\\.[a-zA-Z]{2,6}");
    }
    static boolean estReferenceValide(String ref) {
        return ref != null && ref.matches("PRD-\\d{3}");
    }

    public static void main(String[] args) {
        for (String e : new String[]{"awa.diop@isep.edu.sn", "moussa@gmail", "@wave.com",
                "fatou_sow@orange.sn"}) {
            System.out.println(e + " -> " + estEmailValide(e));
        }
        for (String r : new String[]{"PRD-001", "PRD-1", "prd-001", "PRD-0012"}) {
            System.out.println(r + " -> " + estReferenceValide(r));
        }
    }
}
```

Sortie console

```
awa.diop@isep.edu.sn -> true
moussa@gmail -> false
@wave.com -> false
fatou_sow@orange.sn -> true
PRD-001 -> true
PRD-1 -> false
prd-001 -> false
PRD-0012 -> false
```

`\w` désigne une lettre, un chiffre ou `_`. Valider parfaitement une adresse e-mail par une expression régulière est très difficile ; en pratique, on vérifie un format raisonnable, puis on envoie un e-mail de confirmation.

Corrigé 3.5 — Analyse d'une ligne CSV

LigneVente.java

```java
public class LigneVente {
    static void afficher(String ligne) {
        String[] c = ligne.split(";");
        if (c.length != 6) {
            System.out.println("Ligne invalide");
            return;
        }
        System.out.println("Date      : " + c[0]);
        System.out.println("Commande  : " + c[1]);
        System.out.println("Client    : " + c[2]);
        System.out.println("Quartier  : " + c[3].toUpperCase());
        System.out.println("Paiement  : " + c[4]);
        System.out.println("Montant   : " + c[5] + " FCFA");
    }

    public static void main(String[] args) {
        afficher("2026-09-14;TS-2026-000087;Awa Diop;Médina;WAVE;18500");
        afficher("2026-09-14;TS-2026-000088;Moussa");
    }
}
```

Sortie console

```
Date      : 2026-09-14
Commande  : TS-2026-000087
Client    : Awa Diop
Quartier  : MÉDINA
Paiement  : WAVE
Montant   : 18500 FCFA
Ligne invalide
```

Corrigé 3.6 — Générateur de reçu JSON

RecuJson.java

```java
public class RecuJson {
    static String recuJson(String client, String[] articles, long[] prix, int[] quantites) {
        StringBuilder lignes = new StringBuilder();
        long total = 0;
        for (int i = 0; i < articles.length; i++) {
            long sousTotal = prix[i] * quantites[i];
            total += sousTotal;
            lignes.append("""
                    {"libelle": "%s", "quantite": %d, "prixUnitaire": %d, "sousTotal": %d}"""
                    .formatted(articles[i], quantites[i], prix[i], sousTotal));
            // pas de virgule après le dernier
            if (i < articles.length - 1) lignes.append(",\n    ");
        }
        return """
                {
                  "client": "%s",
                  "lignes": [
                    %s
                  ],
                  "total": %d
                }""".formatted(client, lignes, total);
    }

    public static void main(String[] args) {
        System.out.println(recuJson("Fatou Sow",
                new String[]{"Bissap 1 L", "Pagne wax"}, new long[]{1_500, 7_500},
                        new int[]{3, 2}));
    }
}
```

Sortie console

```
{
  "client": "Fatou Sow",
  "lignes": [
    {"libelle": "Bissap 1 L", "quantite": 3, "prixUnitaire": 1500, "sousTotal": 4500},
    {"libelle": "Pagne wax", "quantite": 2, "prixUnitaire": 7500, "sousTotal": 15000}
  ],
  "total": 19500
}
```

Attention : si un libellé contient un guillemet ", ce JSON « fait main » devient invalide. C'est une des raisons pour lesquelles on utilise une bibliothèque comme Jackson (chapitre 17).

Corrigés du chapitre 4

Corrigé 4.1 — Mention d'examen

Mention.java

```java
public class Mention {
    static String mention(double note) {
        if (note < 0 || note > 20)
```

```
                throw new IllegalArgumentException("Note invalide : " + note);
        if (note < 10) return "Ajourné";
        else if (note < 12) return "Passable";
        else if (note < 14) return "Assez bien";
        else if (note < 16) return "Bien";
        else return "Très bien";
    }

    static String mentionV2(double note) {
        if (note < 0 || note > 20)
                throw new IllegalArgumentException("Note invalide : " + note);
        return switch ((int) note / 2) {
            case 0, 1, 2, 3, 4 -> "Ajourné";      // 0 à 9,99
            case 5 -> "Passable";                  // 10 à 11,99
            case 6 -> "Assez bien";                // 12 à 13,99
            case 7 -> "Bien";                      // 14 à 15,99
            default -> "Très bien";                // 16 à 20
        };
    }

    public static void main(String[] args) {
        for (double n : new double[]{8.5, 10, 13.75, 15.99, 16, 20}) {
            System.out.println(n + " : " + mention(n) + " / " + mentionV2(n));
        }
    }
}
```

Sortie console

```
8.5 : Ajourné / Ajourné
10.0 : Passable / Passable
13.75 : Assez bien / Assez bien
15.99 : Bien / Bien
16.0 : Très bien / Très bien
20.0 : Très bien / Très bien
```

Corrigé 4.2 — Année bissextile

Bissextiles.java

```
public class Bissextiles {
    static boolean estBissextile(int annee) {
        return (annee % 4 == 0 && annee % 100 != 0) || annee % 400 == 0;
    }

    public static void main(String[] args) {
        int compteur = 0;
        for (int a = 1960; a <= 2030; a++) {
            if (!estBissextile(a)) continue;
            System.out.print(a + " ");
            compteur++;
            if (compteur % 10 == 0) System.out.println();
        }
        System.out.println();
    }
}
```

Sortie console

```
1960 1964 1968 1972 1976 1980 1984 1988 1992 1996
2000 2004 2008 2012 2016 2020 2024 2028
```

L'an 2000 est bissextile car il est divisible par 400 (alors que 1900 ne l'était pas). La classe `java.time.Year` offre directement `Year.isLeap(annee)`.

Corrigé 4.3 — Devine le prix

DevinePrix.java

```java
import java.util.Random;
import java.util.Scanner;

public class DevinePrix {
    public static void main(String[] args) {
        Random hasard = new Random();
        int secret = (1 + hasard.nextInt(40)) * 500;       // 500, 1000, ..., 20000
        Scanner sc = new Scanner(System.in);
        int essais = 0, proposition;
        do {
            System.out.print("Votre proposition : ");
            proposition = sc.nextInt();
            essais++;
            if (proposition < secret) System.out.println("C'est plus");
            else if (proposition > secret) System.out.println("C'est moins");
        } while (proposition != secret);
        System.out.println("Bravo ! Trouvé en " + essais + " essai(s).");
    }
}
```

La stratégie optimale est la **dichotomie** : proposer toujours le milieu de l'intervalle restant. Avec 40 valeurs possibles, on trouve en 6 essais au maximum ($2^6 = 64 \geq 40$).

Corrigé 4.4 — Motifs

Motifs.java

```java
public class Motifs {
    public static void main(String[] args) {
        int n = 5;
        // Triangle rectangle
        for (int i = 1; i <= n; i++) {
            System.out.println("*".repeat(i));
        }
        // Pyramide centrée : ligne i -> (n - i) espaces puis (2i - 1) étoiles
        for (int i = 1; i <= n; i++) {
            System.out.println(" ".repeat(n - i) + "*".repeat(2 * i - 1));
        }
        // Losange : la pyramide, puis la pyramide inversée sans sa plus grande ligne
        for (int i = 1; i <= n; i++) System.out.println(" ".repeat(n - i)
                + "*".repeat(2 * i - 1));
        for (int i = n - 1; i >= 1; i--) System.out.println(" ".repeat(n - i)
                + "*".repeat(2 * i - 1));
    }
}
```

Sortie console

```
*
**
***
****
*****
    *
   ***
  *****
 *******
*********
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

Sans `repeat`, on écrirait deux boucles imbriquées : une pour les espaces, une pour les étoiles.

Corrigé 4.5 — Suite de Syracuse

Syracuse.java

```java
import java.util.Scanner;

public class Syracuse {
    public static void main(String[] args) {
        System.out.print("Valeur de départ : ");
        long n = new Scanner(System.in).nextLong();
        long max = n;
        int vol = 0;
        StringBuilder suite = new StringBuilder().append(n);
        while (n != 1) {
            n = n % 2 == 0 ? n / 2 : 3 * n + 1;
            vol++;
            max = Math.max(max, n);
            suite.append(' ').append(n);
        }
        System.out.println();
        System.out.println(suite);
        System.out.println("Temps de vol : " + vol + ", altitude maximale : " + max);
    }
}
```

Sortie console

```
Valeur de départ :
7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
Temps de vol : 16, altitude maximale : 52
```

Avec 27, on obtient bien un temps de vol de 111 et une altitude maximale de 9 232. On utilise `long` car les valeurs intermédiaires peuvent devenir très grandes.

Corrigé 4.6 — Caisse enregistreuse complète

La solution suivante structure le programme en petites méthodes et utilise des tableaux de taille fixe. Au chapitre 11, les tableaux parallèles seront avantageusement remplacés par une `List` de records.

CaisseComplete.java

```java
import java.util.Scanner;
```

```
public class CaisseComplete {
    static final int MAX = 20;
    static final String[] libelles = new String[MAX];
    static final long[] prix = new long[MAX];
    static final int[] quantites = new int[MAX];
    static int nb = 0;
    static int tauxRemise = 0;
    static String code = null;
    static String mode = "non choisi";
    static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        boolean fini = false;
        while (!fini) {
            System.out.println("1.Ajouter 2.Ticket 3.Code promo 4.Paiement 5.Valider");
            int choix = lireEntier();
            switch (choix) {
                case 1 -> ajouter();
                case 2 -> System.out.print(ticket());
                case 3 -> appliquerCode();
                case 4 -> choisirMode();
                case 5 -> fini = valider();
                default -> System.out.println("Choix invalide");
            }
        }
    }

    static int lireEntier() {
        try {
            return Integer.parseInt(sc.nextLine().strip());
        } catch (NumberFormatException e) {
            return -1;
        }
    }

    static void ajouter() {
        if (nb == MAX) { System.out.println("Panier plein"); return; }
        System.out.print("Libellé : ");  libelles[nb] = sc.nextLine();
        System.out.print("Prix : ");     prix[nb] = lireEntier();
        System.out.print("Quantité : "); quantites[nb] = lireEntier();
        if (prix[nb] <= 0 || quantites[nb] <= 0)
                { System.out.println("Saisie invalide"); return; }
        nb++;
    }

    static long sousTotal() {
        long s = 0;
        for (int i = 0; i < nb; i++) s += prix[i] * quantites[i];
        return s;
    }
    static long remise() { return sousTotal() * tauxRemise / 100; }
    static long frais() { return sousTotal() - remise() >= 50_000 ? 0 : 1_500; }
    static long total() { return sousTotal() - remise() + frais(); }

    static String ticket() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < nb; i++) {
            sb.append("%-14s %2d x %6d%n".formatted(libelles[i], quantites[i], prix[i]));
        }
        sb.append("Sous-total %d, remise %d, livraison %d -> TOTAL %d FCFA (%s)%n"
                .formatted(sousTotal(), remise(), frais(), total(), mode));
        return sb.toString();
    }

    static void appliquerCode() {
        System.out.print("Code : ");
        String saisi = sc.nextLine().strip().toUpperCase();
        if (code != null) { System.out.println("Un code est déjà appliqué : " + code); return; }
        int taux = switch (saisi) {
            case "TERANGA10" -> 10;
            case "TABASKI" -> sousTotal() > 30_000 ? 15 : -1;
```

```
            default -> 0;
        };
        switch (taux) {
            case 0 -> System.out.println("Code inconnu");
            case -1 -> System.out.println("TABASKI : minimum 30 000 FCFA d'achat");
            default -> { tauxRemise = taux; code = saisi; System.out.println("Remise de "
                    + taux + " % appliquée"); }
        }
    }

    static void choisirMode() {
        System.out.print("1.Wave 2.Orange Money 3.Espèces : ");
        mode = switch (lireEntier()) {
            case 1 -> "Wave";
            case 2 -> "Orange Money";
            case 3 -> "Espèces";
            default -> { System.out.println("Mode invalide"); yield mode; }
        };
    }

    static boolean valider() {
        if (nb == 0) { System.out.println("Panier vide"); return false; }
        if (mode.equals("non choisi"))
                { System.out.println("Choisissez un mode de paiement"); return false; }
        System.out.print(ticket());
        System.out.println("Commande validée. Merci !");
        return true;
    }
}
```

Sortie console

```
1.Ajouter 2.Ticket 3.Code promo 4.Paiement 5.Valider
Libellé : Prix : Quantité : 1.Ajouter 2.Ticket 3.Code promo 4.Paiement 5.Valider
Libellé : Prix : Quantité : 1.Ajouter 2.Ticket 3.Code promo 4.Paiement 5.Valider
Code : Remise de 15 % appliquée
1.Ajouter 2.Ticket 3.Code promo 4.Paiement 5.Valider
Code : Un code est déjà appliqué : TABASKI
1.Ajouter 2.Ticket 3.Code promo 4.Paiement 5.Valider
Bissap 1 L      4 x   1500
Sac cuir        1 x  25000
Sous-total 31000, remise 4650, livraison 1500 -> TOTAL 27850 FCFA (non choisi)
1.Ajouter 2.Ticket 3.Code promo 4.Paiement 5.Valider
1.Wave 2.Orange Money 3.Espèces : 1.Ajouter 2.Ticket 3.Code promo 4.Paiement 5.Valider
Bissap 1 L      4 x   1500
Sac cuir        1 x  25000
Sous-total 31000, remise 4650, livraison 1500 -> TOTAL 27850 FCFA (Orange Money)
Commande validée. Merci !
```

Corrigés du chapitre 5

Corrigé 5.1 — Minimum, maximum, moyenne

MinMaxMoyenne.java

```
import java.util.Scanner;

public class MinMaxMoyenne {
    static int min(int[] t) {
        int m = t[0];
        for (int v : t) if (v < m) m = v;
        return m;
    }
    static int max(int[] t) {
        int m = t[0];
        for (int v : t) if (v > m) m = v;
        return m;
    }
```

```
    static double moyenne(int[] t) {
        long s = 0;
        for (int v : t) s += v;
        return (double) s / t.length;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Combien de valeurs ? ");
        int n = sc.nextInt();
        int[] t = new int[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Valeur " + (i + 1) + " : ");
            t[i] = sc.nextInt();
        }
        double moy = moyenne(t);
        int auDessus = 0;
        for (int v : t) if (v > moy) auDessus++;
        System.out.println();
        System.out.printf("min=%d max=%d moyenne=%.2f au-dessus=%d%n", min(t),
                max(t), moy, auDessus);
    }
}
```

Sortie console

```
Combien de valeurs ? Valeur 1 : Valeur 2 : Valeur 3 : Valeur 4 : Valeur 5 :
min=4 max=19 moyenne=10.40 au-dessus=2
```

Corrigé 5.2 — Inverser un tableau

Inversion.java

```
import java.util.Arrays;

public class Inversion {
    static void inverser(int[] t) {                 // modifie le tableau reçu
        for (int i = 0, j = t.length - 1; i < j; i++, j--) {
            int tmp = t[i];
            t[i] = t[j];
            t[j] = tmp;
        }
    }
    static int[] inverse(int[] t) {                 // ne modifie pas l'original
        int[] r = new int[t.length];
        for (int i = 0; i < t.length; i++) r[i] = t[t.length - 1 - i];
        return r;
    }

    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4, 5};
        int[] b = inverse(a);
        System.out.println(Arrays.toString(a) + " -> " + Arrays.toString(b));
        inverser(a);
        System.out.println(Arrays.toString(a));
    }
}
```

Sortie console

```
[1, 2, 3, 4, 5] -> [5, 4, 3, 2, 1]
[5, 4, 3, 2, 1]
```

Comme Java passe une copie de la **référence** du tableau, `inverser` agit sur le tableau de l'appelant : c'est un **effet de bord**, pratique mais parfois surprenant. `inverse` est une fonction **pure** : elle ne modifie rien et renvoie un nouveau résultat, ce qui la rend plus sûre et plus facile à tester.

Corrigé 5.3 — Fusion de deux tableaux triés

Fusion.java

```java
import java.util.Arrays;

public class Fusion {
    static int[] fusionner(int[] a, int[] b) {
        int[] r = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length) {
            r[k++] = a[i] <= b[j] ? a[i++] : b[j++];
        }
        while (i < a.length) r[k++] = a[i++];       // reste de a
        while (j < b.length) r[k++] = b[j++];       // reste de b
        return r;
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(fusionner(new int[]{1, 4, 9},
                new int[]{2, 3, 10, 12})));
        System.out.println(Arrays.toString(fusionner(new int[]{}, new int[]{5, 6})));
    }
}
```

Sortie console

```
[1, 2, 3, 4, 9, 10, 12]
[5, 6]
```

La fusion se fait en un seul parcours : sa complexité est O(n + m). C'est l'étape clé du **tri fusion**.

Corrigé 5.4 — Tri par sélection

TriSelection.java

```java
import java.util.Arrays;
import java.util.Random;

public class TriSelection {
    static long comparaisons;

    static void trier(long[] t) {
        for (int i = 0; i < t.length - 1; i++) {
            int iMin = i;
            for (int j = i + 1; j < t.length; j++) {
                comparaisons++;
                if (t[j] < t[iMin]) iMin = j;
            }
            long tmp = t[i]; t[i] = t[iMin]; t[iMin] = tmp;
        }
    }

    public static void main(String[] args) {
        // graine fixe : résultats reproductibles
        Random r = new Random(2026);
        for (int n : new int[]{10, 100, 1_000}) {
            long[] original = new long[n];
            for (int i = 0; i < n; i++) original[i] = r.nextInt(100_000);
            long[] a = Arrays.copyOf(original, n), b = Arrays.copyOf(original, n);
            comparaisons = 0;
            trier(a);
            Arrays.sort(b);
            System.out.println("n=" + n + " : " + comparaisons
                    + " comparaisons, identique à Arrays.sort ? "
                              + Arrays.equals(a, b));
        }
    }
}
```

Sortie console

```
n=10 : 45 comparaisons, identique à Arrays.sort ? true
n=100 : 4950 comparaisons, identique à Arrays.sort ? true
n=1000 : 499500 comparaisons, identique à Arrays.sort ? true
```

Le tri par sélection effectue toujours n(n − 1)/2 comparaisons : sa complexité est **quadratique**, O(n²). Multiplier la taille par 10 multiplie le travail par 100. `Arrays.sort` utilise un algorithme en O(n log n) (un *Quicksort* à double pivot pour les types primitifs).

Corrigé 5.5 — Récursivité

Recursions.java

```java
public class Recursions {
    static int sommeChiffres(int n) {
        return n < 10 ? n : n % 10 + sommeChiffres(n / 10);
    }
    static long puissance(long x, int n) {
        if (n == 0) return 1;
        long moitie = puissance(x, n / 2);
        return n % 2 == 0 ? moitie * moitie : moitie * moitie * x;
    }
    static boolean estPalindrome(String s) {
        if (s.length() <= 1) return true;
        return s.charAt(0) == s.charAt(s.length() - 1)
                && estPalindrome(s.substring(1, s.length() - 1));
    }
    static void afficherBinaire(int n) {
        if (n >= 2) afficherBinaire(n / 2);      // d'abord les bits de poids fort
        System.out.print(n % 2);
    }

    public static void main(String[] args) {
        System.out.println(sommeChiffres(2026));
        System.out.println(puissance(2, 40) + " " + puissance(3, 5));
        System.out.println(estPalindrome("radar") + " " + estPalindrome("dakar"));
        afficherBinaire(221);
        System.out.println();
    }
}
```

Sortie console

```
10
1099511627776 243
true false
11011101
```

L'exponentiation rapide ne fait qu'environ $\log_2(n)$ multiplications au lieu de n.

Corrigé 5.6 — Tableau de bord des ventes

TableauBord.java

```java
public class TableauBord {
    static final String[] PRODUITS = {"Bissap", "Thiakry", "Pagne", "Sac"};
    static final long[] PRIX = {1_500, 1_000, 7_500, 25_000};

    static String jour(int i) {
        return switch (i) {
            case 0 -> "Lundi"; case 1 -> "Mardi"; case 2 -> "Mercredi"; case 3 -> "Jeudi";
            case 4 -> "Vendredi"; case 5 -> "Samedi"; default -> "Dimanche";
        };
    }
    static long caJour(int[][] v, int j) {
```

```
        long s = 0;
        for (int p = 0; p < PRIX.length; p++) s += v[j][p] * PRIX[p];
        return s;
    }
    static int unitesProduit(int[][] v, int p) {
        int s = 0;
        for (int[] ligne : v) s += ligne[p];
        return s;
    }
    static int indiceMax(long[] t) {
        int m = 0;
        for (int i = 1; i < t.length; i++) if (t[i] > t[m]) m = i;
        return m;
    }

    public static void main(String[] args) {
        int[][] ventes = {
            {12, 5, 2, 0}, {8, 9, 1, 1}, {15, 4, 3, 0}, {10, 6, 0, 2},
            {20, 12, 5, 1}, {25, 15, 6, 3}, {5, 3, 1, 0}
        };
        System.out.printf("%-9s %7s %7s %7s %7s %10s%n", "Jour", PRODUITS[0], PRODUITS[1],
                PRODUITS[2], PRODUITS[3], "CA");
        long[] caJours = new long[7];
        for (int j = 0; j < 7; j++) {
            caJours[j] = caJour(ventes, j);
            System.out.printf("%-9s %7d %7d %7d %7d %10d%n", jour(j),
                    ventes[j][0], ventes[j][1], ventes[j][2], ventes[j][3], caJours[j]);
        }
        long[] caProduits = new long[PRIX.length];
        for (int p = 0; p < PRIX.length; p++) caProduits[p] = unitesProduit(ventes,
                p) * PRIX[p];
        System.out.printf("%-9s %7d %7d %7d %7d%n", "Unités", unitesProduit(ventes, 0),
                unitesProduit(ventes, 1),
                unitesProduit(ventes, 2), unitesProduit(ventes, 3));
        int best = indiceMax(caProduits);
        System.out.println("Produit le plus rentable : " + PRODUITS[best] + " ("
                + caProduits[best] + " FCFA)");
        int record = indiceMax(caJours);
        System.out.println("Jour record : " + jour(record) + " (" + caJours[record] + " FCFA)");
    }
}
```

Sortie console

```
Jour       Bissap Thiakry   Pagne     Sac         CA
Lundi          12       5       2       0      38000
Mardi           8       9       1       1      53500
Mercredi       15       4       3       0      49000
Jeudi          10       6       0       2      71000
Vendredi       20      12       5       1     104500
Samedi         25      15       6       3     172500
Dimanche        5       3       1       0      18000
Unités         95      54      18       7
Produit le plus rentable : Sac (175000 FCFA)
Jour record : Samedi (172500 FCFA)
```

Corrigés du chapitre 6

Corrigé 6.1 — Compte Wave

CompteMobile.java

```
public class CompteMobile {
    private final String numero;
    private final String titulaire;
    private long solde;


    public CompteMobile(String numero, String titulaire) {
        this.numero = numero;
```

```
        this.titulaire = titulaire;
    }

    private static void verifierMontant(long montant) {
        if (montant <= 0) throw new IllegalArgumentException("Montant invalide : " + montant);
    }

    public void deposer(long montant) {
        verifierMontant(montant);
        solde += montant;
    }

    public void retirer(long montant) {
        verifierMontant(montant);
        if (montant > solde) {
            throw new IllegalStateException("Solde insuffisant : " + solde + " < " + montant);
        }
        solde -= montant;
    }

    public static long frais(long montant) {
        return Math.min(montant / 100, 5_000);        // 1 %, plafonné à 5 000 FCFA
    }

    public void transferer(CompteMobile destinataire, long montant) {
        verifierMontant(montant);
        if (destinataire == this) throw new IllegalArgumentException("Transfert vers soi-même");
        retirer(montant + frais(montant));             // vérifie le solde AVANT de créditer
        destinataire.deposer(montant);
    }

    public long getSolde() { return solde; }

    @Override
    public String toString() {
        return titulaire + " (" + numero + ") : " + solde + " FCFA";
    }

    public static void main(String[] args) {
        CompteMobile awa = new CompteMobile("771234567", "Awa Diop");
        CompteMobile moussa = new CompteMobile("780001122", "Moussa Ndiaye");
        awa.deposer(100_000);
        awa.transferer(moussa, 30_000);
        System.out.println(awa);
        System.out.println(moussa);
        try {
            moussa.transferer(awa, 30_000);
        } catch (IllegalStateException e) {
            System.out.println("Refusé : " + e.getMessage());
        }
        awa.deposer(1_000_000);
        awa.transferer(moussa, 800_000);               // frais plafonnés
        System.out.println(awa);
    }
}
```

Sortie console

```
Awa Diop (771234567) : 69700 FCFA
Moussa Ndiaye (780001122) : 30000 FCFA
Refusé : Solde insuffisant : 30000 < 30300
Awa Diop (771234567) : 264700 FCFA
```

L'ordre des opérations dans `transferer` est essentiel : on débite l'émetteur (ce qui vérifie le solde) **avant** de créditer le destinataire. Dans l'autre ordre, un échec du débit laisserait de l'argent « créé » sur le compte du destinataire.

Corrigé 6.2 — Étudiant

Etudiant.java

```java
public class Etudiant {
    private static int compteur = 0;

    private final String matricule;
    private final String nom;
    private final String prenom;
    private final double[] notes = new double[5];
    private final boolean[] saisie = new boolean[5];

    public Etudiant(String nom, String prenom) {
        compteur++;
        this.matricule = "ISEP-2026-%03d".formatted(compteur);
        this.nom = nom.toUpperCase();
        this.prenom = prenom;
    }

    public void ajouterNote(int indice, double note) {
        if (indice < 0 || indice >= notes.length)
                throw new IllegalArgumentException("Indice invalide : " + indice);
        if (note < 0 || note > 20)
                throw new IllegalArgumentException("Note invalide : " + note);
        notes[indice] = note;
        saisie[indice] = true;
    }

    public double moyenne() {
        double somme = 0;
        int nb = 0;
        for (int i = 0; i < notes.length; i++) {
            if (saisie[i]) { somme += notes[i]; nb++; }
        }
        if (nb == 0) throw new IllegalStateException("Aucune note pour " + matricule);
        return somme / nb;
    }

    public String mention() {
        double m = moyenne();
        if (m < 10) return "Ajourné";
        if (m < 12) return "Passable";
        if (m < 14) return "Assez bien";
        if (m < 16) return "Bien";
        return "Très bien";
    }

    public static int getNombreEtudiants() { return compteur; }

    @Override
    public String toString() {
        return "%s %s %s : %.2f (%s)".formatted(matricule, prenom, nom, moyenne(), mention());
    }

    public static void main(String[] args) {
        Etudiant e1 = new Etudiant("Diop", "Awa");
        e1.ajouterNote(0, 14); e1.ajouterNote(1, 16.5); e1.ajouterNote(2, 12);
        Etudiant e2 = new Etudiant("Fall", "Ibrahima");
        e2.ajouterNote(0, 9); e2.ajouterNote(1, 11);
        System.out.println(e1);
        System.out.println(e2);
        System.out.println("Étudiants créés : " + Etudiant.getNombreEtudiants());
    }
}
```

Sortie console

```
ISEP-2026-001 Awa DIOP : 14.17 (Bien)
ISEP-2026-002 Ibrahima FALL : 10.00 (Passable)
Étudiants créés : 2
```

Le tableau `saisie` distingue une note absente d'une note égale à 0. Au chapitre 11, une `List<Double>` rendra cette solution plus simple.

Corrigé 6.3 — Fraction immuable

Fraction.java

```java
import java.util.Objects;

public final class Fraction {
    private final long num;
    private final long den;

    public Fraction(long num, long den) {
        if (den == 0) throw new ArithmeticException("Dénominateur nul");
        if (den < 0) { num = -num; den = -den; }      // le signe est porté par le numérateur
        long p = pgcd(Math.abs(num), den);
        this.num = num / p;
        this.den = den / p;
    }

    private static long pgcd(long a, long b) {
        return b == 0 ? (a == 0 ? 1 : a) : pgcd(b, a % b);
    }

    public Fraction plus(Fraction f)      { return new Fraction(num * f.den
            + f.num * den, den * f.den); }
    public Fraction moins(Fraction f)     { return new Fraction(num * f.den - f.num * den,
            den * f.den); }
    public Fraction fois(Fraction f)      { return new Fraction(num * f.num, den * f.den); }
    public Fraction divisePar(Fraction f) { return new Fraction(num * f.den, den * f.num); }
    public double valeur()                { return (double) num / den; }

    @Override public String toString()
            { return den == 1 ? String.valueOf(num) : num + "/" + den; }

    @Override
    public boolean equals(Object o) {
        // formes simplifiées : comparaison directe
        return o instanceof Fraction f && num == f.num && den == f.den;
    }

    @Override public int hashCode() { return Objects.hash(num, den); }

    public static void main(String[] args) {
        Fraction a = new Fraction(1, 2), b = new Fraction(1, 3);
        System.out.println(a.plus(b) + " " + a.moins(b) + " " + a.fois(b) + " "
                + a.divisePar(b));
        System.out.println(new Fraction(6, -4) + " " + new Fraction(10, 5) + " "
                + new Fraction(3, 4).valeur());
        System.out.println(new Fraction(2, 4).equals(new Fraction(-1, -2)));
    }
}
```

Sortie console

```
5/6 1/6 1/6 3/2
-3/2 2 0.75
true
```

Comme les fractions sont toujours stockées sous forme simplifiée avec un dénominateur positif, deux fractions égales ont exactement les mêmes attributs : `equals` et `hashCode` peuvent les comparer directement.

Corrigé 6.4 — Le contrat equals/hashCode

ContratHash.java

```java
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

public class ContratHash {
    static class PointSansHash {
        int x, y;
        PointSansHash(int x, int y) { this.x = x; this.y = y; }
        @Override public boolean equals(Object o) {
            return o instanceof PointSansHash p && x == p.x && y == p.y;
        }
    }
    static class Point {
        // modifiables : volontairement dangereux
        int x, y;
        Point(int x, int y) { this.x = x; this.y = y; }
        @Override public boolean equals(Object o)
                { return o instanceof Point p && x == p.x && y == p.y; }
        @Override public int hashCode() { return Objects.hash(x, y); }
    }

    public static void main(String[] args) {
        Set<PointSansHash> s1 = new HashSet<>();
        s1.add(new PointSansHash(1, 2));
        s1.add(new PointSansHash(1, 2));
        System.out.println("Sans hashCode : taille " + s1.size());

        Set<Point> s2 = new HashSet<>();
        Point p = new Point(1, 2);
        s2.add(p);
        s2.add(new Point(1, 2));
        System.out.println("Avec hashCode : taille " + s2.size());

        p.x = 99;                                          // modification APRÈS l'ajout
        System.out.println("contains(p) : " + s2.contains(p) + ", taille " + s2.size());
    }
}
```

Sortie console

```
Sans hashCode : taille 2
Avec hashCode : taille 1
contains(p) : false, taille 1
```

1. Sans `hashCode` redéfini, deux points égaux ont des codes de hachage différents (hérités d'`Object`) : le `HashSet` les range dans des cases différentes et ne compare même pas leurs contenus. On obtient un « doublon ».
2. Avec un `hashCode` cohérent, le second ajout est détecté comme doublon.
3. Après la modification de `x`, le point se trouve dans la case correspondant à son **ancien** code de hachage : `contains` cherche dans la case du **nouveau** code et ne le trouve plus. L'élément est « perdu » dans l'ensemble. Conclusion : les clés de `HashMap` et les éléments de `HashSet` doivent être **immuables** (records, `String`, enums…).

Corrigé 6.5 — Encapsulation défaillante

Problèmes : tous les attributs sont publics, donc un code extérieur peut modifier `nb` ou `total` sans ajouter d'article (total incohérent), remplacer les tableaux, écrire directement dans `articles` ou `prix` ; `getPrix()` renvoie le tableau interne, qui peut donc être modifié ; `ajouter` ne vérifie ni la capacité (exception au 11e ajout), ni le prix (négatif), ni le libellé ; `total` est une donnée **dérivée** stockée, qui peut se désynchroniser.

Panier.java

```
import java.util.Arrays;

public class Panier {
    private static final int CAPACITE = 10;
    private final String[] articles = new String[CAPACITE];
    private final long[] prix = new long[CAPACITE];
    private int nb;

    public void ajouter(String article, long prixArticle) {
        if (nb == CAPACITE) throw new IllegalStateException("Panier plein");
        if (article == null || article.isBlank())
                throw new IllegalArgumentException("Article obligatoire");
        if (prixArticle <= 0)
                throw new IllegalArgumentException("Prix invalide : " + prixArticle);
        articles[nb] = article;
        prix[nb] = prixArticle;
        nb++;
    }

    public long total() {                                // calculé, jamais stocké
        long t = 0;
        for (int i = 0; i < nb; i++) t += prix[i];
        return t;
    }

    public int taille() { return nb; }

    public long[] getPrix() {
        return Arrays.copyOf(prix, nb);                  // copie défensive
    }

    public static void main(String[] args) {
        Panier p = new Panier();
        p.ajouter("Bissap", 1_500);
        p.ajouter("Pagne", 7_500);
        p.getPrix()[0] = 0;                              // ne modifie que la copie
        System.out.println(p.taille() + " articles, total " + p.total());
    }
}
```

Sortie console

```
2 articles, total 9000
```

Corrigé 6.6 — Bibliothèque de l'ISEP

Structure proposée : `sn.isep.biblio.model` contient `Livre`, `Adherent` et `Emprunt` (record immuable) ; `sn.isep.biblio` contient la classe `Bibliotheque`, qui orchestre les opérations. Chaque règle est vérifiée dans la classe qui détient l'information : le livre sait s'il lui reste des exemplaires, l'adhérent sait combien d'emprunts il a. Pour que le corrigé tienne en un seul fichier testable, les classes sont ici imbriquées.

Bibliotheque.java

```
import java.util.ArrayList;
```

```
import java.util.List;

public class Bibliotheque {
    record Emprunt(Livre livre, Adherent adherent, int jourEmprunt) {
        static final int DUREE = 14;
        int jourRetourPrevu() { return jourEmprunt + DUREE; }
        long penalite(int jourRetour)
                { return Math.max(0, jourRetour - jourRetourPrevu()) * 100L; }
    }

    static class Livre {
        private final String isbn, titre, auteur;
        private int disponibles;
        Livre(String isbn, String titre, String auteur, int exemplaires) {
            if (exemplaires < 1) throw new IllegalArgumentException("Au moins un exemplaire");
            this.isbn = isbn;
            this.titre = titre;
            this.auteur = auteur;
            this.disponibles = exemplaires;
        }
        void sortir() {
            if (disponibles == 0)
                    throw new IllegalStateException("Plus d'exemplaire de « " + titre + " »");
            disponibles--;
        }
        void rentrer() { disponibles++; }
        String getTitre() { return titre; }
    }

    static class Adherent {
        static final int MAX_EMPRUNTS = 3;
        private final int numero;
        private final String nom;
        private final List<Emprunt> enCours = new ArrayList<>();
        Adherent(int numero, String nom) { this.numero = numero; this.nom = nom; }
        void ajouter(Emprunt e) {
            if (enCours.size() >= MAX_EMPRUNTS)
                    throw new IllegalStateException(nom + " a déjà 3 emprunts");
            enCours.add(e);
        }
        Emprunt retirer(Livre livre) {
            for (Emprunt e : enCours) {
                if (e.livre() == livre) { enCours.remove(e); return e; }
            }
            throw new IllegalStateException(nom + " n'a pas emprunté « "
                    + livre.getTitre() + " »");
        }
    }

    public void emprunter(Adherent a, Livre l, int jour) {
        if (a.enCours.size() >= Adherent.MAX_EMPRUNTS)
                throw new IllegalStateException(a.nom + " a déjà 3 emprunts");
        // peut échouer : on n'a encore rien modifié d'autre
        l.sortir();
        a.ajouter(new Emprunt(l, a, jour));
    }

    public long rendre(Adherent a, Livre l, int jour) {
        Emprunt e = a.retirer(l);
        l.rentrer();
        return e.penalite(jour);
    }

    public static void main(String[] args) {
        Bibliotheque bib = new Bibliotheque();
        Livre java = new Livre("978-2", "Java moderne", "A. Mbaye", 1);
        Livre sql = new Livre("978-3", "PostgreSQL", "F. Sow", 2);
        Adherent awa = new Adherent(1, "Awa"), ibou = new Adherent(2, "Ibou");
        bib.emprunter(awa, java, 1);
        bib.emprunter(awa, sql, 1);
        try {
```

```
            bib.emprunter(ibou, java, 2);
        } catch (IllegalStateException e) {
            System.out.println("Refusé : " + e.getMessage());
        }
        System.out.println("Pénalité Java : " + bib.rendre(awa, java, 20) + " FCFA");
        System.out.println("Pénalité SQL  : " + bib.rendre(awa, sql, 10) + " FCFA");
        bib.emprunter(ibou, java, 21);
        System.out.println("Ibou a maintenant « " + ibou.enCours.getFirst().livre()
                .getTitre() + " »");
    }
}
```

Sortie console

```
Refusé : Plus d'exemplaire de « Java moderne »
Pénalité Java : 500 FCFA
Pénalité SQL  : 0 FCFA
Ibou a maintenant « Java moderne »
```

Dans un vrai projet, chaque classe serait publique et placée dans son propre fichier, les jours seraient des `LocalDate` (chapitre 13) et les règles seraient levées sous forme d'exceptions métier (chapitre 9).

Corrigés du chapitre 7

Corrigé 7.1 — Les employés de TerangaShop

Personnel.java

```
public class Personnel {
    static abstract class Employe {
        private final String matricule, nom;
        protected final long salaireBase;
        Employe(String matricule, String nom, long salaireBase) {
            this.matricule = matricule; this.nom = nom; this.salaireBase = salaireBase;
        }
        abstract long salaireMensuel();
        @Override public String toString() { return matricule + " " + nom; }
    }

    static class Vendeur extends Employe {
        private final long chiffreAffaires;
        Vendeur(String m, String n, long base, long ca)
                { super(m, n, base); this.chiffreAffaires = ca; }
        @Override long salaireMensuel() { return salaireBase + chiffreAffaires * 3 / 100; }
        @Override public String toString() { return super.toString() + " (vendeur, CA "
                + chiffreAffaires + ")"; }
    }

    static class Livreur extends Employe {
        private final int livraisons;
        Livreur(String m, String n, long base, int livraisons)
                { super(m, n, base); this.livraisons = livraisons; }
        @Override long salaireMensuel() { return salaireBase + 500L * livraisons; }
        @Override public String toString() { return super.toString() + " (livreur, "
                + livraisons + " livraisons)"; }
    }

    static class Gerant extends Employe {
        Gerant(String m, String n, long base) { super(m, n, base); }
        @Override long salaireMensuel() { return salaireBase + 100_000; }
        @Override public String toString() { return super.toString() + " (gérant)"; }
    }

    public static void main(String[] args) {
        Employe[] equipe = {
            new Gerant("E01", "Mariama Ba", 350_000),
            new Vendeur("E02", "Omar Sy", 150_000, 4_200_000),
```

```
            new Livreur("E03", "Modou Gueye", 100_000, 180),
            new Livreur("E04", "Cheikh Ndiaye", 100_000, 145)
        };
        long masse = 0;
        for (Employe e : equipe) {
            System.out.printf("%-44s %8d FCFA%n", e, e.salaireMensuel());
            masse += e.salaireMensuel();
        }
        System.out.println("Masse salariale : " + masse + " FCFA");
    }
}
```

Sortie console

```
E01 Mariama Ba (gérant)                      450000 FCFA
E02 Omar Sy (vendeur, CA 4200000)            276000 FCFA
E03 Modou Gueye (livreur, 180 livraisons)    190000 FCFA
E04 Cheikh Ndiaye (livreur, 145 livraisons)  172500 FCFA
Masse salariale : 1088500 FCFA
```

Corrigé 7.2 — Formes géométriques

Formes.java

```
public class Formes {
    static abstract class Forme {
        abstract double aire();
        abstract double perimetre();
        String decrire() {
            return "%s : aire %.2f, périmètre %.2f".formatted(getClass().getSimpleName(),
                    aire(), perimetre());
        }
    }
    static class Cercle extends Forme {
        private final double r;
        Cercle(double r) { this.r = r; }
        double aire() { return Math.PI * r * r; }
        double perimetre() { return 2 * Math.PI * r; }
    }
    static class Rectangle extends Forme {
        private final double l, h;
        Rectangle(double l, double h) { this.l = l; this.h = h; }
        double aire() { return l * h; }
        double perimetre() { return 2 * (l + h); }
    }
    static class Triangle extends Forme {
        private final double a, b, c;
        Triangle(double a, double b, double c) {
            if (a + b <= c || a + c <= b || b + c <= a)
                    throw new IllegalArgumentException("Triangle impossible");
            this.a = a; this.b = b; this.c = c;
        }
        double perimetre() { return a + b + c; }
        double aire() {                                  // formule de Héron
            double p = perimetre() / 2;
            return Math.sqrt(p * (p - a) * (p - b) * (p - c));
        }
    }

    public static void main(String[] args) {
        Forme[] formes = {new Cercle(1), new Rectangle(3, 4), new Triangle(3, 4, 5)};
        for (Forme f : formes) System.out.println(f.decrire());
        try { new Triangle(1, 2, 10); } catch (IllegalArgumentException e)
                { System.out.println(e.getMessage()); }
    }
}
```

Sortie console

```
Cercle : aire 3.14, périmètre 6.28
Rectangle : aire 12.00, périmètre 14.00
Triangle : aire 6.00, périmètre 12.00
Triangle impossible
```

Question de réflexion : si `Rectangle` possède `setLargeur` et `setHauteur`, un `Carre` qui en hérite doit les redéfinir pour modifier les deux côtés à la fois. Un code écrit pour des rectangles (« je fixe la largeur à 5, la hauteur à 4, j'attends une aire de 20 ») donnera alors 16 avec un carré : le carré ne peut pas **remplacer** un rectangle sans changer le comportement attendu. C'est une violation du **principe de substitution de Liskov** (le « L » de SOLID). Solutions : formes **immuables** (comme ci-dessus), ou pas d'héritage entre `Carre` et `Rectangle`.

Corrigé 7.3 — Interfaces multiples et méthodes par défaut

ArticleSolde.java

```java
interface Taxable {
    long montantHt();
    default long tva() { return Math.round(montantHt() * 0.18); }
    default String description() { return "article taxable"; }
}

interface Remisable {
    int tauxRemise();
    default long remise(long montant) { return montant * tauxRemise() / 100; }
    default String description() { return "article remisable"; }
}

public class ArticleSolde implements Taxable, Remisable {
    private final long prix;
    private final int taux;
    ArticleSolde(long prix, int taux) { this.prix = prix; this.taux = taux; }

    @Override public long montantHt() { return prix - remise(prix); }
    @Override public int tauxRemise() { return taux; }

    // obligatoire : conflit entre deux défauts
    @Override
    public String description() {
        return Taxable.super.description() + " et " + Remisable.super.description();
    }

    public static void main(String[] args) {
        ArticleSolde a = new ArticleSolde(10_000, 20);
        System.out.println(a.description() + " : HT " + a.montantHt() + ", TVA " + a.tva());
    }
}
```

Sortie console

```
article taxable et article remisable : HT 8000, TVA 1440
```

Sans la redéfinition de `description()`, la compilation échoue : *class ArticleSolde inherits unrelated defaults for description() from types Taxable and Remisable*. Java refuse de choisir à votre place ; la syntaxe `Interface.super.methode()` permet d'appeler explicitement l'une ou l'autre version.

Corrigé 7.4 — Prédire l'affichage

Quiz.java

```
class A {
    String nom = "A";
    A() { afficher(); }
    void afficher() { System.out.println("A.afficher " + nom); }
    static void statique() { System.out.println("A.statique"); }
}
class B extends A {
    String nom = "B";
    B() { super(); afficher(); }
    @Override void afficher() { System.out.println("B.afficher " + nom); }
    static void statique() { System.out.println("B.statique"); }
}
public class Quiz {
    public static void main(String[] args) {
        A a = new B();
        System.out.println(a.nom);
        a.statique();
        ((B) a).statique();
    }
}
```

Sortie console

```
B.afficher null
B.afficher B
A
A.statique
B.statique
```

1. Le constructeur de `A` appelle `afficher()` : la liaison dynamique choisit **`B.afficher`**, alors que l'attribut `nom` de `B` n'est pas encore initialisé (les attributs de la sous-classe sont initialisés **après** le retour de `super()`) : d'où `null`. C'est pourquoi il ne faut jamais appeler une méthode redéfinissable dans un constructeur.
2. Le constructeur de `B` appelle ensuite `afficher()` : `nom` vaut maintenant `"B"`.
3. Les **attributs** ne sont pas polymorphes : `a.nom` utilise le type déclaré de `a` (`A`). `B` possède deux attributs `nom` ; celui de `B` *masque* celui de `A`.
4. Les méthodes `static` non plus : `a.statique()` est résolu à la compilation selon le type déclaré (`A`). On parle de **masquage**, pas de redéfinition (d'où l'absence de `@Override`, qui serait refusé).

Corrigé 7.5 — Builder pour Produit

ProduitBuilder.java

```
public class ProduitBuilder {
    record Produit(String reference, String libelle, long prix, int stock, String categorie,
            String description) { }

    static Builder produit(String reference, String libelle)
            { return new Builder(reference, libelle); }

    static class Builder {
        private final String reference, libelle;
        private long prix = 1_000;
        private int stock = 0;
        private String categorie = "Divers";
        private String description = "";

        private Builder(String reference, String libelle)
                { this.reference = reference; this.libelle = libelle; }
```

```
        Builder prix(long p) { this.prix = p; return this; }
        Builder stock(int s) { this.stock = s; return this; }
        Builder categorie(String c) { this.categorie = c; return this; }
        Builder description(String d) { this.description = d; return this; }

        Produit construire() {                          // toute la validation au même endroit
            if (reference == null || !reference.matches("PRD-\\d{3}"))
                throw new IllegalArgumentException("Référence invalide : " + reference);
            if (libelle == null || libelle.isBlank())
                    throw new IllegalArgumentException("Libellé obligatoire");
            if (prix <= 0) throw new IllegalArgumentException("Prix invalide");
            if (stock < 0) throw new IllegalArgumentException("Stock invalide");
            return new Produit(reference, libelle.strip(), prix, stock, categorie, description);
        }
    }

    public static void main(String[] args) {
        System.out.println(produit("PRD-001", "Bissap 1 L").prix(1_500).stock(40)
                .categorie("Boisson").construire());
        System.out.println(produit("PRD-002", "Thiakry").construire());
        try {
            produit("PRD-3", "Erreur").construire();
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Sortie console

```
Produit[reference=PRD-001, libelle=Bissap 1 L, prix=1500, stock=40, categorie=Boisson, description=]
Produit[reference=PRD-002, libelle=Thiakry, prix=1000, stock=0, categorie=Divers, description=]
Référence invalide : PRD-3
```

Corrigé 7.6 — Notifications multicanal

Multicanal.java

```
import java.util.List;

public class Multicanal {
    interface Notificateur {
        void envoyer(String destinataire, String message);
    }

    static class NotificateurSms implements Notificateur {
        public void envoyer(String dest, String msg) {
            String court = msg.length() > 160 ? msg.substring(0, 157) + "..." : msg;
            System.out.println("[SMS " + dest + "] " + court);
        }
    }
    static class NotificateurEmail implements Notificateur {
        public void envoyer(String dest, String msg) {
            System.out.println("[E-mail " + dest + "] Objet : TerangaShop | " + msg
                    + " | -- L'équipe TerangaShop");
        }
    }
    static class NotificateurWhatsApp implements Notificateur {
        public void envoyer(String dest, String msg) {
            System.out.println("[WhatsApp " + dest + "] " + msg);
        }
    }

    /** Composite : se comporte comme UN notificateur, mais délègue à plusieurs. */
    static class NotificateurMultiple implements Notificateur {
        private final List<Notificateur> canaux;
        NotificateurMultiple(Notificateur... canaux) { this.canaux = List.of(canaux); }
        public void envoyer(String dest, String msg) {
            for (Notificateur n : canaux) n.envoyer(dest, msg);
```

```
        }
    }

    static class ServiceCommande {
        private final Notificateur notificateur;
        // injection
        ServiceCommande(Notificateur notificateur) { this.notificateur = notificateur; }
        void valider(String numero, String client) {
            // ... règles métier, enregistrement ...
            notificateur.envoyer(client, "Votre commande " + numero + " est validée.");
        }
    }

    public static void main(String[] args) {
        new ServiceCommande(new NotificateurSms()).valider("TS-001", "771234567");
        new ServiceCommande(new NotificateurMultiple(new NotificateurSms(),
                new NotificateurEmail(),
                new NotificateurWhatsApp())).valider("TS-002", "awa@exemple.sn");
    }
}
```

Sortie console

```
[SMS 771234567] Votre commande TS-001 est validée.
[SMS awa@exemple.sn] Votre commande TS-002 est validée.
[E-mail awa@exemple.sn] Objet : TerangaShop | Votre commande TS-002 est validée. | -- L'équipe TerangaShop
[WhatsApp awa@exemple.sn] Votre commande TS-002 est validée.
```

`ServiceCommande` n'a pas changé d'une ligne entre les deux utilisations : il dépend de l'**abstraction** `Notificateur`, pas des implémentations. Dans un vrai système, chaque canal utiliserait naturellement l'adresse adaptée (téléphone ou e-mail) du client.

Corrigés du chapitre 8

Corrigé 8.1 — Records de base

RecordsBase.java

```
public class RecordsBase {
    record Montant(long valeur, String devise) {
        Montant {
            if (valeur < 0) throw new IllegalArgumentException("Montant négatif");
            devise = devise.strip().toUpperCase();
        }
        Montant plus(Montant autre) {
            if (!devise.equals(autre.devise)) throw new IllegalArgumentException(devise
                    + " ≠ " + autre.devise);
            return new Montant(valeur + autre.valeur, devise);
        }
        Montant fois(int n) { return new Montant(valeur * n, devise); }
    }

    record Periode(int anneeDebut, int anneeFin) {
        Periode {
            if (anneeFin < anneeDebut) throw new IllegalArgumentException("Période invalide");
        }
        int duree() { return anneeFin - anneeDebut; }
        boolean contient(int annee) { return annee >= anneeDebut && annee <= anneeFin; }
    }

    public static void main(String[] args) {
        Montant m = new Montant(1_500, "xof");
        System.out.println(m.fois(3).plus(new Montant(500, " XOF ")));
        System.out.println(m.equals(new Montant(1_500, "XOF")) + " "
                + (m.hashCode() == new Montant(1_500, "XOF").hashCode()));
        try { m.plus(new Montant(10, "EUR")); } catch (IllegalArgumentException e)
                { System.out.println(e.getMessage()); }
```

```
        Periode p = new Periode(2024, 2027);
        System.out.println(p + " durée=" + p.duree() + " contient 2026 ? " + p.contient(2026));
    }
}
```

Sortie console

```
Montant[valeur=5000, devise=XOF]
true true
XOF ≠ EUR
Periode[anneeDebut=2024, anneeFin=2027] durée=3 contient 2026 ? true
```

Corrigé 8.2 — Enum Categorie

Categories.java

```
import java.util.EnumMap;
import java.util.List;
import java.util.Map;

public class Categories {
    enum Categorie {
        ALIMENTAIRE("Alimentaire", 0), TEXTILE("Textile", 18), ARTISANAT("Artisanat", 18),
                COSMETIQUE("Cosmétique", 18);

        private final String libelle;
        private final int tauxTva;
        Categorie(String libelle, int tauxTva)
                { this.libelle = libelle; this.tauxTva = tauxTva; }
        long prixTtc(long prixHt) { return prixHt + Math.round(prixHt * tauxTva / 100.0); }
        String libelle() { return libelle; }
    }

    record Vente(String produit, Categorie categorie, long montantHt) { }

    public static void main(String[] args) {
        List<Vente> ventes = List.of(
            new Vente("Riz local 5 kg", Categorie.ALIMENTAIRE, 4_500),
            new Vente("Pagne wax", Categorie.TEXTILE, 7_500),
            new Vente("Sac cuir", Categorie.ARTISANAT, 25_000),
            new Vente("Thiakry", Categorie.ALIMENTAIRE, 1_000),
            new Vente("Boubou", Categorie.TEXTILE, 35_000)
        );
        Map<Categorie, Long> caTtc = new EnumMap<>(Categorie.class);   // ordre des constantes
        for (Vente v : ventes) {
            caTtc.merge(v.categorie(), v.categorie().prixTtc(v.montantHt()), Long::sum);
        }
        caTtc.forEach((c, total) -> System.out.println(c.libelle() + " : "
                + total + " FCFA TTC"));
    }
}
```

Sortie console

```
Alimentaire : 5500 FCFA TTC
Textile : 50150 FCFA TTC
Artisanat : 29500 FCFA TTC
```

Corrigé 8.3 — Refactoriser avec le pattern matching

Aires.java

```
public class Aires {
    sealed interface Forme permits Cercle, Rectangle, Triangle { }
    record Cercle(double rayon) implements Forme { }
    record Rectangle(double largeur, double hauteur) implements Forme { }
    record Triangle(double base, double hauteur) implements Forme { }
```

```
    static double aire(Forme forme) {
        return switch (forme) {
            case Cercle(var r) -> Math.PI * r * r;
            case Rectangle(var l, var h) when l == h -> {
                System.out.println("(c'est un carré)");
                yield l * h;
            }
            case Rectangle(var l, var h) -> l * h;
            case Triangle(var b, var h) -> b * h / 2;
        // exhaustif : plus besoin du cas « forme inconnue »
        };
    }

    public static void main(String[] args) {
        System.out.printf("%.2f%n", aire(new Cercle(2)));
        System.out.printf("%.2f%n", aire(new Rectangle(3, 3)));
        System.out.printf("%.2f%n", aire(new Triangle(4, 5)));
    }
}
```

Sortie console

```
12.57
(c'est un carré)
9.00
10.00
```

Corrigé 8.4 — Machine à états des livraisons

SuiviLivraison.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Set;

public class SuiviLivraison {
    enum EtapeLivraison {
        PREPARATION, EN_ROUTE, LIVREE, ECHEC, RETOUR_ENTREPOT;

        Set<EtapeLivraison> suivantes() {
            return switch (this) {
                case PREPARATION -> Set.of(EN_ROUTE);
                case EN_ROUTE -> Set.of(LIVREE, ECHEC);
                case ECHEC -> Set.of(EN_ROUTE, RETOUR_ENTREPOT);
                case LIVREE, RETOUR_ENTREPOT -> Set.of();
            };
        }
    }

    private final List<EtapeLivraison> historique =
            new ArrayList<>(List.of(EtapeLivraison.PREPARATION));
    private int echecs = 0;

    public void passerA(EtapeLivraison suivante) {
        EtapeLivraison actuelle = historique.getLast();
        if (!actuelle.suivantes().contains(suivante)) {
            throw new IllegalStateException("Transition interdite : " + actuelle
                    + " -> " + suivante);
        }
        if (actuelle == EtapeLivraison.ECHEC
                && suivante == EtapeLivraison.EN_ROUTE && echecs >= 2) {
            throw new IllegalStateException("Deux échecs : retour à l'entrepôt obligatoire");
        }
        if (suivante == EtapeLivraison.ECHEC) echecs++;
        historique.add(suivante);
    }
```

```
    public static void main(String[] args) {
        SuiviLivraison s = new SuiviLivraison();
        s.passerA(EtapeLivraison.EN_ROUTE);
        s.passerA(EtapeLivraison.ECHEC);
        s.passerA(EtapeLivraison.EN_ROUTE);
        s.passerA(EtapeLivraison.ECHEC);
        try {
            s.passerA(EtapeLivraison.EN_ROUTE);
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
        s.passerA(EtapeLivraison.RETOUR_ENTREPOT);
        System.out.println(s.historique);
    }
}
```

Sortie console

```
Deux échecs : retour à l'entrepôt obligatoire
[PREPARATION, EN_ROUTE, ECHEC, EN_ROUTE, ECHEC, RETOUR_ENTREPOT]
```

Corrigé 8.5 — Évaluateur d'expressions

Expressions.java

```
public class Expressions {
    sealed interface Expr permits Nombre, Addition, Multiplication, Negation { }
    record Nombre(double v) implements Expr { }
    record Addition(Expr g, Expr d) implements Expr { }
    record Multiplication(Expr g, Expr d) implements Expr { }
    record Negation(Expr e) implements Expr { }

    static double evaluer(Expr e) {
        return switch (e) {
            case Nombre(var v) -> v;
            case Addition(var g, var d) -> evaluer(g) + evaluer(d);
            case Multiplication(var g, var d) -> evaluer(g) * evaluer(d);
            case Negation(var x) -> -evaluer(x);
        };
    }

    static String afficher(Expr e) {
        return switch (e) {
            case Nombre(var v) -> v == Math.floor(v) ? String.valueOf((long) v)
                    : String.valueOf(v);
            case Addition(var g, var d) -> "(" + afficher(g) + " + " + afficher(d) + ")";
            case Multiplication(var g, var d) -> afficher(g) + " * " + afficher(d);
            case Negation(var x) -> "-" + afficher(x);
        };
    }

    static Expr simplifier(Expr e) {
        return switch (e) {
            case Nombre n -> n;
            case Negation(Negation(var x)) -> simplifier(x);                    // -(-x) = x
            case Negation(var x) -> new Negation(simplifier(x));
            case Addition(var g, var d) -> {
                Expr sg = simplifier(g), sd = simplifier(d);
                if (sd instanceof Nombre(var v) && v == 0) yield sg;            // x + 0 = x
                if (sg instanceof Nombre(var v) && v == 0) yield sd;            // 0 + x = x
                yield new Addition(sg, sd);
            }
            case Multiplication(var g, var d) -> {
                Expr sg = simplifier(g), sd = simplifier(d);
                if (sg instanceof Nombre(var v) && v == 0 || sd instanceof Nombre(var w)
                        && w == 0) yield new Nombre(0);
                if (sd instanceof Nombre(var v) && v == 1) yield sg;            // x * 1 = x
                if (sg instanceof Nombre(var v) && v == 1) yield sd;
                yield new Multiplication(sg, sd);
            }
```

```
        };
    }

    public static void main(String[] args) {
        Expr e = new Multiplication(new Addition(new Nombre(2), new Nombre(3)),
                new Negation(new Negation(new Nombre(4))));
        System.out.println(afficher(e) + " = " + evaluer(e));
        Expr s = simplifier(e);
        System.out.println("Simplifiée : " + afficher(s) + " = " + evaluer(s));
        Expr f = new Addition(new Multiplication(new Nombre(7), new Nombre(1)), new Nombre(0));
        System.out.println(afficher(f) + "  ->  " + afficher(simplifier(f)));
    }
}
```

Sortie console

```
(2 + 3) * --4 = 20.0
Simplifiée : (2 + 3) * 4 = 20.0
(7 * 1 + 0)  ->  7
```

Le motif imbriqué `Negation(Negation(var x))` montre toute la puissance des *record patterns* : il reconnaît une forme **à plusieurs niveaux** en une seule ligne.

Corrigé 8.6 — Moteur de promotions

Promotions.java

```
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

public class Promotions {
    static final long FRAIS_LIVRAISON = 1_500;

    record Ligne(String reference, long prixUnitaire, int quantite) {
        long sousTotal() { return prixUnitaire * quantite; }
    }
    record Panier(String quartier, List<Ligne> lignes) {
        long total() { return lignes.stream().mapToLong(Ligne::sousTotal).sum(); }
    }

    sealed interface Promotion permits PourcentageSurTotal, RemiseFixe, ArticleOffert,
            LivraisonOfferte { }
    record PourcentageSurTotal(int taux, long minimumAchat) implements Promotion { }
    record RemiseFixe(long montant, long minimumAchat) implements Promotion { }
    record ArticleOffert(String reference, int quantiteMinimum) implements Promotion { }
    record LivraisonOfferte(String quartier) implements Promotion { }

    static long economie(Promotion p, Panier panier) {
        long total = panier.total();
        return switch (p) {
            case PourcentageSurTotal(var taux, var min) when total >= min -> total * taux / 100;
            case RemiseFixe(var montant, var min) when total >= min -> Math.min(montant, total);
            case ArticleOffert(var ref, var qMin) -> panier.lignes().stream()
                    .filter(l -> l.reference().equals(ref) && l.quantite() >= qMin)
                    .mapToLong(Ligne::prixUnitaire)
                    .findFirst().orElse(0);
            case LivraisonOfferte(var q) when q.equalsIgnoreCase(panier.quartier())
                    -> FRAIS_LIVRAISON;
            // conditions non remplies
            case PourcentageSurTotal _, RemiseFixe _, LivraisonOfferte _ -> 0;
        };
    }

    static Optional<Promotion> meilleure(List<Promotion> promotions, Panier panier) {
        return promotions.stream()
                .filter(p -> economie(p, panier) > 0)
                .max(Comparator.comparingLong(p -> economie(p, panier)));
```

```
    }

    public static void main(String[] args) {
        Panier panier = new Panier("Médina", List.of(
                new Ligne("PRD-001", 1_500, 6), new Ligne("PRD-014", 7_500, 2)));
        List<Promotion> promos = List.of(
                new PourcentageSurTotal(10, 30_000), new RemiseFixe(2_000, 20_000),
                new ArticleOffert("PRD-001", 5), new LivraisonOfferte("Médina"));
        System.out.println("Total du panier : " + panier.total());
        promos.forEach(p -> System.out.println(p + " -> " + economie(p, panier)));
        System.out.println("Meilleure : " + meilleure(promos, panier).orElseThrow());
    }
}
```

Sortie console

```
Total du panier : 24000
PourcentageSurTotal[taux=10, minimumAchat=30000] -> 0
RemiseFixe[montant=2000, minimumAchat=20000] -> 2000
ArticleOffert[reference=PRD-001, quantiteMinimum=5] -> 1500
LivraisonOfferte[quartier=Médina] -> 1500
Meilleure : RemiseFixe[montant=2000, minimumAchat=20000]
```

Le dernier `case` regroupe plusieurs motifs de type avec des variables anonymes `_` : c'est ce qui rend le `switch` exhaustif lorsque les gardes `when` ne sont pas satisfaites.

Corrigés du chapitre 9

Corrigé 9.1 — Suivre le flux d'exécution

FluxFinally.java

```
public class FluxFinally {
    @SuppressWarnings("finally")
    static int test(int cas) {
        try {
            System.out.println("try");
            if (cas == 1) throw new IllegalStateException("cas 1");
            if (cas == 2) return 20;
            return 10;
        } catch (IllegalStateException e) {
            System.out.println("catch " + e.getMessage());
            return 30;
        } finally {
            System.out.println("finally");
            if (cas == 2) return 99;
        }
    }

    public static void main(String[] args) {
        for (int cas = 0; cas <= 2; cas++) {
            System.out.println("-> test(" + cas + ") = " + test(cas));
        }
    }
}
```

Sortie console

```
try
finally
-> test(0) = 10
try
catch cas 1
finally
-> test(1) = 30
try
finally
-> test(2) = 99
```

Le bloc `finally` s'exécute toujours, **après** l'évaluation de la valeur du `return` mais **avant** que la méthode ne rende la main. Un `return` placé dans `finally` **remplace** la valeur déjà calculée (et ferait même disparaître une exception en cours) : c'est une très mauvaise pratique, que le compilateur signale d'ailleurs par un avertissement.

Corrigé 9.2 — Saisie contrôlée

SaisieControlee.java

```java
import java.util.Scanner;

public class SaisieControlee {
    static int lireEntier(Scanner sc, String invite, int min, int max) {
        while (true) {
            System.out.print(invite);
            String ligne = sc.nextLine().strip();
            try {
                int valeur = Integer.parseInt(ligne);
                if (valeur >= min && valeur <= max) return valeur;
                System.out.println("  La valeur doit être comprise entre "
                        + min + " et " + max);
            } catch (NumberFormatException e) {
                System.out.println("  « " + ligne + " » n'est pas un nombre entier");
            }
        }
    }

    public static void main(String[] args) {
        int q = lireEntier(new Scanner(System.in), "Quantité (1-100) : ", 1, 100);
        System.out.println("Quantité retenue : " + q);
    }
}
```

Sortie console

```
Quantité (1-100) :   « abc » n'est pas un nombre entier
Quantité (1-100) :   La valeur doit être comprise entre 1 et 100
Quantité (1-100) : Quantité retenue : 12
```

Corrigé 9.3 — Exceptions du compte mobile

Version **vérifiée** : `class SoldeInsuffisantException extends Exception`. Il faut alors ajouter `throws SoldeInsuffisantException` à `retirer`, **et** à `transferer` (qui appelle `retirer`), puis intercepter l'exception dans chaque appelant, même dans ceux qui ne peuvent rien en faire. Version **non vérifiée** (ci-dessous) : les signatures restent simples et l'exception est traitée là où c'est utile.

CompteAvecExceptions.java

```java
public class CompteAvecExceptions {
    static class SoldeInsuffisantException extends RuntimeException {
        private final long solde, demande;
        SoldeInsuffisantException(long solde, long demande) {
```

```
            super("Solde insuffisant : " + solde + " FCFA disponibles, "
                    + demande + " demandés");
            this.solde = solde;
            this.demande = demande;
        }
        long manque() { return demande - solde; }
    }

    private long solde;
    void deposer(long m) { solde += m; }
    void retirer(long m) {
        if (m > solde) throw new SoldeInsuffisantException(solde, m);
        solde -= m;
    }

    public static void main(String[] args) {
        var compte = new CompteAvecExceptions();
        compte.deposer(5_000);
        try {
            compte.retirer(8_000);
        } catch (SoldeInsuffisantException e) {
            System.out.println(e.getMessage() + " (il manque " + e.manque() + " FCFA)");
        }
    }
}
```

Sortie console

```
Solde insuffisant : 5000 FCFA disponibles, 8000 demandés (il manque 3000 FCFA)
```

On peut défendre les deux choix. Un solde insuffisant est une situation **prévisible** que l'appelant devrait gérer, ce qui plaide pour une exception vérifiée dans une bibliothèque bancaire. Dans une application backend en couches, on préfère une exception **non vérifiée** : elle remonte jusqu'au gestionnaire global qui la traduit en réponse HTTP 409, sans alourdir toutes les signatures intermédiaires.

Corrigé 9.4 — Ressources et exceptions supprimées

Supprimees.java

```
public class Supprimees {
    static class FichierSimule implements AutoCloseable {
        @Override
        public void close() {
            throw new IllegalStateException("fermeture impossible");
        }
    }

    public static void main(String[] args) {
        try (FichierSimule f = new FichierSimule()) {
            throw new IllegalArgumentException("erreur de lecture");
        } catch (RuntimeException e) {
            System.out.println("Exception principale : " + e.getMessage());
            for (Throwable s : e.getSuppressed()) {
                System.out.println("Exception supprimée  : " + s.getMessage());
            }
        }
    }
}
```

Sortie console

```
Exception principale : erreur de lecture
Exception supprimée  : fermeture impossible
```

Avec un `try/finally` classique appelant `close()` dans le `finally`, l'exception de `close()` **remplacerait** l'exception principale : l'erreur de lecture, pourtant la vraie cause du problème, serait perdue. *Try-with-resources* conserve la cause principale et attache l'autre comme exception supprimée.

Corrigé 9.5 — Import tolérant aux erreurs

ImportCommandes.java

```
import java.util.ArrayList;
import java.util.List;

public class ImportCommandes {
    record Ligne(String numero, String client, int quantite, long prixUnitaire) {
        Ligne {
            if (quantite <= 0) throw new IllegalArgumentException("quantité négative ou nulle");
            if (prixUnitaire <= 0) throw new IllegalArgumentException("prix négatif ou nul");
        }
        long total() { return quantite * prixUnitaire; }
    }

    static Ligne analyser(String texte) {
        String[] c = texte.split(";");
        if (c.length != 4) throw new IllegalArgumentException(c.length
                + " champs au lieu de 4");
        int quantite;
        try {
            quantite = Integer.parseInt(c[2].strip());
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("quantité non numérique « " + c[2] + " »");
        }
        return new Ligne(c[0], c[1], quantite, Long.parseLong(c[3].strip()));
    }

    public static void main(String[] args) {
        String[] lignes = {
            "TS-101;Awa Diop;2;7500",
            "TS-102;Moussa Ndiaye;trois;1500",
            "TS-103;Fatou Sow;1",
            "TS-104;Ibrahima Fall;4;-200",
            "TS-105;Aminata Ba;1;25000"
        };
        long total = 0;
        List<String> erreurs = new ArrayList<>();
        for (int i = 0; i < lignes.length; i++) {
            try {
                total += analyser(lignes[i]).total();
            } catch (IllegalArgumentException e) {
                erreurs.add("Ligne " + (i + 1) + " : " + e.getMessage());
            }
        }
        System.out.println("Total des lignes valides : " + total + " FCFA");
        System.out.println(erreurs.size() + " erreur(s) :");
        erreurs.forEach(e -> System.out.println("  " + e));
    }
}
```

Sortie console

```
Total des lignes valides : 40000 FCFA
3 erreur(s) :
  Ligne 2 : quantité non numérique « trois »
  Ligne 3 : 3 champs au lieu de 4
  Ligne 4 : prix négatif ou nul
```

Corrigé 9.6 — Gestionnaire global d'erreurs

GestionnaireErreurs.java

```
import java.util.List;
import java.util.function.Supplier;

public class GestionnaireErreurs {
    record ErreurApi(int statut, String code, String message) {
        String json() {
            return """
                {"statut": %d, "code": "%s", "message": "%s"}""".formatted(statut, code, message);
        }
    }

    static abstract class TerangaShopException extends RuntimeException {
        TerangaShopException(String m) { super(m); }
    }
    static class ProduitIntrouvableException extends TerangaShopException {
        ProduitIntrouvableException(String ref) { super("Aucun produit " + ref); }
    }
    static class StockInsuffisantException extends TerangaShopException {
        StockInsuffisantException(String ref) { super("Stock insuffisant pour " + ref); }
    }
    static class PaiementRefuseException extends TerangaShopException {
        PaiementRefuseException(String m) { super("Paiement refusé : " + m); }
    }

    static ErreurApi traduire(Throwable t) {
        return switch (t) {
            case ProduitIntrouvableException e ->
                    new ErreurApi(404, "PRODUIT_INTROUVABLE", e.getMessage());
            case StockInsuffisantException e ->
                    new ErreurApi(409, "STOCK_INSUFFISANT", e.getMessage());
            case PaiementRefuseException e ->
                    new ErreurApi(502, "PAIEMENT_REFUSE", e.getMessage());
            case IllegalArgumentException e ->
                    new ErreurApi(400, "REQUETE_INVALIDE", e.getMessage());
            default -> new ErreurApi(500, "ERREUR_INTERNE", "Une erreur inattendue est survenue");
        };
    }

    static String controleur(Supplier<String> operation) {
        try {
            return "200 " + operation.get();
        } catch (RuntimeException e) {
            ErreurApi erreur = traduire(e);
            return erreur.statut() + " " + erreur.json();
        }
    }

    public static void main(String[] args) {
        List<Supplier<String>> operations = List.of(
            () -> "{\"reference\": \"PRD-001\"}",
            () -> { throw new ProduitIntrouvableException("PRD-999"); },
            () -> { throw new StockInsuffisantException("PRD-014"); },
            () -> { throw new IllegalArgumentException("quantité invalide : -2"); },
            () -> { throw new NullPointerException("champ client nul dans "
                    + "ServiceCommande.java:42"); }
        );
        operations.forEach(op -> System.out.println(controleur(op)));
    }
}
```

Sortie console

```
200 {"reference": "PRD-001"}
404 {"statut": 404, "code": "PRODUIT_INTROUVABLE", "message": "Aucun produit PRD-999"}
409 {"statut": 409, "code": "STOCK_INSUFFISANT", "message": "Stock insuffisant pour PRD-014"}
400 {"statut": 400, "code": "REQUETE_INVALIDE", "message": "quantité invalide : -2"}
500 {"statut": 500, "code": "ERREUR_INTERNE", "message": "Une erreur inattendue est survenue"}
```

Pour la dernière opération, le message technique (nom de classe, numéro de ligne) n'est **pas** transmis au client : il aiderait un attaquant. Il doit en revanche être journalisé côté serveur, avec la trace complète.

Corrigés du chapitre 10

Corrigé 10.1 — Méthodes génériques simples

GeneriquesSimples.java

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Objects;

public class GeneriquesSimples {
    static <T> int compter(T[] tableau, T valeur) {
        int n = 0;
        for (T t : tableau) if (Objects.equals(t, valeur)) n++;
        return n;
    }
    static <T> void echanger(T[] tableau, int i, int j) {
        T tmp = tableau[i];
        tableau[i] = tableau[j];
        tableau[j] = tmp;
    }
    static <T> List<T> repeter(T valeur, int n) {
        List<T> r = new ArrayList<>();
        for (int i = 0; i < n; i++) r.add(valeur);
        return r;
    }

    public static void main(String[] args) {
        String[] zones = {"Médina", "Fann", "Médina", null};
        Integer[] notes = {12, 15, 12, 12};
        System.out.println(compter(zones, "Médina") + " " + compter(zones, null) + " "
                + compter(notes, 12));
        echanger(zones, 0, 1);
        System.out.println(Arrays.toString(zones));
        System.out.println(repeter("TS", 3) + " " + repeter(0, 4));
    }
}
```

Sortie console

```
2 1 3
[Fann, Médina, Médina, null]
[TS, TS, TS] [0, 0, 0, 0]
```

Corrigé 10.2 — Minimum et maximum

MinMax.java

```java
import java.util.List;

public class MinMax {
    record Paire<A, B>(A premier, B second) { }

    static <T extends Comparable<? super T>> Paire<T, T> minMax(List<T> liste) {
```

```
        if (liste.isEmpty()) throw new IllegalArgumentException("Liste vide");
        T min = liste.getFirst(), max = liste.getFirst();
        for (T e : liste) {
            if (e.compareTo(min) < 0) min = e;
            if (e.compareTo(max) > 0) max = e;
        }
        return new Paire<>(min, max);
    }

    static class Produit implements Comparable<Produit> {
        final long prix;
        Produit(long prix) { this.prix = prix; }
        public int compareTo(Produit o) { return Long.compare(prix, o.prix); }
        public String toString() { return getClass().getSimpleName() + "(" + prix + ")"; }
    }
    // Comparable<Produit>, pas Comparable<ProduitSolde>
    static class ProduitSolde extends Produit {
        ProduitSolde(long prix) { super(prix); }
    }

    public static void main(String[] args) {
        System.out.println(minMax(List.of(7_500, 1_500, 25_000)));
        System.out.println(minMax(List.of(new ProduitSolde(900), new ProduitSolde(300))));
    }
}
```

Sortie console

```
Paire[premier=1500, second=25000]
Paire[premier=ProduitSolde(300), second=ProduitSolde(900)]
```

`ProduitSolde` hérite de `Comparable<Produit>` : il n'implémente **pas** `Comparable<ProduitSolde>`. Avec la borne `T extends Comparable<T>`, l'appel `minMax(List<ProduitSolde>)` serait refusé. La borne `Comparable<? super T>` accepte tout type qui sait se comparer à un de ses **super-types** : c'est pourquoi l'API standard l'utilise (par exemple dans `Collections.max`).

Corrigé 10.3 — Une pile générique

Pile.java

```
import java.util.Arrays;

public class Pile<E> {
    private Object[] elements = new Object[4];
    private int taille = 0;

    public void empiler(E e) {
        // doublement
        if (taille == elements.length) elements = Arrays.copyOf(elements, taille * 2);
        elements[taille++] = e;
    }

    @SuppressWarnings("unchecked")                  // sûr : on n'y a rangé que des E
    public E sommet() {
        if (estVide()) throw new IllegalStateException("Pile vide");
        return (E) elements[taille - 1];
    }

    public E depiler() {
        E e = sommet();
        elements[--taille] = null;                  // évite de retenir l'objet en mémoire
        return e;
    }

    public boolean estVide() { return taille == 0; }
    public int taille() { return taille; }
```

```
    static boolean bienParenthesee(String expr) {
        Pile<Character> pile = new Pile<>();
        for (char c : expr.toCharArray()) {
            switch (c) {
                case '(', '[', '{' -> pile.empiler(c);
                case ')', ']', '}' -> {
                    if (pile.estVide()) return false;
                    char ouvrante = pile.depiler();
                    if ("([{".indexOf(ouvrante) != ")]}".indexOf(c)) return false;
                }
                default -> { }
            }
        }
        return pile.estVide();
    }

    public static void main(String[] args) {
        for (String e : new String[]{"{[(a+b)*c]}", "(a+b]", "((a)", "f(x[2]) + {y}"}) {
            System.out.println(e + " -> " + bienParenthesee(e));
        }
    }
}
```

Sortie console

```
{[(a+b)*c]} -> true
(a+b] -> false
((a) -> false
f(x[2]) + {y} -> true
```

Corrigé 10.4 — Producteurs et consommateurs

Pecs.java

```
import java.util.ArrayList;
import java.util.List;

public class Pecs {
    static <T> void copier(List<? super T> destination, List<? extends T> source) {
        for (T element : source) destination.add(element);
    }

    public static void main(String[] args) {
        List<Integer> entiers = List.of(1, 2, 3);
        List<Number> nombres = new ArrayList<>(List.of(0.5));
        List<Object> objets = new ArrayList<>(List.of("texte"));
        copier(nombres, entiers);          // T = Integer : Number est un super-type d'Integer
        copier(objets, entiers);           // Object aussi
        System.out.println(nombres + " " + objets);
    }
}
```

Sortie console

```
[0.5, 1, 2, 3] [texte, 1, 2, 3]
```

Une `List<? extends Number>` peut désigner une `List<Integer>`, une `List<Double>`… Le compilateur ne sait pas laquelle : ajouter un `Double` pourrait corrompre une liste d'`Integer`, il refuse donc **tout** ajout (sauf `null`). En revanche, tout ce qu'on en lit est au moins un `Number`.

Corrigé 10.5 — Un cache à capacité limitée

La classe `LinkedHashMap` conserve l'ordre d'insertion et offre une méthode `removeEldestEntry`, appelée après chaque ajout, qu'il suffit de redéfinir pour supprimer l'entrée la plus ancienne.

Cache.java

```java
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Optional;

public class Cache<K, V> {
    private final Map<K, V> donnees;
    private long hits, miss;

    public Cache(int capacite) {
        this.donnees = new LinkedHashMap<>() {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> plusAncienne) {
                return size() > capacite;
            }
        };
    }

    public void put(K cle, V valeur) { donnees.put(cle, valeur); }

    public Optional<V> get(K cle) {
        V v = donnees.get(cle);
        if (v == null) miss++; else hits++;
        return Optional.ofNullable(v);
    }

    public double tauxSucces() {
        long total = hits + miss;
        return total == 0 ? 0 : 100.0 * hits / total;
    }

    public static void main(String[] args) {
        Cache<String, Long> prix = new Cache<>(2);
        prix.put("PRD-001", 1_500L);
        prix.put("PRD-014", 7_500L);
        prix.put("PRD-020", 25_000L);                  // PRD-001 est évincé
        System.out.println(prix.get("PRD-001").orElse(-1L) + " "
                + prix.get("PRD-014").orElse(-1L)
                           + " " + prix.get("PRD-020").orElse(-1L));
        System.out.printf("Taux de succès : %.1f %%%n", prix.tauxSucces());
    }
}
```

Sortie console

```
-1 7500 25000
Taux de succès : 66.7 %
```

Avec `new LinkedHashMap<>(16, 0.75f, true)`, l'ordre devient celui des **accès** : on obtient un cache **LRU** (*Least Recently Used*), qui évince l'entrée la moins récemment utilisée.

Corrigé 10.6 — Un type Résultat

ResultatDemo.java

```java
import java.util.function.Function;

public class ResultatDemo {
    sealed interface Resultat<T> permits Succes, Echec {
        default <R> Resultat<R> map(Function<? super T, ? extends R> f) {
            return switch (this) {
                case Succes<T>(var v) -> new Succes<>(f.apply(v));
                case Echec<T>(var e) -> new Echec<>(e);
            };
        }
        default <R> Resultat<R> flatMap(Function<? super T, Resultat<R>> f) {
            return switch (this) {
```

```
                case Succes<T>(var v) -> f.apply(v);
                case Echec<T>(var e) -> new Echec<>(e);
            };
        }
        default T orElse(T defaut) {
            return this instanceof Succes<T>(var v) ? v : defaut;
        }
    }
    record Succes<T>(T valeur) implements Resultat<T> { }
    record Echec<T>(String erreur) implements Resultat<T> { }

    record Commande(String telephone, String quartier, long montant) { }

    static Resultat<Commande> verifierTelephone(Commande c) {
        return c.telephone().matches("7[05678]\\d{7}") ? new Succes<>(c)
                : new Echec<>("téléphone invalide");
    }
    static Resultat<Commande> verifierQuartier(Commande c) {
        return c.quartier().isBlank() ? new Echec<>("quartier manquant") : new Succes<>(c);
    }
    static Resultat<Commande> verifierMontant(Commande c) {
        return c.montant() >= 2_000 ? new Succes<>(c)
                : new Echec<>("montant minimum : 2000 FCFA");
    }

    static Resultat<String> valider(Commande c) {
        return verifierTelephone(c)
                .flatMap(ResultatDemo::verifierQuartier)
                .flatMap(ResultatDemo::verifierMontant)
                .map(ok -> "commande acceptée pour " + ok.quartier());
    }

    public static void main(String[] args) {
        System.out.println(valider(new Commande("771234567", "Fann", 5_000)));
        System.out.println(valider(new Commande("331234567", "", 500)));
        System.out.println(valider(new Commande("781234567", "Yoff", 500)));
        System.out.println(valider(new Commande("781234567", "", 500)).orElse("refusée"));
    }
}
```

Sortie console

```
Succes[valeur=commande acceptée pour Fann]
Echec[erreur=téléphone invalide]
Echec[erreur=montant minimum : 2000 FCFA]
refusée
```

La chaîne s'arrête à la première erreur sans aucun `if` ni exception dans `valider` : chaque `flatMap` sur un `Echec` se contente de le propager.

Corrigés du chapitre 11

Corrigé 11.1 — Manipulations de listes

Prenoms.java

```java
import java.util.*;

public class Prenoms {
    public static void main(String[] args) {
        List<String> brut = List.of("Awa", "moussa", "Fatou", "awa", "Ibrahima",
                "Fatou", "aminata");
        List<String> normalises = new ArrayList<>();
        for (String p : brut) {
            normalises.add(Character.toUpperCase(p.charAt(0)) + p.substring(1).toLowerCase());
        }
        List<String> uniques = new ArrayList<>(new LinkedHashSet<>(normalises));
        System.out.println(uniques);
        uniques.sort(Comparator.naturalOrder());
        System.out.println(uniques);
        uniques.sort(Comparator.comparing(String::length).reversed()
                .thenComparing(Comparator.naturalOrder()));
        System.out.println(uniques);
        System.out.println(uniques.getFirst() + " ... " + uniques.getLast());
    }
}
```

Sortie console

```
[Awa, Moussa, Fatou, Ibrahima, Aminata]
[Aminata, Awa, Fatou, Ibrahima, Moussa]
[Ibrahima, Aminata, Moussa, Fatou, Awa]
Ibrahima ... Awa
```

Corrigé 11.2 — Fréquence des mots

Frequences.java

```java
import java.util.*;

public class Frequences {
    public static void main(String[] args) {
        String texte = """
            La boutique livre le bissap, le thiakry et le café. La boutique livre à Médina,
            à Fann et à Yoff ; le bissap et le café sont les produits les plus vendus.""";
        Map<String, Integer> compte = new HashMap<>();
        // tout ce qui n'est pas une lettre sépare
        for (String mot : texte.toLowerCase().split("[^\\p{L}]+")) {
            if (!mot.isEmpty()) compte.merge(mot, 1, Integer::sum);
        }
        List<Map.Entry<String, Integer>> entrees = new ArrayList<>(compte.entrySet());
        entrees.sort(Map.Entry.<String, Integer>comparingByValue().reversed()
                            .thenComparing(Map.Entry.comparingByKey()));
        for (var e : entrees.subList(0, 5)) {
            System.out.println(e.getKey() + " : " + e.getValue());
        }
    }
}
```

Sortie console

```
le : 5
et : 3
à : 3
bissap : 2
boutique : 2
```

`\p{L}` désigne n'importe quelle lettre Unicode, accentuée ou non : `[^\p{L}]+` découpe donc sur les espaces **et** la ponctuation.

Corrigé 11.3 — Annuaire des clients par quartier

Annuaire.java

```java
import java.util.*;

public class Annuaire {
    private final Map<String, SortedSet<String>> parQuartier = new TreeMap<>();

    public void ajouter(String nom, String quartier) {
        parQuartier.computeIfAbsent(quartier, q -> new TreeSet<>()).add(nom);
    }

    public boolean retirer(String nom) {
        for (Iterator<Map.Entry<String, SortedSet<String>>> it = parQuartier.entrySet()
                .iterator(); it.hasNext(); ) {
            var entree = it.next();
            if (entree.getValue().remove(nom)) {
                if (entree.getValue().isEmpty()) it.remove();       // quartier devenu vide
                return true;
            }
        }
        return false;
    }

    public SortedSet<String> clientsDe(String quartier) {
        return Collections.unmodifiableSortedSet(parQuartier.getOrDefault(quartier,
                new TreeSet<>()));
    }

    public Optional<String> quartierLePlusActif() {
        return parQuartier.entrySet().stream()
                .max(Comparator.comparingInt(e -> e.getValue().size()))
                .map(Map.Entry::getKey);
    }

    @Override public String toString() { return parQuartier.toString(); }

    public static void main(String[] args) {
        Annuaire a = new Annuaire();
        a.ajouter("Awa", "Médina"); a.ajouter("Fatou", "Médina"); a.ajouter("Ibou", "Yoff");
        a.ajouter("Moussa", "Rufisque"); a.ajouter("Aminata", "Médina");
        System.out.println(a);
        a.retirer("Ibou");
        System.out.println(a + " / plus actif : " + a.quartierLePlusActif().orElse("-"));
        System.out.println(a.clientsDe("Médina") + " " + a.clientsDe("Fann"));
    }
}
```

Sortie console

```
{Médina=[Aminata, Awa, Fatou], Rufisque=[Moussa], Yoff=[Ibou]}
{Médina=[Aminata, Awa, Fatou], Rufisque=[Moussa]} / plus actif : Médina
[Aminata, Awa, Fatou] []
```

Corrigé 11.4 — Service client

ServiceClient.java

```java
import java.util.*;

public class ServiceClient {
    record Demande(int numero, String client, boolean clientPremium, int minuteArrivee) { }
```

```
    public static void main(String[] args) {
        List<Demande> arrivees = List.of(
            new Demande(1, "Awa", false, 0), new Demande(2, "Moussa", true, 1),
            new Demande(3, "Fatou", false, 2), new Demande(4, "Ibou", true, 4),
            new Demande(5, "Aminata", false, 5));
        Comparator<Demande> ordre = Comparator.comparing(Demande::clientPremium).reversed()
                                              .thenComparingInt(Demande::minuteArrivee);
        PriorityQueue<Demande> file = new PriorityQueue<>(ordre);
        int prochaine = 0, minute = 0;
        long attenteTotale = 0;
        while (prochaine < arrivees.size() || !file.isEmpty()) {
            // les demandes arrivées à cette minute entrent dans la file
            while (prochaine < arrivees.size() && arrivees.get(prochaine)
                    .minuteArrivee() <= minute) {
                file.add(arrivees.get(prochaine++));
            }
            if (file.isEmpty()) { minute = arrivees.get(prochaine).minuteArrivee(); continue; }
            Demande d = file.poll();
            int attente = minute - d.minuteArrivee();
            attenteTotale += attente;
            System.out.printf("minute %2d : %-8s %s attente %d min%n", minute, d.client(),
                              d.clientPremium() ? "(premium)" : "         ", attente);
            minute += 3;                                        // durée de traitement
        }
        System.out.printf("Attente moyenne : %.1f min%n",
                (double) attenteTotale / arrivees.size());
    }
}
```

Sortie console

```
minute  0 : Awa               attente 0 min
minute  3 : Moussa   (premium) attente 2 min
minute  6 : Ibou     (premium) attente 2 min
minute  9 : Fatou             attente 7 min
minute 12 : Aminata           attente 7 min
Attente moyenne : 3.6 min
```

Corrigé 11.5 — Chasse aux bogues

- **(a)** `List.of` crée une liste **immuable** : `add` lève `UnsupportedOperationException`. Correction : `new ArrayList<>(List.of("Plateau", "Médina"))`.
- **(b)** `ids.remove(3)` supprime l'élément d'**indice** 3, qui n'existe pas : `IndexOutOfBoundsException`. Pour supprimer la valeur 3 : `ids.remove(Integer.valueOf(3))`.
- **(c)** Supprimer pendant une boucle *for-each* lève une `ConcurrentModificationException`. Correction : `clients.removeIf(String::isBlank);`.
- **(d)** `stock.get("PRD-001")` renvoie `null` (clé absente) ; l'*unboxing* vers `int` lève une `NullPointerException`. Correction : `int s = stock.getOrDefault("PRD-001", 0);`.
- **(e)** Les tableaux n'ont pas d'`equals`/`hashCode` fondés sur leur contenu (ils héritent de ceux d'`Object`) : deux tableaux de même contenu sont différents pour un `HashSet`. Correction : utiliser un record, `Set<Point> dejaVus` avec `dejaVus.add(new Point(1, 2))`, ou une `List<Integer>` (qui, elle, compare les contenus).

Corrigé 11.6 — Gestion de stock avec historique

GestionStock.java

```
import java.util.*;
```

```java
public class GestionStock {
    record Mouvement(String reference, int quantite, String motif) { }

    private final Map<String, Deque<Mouvement>> mouvements = new HashMap<>();

    public int stock(String reference) {
        int s = 0;
        for (Mouvement m : mouvements.getOrDefault(reference,
                new ArrayDeque<>())) s += m.quantite();
        return s;
    }

    public void enregistrer(Mouvement m) {
        if (stock(m.reference()) + m.quantite() < 0) {
            throw new IllegalStateException("Stock insuffisant pour " + m.reference());
        }
        mouvements.computeIfAbsent(m.reference(), r -> new ArrayDeque<>()).addLast(m);
    }

    public Mouvement annulerDernier(String reference) {
        Deque<Mouvement> pile = mouvements.get(reference);
        if (pile == null || pile.isEmpty())
                throw new IllegalStateException("Aucun mouvement pour " + reference);
        return pile.removeLast();
    }

    public SortedMap<String, Integer> sousLeSeuil(int seuil) {
        SortedMap<String, Integer> r = new TreeMap<>();
        for (String ref : mouvements.keySet()) {
            int s = stock(ref);
            if (s < seuil) r.put(ref, s);
        }
        return r;
    }

    public List<Mouvement> historique(String reference) {
        return List.copyOf(mouvements.getOrDefault(reference, new ArrayDeque<>()).reversed());
    }

    public static void main(String[] args) {
        GestionStock g = new GestionStock();
        g.enregistrer(new Mouvement("PRD-001", 50, "réception"));
        g.enregistrer(new Mouvement("PRD-001", -46, "ventes"));
        g.enregistrer(new Mouvement("PRD-014", 10, "réception"));
        g.enregistrer(new Mouvement("PRD-014", -2, "vente"));
        try {
            g.enregistrer(new Mouvement("PRD-014", -20, "vente"));
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Sous le seuil de 5 : " + g.sousLeSeuil(5));
        System.out.println("Annulé : " + g.annulerDernier("PRD-001").motif()
                + ", stock PRD-001 = " + g.stock("PRD-001"));
        g.historique("PRD-014").forEach(System.out::println);
    }
}
```

Sortie console

```
Stock insuffisant pour PRD-014
Sous le seuil de 5 : {PRD-001=4}
Annulé : ventes, stock PRD-001 = 50
Mouvement[reference=PRD-014, quantite=-2, motif=vente]
Mouvement[reference=PRD-014, quantite=10, motif=réception]
```

Corrigés du chapitre 12

Corrigé 12.1 — Des classes anonymes aux lambdas

VersLambdas.java

```java
import java.util.*;
import java.util.function.*;

public class VersLambdas {
    public static void main(String[] args) {
        Runnable r = () -> System.out.println("Livraison");
        Comparator<String> c = (a, b) -> a.length() - b.length();      // ou mieux :
        Comparator<String> c2 = Comparator.comparingInt(String::length);
        // ou s -> Integer.parseInt(s)
        Function<String, Integer> f = Integer::parseInt;
        // ou () -> new ArrayList<>()
        Supplier<List<String>> s = ArrayList::new;
        BiFunction<Long, Integer, Long> total = (prix, qte) -> prix * qte;

        r.run();
        System.out.println(c.compare("Fann", "Médina") < 0 && c2.compare("Fann", "Médina") < 0);
        System.out.println(f.apply("42") + 1);
        System.out.println(s.get().isEmpty());
        System.out.println(total.apply(1_500L, 3));
    }
}
```

Sortie console

```
Livraison
true
43
true
4500
```

La lambda `(prix, qte) -> prix * qte` n'a pas d'équivalent en référence de méthode simple : aucune méthode existante ne fait ce calcul.

Corrigé 12.2 — Filtres combinables

Recherche.java

```java
import java.util.List;
import java.util.function.Predicate;

public class Recherche {
    record Produit(String libelle, String categorie, long prix, int stock) { }

    static Predicate<Produit> categorie(String c)
            { return p -> p.categorie().equalsIgnoreCase(c); }
    static Predicate<Produit> prixEntre(long min, long max)
            { return p -> p.prix() >= min && p.prix() <= max; }
    static Predicate<Produit> enStock() { return p -> p.stock() > 0; }
    static Predicate<Produit> libelleContient(String motCle) {
        String m = motCle.toLowerCase();
        return p -> p.libelle().toLowerCase().contains(m);
    }

    static List<Produit> rechercher(List<Produit> catalogue,
            List<Predicate<Produit>> criteres) {
        // neutre : toujours vrai
        Predicate<Produit> tous = criteres.stream().reduce(p -> true, Predicate::and);
        return catalogue.stream().filter(tous).toList();
    }

    public static void main(String[] args) {
```

```
        List<Produit> catalogue = List.of(
            new Produit("Pagne wax", "Textile", 7_500, 12), new Produit("Pagne bazin",
                    "Textile", 18_000, 0),
            new Produit("Boubou brodé", "Textile", 35_000, 3), new Produit("Bissap 1 L",
                    "Boisson", 1_500, 40));
        System.out.println(rechercher(catalogue, List.of(categorie("textile"), enStock())));
        System.out.println(rechercher(catalogue, List.of(libelleContient("PAGNE"),
                prixEntre(5_000, 20_000))));
        System.out.println(rechercher(catalogue, List.of()).size());
    }
}
```

Sortie console

```
[Produit[libelle=Pagne wax, categorie=Textile, prix=7500, stock=12], Produit[libelle=Boubou brodé,
categorie=Textile, prix=35000, stock=3]]
[Produit[libelle=Pagne wax, categorie=Textile, prix=7500, stock=12], Produit[libelle=Pagne bazin,
categorie=Textile, prix=18000, stock=0]]
4
```

Corrigé 12.3 — Optional en cascade

OptionalCascade.java

```
import java.util.Optional;

public class OptionalCascade {
    record Coordonnees(double lat, double lon) {
        // approximation suffisante à l'échelle de Dakar
        double distanceKm(Coordonnees o) {
            double dx = (o.lon - lon) * 111.32 * Math.cos(Math.toRadians(lat));
            double dy = (o.lat - lat) * 110.57;
            return Math.sqrt(dx * dx + dy * dy);
        }
    }
    record Adresse(String quartier, Coordonnees gps) {
        Optional<Coordonnees> coordonnees() { return Optional.ofNullable(gps); }
    }
    record Client(String nom, Adresse adr) {
        Optional<Adresse> adresse() { return Optional.ofNullable(adr); }
    }

    static final Coordonnees ENTREPOT = new Coordonnees(14.7167, -17.4677);

    static double distanceDepuisEntrepot(Client c) {
        return c.adresse()
                .flatMap(Adresse::coordonnees)                // Optional<Coordonnees>
                .map(ENTREPOT::distanceKm)                    // Optional<Double>
                .orElseThrow(() -> new IllegalStateException("Position inconnue pour "
                        + c.nom()));
    }

    public static void main(String[] args) {
        Client awa = new Client("Awa", new Adresse("Almadies",
                new Coordonnees(14.7453, -17.5130)));
        Client ibou = new Client("Ibou", new Adresse("Yoff", null));
        Client moussa = new Client("Moussa", null);
        System.out.printf("Awa : %.1f km%n", distanceDepuisEntrepot(awa));
        for (Client c : new Client[]{ibou, moussa}) {
            try {
                distanceDepuisEntrepot(c);
            } catch (IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

Sortie console

```
Awa : 5.8 km
Position inconnue pour Ibou
Position inconnue pour Moussa
```

Corrigé 12.4 — Statistiques de promotion

StatsPromotion.java

```java
import java.util.*;
import java.util.stream.Collectors;

public class StatsPromotion {
    record Etudiant(String nom, String filiere, double moyenne) { }

    public static void main(String[] args) {
        List<Etudiant> promo = List.of(
            new Etudiant("Awa", "Backend", 15.5), new Etudiant("Moussa", "Backend", 12.0),
            new Etudiant("Fatou", "Réseaux", 9.5), new Etudiant("Ibou", "Réseaux", 13.25),
            new Etudiant("Aminata", "Backend", 17.0), new Etudiant("Cheikh", "Réseaux", 8.0));

        double generale = promo.stream().mapToDouble(Etudiant::moyenne).average().orElse(0);
        System.out.printf("Moyenne générale : %.2f%n", generale);

        List<String> top3 = promo.stream().sorted(Comparator.comparingDouble(Etudiant::moyenne)
                .reversed())
                                .limit(3).map(Etudiant::nom).toList();
        System.out.println("Top 3 : " + top3);

        Map<String, Long> parFiliere = promo.stream()
                .collect(Collectors.groupingBy(Etudiant::filiere, TreeMap::new,
                        Collectors.counting()));
        System.out.println("Effectifs : " + parFiliere);

        Comparator<Etudiant> parMoyenne = Comparator.comparingDouble(Etudiant::moyenne);
        Map<String, String> majors = promo.stream()
                .collect(Collectors.groupingBy(Etudiant::filiere, TreeMap::new,
                        Collectors.collectingAndThen(Collectors.maxBy(parMoyenne),
                                                     e -> e.orElseThrow().nom())));
        System.out.println("Majors : " + majors);

        Map<Boolean, Long> admis = promo.stream()
                .collect(Collectors.partitioningBy(e -> e.moyenne() >= 10,
                        Collectors.counting()));
        System.out.println("Admis : " + admis.get(true) + ", ajournés : " + admis.get(false));

        String liste = promo.stream().filter(e -> e.moyenne() >= 10)
                .sorted(Comparator.comparingDouble(Etudiant::moyenne).reversed())
                .map(e -> "%s (%.1f)".formatted(e.nom(), e.moyenne()))
                .collect(Collectors.joining(", "));
        System.out.println(liste);
    }
}
```

Sortie console

```
Moyenne générale : 12.54
Top 3 : [Aminata, Awa, Ibou]
Effectifs : {Backend=3, Réseaux=3}
Majors : {Backend=Aminata, Réseaux=Ibou}
Admis : 4, ajournés : 2
Aminata (17.0), Awa (15.5), Ibou (13.3), Moussa (12.0)
```

`collectingAndThen` applique une transformation finale au résultat d'un collecteur : ici, extraire le nom de l'`Optional<Etudiant>` renvoyé par `maxBy`.

Corrigé 12.5 — Fréquence des mots, version stream

FrequencesStream.java

```java
import java.util.Arrays;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

public class FrequencesStream {
    public static void main(String[] args) {
        String texte = """
            La boutique livre le bissap, le thiakry et le café. La boutique livre à Médina,
            à Fann et à Yoff ; le bissap et le café sont les produits les plus vendus.""";
        Arrays.stream(texte.toLowerCase().split("[^\\p{L}]+"))
              .filter(m -> !m.isEmpty())
              .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
              .entrySet().stream()
              .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                      .thenComparing(Map.Entry.comparingByKey()))
              .limit(5)
              .forEach(e -> System.out.println(e.getKey() + " : " + e.getValue()));
    }
}
```

Sortie console

```
le : 5
et : 3
à : 3
bissap : 2
boutique : 2
```

La version stream est plus courte et se lit comme une description du traitement ; la version impérative est plus facile à déboguer pas à pas. Les deux sont acceptables : l'important est la lisibilité pour l'équipe.

Corrigé 12.6 — Rapport de ventes mensuel

Les données sont générées avec une graine fixe pour que le rapport soit reproductible.

RapportMensuel.java

```java
import java.util.*;
import java.util.stream.*;

public class RapportMensuel {
    record Vente(int jour, String quartier, String produit, String categorie, int quantite,
            long prixUnitaire, String mode) {
        long montant() { return quantite * prixUnitaire; }
    }

    static List<Vente> generer() {
        String[][] produits = {{"Bissap", "Boisson", "1500"}, {"Café Touba", "Boisson", "2000"},
                {"Pagne wax", "Textile", "7500"}, {"Sac cuir", "Artisanat", "25000"},
                        {"Thiakry", "Alimentaire", "1000"}};
        String[] quartiers = {"Médina", "Fann", "Yoff", "Rufisque"};
        String[] modes = {"WAVE", "ORANGE_MONEY", "ESPECES"};
        Random r = new Random(2026);
        List<Vente> ventes = new ArrayList<>();
        for (int i = 0; i < 120; i++) {
            String[] p = produits[r.nextInt(produits.length)];
            ventes.add(new Vente(1 + r.nextInt(30), quartiers[r.nextInt(4)], p[0], p[1],
                    1 + r.nextInt(4), Long.parseLong(p[2]), modes[r.nextInt(3)]));
        }
        return ventes;
    }

    public static void main(String[] args) {
        List<Vente> ventes = generer();
```

```
        long total = ventes.stream().mapToLong(Vente::montant).sum();
        System.out.printf("CA total : %d FCFA, moyenne par jour : %d FCFA%n",
                total, total / 30);

        System.out.println("CA par catégorie :");
        ventes.stream().collect(Collectors.groupingBy(Vente::categorie,
                Collectors.summingLong(Vente::montant)))
              .entrySet().stream().sorted(Map.Entry.<String, Long>comparingByValue().reversed())
              .forEach(e -> System.out.printf("  %-12s %8d%n", e.getKey(), e.getValue()));

        System.out.println("Produit le plus vendu par quartier :");
        ventes.stream().collect(Collectors.groupingBy(Vente::quartier, TreeMap::new,
                        Collectors.groupingBy(Vente::produit,
                                Collectors.summingInt(Vente::quantite))))
              .forEach((q, parProduit) -> {
                  var best = Collections.max(parProduit.entrySet(),
                          Map.Entry.comparingByValue());
                  System.out.printf("  %-9s %s (%d unités)%n", q,
                          best.getKey(), best.getValue());
              });

        long[] caJour = new long[31];
        ventes.forEach(v -> caJour[v.jour()] += v.montant());
        int record = IntStream.rangeClosed(1, 30).boxed()
                .max(Comparator.comparingLong(j -> caJour[j])).orElseThrow();
        System.out.println("Meilleur jour : " + record + " (" + caJour[record] + " FCFA)");

        List<Long> serie = LongStream.rangeClosed(1, 30).map(j -> caJour[(int) j])
                .boxed().toList();
        List<Long> moyenne7 = serie.stream().gather(Gatherers.windowSliding(7))
                .map(f -> f.stream().mapToLong(Long::longValue).sum() / 7).toList();
        System.out.println("Moyenne mobile 7 j (5 premières) : " + moyenne7.subList(0, 5));

        List<Integer> baisses = IntStream.rangeClosed(2, 30)
                .filter(j -> caJour[j] < caJour[j - 1]).boxed().toList();
        System.out.println("Jours en baisse : " + baisses.size() + " -> " + baisses);
    }
}
```

Sortie console

```
CA total : 1553500 FCFA, moyenne par jour : 51783 FCFA
CA par catégorie :
  Artisanat      925000
  Textile        292500
  Boisson        272000
  Alimentaire     64000
Produit le plus vendu par quartier :
  Fann      Bissap (24 unités)
  Médina    Thiakry (21 unités)
  Rufisque  Bissap (40 unités)
  Yoff      Sac cuir (20 unités)
Meilleur jour : 24 (186500 FCFA)
Moyenne mobile 7 j (5 premières) : [53500, 56642, 72000, 67214, 57571]
Jours en baisse : 13 -> [2, 6, 8, 10, 12, 16, 17, 19, 20, 23, 25, 27, 29]
```

La dernière statistique est ici calculée avec un `IntStream` sur les indices, plus simple qu'une fenêtre glissante de taille 2 ; les deux approches sont correctes.

Corrigés du chapitre 13

Corrigé 13.1 — Anniversaire

Pour pouvoir tester le programme, la date du jour est un **paramètre** (en production, on passerait `LocalDate.now()` ou, mieux, une `Clock`).

Anniversaire.java

```java
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;
import java.time.temporal.ChronoUnit;
import java.util.Locale;

public class Anniversaire {
    static final DateTimeFormatter SAISIE =
            DateTimeFormatter.ofPattern("dd/MM/uuuu").withResolverStyle(ResolverStyle.STRICT);
    static final DateTimeFormatter LONG_FR = DateTimeFormatter.ofPattern("EEEE d MMMM yyyy",
            Locale.FRENCH);

    static void analyser(String saisie, LocalDate aujourdhui) {
        try {
            LocalDate naissance = LocalDate.parse(saisie, SAISIE);
            Period age = Period.between(naissance, aujourdhui);
            System.out.printf("Âge : %d ans, %d mois, %d jours%n", age.getYears(),
                    age.getMonths(), age.getDays());
            System.out.println("Né(e) un " + naissance.getDayOfWeek()
                    .getDisplayName(java.time.format.TextStyle.FULL, Locale.FRENCH));
            LocalDate prochain = naissance.withYear(aujourdhui.getYear());
            if (prochain.isBefore(aujourdhui)) prochain = prochain.plusYears(1);
            System.out.println("Prochain anniversaire : " + prochain.format(LONG_FR)
                               + ", dans " + ChronoUnit.DAYS.between(aujourdhui,
                                       prochain) + " jours");
        } catch (DateTimeParseException e) {
            System.out.println("Date invalide : " + saisie);
        }
    }

    public static void main(String[] args) {
        LocalDate aujourdhui = LocalDate.of(2026, 9, 27);
        analyser("04/04/2003", aujourdhui);
        analyser("31/02/2004", aujourdhui);
    }
}
```

Sortie console

```
Âge : 23 ans, 5 mois, 23 jours
Né(e) un vendredi
Prochain anniversaire : dimanche 4 avril 2027, dans 189 jours
Date invalide : 31/02/2004
```

Attention au mode de résolution : par défaut (`ResolverStyle.SMART`), le 31/02/2004 serait silencieusement « corrigé » en 29/02/2004. Le mode `STRICT` (qui exige le symbole `uuuu` pour l'année) rejette les dates impossibles. Par ailleurs, `withYear` gère le cas du 29 février : pour une année non bissextile, la date devient le 28 février.

Corrigé 13.2 — Calendrier des tournées

Tournees.java

```
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.util.List;
import java.util.Set;
import java.util.stream.Stream;

public class Tournees {
    static List<LocalDate> prochainesTournees(LocalDate depuis, int n, Set<LocalDate> feries) {
        return Stream.iterate(depuis.plusDays(1), d -> d.plusDays(1))
                .filter(d -> d.getDayOfWeek() == DayOfWeek.TUESDAY
                        || d.getDayOfWeek() == DayOfWeek.FRIDAY)
                .filter(d -> !feries.contains(d))
                .limit(n)
                .toList();
    }

    public static void main(String[] args) {
        Set<LocalDate> feries = Set.of(LocalDate.of(2027, 4, 2), LocalDate.of(2026, 11, 1));
        System.out.println(prochainesTournees(LocalDate.of(2026, 9, 27), 5, feries));
        System.out.println(prochainesTournees(LocalDate.of(2027, 3, 28), 3, feries));
    }
}
```

Sortie console

```
[2026-09-29, 2026-10-02, 2026-10-06, 2026-10-09, 2026-10-13]
[2027-03-30, 2027-04-06, 2027-04-09]
```

`Stream.iterate` produit une suite **infinie** de dates ; c'est `limit(n)` qui l'arrête. (Le 2 avril 2027, un vendredi, a été déclaré férié pour l'exemple.)

Corrigé 13.3 — Respect des délais

Delais.java

```
import java.time.*;
import java.util.*;
import java.util.stream.Collectors;

public class Delais {
    record Livraison(String numero, LocalDateTime commandee, LocalDateTime livree) {
        Duration delai() { return Duration.between(commandee, livree); }
    }

    public static void main(String[] args) {
        List<Livraison> livraisons = List.of(
            new Livraison("TS-1", LocalDateTime.of(2026, 9, 14, 9, 0),
                    LocalDateTime.of(2026, 9, 15, 11, 30)),
            new Livraison("TS-2", LocalDateTime.of(2026, 9, 14, 18, 0),
                    LocalDateTime.of(2026, 9, 17, 10, 0)),
            new Livraison("TS-3", LocalDateTime.of(2026, 9, 15, 8, 15),
                    LocalDateTime.of(2026, 9, 16, 16, 45)),
            new Livraison("TS-4", LocalDateTime.of(2026, 9, 18, 20, 0),
                    LocalDateTime.of(2026, 9, 21, 12, 0)),
            new Livraison("TS-5", LocalDateTime.of(2026, 9, 21, 10, 0),
                    LocalDateTime.of(2026, 9, 22, 9, 0)));
        Duration limite = Duration.ofHours(48);

        long moyenneMin = (long) livraisons.stream().mapToLong(l -> l.delai().toMinutes())
                .average().orElse(0);
        System.out.printf("Délai moyen : %d h %02d min%n", moyenneMin / 60, moyenneMin % 60);

        long aTemps = livraisons.stream().filter(l -> l.delai().compareTo(limite) <= 0).count();
        System.out.printf("Livrées en moins de 48 h : %.0f %%%n",
```

```
                100.0 * aTemps / livraisons.size());

        livraisons.stream().filter(l -> l.delai().compareTo(limite) > 0)
                  .sorted(Comparator.comparing(Livraison::delai).reversed())
                  .forEach(l -> System.out.println("En retard : " + l.numero() + " ("
                          + l.delai().toHours() + " h)"));

        Map<DayOfWeek, Double> parJour = livraisons.stream().collect(Collectors.groupingBy(
                l -> l.commandee().getDayOfWeek(), TreeMap::new,
                Collectors.averagingLong(l -> l.delai().toMinutes())));
        parJour.forEach((jour,
                minutes) -> System.out.printf("  commandes du %s : %.1f h en moyenne%n",
                jour.getDisplayName(java.time.format.TextStyle.FULL,
                        Locale.FRENCH), minutes / 60));
    }
}
```

Sortie console

```
Délai moyen : 42 h 00 min
Livrées en moins de 48 h : 60 %
En retard : TS-2 (64 h)
En retard : TS-4 (64 h)
  commandes du lundi : 37.8 h en moyenne
  commandes du mardi : 32.5 h en moyenne
  commandes du vendredi : 64.0 h en moyenne
```

Corrigé 13.4 — Journal de caisse quotidien

JournalCaisse.java

```
import java.io.IOException;
import java.nio.file.*;
import java.time.LocalDate;
import java.util.List;

public class JournalCaisse {
    private final Path dossier;

    public JournalCaisse(Path dossier) { this.dossier = dossier; }

    private Path fichier(LocalDate jour) {
        // LocalDate.toString() : AAAA-MM-JJ
        return dossier.resolve("ventes-" + jour + ".csv");
    }

    public void enregistrer(LocalDate jour, String numero, long montant) throws IOException {
        Path f = fichier(jour);
        if (Files.notExists(f)) Files.writeString(f, "numero;montant\n");
        Files.writeString(f, numero + ";" + montant + "\n", StandardOpenOption.APPEND);
    }

    public long totalDuJour(LocalDate jour) throws IOException {
        Path f = fichier(jour);
        if (Files.notExists(f)) return 0;
        List<String> lignes = Files.readAllLines(f);
        return lignes.stream().skip(1).mapToLong(l -> Long.parseLong(l.split(";")[1])).sum();
    }

    public static void main(String[] args) throws IOException {
        Path tmp = Files.createTempDirectory("caisse");
        JournalCaisse j = new JournalCaisse(tmp);
        LocalDate lundi = LocalDate.of(2026, 9, 14);
        j.enregistrer(lundi, "TS-1", 22_500);
        j.enregistrer(lundi, "TS-2", 8_000);
        j.enregistrer(lundi.plusDays(1), "TS-3", 51_500);
        System.out.println(lundi + " : " + j.totalDuJour(lundi));
        System.out.println(lundi.plusDays(1) + " : " + j.totalDuJour(lundi.plusDays(1)));
```

```
        System.out.println(lundi.plusDays(2) + " : " + j.totalDuJour(lundi.plusDays(2)));
        try (var fichiers = Files.list(tmp)) {
            System.out.println(fichiers.map(p -> p.getFileName().toString()).sorted().toList());
        }
    }
}
```

Sortie console

```
2026-09-14 : 30500
2026-09-15 : 51500
2026-09-16 : 0
[ventes-2026-09-14.csv, ventes-2026-09-15.csv]
```

Corrigé 13.5 — Inventaire d'un dossier

Pour que le résultat soit reproductible, le programme crée lui-même un dossier d'exemple quand aucun argument n'est fourni.

Inventaire.java

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.FileTime;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.*;
import java.util.stream.*;

public class Inventaire {
    static String extension(Path p) {
        String n = p.getFileName().toString();
        int i = n.lastIndexOf('.');
        return i < 0 ? "(aucune)" : n.substring(i + 1).toLowerCase();
    }

    public static void main(String[] args) throws IOException {
        Path racine = args.length > 0 ? Path.of(args[0]) : exemple();
        List<Path> fichiers;
        try (Stream<Path> s = Files.walk(racine)) {
            fichiers = s.filter(Files::isRegularFile).toList();
        }
        Map<String, Long> parExtension = fichiers.stream()
                .collect(Collectors.groupingBy(Inventaire::extension, Collectors.counting()));
        parExtension.entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                        .thenComparing(Map.Entry.comparingByKey()))
                .forEach(e -> System.out.println(e.getKey() + " : " + e.getValue()));

        long totalOctets = 0;
        for (Path f : fichiers) totalOctets += Files.size(f);
        System.out.printf("Taille totale : %.1f Ko%n", totalOctets / 1024.0);

        System.out.println("Plus volumineux : " + fichiers.stream()
                .sorted(Comparator.comparingLong((Path f) -> taille(f)).reversed())
                .limit(2).map(f -> f.getFileName().toString()).toList());

        Instant ilYa7Jours = Instant.now().minus(7, ChronoUnit.DAYS);
        long recents = fichiers.stream().filter(f -> dateModif(f).isAfter(ilYa7Jours)).count();
        System.out.println("Modifiés depuis moins de 7 jours : " + recents);
    }

    static long taille(Path f) {
        try { return Files.size(f); } catch (IOException e)
                { throw new java.io.UncheckedIOException(e); }
    }
    static Instant dateModif(Path f) {
        try { return Files.getLastModifiedTime(f).toInstant(); } catch (IOException e)
```

```
                { throw new java.io.UncheckedIOException(e); }
    }

    static Path exemple() throws IOException {
        Path d = Files.createTempDirectory("inventaire");
        Files.createDirectories(d.resolve("src/main"));
        Files.writeString(d.resolve("src/main/App.java"), "x".repeat(3_000));
        Files.writeString(d.resolve("src/main/Produit.java"), "x".repeat(1_200));
        Files.writeString(d.resolve("README.md"), "x".repeat(500));
        Files.writeString(d.resolve("catalogue.csv"), "x".repeat(4_000));
        Path ancien = Files.writeString(d.resolve("ventes-2025.csv"), "x".repeat(2_000));
        Files.setLastModifiedTime(ancien, FileTime.from(Instant.parse("2025-12-31T23:00:00Z")));
        return d;
    }
}
```

Sortie console

```
csv : 2
java : 2
md : 1
Taille totale : 10.4 Ko
Plus volumineux : [catalogue.csv, App.java]
Modifiés depuis moins de 7 jours : 4
```

Remarquez les méthodes `taille` et `dateModif` : les méthodes de `Files` lèvent des `IOException` **vérifiées**, interdites dans une lambda de stream. On les enveloppe dans une `UncheckedIOException`.

Corrigé 13.6 — Import configurable avec rapport d'erreurs

Structure de la solution : une classe `ConfigImport` (record) chargée depuis `import.properties` ; une méthode `validerLigne` qui renvoie soit un `Produit`, soit une erreur (on peut réutiliser le type `Resultat` de l'exercice 10.6) ; une boucle sur `Files.list(dossierEntree)` filtrée sur `.csv` ; une `LinkedHashMap<String, Produit>` pour la fusion (la dernière ligne gagne) ; enfin l'écriture des deux fichiers de sortie. Voici le cœur du traitement :

ImportConfigurable.java

```
import java.io.IOException;
import java.io.StringReader;
import java.nio.file.*;
import java.util.*;

public class ImportConfigurable {
    record ConfigImport(Path entree, Path sortie, String separateur, Set<String> categories) {
        static ConfigImport depuis(Properties p) {
            return new ConfigImport(Path.of(p.getProperty("dossier.entree")),
                    Path.of(p.getProperty("dossier.sortie")),
                    p.getProperty("separateur", ";"),
                            Set.of(p.getProperty("categories").split(",")));
        }
    }
    record Produit(String reference, String libelle, String categorie, long prix, int stock) { }
    record Erreur(String fichier, int ligne, String message) { }

    private static final System.Logger LOG = System.getLogger("import");

    static Produit valider(String[] c, ConfigImport cfg) {
        if (c.length != 5) throw new IllegalArgumentException("5 champs attendus");
        if (!c[0].matches("PRD-\\d{3}"))
                throw new IllegalArgumentException("référence invalide " + c[0]);
        if (!cfg.categories().contains(c[2]))
                throw new IllegalArgumentException("catégorie inconnue " + c[2]);
        long prix = Long.parseLong(c[3]);
        int stock = Integer.parseInt(c[4]);
        if (prix <= 0 || stock < 0)
```

```
                throw new IllegalArgumentException("prix ou stock invalide");
            return new Produit(c[0], c[1], c[2], prix, stock);
        }

        static void importer(ConfigImport cfg) throws IOException {
            Map<String, Produit> catalogue = new LinkedHashMap<>();
            List<Erreur> erreurs = new ArrayList<>();
            List<Path> fichiers;
            try (var s = Files.list(cfg.entree())) {
                fichiers = s.filter(p -> p.toString().endsWith(".csv")).sorted().toList();
            }
            for (Path f : fichiers) {
                List<String> lignes = Files.readAllLines(f);
                for (int i = 1; i < lignes.size(); i++) {                    // ligne 0 = en-tête
                    try {
                        Produit p = valider(lignes.get(i).split(cfg.separateur(), -1), cfg);
                        if (catalogue.put(p.reference(), p) != null) {
                            LOG.log(System.Logger.Level.WARNING,
                                    "Doublon {0} remplacé", p.reference());
                        }
                    } catch (IllegalArgumentException e) {
                        erreurs.add(new Erreur(f.getFileName().toString(), i + 1, e.getMessage()));
                    }
                }
            }
            Files.createDirectories(cfg.sortie());
            List<String> sortie = new ArrayList<>(List.of("reference;libelle;categorie;prix;stock"));
            catalogue.values().forEach(p -> sortie.add(String.join(";", p.reference(),
                    p.libelle(), p.categorie(),
                    String.valueOf(p.prix()), String.valueOf(p.stock()))));
            Files.write(cfg.sortie().resolve("catalogue-consolide.csv"), sortie);
            List<String> rapport = new ArrayList<>(List.of("fichier;ligne;message"));
            erreurs.forEach(e -> rapport.add(e.fichier() + ";" + e.ligne() + ";" + e.message()));
            Files.write(cfg.sortie().resolve("erreurs.csv"), rapport);
            System.out.println("Produits consolidés : " + catalogue.size() + ", erreurs : "
                    + erreurs.size());
            rapport.stream().skip(1).forEach(l -> System.out.println("  " + l));
        }

        public static void main(String[] args) throws IOException {
            Path base = Files.createTempDirectory("import");
            Path entree = Files.createDirectories(base.resolve("entree"));
            Files.writeString(entree.resolve("a.csv"),
                    "ref;lib;cat;prix;stock\nPRD-001;Bissap;Boisson;1500;40\nPRD-2;X;Boisson;1;1\n");
            Files.writeString(entree.resolve("b.csv"),
                    "ref;lib;cat;prix;stock\nPRD-001;Bissap 1 "
                            + "L;Boisson;1600;35\nPRD-014;Pagne;Mode;7500;3\n");
            Properties props = new Properties();
            props.load(new StringReader("dossier.entree=" + entree + "\ndossier.sortie="
                    + base.resolve("sortie")
                    + "\ncategories=Boisson,Textile,Artisanat\n"));
            importer(ConfigImport.depuis(props));
        }
    }
```

Sortie console

```
Produits consolidés : 1, erreurs : 2
  a.csv;3;référence invalide PRD-2
  b.csv;3;catégorie inconnue Mode
```

(Le message d'avertissement sur le doublon est écrit par `System.Logger` sur la sortie d'erreur.)

Corrigés du chapitre 14

Corrigé 14.1 — Observer l'ordonnancement

Ordonnancement.java

```java
import java.util.concurrent.ThreadLocalRandom;

public class Ordonnancement {
    public static void main(String[] args) throws InterruptedException {
        Thread[] caisses = new Thread[3];
        String[] noms = {"Wave", "Orange Money", "Espèces"};
        for (int i = 0; i < 3; i++) {
            String nom = noms[i];
            caisses[i] = new Thread(() -> {
                for (int k = 1; k <= 5; k++) {
                    System.out.println(nom + " " + k);
                    try {
                        Thread.sleep(ThreadLocalRandom.current().nextInt(100));
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
            });
            caisses[i].start();
        }
        for (Thread t : caisses) t.join();              // attendre les trois threads
        System.out.println("Clôture de la caisse");
    }
}
```

Exemple de sortie (varie d'une exécution à l'autre)

```
Wave 1
Orange Money 1
Espèces 1
Espèces 2
Wave 2
Orange Money 2
...
Clôture de la caisse
```

L'ordre des lignes change à chaque exécution. Sans les `join()`, « Clôture de la caisse » pourrait s'afficher avant la fin des autres threads, puisque le thread `main` continue son exécution indépendamment.

Corrigé 14.2 — Le compte partagé

Dans la version non protégée, deux threads peuvent tous deux lire un solde de 100 FCFA, constater qu'il suffit, puis retirer chacun 100 FCFA : le solde devient négatif. Les deux corrections :

ComptePartage.java

```java
import java.util.concurrent.atomic.AtomicLong;

public class ComptePartage {
    static class CompteSynchronise {
        private long solde;
        CompteSynchronise(long solde) { this.solde = solde; }
        // vérification + retrait : indivisibles
        synchronized boolean retirer(long montant) {
            if (montant > solde) return false;
            solde -= montant;
            return true;
        }
        synchronized long solde() { return solde; }
    }
```

```
    static class CompteAtomique {
        private final AtomicLong solde;
        CompteAtomique(long solde) { this.solde = new AtomicLong(solde); }
        boolean retirer(long montant) {
            while (true) {                                  // boucle « compare-and-set »
                long actuel = solde.get();
                if (montant > actuel) return false;
                if (solde.compareAndSet(actuel, actuel - montant)) return true;
                // un autre thread a modifié le solde entre-temps : on recommence
            }
        }
        long solde() { return solde.get(); }
    }

    public static void main(String[] args) throws InterruptedException {
        CompteSynchronise cs = new CompteSynchronise(150_000);
        CompteAtomique ca = new CompteAtomique(150_000);
        Runnable retraits = () -> {
            for (int i = 0; i < 1_000; i++) { cs.retirer(100); ca.retirer(100); }
        };
        Thread t1 = new Thread(retraits), t2 = new Thread(retraits);
        t1.start(); t2.start(); t1.join(); t2.join();
        System.out.println("synchronized : " + cs.solde());
        System.out.println("AtomicLong   : " + ca.solde());
    }
}
```

Sortie console

```
synchronized : 0
AtomicLong   : 0
```

2 000 retraits de 100 FCFA demandés pour 150 000 FCFA disponibles : seuls 1 500 réussissent, et le solde final vaut exactement 0, jamais moins.

Corrigé 14.3 — Somme parallèle

SommeParallele.java

```
import java.util.*;
import java.util.concurrent.*;

public class SommeParallele {
    public static void main(String[] args) throws Exception {
        long[] t = new Random(1).longs(20_000_000, 0, 1_000).toArray();

        long d0 = System.nanoTime();
        long s1 = 0;
        for (long v : t) s1 += v;
        long d1 = System.nanoTime();

        long s2 = 0;
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            List<Future<Long>> parts = new ArrayList<>();
            int taille = t.length / 8;
            for (int k = 0; k < 8; k++) {
                int debut = k * taille, fin = (k == 7) ? t.length : debut + taille;
                parts.add(pool.submit(() -> {
                    long s = 0;
                    for (int i = debut; i < fin; i++) s += t[i];
                    return s;
                }));
            }
            for (Future<Long> f : parts) s2 += f.get();
        }
        long d2 = System.nanoTime();
        long s3 = Arrays.stream(t).parallel().sum();
```

```
        long d3 = System.nanoTime();

        System.out.println("Sommes identiques : " + (s1 == s2 && s2 == s3));
        System.out.printf("boucle %d ms, 8 tâches %d ms, stream parallèle %d ms%n",
                (d1 - d0) / 1_000_000, (d2 - d1) / 1_000_000, (d3 - d2) / 1_000_000);
    }
}
```

Exemple de sortie (varie d'une exécution à l'autre)

```
Sommes identiques : true
boucle 21 ms, 8 tâches 9 ms, stream parallèle 6 ms
```

Les gains dépendent du nombre de cœurs de la machine. Le calcul est ici **limité par le processeur** (*CPU-bound*) : aucun thread n'attend d'entrée/sortie. Les threads virtuels, qui brillent quand les tâches **attendent**, n'apporteraient donc rien : le nombre de cœurs reste la limite.

Corrigé 14.4 — Comparateur de transporteurs

Pour que le corrigé soit reproductible, les délais et l'échec sont fixés au lieu d'être tirés au hasard.

Transporteurs.java

```
import java.util.*;
import java.util.concurrent.*;

public class Transporteurs {
    record Offre(String transporteur, long prix) { }

    static Offre interroger(String nom, long delaiMs, long prix, boolean echoue) {
        try { Thread.sleep(delaiMs); } catch (InterruptedException e)
                { Thread.currentThread().interrupt(); }
        if (echoue) throw new IllegalStateException(nom + " indisponible");
        return new Offre(nom, prix);
    }

    public static void main(String[] args) {
        try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
            List<CompletableFuture<Optional<Offre>>> appels = List.of(
                appel(ex, "Yobanté Express", 300, 1_800, false),
                appel(ex, "Jakarta Moto", 1_400, 1_200, false),       // trop lent
                appel(ex, "Livraison Pro", 200, 1_000, true));        // en panne
            CompletableFuture.allOf(appels.toArray(CompletableFuture[]::new)).join();

            Offre meilleure = appels.stream().map(CompletableFuture::join)
                    .flatMap(Optional::stream)
                    .min(Comparator.comparingLong(Offre::prix))
                    .orElse(new Offre("tarif par défaut", 2_000));
            System.out.println("Meilleure offre : " + meilleure);
        }
    }

    static CompletableFuture<Optional<Offre>> appel(ExecutorService ex, String nom, long delai,
            long prix, boolean echoue) {
        return CompletableFuture.supplyAsync(() -> Optional.of(interroger(nom, delai,
                prix, echoue)), ex)
                // pas de réponse en 1 s : ignoré
                .completeOnTimeout(Optional.empty(), 1, TimeUnit.SECONDS)
                .exceptionally(e -> {
                    System.out.println("Écarté : " + e.getCause().getMessage());
                    return Optional.empty();
                });
    }
}
```

Sortie console

```
Écarté : Livraison Pro indisponible
Meilleure offre : Offre[transporteur=Yobanté Express, prix=1800]
```

Jakarta Moto proposait le meilleur prix, mais n'a pas répondu dans la seconde : son offre est ignorée.

Corrigé 14.5 — Chaîne de préparation

ChainePreparation.java

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class ChainePreparation {
    static final String FIN = "FIN";

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<String> aPreparer = new LinkedBlockingQueue<>();
        BlockingQueue<String> aLivrer = new LinkedBlockingQueue<>();
        AtomicInteger livrees = new AtomicInteger();
        long debut = System.nanoTime();

        Thread reception = Thread.ofVirtual().start(() -> {
            for (int i = 1; i <= 20; i++) aPreparer.add("TS-" + i);
            // une pilule par préparateur
            aPreparer.add(FIN); aPreparer.add(FIN);
        });
        Runnable preparateur = () -> {
            try {
                for (String c = aPreparer.take(); !c.equals(FIN); c = aPreparer.take()) {
                    Thread.sleep(200);                          // préparation
                    aLivrer.add(c);
                }
                aLivrer.add(FIN);                               // signale sa fin au livreur
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        };
        Thread p1 = Thread.ofVirtual().start(preparateur), p2 = Thread.ofVirtual()
                .start(preparateur);
        Thread livreur = Thread.ofVirtual().start(() -> {
            try {
                int finsRecues = 0;
                // attend la fin des DEUX préparateurs
                while (finsRecues < 2) {
                    String c = aLivrer.take();
                    if (c.equals(FIN)) finsRecues++; else livrees.incrementAndGet();
                }
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        for (Thread t : new Thread[]{reception, p1, p2, livreur}) t.join();
        long ms = (System.nanoTime() - debut) / 1_000_000;
        System.out.println(livrees.get() + " commandes livrées");
        System.out.println("Environ 2 s (20 × 200 ms / 2 préparateurs) ? "
                + (ms >= 1_900 && ms < 3_000));
    }
}
```

Sortie console

```
20 commandes livrées
Environ 2 s (20 × 200 ms / 2 préparateurs) ? true
```

Corrigé 14.6 — Banc d'essai : plateforme contre virtuels

BancEssai.java

```
import java.util.concurrent.*;
import java.util.stream.IntStream;
```

```
public class BancEssai {
    static final ScopedValue<Integer> REQUETE = ScopedValue.newInstance();
    static final Semaphore CONNEXIONS_BD = new Semaphore(50);

    static void appel(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException e)
                { Thread.currentThread().interrupt(); }
    }

    static void traiterRequete(boolean limiterBd) {
        if (limiterBd) {
            CONNEXIONS_BD.acquireUninterruptibly();
            try { appel(50); } finally { CONNEXIONS_BD.release(); }
        } else {
            appel(50);                                  // base de données
        }
        appel(50);                                      // service de stock
        appel(50);                                      // opérateur de paiement
        if (REQUETE.get() % 1_000 == 0) System.out.println("  requête "
                + REQUETE.get() + " terminée");
    }

    static long mesurer(ExecutorService executeur, boolean limiterBd) {
        long debut = System.nanoTime();
        try (executeur) {
            IntStream.range(0, 5_000).forEach(i -> executeur.submit(() ->
                    ScopedValue.where(REQUETE, i).run(() -> traiterRequete(limiterBd))));
        }
        return (System.nanoTime() - debut) / 1_000_000;
    }

    public static void main(String[] args) {
        System.out.println("Pool de 100 threads de plateforme : "
                + mesurer(Executors.newFixedThreadPool(100), false) + " ms");
        System.out.println("Threads virtuels                  : "
                + mesurer(Executors.newVirtualThreadPerTaskExecutor(), false) + " ms");
        System.out.println("Virtuels + 50 connexions BD       : "
                + mesurer(Executors.newVirtualThreadPerTaskExecutor(), true) + " ms");
    }
}
```

Exemple de sortie (varie d'une exécution à l'autre)

```
  requête 0 terminée
  ...
Pool de 100 threads de plateforme : 7650 ms
  ...
Threads virtuels                  : 290 ms
  ...
Virtuels + 50 connexions BD       : 5210 ms
```

Avec 100 threads de plateforme, les 5 000 requêtes de 150 ms passent par « vagues » de 100 : environ 5 000 / 100 × 150 ms ≈ 7,5 s. Avec des threads virtuels, toutes les requêtes attendent en même temps : un peu plus de 150 ms. La limitation à 50 accès simultanés à la base (le `Semaphore`) redevient le goulot d'étranglement : 5 000 / 50 × 50 ms ≈ 5 s. C'est la situation réelle d'un serveur : le nombre de connexions à la base, et non le nombre de threads, limite le débit.

Corrigés du chapitre 15

Corrigé 15.1 — Premier projet Maven

Terminal

```
mvn archetype:generate -DgroupId=sn.isep -DartifactId=outils-caisse \
    -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
cd outils-caisse
```

Dans le `pom.xml` généré, remplacez les propriétés et la dépendance de test par celles du chapitre 15 (`maven.compiler.release` à 25, `junit-jupiter` en version 6.1.0 avec la portée `test`), placez `Tarification.java` dans `src/main/java/sn/isep/` en ajoutant la ligne `package sn.isep;`, puis :

Terminal

```
mvn package
java -cp target/outils-caisse-1.0-SNAPSHOT.jar sn.isep.Tarification
mvn dependency:tree
```

L'arbre montre `junit-jupiter` et ses dépendances **transitives** (`junit-jupiter-api`, `junit-jupiter-params`, `junit-jupiter-engine`, `junit-platform-commons`, `opentest4j`, `apiguardian-api`…), toutes en portée `test` : aucune n'est incluse dans le `.jar` de l'application.

Corrigé 15.2 — Tester une classe existante

FractionTest.java

```java
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;

class FractionTest {

    @ParameterizedTest(name = "{0}/{1} s'écrit {2}")
    @CsvSource({"2, 4, 1/2", "6, -4, -3/2", "-6, -4, 3/2", "10, 5, 2", "0, 7, 0"})
    void simplificationEtSigne(long num, long den, String attendu) {
        assertEquals(attendu, new Fraction(num, den).toString());
    }

    @Test
    void operations() {
        Fraction a = new Fraction(1, 2), b = new Fraction(1, 3);
        assertAll(
            () -> assertEquals(new Fraction(5, 6), a.plus(b)),
            () -> assertEquals(new Fraction(1, 6), a.moins(b)),
            () -> assertEquals(new Fraction(1, 6), a.fois(b)),
            () -> assertEquals(new Fraction(3, 2), a.divisePar(b)),
            () -> assertEquals(0.5, a.valeur(), 1e-9)
        );
    }

    @Test
    void egaliteEtHashCode() {
        assertEquals(new Fraction(2, 4), new Fraction(-1, -2));
        assertEquals(new Fraction(2, 4).hashCode(), new Fraction(-1, -2).hashCode());
        assertNotEquals(new Fraction(1, 2), new Fraction(1, 3));
    }

    @Test
    void denominateurNul() {
        assertThrows(ArithmeticException.class, () -> new Fraction(1, 0));
        assertThrows(ArithmeticException.class, () -> new Fraction(1, 2)
                .divisePar(new Fraction(0, 5)));
```

```
    }
}
```

Le dernier test vérifie un cas limite facile à oublier : diviser par la fraction nulle revient à créer une fraction de dénominateur nul.

Corrigé 15.3 — Tests paramétrés et cas limites

Le test avec `null` échoue avec une `NullPointerException` : la méthode du chapitre 3 appelle `replaceAll` sans vérifier son argument. Version corrigée et tests :

Telephone.java

```java
public final class Telephone {
    private Telephone() { }

    public static boolean estMobileValide(String numero) {
        if (numero == null) return false;
        String nettoye = numero.replaceAll("[\\s.-]", "");
        return nettoye.matches("(\\+221)?7[05678]\\d{7}");
    }
}
```

TelephoneTest.java

```java
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

class TelephoneTest {
    @ParameterizedTest
    @ValueSource(strings = {"771234567", "77 123 45 67", "+221771234567", "+221 78-000-11-22",
            "70.555.44.33", "76 000 00 00"})
    void numerosValides(String numero) {
        assertTrue(Telephone.estMobileValide(numero), numero);
    }

    @ParameterizedTest
    @ValueSource(strings = {"", "33 820 00 00", "76123456", "7712345678", "79 123 45 67",
            "+33 6 12 34 56 78", "77 12A 45 67"})
    void numerosInvalides(String numero) {
        assertFalse(Telephone.estMobileValide(numero), numero);
    }

    @Test
    void nullEstInvalide() {
        assertFalse(Telephone.estMobileValide(null));
    }
}
```

Corrigé 15.4 — TDD : le calcul des points de fidélité

Les étapes du cycle, dans l'ordre : (1) test « 2 500 FCFA donnent 2 points » → rouge, puis `return (int) (montant / 1000)` → vert ; (2) test « le vendredi, les points sont doublés » → rouge, ajout du paramètre `jour` et du doublement → vert, puis refactorisation des tests existants ; (3) test « bonus de 50 points pour la première commande » → rouge, puis vert ; (4) test « plafond de 500 points » → rouge, ajout de `Math.min` → vert ; (5) refactorisation finale : constantes nommées. Voici le résultat final :

CalculPoints.java

```java
import java.time.DayOfWeek;

public class CalculPoints {
    static final int TRANCHE = 1_000;
    static final int BONUS_PREMIERE_COMMANDE = 50;
    static final int PLAFOND = 500;

    public int points(long montant, DayOfWeek jour, boolean premiereCommande) {
        int points = (int) (montant / TRANCHE);
        if (jour == DayOfWeek.FRIDAY) points *= 2;
        if (premiereCommande) points += BONUS_PREMIERE_COMMANDE;
        return Math.min(points, PLAFOND);
    }
}
```

CalculPointsTest.java

```java
import org.junit.jupiter.api.Test;
import static java.time.DayOfWeek.*;
import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculPointsTest {
    private final CalculPoints calcul = new CalculPoints();

    @Test void unPointParTranche()   { assertEquals(2, calcul.points(2_500, MONDAY, false)); }
    @Test void moinsDUneTranche()    { assertEquals(0, calcul.points(999, MONDAY, false)); }
    @Test void doubleLeVendredi()    { assertEquals(4, calcul.points(2_500, FRIDAY, false)); }
    @Test void bonusPremiere()       { assertEquals(52, calcul.points(2_500, MONDAY, true)); }
    @Test void bonusNonDouble()      { assertEquals(54, calcul.points(2_500, FRIDAY, true)); }
    @Test void plafond500()          { assertEquals(500,
            calcul.points(900_000, FRIDAY, true)); }
}
```

Le test `bonusNonDouble` n'était pas prévu par l'énoncé : c'est en écrivant les tests qu'on se pose la question « le bonus est-il doublé le vendredi ? ». Le TDD fait apparaître les ambiguïtés des spécifications, qu'il faut alors trancher avec le client.

Corrigé 15.5 — Tester un service avec des doublures

ServiceCommande.java

```java
public class ServiceCommande {
    public interface CatalogueRepository {
        int stock(String reference);
        long prix(String reference);
        void ajusterStock(String reference, int delta);
    }
    public interface PasserellePaiement {
        void debiter(String telephone, long montant);        // lève PaiementRefuseException
    }
    public interface Notificateur {
        void envoyer(String telephone, String message);
    }
    public static class StockInsuffisantException extends RuntimeException {
        public StockInsuffisantException(String ref) { super("Stock insuffisant : " + ref); }
    }
    public static class PaiementRefuseException extends RuntimeException {
        public PaiementRefuseException(String motif) { super("Paiement refusé : " + motif); }
    }

    private final CatalogueRepository catalogue;
    private final PasserellePaiement paiement;
    private final Notificateur notificateur;

    public ServiceCommande(CatalogueRepository c, PasserellePaiement p, Notificateur n) {
        this.catalogue = c; this.paiement = p; this.notificateur = n;
```

```
    }

    public long passer(String telephone, String reference, int quantite) {
        if (catalogue.stock(reference) < quantite)
                throw new StockInsuffisantException(reference);
        long montant = catalogue.prix(reference) * quantite;
        catalogue.ajusterStock(reference, -quantite);
        try {
            paiement.debiter(telephone, montant);
        } catch (PaiementRefuseException e) {
            catalogue.ajusterStock(reference, quantite);         // compensation
            notificateur.envoyer(telephone, "Paiement refusé, commande annulée.");
            throw e;
        }
        notificateur.envoyer(telephone, "Commande confirmée : " + montant + " FCFA.");
        return montant;
    }
}
```

ServiceCommandeTest.java

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.*;
import static org.junit.jupiter.api.Assertions.*;

class ServiceCommandeTest {
    /** Faux dépôt en mémoire. */
    static class FauxCatalogue implements ServiceCommande.CatalogueRepository {
        final Map<String, Integer> stocks = new HashMap<>(Map.of("PRD-001", 10));
        public int stock(String r) { return stocks.getOrDefault(r, 0); }
        public long prix(String r) { return 1_500; }
        public void ajusterStock(String r, int delta) { stocks.merge(r, delta, Integer::sum); }
    }
    /** Passerelle configurable : réussit ou échoue, et compte les appels. */
    static class FaussePasserelle implements ServiceCommande.PasserellePaiement {
        boolean refuser = false;
        int appels = 0;
        public void debiter(String tel, long montant) {
            appels++;
            if (refuser) throw new ServiceCommande.PaiementRefuseException("solde insuffisant");
        }
    }
    /** Espion de notifications. */
    static class Espion implements ServiceCommande.Notificateur {
        final List<String> messages = new ArrayList<>();
        public void envoyer(String tel, String msg) { messages.add(msg); }
    }

    FauxCatalogue catalogue; FaussePasserelle passerelle; Espion espion; ServiceCommande service;

    @BeforeEach
    void preparer() {
        catalogue = new FauxCatalogue();
        passerelle = new FaussePasserelle();
        espion = new Espion();
        service = new ServiceCommande(catalogue, passerelle, espion);
    }

    @Test
    void commandeReussie() {
        assertEquals(4_500, service.passer("771234567", "PRD-001", 3));
        assertEquals(7, catalogue.stock("PRD-001"));
        assertEquals(List.of("Commande confirmée : 4500 FCFA."), espion.messages);
    }

    @Test
    void stockInsuffisantAucunPaiementTente() {
        assertThrows(ServiceCommande.StockInsuffisantException.class,
                () -> service.passer("771234567", "PRD-001", 11));
```

```
        assertEquals(0, passerelle.appels);
        assertEquals(10, catalogue.stock("PRD-001"));
    }

    @Test
    void paiementRefuseStockRestaureEtClientPrevenu() {
        passerelle.refuser = true;
        assertThrows(ServiceCommande.PaiementRefuseException.class,
                () -> service.passer("771234567", "PRD-001", 3));
        assertEquals(10, catalogue.stock("PRD-001"));
        assertEquals(List.of("Paiement refusé, commande annulée."), espion.messages);
    }
}
```

Corrigés du chapitre 16

Les programmes suivants s'exécutent sur la base obtenue à la fin du chapitre 16 et réutilisent les classes `Bd` et `Produit` de ce chapitre.

Corrigé 16.1 — Premières requêtes

Requetes.java

```
import java.sql.*;

public class Requetes {
    static void afficher(Connection cx, String titre, String sql,
            Object... params) throws SQLException {
        System.out.println("== " + titre);
        try (PreparedStatement ps = cx.prepareStatement(sql)) {
            for (int i = 0; i < params.length; i++) ps.setObject(i + 1, params[i]);
            try (ResultSet rs = ps.executeQuery()) {
                int n = rs.getMetaData().getColumnCount();          // métadonnées du résultat
                while (rs.next()) {
                    StringBuilder ligne = new StringBuilder("  ");
                    for (int c = 1; c <= n; c++) ligne.append(rs.getString(c))
                            .append(c < n ? " | " : "");
                    System.out.println(ligne);
                }
            }
        }
    }

    public static void main(String[] args) throws SQLException {
        try (Connection cx = Bd.connexion()) {
            afficher(cx, "Clients", "SELECT nom, quartier FROM client ORDER BY nom");
            afficher(cx, "Produits par catégorie",
                    "SELECT categorie, COUNT(*) FROM produit GROUP BY categorie ORDER BY "
                            + "categorie");
            afficher(cx, "Plus cher par catégorie", """
                    SELECT DISTINCT ON (categorie) categorie, libelle, prix
                    FROM produit ORDER BY categorie, prix DESC""");
            // en pratique : saisi au clavier
            String mot = "karité";
            afficher(cx, "Recherche « " + mot + " »",
                    "SELECT reference, libelle FROM produit WHERE libelle ILIKE ?", "%" + mot
                            + "%");
        }
    }
}
```

Sortie console

```
== Clients
  Awa Diop | Médina
  Moussa Ndiaye | Rufisque
== Produits par catégorie
  Alimentaire | 1
  Artisanat | 2
  Boisson | 2
  Cosmétique | 1
  Textile | 2
== Plus cher par catégorie
  Alimentaire | Thiakry 500 g | 1000
  Artisanat | Sac en cuir de Ngaye | 25000
  Boisson | Café Touba 250 g | 2000
  Cosmétique | Beurre de karité | 3000
  Textile | Boubou brodé | 38500
== Recherche « karité »
  PRD-030 | Beurre de karité
```

Le caractère % fait partie de la **valeur** du paramètre, pas du texte SQL : la requête reste protégée contre les injections.

Corrigé 16.2 — Mise à jour sécurisée

Promotion.java

```java
import java.sql.*;

public class Promotion {
    static int appliquerPromotion(Connection cx, String categorie,
            int pourcentage) throws SQLException {
        if (pourcentage < 1 || pourcentage > 50)
            throw new IllegalArgumentException("Pourcentage hors de [1, 50]");
        String sql = "UPDATE produit SET prix = GREATEST(100, prix * (100 - ?) / 100) WHERE "
                + "categorie = ?";
        try (PreparedStatement ps = cx.prepareStatement(sql)) {
            ps.setInt(1, pourcentage);
            ps.setString(2, categorie);
            return ps.executeUpdate();
        }
    }

    public static void main(String[] args) throws SQLException {
        try (Connection cx = Bd.connexion()) {
            System.out.println(appliquerPromotion(cx, "Artisanat", 20)
                    + " produit(s) modifié(s)");
            try (PreparedStatement ps = cx.prepareStatement(
                    "SELECT reference, prix FROM produit WHERE categorie = 'Artisanat' ORDER "
                            + "BY reference");
                 ResultSet rs = ps.executeQuery()) {
                while (rs.next()) System.out.println(rs.getString(1) + " -> " + rs.getLong(2));
            }
            try {
                appliquerPromotion(cx, "Textile", 80);
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

Sortie console

```
2 produit(s) modifié(s)
PRD-020 -> 20000
PRD-021 -> 4800
Pourcentage hors de [1, 50]
```

Corrigé 16.3 — DAO des clients

Cette fois, la source de données est le `PGSimpleDataSource` fourni par le pilote PostgreSQL (sans pool) : le DAO ne voit que l'interface `DataSource` et fonctionnerait à l'identique avec HikariCP.

ClientDaoDemo.java

```
import org.postgresql.ds.PGSimpleDataSource;
import javax.sql.DataSource;
import java.sql.*;
import java.util.*;

public class ClientDaoDemo {
    record Client(Long id, String nom, String telephone, String quartier) { }

    static class TelephoneDejaUtiliseException extends RuntimeException {
        TelephoneDejaUtiliseException(String tel) { super("Téléphone déjà utilisé : " + tel); }
    }

    interface ClientDao {
        Client creer(Client c);
        Optional<Client> trouverParId(long id);
        Optional<Client> trouverParTelephone(String tel);
        List<Client> trouverParQuartier(String quartier);
        boolean changerQuartier(long id, String quartier);
    }

    static class JdbcClientDao implements ClientDao {
        private final DataSource ds;
        JdbcClientDao(DataSource ds) { this.ds = ds; }

        public Client creer(Client c) {
            String sql = "INSERT INTO client (nom, telephone, quartier) VALUES (?, ?, ?) "
                    + "RETURNING id";
            try (Connection cx = ds.getConnection();
                 PreparedStatement ps = cx.prepareStatement(sql)) {
                ps.setString(1, c.nom()); ps.setString(2,
                        c.telephone()); ps.setString(3, c.quartier());
                try (ResultSet rs = ps.executeQuery()) {
                    rs.next();
                    return new Client(rs.getLong(1), c.nom(), c.telephone(), c.quartier());
                }
            } catch (SQLException e) {
                if ("23505".equals(e.getSQLState()))
                        throw new TelephoneDejaUtiliseException(c.telephone());
                throw new IllegalStateException("Erreur d'accès aux données", e);
            }
        }
        public Optional<Client> trouverParId(long id) {
            return requete("SELECT id, nom, telephone, quartier FROM client WHERE id = ?", id)
                    .stream().findFirst();
        }
        public Optional<Client> trouverParTelephone(String tel) {
            return requete("SELECT id, nom, telephone, quartier FROM client WHERE telephone = "
                    + "?", tel).stream().findFirst();
        }
        public List<Client> trouverParQuartier(String q) {
            return requete("SELECT id, nom, telephone, quartier FROM client WHERE quartier = "
                    + "? ORDER BY nom", q);
        }
        public boolean changerQuartier(long id, String quartier) {
            try (Connection cx = ds.getConnection();
                 PreparedStatement ps = cx.prepareStatement("UPDATE client SET quartier = ? "
                         + "WHERE id = ?")) {
                ps.setString(1, quartier); ps.setLong(2, id);
                return ps.executeUpdate() == 1;
            } catch (SQLException e) {
                throw new IllegalStateException("Erreur d'accès aux données", e);
            }
        }
        private List<Client> requete(String sql, Object param) {
```

```
            try (Connection cx = ds.getConnection();
                 PreparedStatement ps = cx.prepareStatement(sql)) {
                ps.setObject(1, param);
                try (ResultSet rs = ps.executeQuery()) {
                    List<Client> r = new ArrayList<>();
                    while (rs.next()) r.add(new Client(rs.getLong("id"), rs.getString("nom"),
                            rs.getString("telephone"), rs.getString("quartier")));
                    return r;
                }
            } catch (SQLException e) {
                throw new IllegalStateException("Erreur d'accès aux données", e);
            }
        }
    }

    public static void main(String[] args) {
        PGSimpleDataSource ds = new PGSimpleDataSource();
        ds.setUrl(Bd.URL); ds.setUser(Bd.UTILISATEUR); ds.setPassword(Bd.MOT_DE_PASSE);
        ClientDao dao = new JdbcClientDao(ds);

        Client fatou = dao.creer(new Client(null, "Fatou Sow", "761112233", "Médina"));
        System.out.println("Créée : " + fatou);
        try {
            dao.creer(new Client(null, "Autre", "761112233", "Fann"));
        } catch (TelephoneDejaUtiliseException e) {
            System.out.println(e.getMessage());
        }
        dao.changerQuartier(fatou.id(), "Fann");
        System.out.println(dao.trouverParTelephone("761112233").orElseThrow());
        System.out.println("Médina : " + dao.trouverParQuartier("Médina"));
        System.out.println("Id 99 : " + dao.trouverParId(99).isPresent());
    }
}
```

Sortie console

```
Créée : Client[id=4, nom=Fatou Sow, telephone=761112233, quartier=Médina]
Téléphone déjà utilisé : 761112233
Client[id=4, nom=Fatou Sow, telephone=761112233, quartier=Fann]
Médina : [Client[id=1, nom=Awa Diop, telephone=771234567, quartier=Médina]]
Id 99 : false
```

L'identifiant attribué est 4 et non 3 : lors de la tentative d'insertion refusée du chapitre 16 (téléphone en double), PostgreSQL avait déjà consommé la valeur 3 de la séquence. Les identifiants générés sont **uniques**, mais pas forcément **consécutifs** : ne leur donnez jamais de signification métier.

Corrigé 16.4 — Import massif

ImportMassif.java

```
import java.sql.*;

public class ImportMassif {
    static final int N = 10_000;

    static long mesurer(String titre, Connection cx, boolean transaction,
            int tailleLot) throws SQLException {
        try (Statement st = cx.createStatement()) {
            st.execute("CREATE TEMP TABLE IF NOT EXISTS essai (ref VARCHAR(10), prix BIGINT)");
            st.execute("TRUNCATE essai");
        }
        long debut = System.nanoTime();
        cx.setAutoCommit(!transaction);
        try (PreparedStatement ps = cx.prepareStatement("INSERT INTO essai VALUES (?, ?)")) {
            for (int i = 0; i < N; i++) {
                ps.setString(1, "P" + i);
                ps.setLong(2, 1_000 + i);
                if (tailleLot == 0) ps.executeUpdate();
```

```
                else {
                    ps.addBatch();
                    if ((i + 1) % tailleLot == 0) ps.executeBatch();
                }
            }
            if (tailleLot > 0) ps.executeBatch();
        }
        if (transaction) cx.commit();
        cx.setAutoCommit(true);
        long ms = (System.nanoTime() - debut) / 1_000_000;
        System.out.printf("%-35s %6d ms%n", titre, ms);
        return ms;
    }

    public static void main(String[] args) throws SQLException {
        try (Connection cx = Bd.connexion()) {
            mesurer("Une requête par ligne, auto-commit", cx, false, 0);
            mesurer("Une requête par ligne, 1 transaction", cx, true, 0);
            mesurer("Lots de 500, 1 transaction", cx, true, 500);
        }
    }
}
```

Résultats typiques sur un poste de développement (base locale) : quelques secondes en auto-commit (chaque ligne est une transaction, avec une écriture synchrone sur disque), nettement moins dans une seule transaction, et quelques centaines de millisecondes par lots (le nombre d'allers-retours réseau est divisé par 500). Sur une base distante, l'écart est encore plus spectaculaire. Conclusion : pour les imports, **une transaction et des lots**. (PostgreSQL offre aussi la commande `COPY`, accessible via l'API `CopyManager` du pilote, encore plus rapide.)

Corrigé 16.5 — Annulation de commande

AnnulerCommande.java

```
import java.sql.*;

public class AnnulerCommande {
    static void annulerCommande(Connection cx, long commandeId) throws SQLException {
        boolean ac = cx.getAutoCommit();
        cx.setAutoCommit(false);
        try {
            String statut;
            try (PreparedStatement ps = cx.prepareStatement("SELECT statut FROM commande "
                    + "WHERE id = ? FOR UPDATE")) {
                // FOR UPDATE : verrouille la ligne
                ps.setLong(1, commandeId);
                try (ResultSet rs = ps.executeQuery()) {
                    if (!rs.next())
                            throw new IllegalArgumentException("Commande introuvable : "
                                    + commandeId);
                    statut = rs.getString(1);
                }
            }
            if (!statut.equals("EN_ATTENTE") && !statut.equals("PAYEE")) {
                throw new IllegalStateException("Impossible d'annuler une commande " + statut);
            }
            try (PreparedStatement ps = cx.prepareStatement("""
                    UPDATE produit p SET stock = p.stock + l.quantite
                    FROM ligne_commande l
                    WHERE l.commande_id = ? AND l.reference = p.reference""")) {
                ps.setLong(1, commandeId);
                System.out.println(ps.executeUpdate() + " produit(s) remis en stock");
            }
            try (PreparedStatement ps = cx.prepareStatement("UPDATE commande SET statut = "
                    + "'ANNULEE' WHERE id = ?")) {
                ps.setLong(1, commandeId);
                ps.executeUpdate();
            }
```

```
            cx.commit();
        } catch (SQLException | RuntimeException e) {
            cx.rollback();
            throw e;
        } finally {
            cx.setAutoCommit(ac);
        }
    }

    public static void main(String[] args) throws SQLException {
        try (Connection cx = Bd.connexion()) {
            annulerCommande(cx, 1);
            try (Statement st = cx.createStatement();
                 ResultSet rs = st.executeQuery("SELECT reference, stock FROM produit WHERE "
                         + "reference IN ('PRD-001','PRD-030') ORDER BY 1")) {
                while (rs.next()) System.out.println(rs.getString(1) + " = " + rs.getInt(2));
            }
            try {
                annulerCommande(cx, 1);
            } catch (IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

Sortie console

```
2 produit(s) remis en stock
PRD-001 = 40
PRD-030 = 21
Impossible d'annuler une commande ANNULEE
```

La remise en stock se fait en **une seule** requête `UPDATE … FROM` (syntaxe PostgreSQL), qui joint les lignes de la commande aux produits. Le `SELECT … FOR UPDATE` empêche une autre transaction de modifier la commande pendant l'annulation.

Corrigé 16.6 — Tableau de bord SQL

Chaque statistique correspond à une requête et à un record. Voici les requêtes ; le code Java suit le modèle de `LireProduits` (chapitre 16) avec un record par type de ligne.

SQL

```sql
-- CA par jour sur 30 jours -> record CaJour(LocalDate jour, long ca)
SELECT date_trunc('day', cree_le)::date AS jour, SUM(total) AS ca
FROM commande
WHERE statut <> 'ANNULEE' AND cree_le >= now() - INTERVAL '30 days'
GROUP BY jour ORDER BY jour;

-- Top 5 des produits -> record TopProduit(String ref, String libelle, long qte, long montant)
SELECT p.reference, p.libelle, SUM(l.quantite) AS qte,
       SUM(l.quantite * l.prix_unitaire) AS montant
FROM ligne_commande l JOIN produit p ON p.reference = l.reference
JOIN commande c ON c.id = l.commande_id AND c.statut <> 'ANNULEE'
GROUP BY p.reference, p.libelle
ORDER BY qte DESC, montant DESC LIMIT 5;

-- Panier moyen par mode -> record PanierMoyen(String mode, long moyenne, long nb)
SELECT mode_paiement, ROUND(AVG(total)) AS moyenne, COUNT(*) AS nb
FROM commande WHERE statut <> 'ANNULEE'
GROUP BY mode_paiement ORDER BY moyenne DESC;

-- Clients sans commande -> record Client(...)
SELECT cl.id, cl.nom, cl.telephone
FROM client cl LEFT JOIN commande c ON c.client_id = cl.id
WHERE c.id IS NULL;
```

```sql
-- Taux d'annulation par quartier -> record TauxAnnulation(String quartier, double taux)
SELECT cl.quartier,
       ROUND(100.0 * COUNT(*) FILTER (WHERE c.statut = 'ANNULEE') / COUNT(*), 1) AS taux
FROM commande c JOIN client cl ON cl.id = c.client_id
GROUP BY cl.quartier ORDER BY taux DESC;
```

Pour le jeu de données, générez au moins 50 commandes avec un `Random` à graine fixe (comme dans le corrigé 12.6), en insérant commandes et lignes par lots dans une seule transaction, et en variant `cree_le` (`now() - (? * INTERVAL '1 day')`).

Corrigés du chapitre 17

Corrigé 17.1 — Concevoir des routes REST

Opération	Requête	Codes possibles	Idempotente
Livraisons du jour d'un livreur	`GET /api/livreurs/{id}/livraisons?date=2026-09-27`	200, 404	oui (lecture)
Affecter une commande à un livreur	`PUT /api/commandes/{numero}/livreur` avec `{"livreurId": 7}`	200, 404, 409	oui : réaffecter au même livreur ne change rien
Signaler une livraison effectuée	`POST /api/livraisons/{id}/evenements` avec `{"type": "LIVREE"}`	201, 404, 409	non
Signaler un échec	`POST /api/livraisons/{id}/evenements` avec `{"type": "ECHEC", "motif": "absent"}`	201, 400, 404, 409	non
Historique des statuts	`GET /api/livraisons/{id}/evenements`	200, 404	oui

Modéliser les changements de statut comme des **événements** que l'on ajoute (`POST`) plutôt que comme une modification directe du statut conserve l'historique et correspond bien au métier. Le code 409 signale une transition interdite (par exemple « livrée » après « retour à l'entrepôt »).

Corrigé 17.2 — Client d'API publique

Exemple avec l'API météo gratuite et sans clé Open-Meteo (le programme ne peut évidemment fonctionner qu'avec un accès à Internet) :

`MeteoDakar.java`

```java
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.net.URI;
import java.net.http.*;
import java.time.Duration;

public class MeteoDakar {
    @JsonIgnoreProperties(ignoreUnknown = true)
    record Actuel(String time, double temperature_2m, double wind_speed_10m) { }
    @JsonIgnoreProperties(ignoreUnknown = true)
    record Reponse(double latitude, double longitude, Actuel current) { }

    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newBuilder()
                .connectTimeout(Duration.ofSeconds(5)).build();
        HttpRequest req = HttpRequest.newBuilder(URI.create(
                "https://api.open-meteo.com/v1/forecast?latitude=14.69&longitude=-17.44"
                + "&current=temperature_2m,wind_speed_10m"))
                .timeout(Duration.ofSeconds(10)).header("Accept", "application/json").build();
```

```
        try {
            HttpResponse<String> r = client.send(req, HttpResponse.BodyHandlers.ofString());
            if (r.statusCode() != 200) {
                System.out.println("Réponse inattendue : " + r.statusCode());
                return;
            }
            Reponse meteo = new ObjectMapper().readValue(r.body(), Reponse.class);
            System.out.printf("Dakar, %s : %.1f °C, vent %.0f km/h%n",
                    meteo.current().time(), meteo.current().temperature_2m(), meteo.current()
                            .wind_speed_10m());
        } catch (HttpTimeoutException e) {
            System.out.println("Délai dépassé");
        } catch (com.fasterxml.jackson.core.JsonProcessingException e) {
            System.out.println("JSON inattendu : " + e.getOriginalMessage());
        }
    }
}
```

Corrigé 17.3 — Compléter la mini-API

Les ajouts dans `ProduitControleur.traiter` (les nouvelles routes) et dans le service :

Java

```
case "PATCH 4" -> {
    // seuls les champs présents
    JsonNode modifs = json.readTree(ex.getRequestBody());
    Produit p = service.obtenir(segments[3]);
    long prix = modifs.has("prix") ? modifs.get("prix").asLong() : p.prix();
    int stock = modifs.has("stock") ? modifs.get("stock").asInt() : p.stock();
    envoyer(ex, 200, service.remplacer(new Produit(p.reference(), p.libelle(), prix, stock)));
}
case "GET 3" -> {
    int page = parametre(ex, "page").map(Integer::parseInt).orElse(0);
    int taille = Math.min(50, parametre(ex, "taille").map(Integer::parseInt).orElse(20));
    List<Produit> tous = service.lister(parametre(ex, "prixMax")
            .map(Long::valueOf).orElse(null));
    Page<Produit> resultat = paginer(tous, page, taille);          // méthode du chapitre 10
    ex.getResponseHeaders().add("X-Total-Count", String.valueOf(tous.size()));
    envoyer(ex, 200, resultat);
}
```

Pour la journalisation, on enveloppe le traitement dans le contexte du serveur :

Java

```
serveur.createContext("/api/produits", ex -> {
    long debut = System.nanoTime();
    controleur.traiter(ex);
    LOG.log(System.Logger.Level.INFO, "{0} {1} -> {2} ({3} ms)", ex.getRequestMethod(),
            ex.getRequestURI(), ex.getResponseCode(), (System.nanoTime() - debut) / 1_000_000);
});
```

La méthode `remplacer` du service vérifie que le produit existe (sinon 404) et que le prix est positif, puis appelle `depot.put`.

Corrigé 17.4 — Brancher la base de données

Le service ne dépend plus d'une `Map` mais de l'interface `ProduitDao` (chapitre 16), reçue dans son constructeur :

Java

```
static class ProduitService {
    private final ProduitDao dao;
    ProduitService(ProduitDao dao) { this.dao = dao; }
```

```
    Produit obtenir(String ref) {
        return dao.trouverParReference(ref)
                  .orElseThrow(() -> new ApiException(404, "PRODUIT_INTROUVABLE",
                          "Aucun produit " + ref));
    }
    Produit creer(Produit p) {
        // ... validations inchangées ...
        if (dao.trouverParReference(p.reference()).isPresent())
            throw new ApiException(409, "PRODUIT_EXISTANT", "Le produit " + p.reference()
                    + " existe déjà");
        dao.enregistrer(p);
        return p;
    }
}
// Dans le contrôleur, un catch supplémentaire, AVANT celui de RuntimeException :
} catch (AccesDonneesException e) {
    envoyer(ex, 503, new ErreurApi(503, "BASE_INDISPONIBLE",
            "Service momentanément indisponible"));
}
// Au démarrage :
var dataSource = new HikariDataSource(config);
var controleur = new ProduitControleur(new ProduitService(new JdbcProduitDao(dataSource)));
```

Remarque : le test « existe déjà » suivi de l'insertion n'est pas atomique ; deux requêtes simultanées pourraient passer le test. La contrainte `PRIMARY KEY` de la base garantit malgré tout l'unicité : il suffit de traduire l'erreur `23505` en 409.

Corrigé 17.5 — API de commandes et test de charge

Structure : un record `PasserCommandeRequete(long clientId, String modePaiement, List<LigneRequete> articles)` ; un `CommandeControleur` qui désérialise la requête, appelle un `CommandeService` qui emprunte une connexion au pool et appelle `passerCommande` (chapitre 16) ; la `StockInsuffisantException` est traduite en 409. Pour le test de charge :

Java

```
HttpClient client = HttpClient.newBuilder()
        .executor(Executors.newVirtualThreadPerTaskExecutor()).build();
String corps = """
    {"clientId": 1, "modePaiement": "WAVE",
     "articles": [{"reference": "PRD-001", "quantite": 1}]}""";
List<CompletableFuture<Integer>> reponses = IntStream.range(0, 1_000)
        .mapToObj(i -> client.sendAsync(HttpRequest.newBuilder(URI.create(base
                + "/api/commandes"))
                        .header("Content-Type", "application/json")
                        .POST(HttpRequest.BodyPublishers.ofString(corps)).build(),
                HttpResponse.BodyHandlers.discarding())
                .thenApply(HttpResponse::statusCode))
        .toList();
Map<Integer, Long> parCode = reponses.stream().map(CompletableFuture::join)
        .collect(Collectors.groupingBy(c -> c, TreeMap::new, Collectors.counting()));
System.out.println(parCode);     // par exemple {201=40, 409=960} avec un stock initial de 40
```

Le stock final doit valoir exactement 0 et le nombre de réponses 201 doit être égal au stock initial : c'est l'`UPDATE … WHERE stock >= ?` atomique qui le garantit, quelle que soit la concurrence.

Corrigés du chapitre 18

Corrigé 18.1 — Premier contrôleur

Java

```java
@RestController
@RequestMapping("/api")
public class BonjourControleur {
    public record Bonjour(String message, LocalDate date) { }

    @GetMapping("/bonjour")
    public Bonjour bonjour(@RequestParam(defaultValue = "visiteur") String nom) {
        return new Bonjour("Bonjour " + nom + ", bienvenue sur TerangaShop", LocalDate.now());
    }
}

@WebMvcTest(BonjourControleur.class)
class BonjourControleurTest {
    @Autowired MockMvcTester mvc;

    @Test
    void bonjourAvecNom() {
        assertThat(mvc.get().uri("/api/bonjour?nom=Awa"))
                .hasStatusOk()
                .bodyJson().extractingPath("$.message")
                        .isEqualTo("Bonjour Awa, bienvenue sur TerangaShop");
    }
}
```

Jackson sérialise le `LocalDate` au format ISO (`"2026-09-27"`) grâce à la configuration par défaut de Spring Boot.

Corrigé 18.2 — CRUD des clients

Points clés de la solution :

Java

```java
@Entity
public class Client {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, length = 100) private String nom;
    @Column(nullable = false, unique = true, length = 9) private String telephone;
    @Column(nullable = false, length = 60) private String quartier;
    protected Client() { }
    // constructeur, accesseurs, méthode demenager(String quartier)
}

public interface ClientRepository extends JpaRepository<Client, Long> {
    Optional<Client> findByTelephone(String telephone);
    List<Client> findByQuartierOrderByNom(String quartier);
    boolean existsByTelephone(String telephone);
}

public record CreerClient(
        @NotBlank @Size(max = 100) String nom,
        @NotBlank @Pattern(regexp = "7[05678]\\d{7}",
                message = "numéro mobile sénégalais attendu") String telephone,
        @NotBlank String quartier) { }

// Dans ClientService (@Service @Transactional) :
public Client inscrire(CreerClient demande) {
    if (repository.existsByTelephone(demande.telephone())) {
        // -> 409 dans le GestionnaireErreurs
        throw new TelephoneDejaUtiliseException(demande.telephone());
```

```
    }
    return repository.save(new Client(demande.nom(), demande.telephone(), demande.quartier()));
}

@PatchMapping("/{id}/quartier")
public ClientReponse demenager(@PathVariable long id,
        @RequestBody @Valid ChangerQuartier demande) {
    return ClientReponse.depuis(service.demenager(id, demande.quartier()));
}
```

Corrigé 18.3 — Pagination et tri

Java

```
@GetMapping
public Page<ProduitReponse> lister(@RequestParam(required = false) Long prixMin,
                                   @RequestParam(required = false) Long prixMax,
                                   @PageableDefault(size = 20,
                                           sort = "reference") Pageable pageable) {
    Pageable borne = PageRequest.of(pageable.getPageNumber(), Math.min(pageable.getPageSize(),
            50), pageable.getSort());
    long min = prixMin == null ? 0 : prixMin;
    long max = prixMax == null ? Long.MAX_VALUE : prixMax;
    return repository.findByPrixBetween(min, max, borne).map(ProduitReponse::depuis);
}
// Dans ProduitRepository :
Page<Produit> findByPrixBetween(long min, long max, Pageable pageable);
```

Spring Data traduit `Pageable` en `ORDER BY … LIMIT … OFFSET …` et exécute une requête `COUNT` pour le nombre total d'éléments. Pour sérialiser une `Page` de façon stable, Spring Boot recommande d'activer `@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)` ou de renvoyer votre propre record de page (comme `Page<T>` au chapitre 10).

Corrigé 18.4 — Tests du service et du contrôleur

Java

```
@Test
void creerUnProduitExistantEstRefuse() {
    when(repository.existsById("PRD-001")).thenReturn(true);
    var demande = new CreerProduit("PRD-001", "Bissap", "Boisson", 1_500, 10);
    assertThatThrownBy(() -> service.creer(demande))
            .isInstanceOf(ProduitExistantException.class);
    verify(repository, never()).save(any());
}

@Test
void creationInvalideDonne400AvecLesChamps() {
    assertThat(mvc.post().uri("/api/produits").contentType(MediaType.APPLICATION_JSON)
                  .content("""
                      {"reference": "X1", "libelle": "", "categorie": "Textile",
                       "prix": -5, "stock": 3}"""))
            .hasStatus(HttpStatus.BAD_REQUEST)
            .bodyJson().extractingPath("$.erreurs.prix").isNotNull();
}

@Test
void creationValideDonne201EtLocation() {
    when(service.creer(any())).thenReturn(new Produit("PRD-050", "Tapis",
            "Artisanat", 12_000, 6));
    assertThat(mvc.post().uri("/api/produits").contentType(MediaType.APPLICATION_JSON)
                  .content("""
                      {"reference": "PRD-050", "libelle": "Tapis", "categorie": "Artisanat",
                       "prix": 12000, "stock": 6}"""))
            .hasStatus(HttpStatus.CREATED)
            .hasHeader("Location", "/api/produits/PRD-050");
}
```

Corrigé 18.5 — Passer une commande avec Spring

Éléments de correction : `Commande` possède `@ManyToOne(optional = false) Client client`, `@Enumerated(EnumType.STRING) StatutCommande statut` et `@OneToMany(mappedBy = "commande", cascade = CascadeType.ALL, orphanRemoval = true) List<LigneCommande> lignes`. La méthode `CommandeService.passer` (annotée `@Transactional`) charge chaque produit avec `findById`, appelle `produit.retirer(quantite)`, crée les lignes, puis `commandeRepository.save(commande)`. Si `retirer` lève `StockInsuffisantException` (non vérifiée) sur le dernier article, Spring effectue un **rollback** : les modifications des produits précédents, suivies par Hibernate mais pas encore validées, ne sont jamais écrites. Le test d'intégration (`@SpringBootTest` sur une base PostgreSQL de test, idéalement avec Testcontainers) vérifie, après l'exception, que les stocks relus en base n'ont pas changé.

Avec un champ `@Version long version` dans `Produit`, Hibernate ajoute `WHERE version = ?` à chaque `UPDATE` et incrémente la version. Si deux transactions ont lu la même version et modifient toutes deux le produit, la seconde ne met à jour aucune ligne : Hibernate lève une `OptimisticLockException` (traduite par Spring en `ObjectOptimisticLockingFailureException`), que l'on peut traduire en 409 ou traiter par une nouvelle tentative. C'est le **verrouillage optimiste**, adapté aux conflits rares ; l'alternative est le verrouillage pessimiste (`SELECT … FOR UPDATE`, annotation `@Lock`).